WNDRVR
SUPPORT NETWORK

VXWORKS 7 FILE SYSTEMS AND I/O PROGRAMMER'S GUIDE, SR0650 EDITION 2

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 File Systems and I/O Programmer's Guide, SR0650 Edition 2

28 August 2020

1. ABOUT THE VXWORKS I/O AND FILE SYSTEMS

1. Overview of VxWorks File Systems and I/O Devices

VxWorks® provides several types of file systems that are suitable for different types of applications. The file systems can be used simultaneously, and in most cases in multiple instances, for a single VxWorks system. The VxWorks I/O system is a simple, uniform, device-independent interface to any kind of device.

This includes:

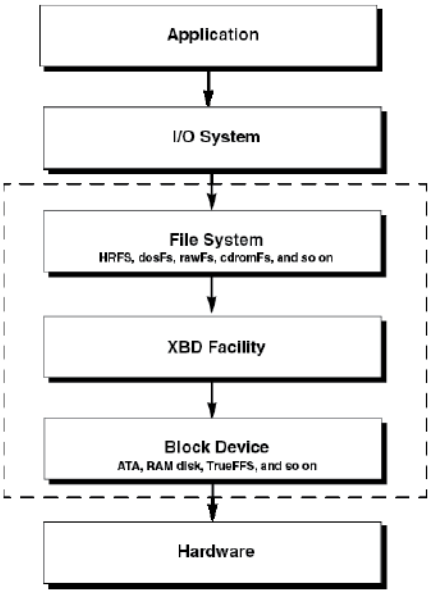
- character-oriented devices such as terminals and communications lines
- devices representing various sorts of file systems
- virtual devices such as intertask *pipes* and *sockets*
- monitor and control devices such as digital and analog I/O devices
- network devices that give access to remote devices

The VxWorks I/O system provides standard C libraries for both basic and buffered I/O. The basic I/O libraries are Posix-compatible; the buffered I/O libraries are ANSI C compatible. For information about VxWorks devices, see *VxWorks Devices Overview on page 167*.

The I/O system does not interface with block devices such as disks directly. Instead, it interfaces with devices that represent filesystems on these devices. The filesystem driver code interacts with the block devices at a lower level.

Most VxWorks file systems rely on the extended block device (XBD) facility for a standard I/O interface between the file system and device drivers. This standard interface allows you to write your own file system for VxWorks, and freely mix file systems and device drivers. For information about the XBD facility, see *Extended Block Device Facility: XBD Overview on page 79*.

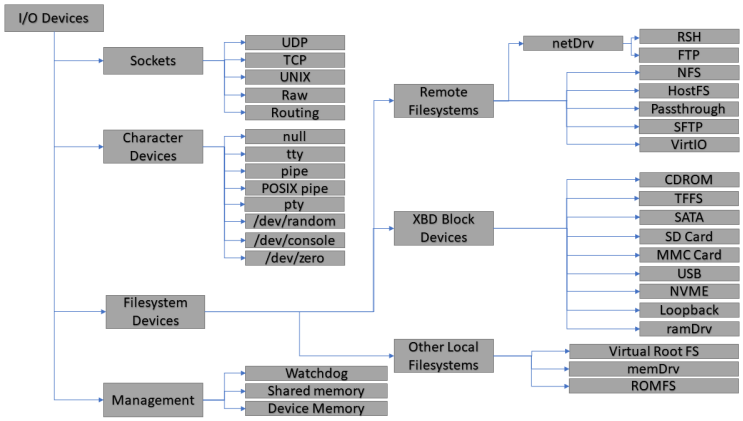
File systems used for removable devices use the file system monitor to automatically detect device insertion and instantiation of the appropriate file system on the device. For more information, see File System Monitor on page 80.



The relationship between applications, file systems, I/O facilities, device drivers and hardware devices is illustrated above. Note that this illustration is relevant for the HRFS, dosFs, rawFs, and cdromFs file systems. The dotted line indicates the elements that must be configured and instantiated to create a specific, functional run-time file system.

VxWorks I/O Devices

VxWorks supports a variety of device types and the VxWorks I/O system allows different device drivers to handle the seven basic I/O functions. The figure below shows how file systems and I/O devices are represented in VxWorks. For more information, see VxWorks Devices and Driver Model Overview on page 167.



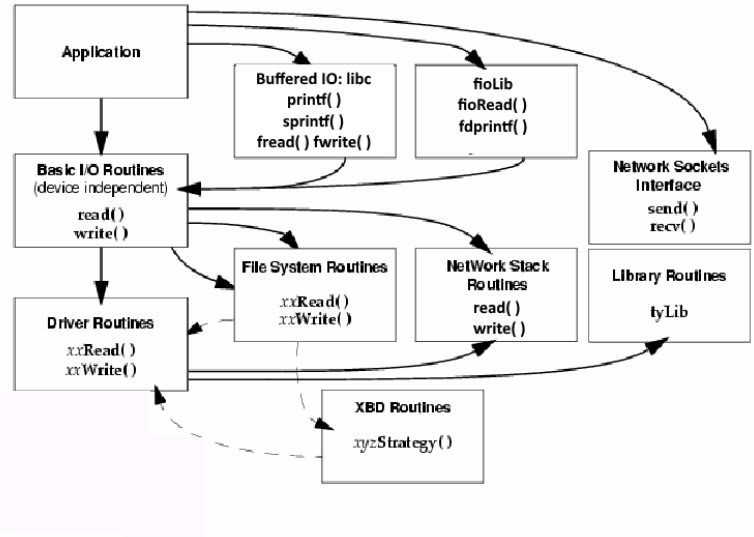
VxWorks File Systems

To see the relationship between the file systems and devices, see the table in I/O System and Device Naming Conventions on page 4.

Kernel I/O

The VxWorks I/O system is based upon the POSIX open()/read()/write()/ioctl()/close() interface. The I/O system implementation originated at a time when VxWorks was kernel-only and required only a very simple I/O system, with few protections and limited other POSIX support. Since the introduction of RTP support in VxWorks 6, the desire for more complete POSIX support within VxWorks has grown and the I/O system is gradually being adapted to more broadly support and comply with POSIX expectations. For more information, see Differences Between the User Mode I/O System and the Kernel I/O System on page 163.

The diagram in the figure below illustrates the relationships between the different elements of the VxWorks kernel I/O system.



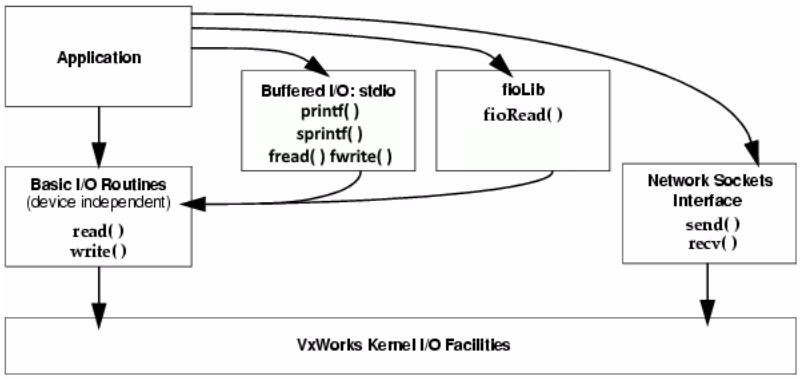
Note: The dotted lines indicate that the XBD facility is required for some file systems. For example, HRFS and dosFs require XBD, while ROMFS has its own interface to drivers. For more information, see Extended Block Device Facility: XBD Overview on page 79.

Note: The I/O system driver-level functions for read( ), write( ), ioctl( ), and close( ) on sockets are implemented by sockLib, and call functions provided by the sockLib back end (for example, IPNET, RTNET, CAN sockets, UN sockets, and so on).

I/O System and RTP Applications

The figure below illustrates the relationships between the different elements of the VxWorks I/O system available to real-time processes (RTPs). For more information, see I/O Devices, Named Files, and File Systems on page 137.

Figure 4. I/O System for RTPs (Processes)



Note: In the user space, functions such as read( ), write( ), send( ), and recv( ) are either system calls or small wrappers around system calls; while buffered I/O functions such as fread( ), printf( ), and so on work in the user space to support the buffering.

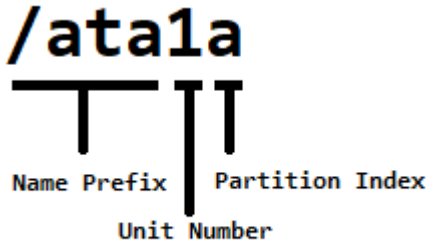
WNDRVR

2. I/O System and Device Naming Conventions

Legacy and the current naming conventions can be used in VxWorks as of SR0620.

Block Device Naming Convention

As of SR0620, VxWorks block devices use the following naming convention:



Name Prefix

The name prefix is **/ata** for SATA/ATA devices, **/bd** for USB storage device, **/sd** for SD Card storage device, and so on. For more information, see the table below.

Unit Number, Interface Number, or Logical Unit Number (LUN)

The unit number is the hardware interface index; it may start from **0** or **1**, depending on the hardware driver. The LUN may not be continuous. For USB devices, every physical device occupies 16 LUNs. For example, the first device is **/bd0**, the second device is **/bd16**. This rule applies to all USB storage devices such as USB disk and card readers.

Partition Index

The partition index is an alpha character from **a** to **z** to indicate from first partition to 26th partition. If the partition index is greater than **z**, it will start with **aa** to **az**, **ba** to **bz**, and so on.

File Storage Devices Names

The following table shows device names used when opening a file using an absolute path. Both the current and legacy naming convention are shown (for more information, Linking to Legacy Device Names on page 14). The variable portion in the name are identified by italics. This list does not include all possible file storage devices. For detailed information on filesystem device names, see I/O Devices, Named Files, and File Systems on page 137.

Note: A XBD block device can be formatted with either the DOSFS or HRFS filesystem. For more information, see Extended Block Device Facility: XBD Overview on page 79.

Device Name	XBD Block Device?	Legacy (Current/Strict) Device Name Example	Notes
True Flash File System NOR flash	Yes	/tffs1 (/tffs1)	For more information, see True Flash File System (TrueFFS) on page 106.
True Flash File System NAND flash	Yes	/ftl0:1 (/ftl0a)	For more information, see NAND Flash Translation Layer (NFTL) on page 121.

WNDRVR

Device Name	XBD Block Device?	Legacy (Current/Strict) Device Name Example	Notes
CDROM	Yes	/cdrom0 (/cdrom0)	Can be used (ISO9660) optical drives, also called CDROM. See cdromFs on page 73
SATA driver	Yes	/ata1:2 (/ata1b)	The device name can be changed to user defined values by using the **INCLUDE_SATA_DISK_NAME_CFG** component.
netDrv	No	remotename: (/host.remotename)	Remote filesystem on the network boot server using FTP or RSH depending on the boot parameters. For more information, see Non-NFS Network Device File Systems on page 88. Current name only applies if **INCLUDE_PATHNAME_RESOLUTION_STRICT** is not enabled.
VxSim Passthrough File System (PassFS)	No	mycomputer: (/host.mycomputer)	Only available with the VxWorks simulator. On Windows the computer name is always **/host** and on Linux it is the workstation name. For more information, see Accessing Remote File Systems from VxWorks on page 87. Current name only applies if **INCLUDE_PATHNAME_RESOLUTION_STRICT** is not enabled.
Non-Voltile Memory	Yes	/nvme1:3 (/nvme1c)	For more information, see RAM Drives on page 85.
USB Storage device	Yes	/bd16:1 (/bd16a)	
SD card	Yes	/sd0:1 (/sd0a)	
MultiMediaCard (MMC)	Yes	/mmc0:2 (/mmc0b)	
RAM drive	Yes	/ram0 (/ram0)	Device name is selected at creation. For more information, see RAM Drives on page 85.
ROM drive	No	/romfs (/romfs)	RomFS is a simple block device and read only filesystem. The device name can be selected at creation. For more information, see ROMFS on page 37.
HostFs	No	/hostfs (/hostfs)	Remote filesystem that uses the debugger connection as transport. For more information, see HostFs on page 104.

WNDRVR

Device Name	XBD Block Device?	Legacy (Current/Strict) Device Name Example	Notes
Loopback filesystem	Yes	/user.defined (/user.defined)	Used with encrypted filesystems and ISO image files (ISO9660). For more information, see Loop File System and Tcplay on page 126.
Network File System (NFS)	No	/user.defined (/user.defined)	The device name is selected at creation. For more information, see NFS and Remote File Systems on page 86.

Serial Port Devices

These are examples of serial port devices (where the variable portion in the name is identified by italics):

- traditional on-board serial ports: /ttyS0
- USB serial ports: /ttyUSB0
- virtual I/O serial ports: /ttyVIO0

2. CONFIGURING THE VXWORKS I/O SYSTEM

1. VxWorks I/O Facility Configuration

VxWorks provides a variety of I/O facilities that can be included or removed from the operating system in the VIP.

These facilities are described in the VxWorks I/O Device and File System Facilities and Programming I/O in Applications sections of this guide.

Primary I/O Components

The **INCLUDE_IO_SYSTEM** component provides the core components of the system and enables many of the components listed below.

Component	Description
SELECT_PATHNAME_RESOLUTION	I/O system device and pathname resolution model. You must select one of two sub-components: **INCLUDE_PATHNAME_RESOLUTION_LEGACY** (default, except in CERT builds) **INCLUDE_PATHNAME_RESOLUTION_STRICT** (default for CERT) For more information, see Enforcing Strict I/O Path and Device Naming Conventions on page 11.
INCLUDE_IO_BASIC	Basic I/O functionality
INCLUDE_IO_FILE_SYSTEM	File system support
INCLUDE_POSIX_DIRLIB	POSIX directory utilities
INCLUDE_IO_REMOVABLE	Support for removable file systems
INCLUDE_IO_POSIX	POSIX I/O support
INCLUDE_IO_RTP	I/O support for RTPs
INCLUDE_IO_MISC	Miscellaneous I/O functions that are no longer referenced but are provided for backwards compatibility
INCLUDE_POSIX_AIO	Asynchronous I/O support
INCLUDE_POSIX_AIO_SHOW	Asynchronous I/O show function
INCLUDE_POSIX_UMASK	Adds support for the POSIX file mode creation mask (umask( ) function) in the kernel environment

WNDRVR

Other I/O Library Components

Table 1. Other Library Components

Component	Description
INCLUDE_PIPES	Enables the use of VxWorks pipes. For more information, see the *VxWorks 7 Application Programmer's Guide: Pipes*.
INCLUDE_SELECT	Provides the kernel selectLib library which includes select() and related functions.
INCLUDE_RAMDRV	Includes a block device driver for RAM disk devices. This component does not create such a device or install a filesystem on it; instead, the **INCLUDE_RAM_DISK** component uses the newer **xbdRamDisk** driver and actually creates a RAM disk filesystem.
INCLUDE_MEMDRV	Enables a driver that can create a device representing a fixed-size block of RAM or a fixed-layout static filesystem of directories and files representing RAM blocks.
INCLUDE_PTYDRV	Enables the pseudo-terminal (pty) driver allowing creation of pty devices for communication between contexts.
INCLUDE_TYLIB	Support library for pseudo-terminal (pty) and terminal (tty) drivers.
INCLUDE_TTY_DEV	Upper-level driver for terminal (tty) devices (depending on lower-level serial device driver).
INCLUDE_POSIX_PIPES	Provides POSIX pipe(), pipe2() and mkfifo() support.

2. VxWorks I/O Global Configuration Parameters

The I/O system manages file descriptors as kernel objects. These are predefined system maximums for active file descriptors and apply to all devices and file systems within the global system used.

Item	Description
NUM_FILES	Defines the maximum number of open file descriptors in the kernel, and system wide. Note: In addition to the system maximum NUM_FILES, there are also individual limits on different types of file descriptors. These individual limits, some mentioned in this table, will limit the number of successful opens on the device they manage.

WNDRVR

Item	Description
	In the Workbench Kernel Configuration editor, the parameter is found in **Operating system (default) > IO system (default) > IO subsystem > Basic IO System > maximum number of open files in the kernel**. The default is NUM_FILES = 50. For more information see, select().
RTP_FD_NUM_MAX	Defines the total number of file descriptors for all the I/O devices in an RTP. Note: The limit can be changed at runtime using rtpIoTableSizeSet( ) or setprlimit( ). In the Workbench Kernel Configuration editor, the parameter is found in **Operating system (default) > Real-time process (RTP) > Real-time process (default) > Number of entries in an RTP fd table**. The default is RTP_FD_NUM_MAX = 20.
FOPEN_MAX	The current limit, in the current context, on the number of file streams (or FILE pointers) created with the fopen( ) family of functions (as specified in the C standard). Note: FOPEN_MAX is **not** used as a configuration parameter in VxWorks.
FOPEN_MAX_KERNEL	Defines the maximum number of streams (or FILE pointers) open in the kernel context. Note: This is VxWorks equivalent of FOPEN_MAX. In the Workbench Source Build Configuration editor, the parameter is found in **os > lang_lib > LIBC_STD_x_x_x_x Menu > Max File pointers in kernel**. The default is FOPEN_MAX_KERNEL = 50.
FOPEN_MAX_RTP	Defines the maximum number of streams (or FILE pointers) open for each individual RTP. Note: This is the VxWorks equivalent of FOPEN_MAX. In the Workbench Source Build Configuration editor, the parameter is found in **os > lang_lib > LIBC_STD_x_x_x_x Menu > Max File pointers per RTP**. The default is FOPEN_MAX_RTP = 20.
FD_SET_SIZE	Configures the number of file descriptors manageable by select( ). Note: Increasing this number may decrease the performance of select( ) calls that blindly pass FD_SETSIZE as the first parameter to select( ). **Caution:** The value of FD_SETSIZE is the greater of the values FD_SET_SIZE and NET_SOCK_MAX. By default, these have the same size, but if NET_SOCK_MAX is bigger, FD_SETSIZE would be set to NET_SOCK_MAX rather than to FD_SET_SIZE. As an alternative the function epoll( ) is available for network sockets and is more performant, and unaffected by FD_SET_SIZE. In the Workbench Source Build Configuration editor, the parameter is found in **os > core-x.x.x.x > CORE_x_x_x_x Menu > Value for FD_SETSIZE**. The default is FD_SET_SIZE = 2048. For more information see, select().

WNDRVR

Item	Description
SELECT_FDSET_OVERSIZE_ALLOW	Allows FD Tables to be larger than FD_SETSIZE **Caution**: Changing this value to TRUE has implications for binary compatibility between releases because this allows the size of the **fd_set** structure type to become larger and some applications using select( ) may not expect this. In the Workbench Kernel Configuration editor, the parameter is found in **Operating system > IO system > Select > Allow FD tables larger than FD_SETSIZE**. The default is SELECT_FDSET_OVERSIZE_ALLOW = FALSE. For more information see, select().

3. VxWorks I/O Device or File System Configuration Parameters

The I/O system manages file descriptors for specific I/O device types or file system types. These are predefined maximums for active file descriptors, I/O device types or file system types. These work with the Global Configuration Parameters to determine available maximums.

Item	Description
NET_SOCK_MAX	Configures the maximum number of sockets system wide. Each network socket uses a file descriptor. Network configuration commands may temporarily use an additional socket file descriptor such as routing commands. In the Workbench Source Build Configuration editor, the parameter is found in **net > Enable networking stack core libraries x_x_x_x > IPNET_COREIP_x_x_x_x Menu > Maximum number of sockets**. The default is NET_SOCK_MAX = 2048. For additional information see, VxWorks 7 Sockets Programmer's Guide.
DOSFS_DEFAULT_MAX_FILES	Sets the limit for the number of open files. In the Workbench Kernel Configuration editor, the parameter is found in **Storage > File system > DosFs file system components (dosFs2) > dosfs Fle System Main Module (dosFs2) > Default maximum open files (DosFs)**. The default is DOSFS_DEFAULT_MAX_FILES = 20.
HRFS_DEFAULT_MAX_FILES	Sets the limit for the number of files that can be open on an HRFS volume. In the Workbench Kernel Configuration editor, the parameter is found in **Storage > File system > Highly Reliable File System Components (HRFS) > Read-Only Highly Reliable Files System > Default Max Open Files (HRFS)**. The default is HRFS_DEFAULT_MAX_FILES = 10. For more information see, Maximum Number of Files and Directories on page 31.
PIPE_MAX_OPEN_FDS	Sets the limit for the number of open file descriptors for each pipe device. In the Workbench Kernel Configuration editor, the parameter is found in **Operating system > IO system > Pipes (defualt) > maximum open fds for each pipe device**. The default is PIPE_MAX_OPEN_FDS = 16.
ROMFS_MAX_OPEN_FILES	Sets the limit for the number of open files.

WNDRVR

Item	Description
	In the Workbench Kernel Configuration editor, the parameter is found in **Storage > File system > ROMFS file system > ROMFS file system driver > Default maximum open files (ROMFS)**. The default is ROMFS_MAX_OPEN_FILES = NUM_FILES.
NUM_VDFS_FILES	Sets the limit for the number of open VDFS files. In the Workbench Kernel Configuration editor, the parameter is found in **Edit Kernel ConfigurationStorage > File system > Virtual Disk file system > Virtual Disk filesystem > max # open VDFS files**. The default is NUM_VDFS_FILES = 64.

4. Enforcing Strict I/O Path and Device Naming Conventions

You must enable one of two components to specify whether the legacy VxWorks pathname resolution model (default setting) or the strict POSIX pathname handling is used in your VIP.

Overview

Historically in VxWorks, pathname resolution and device naming rules have been different from those required by POSIX. The historical rules aim to allow (but not enforce) POSIX style pathnames, while also supporting DOS-style drive letters, for example **C:**, or rcp-style host prefixes such as **server:**. This resulted in effective rules that were in some aspects more complicated than POSIX rules for pathname resolution, but at the same time less restrictive. For example, device look-up was based on longest name prefix match and the device name did not have to match whole components of the pathname being resolved. Furthermore, pathnames starting with a slash [**/**] could be interpreted as relative; whether this occurred or not depended on both the current working directory and what devices were present in the system. These departures from POSIX behavior may hinder porting POSIX reliant code to VxWorks. To address the aforementioned pathname and device naming issues, you must choose one of these two VIP components:

INCLUDE_PATHNAME_RESOLUTION_STRICT (default setting in CERT)

The strict configuration enforces strict rules for pathname resolution and device naming. Specifically, all device names must start with [**/**], partial component pathname matching does not occur, and determining whether or not a pathname is absolute is based upon the first character of the pathname.

INCLUDE_PATHNAME_RESOLUTION_LEGACY (default setting except in CERT)

The legacy configuration has less restrictive pathname resolution and device naming rules. The component has a parameter DEV_NAME_POLICY that specifies the device naming requirements. The default settings are more restrictive than in previous VxWorks releases, however, common device names such as **host:** are allowed (where the device name does not begin with a [**/**].

Note: The legacy configuration is currently the default setting so you can update your code in preparation for strict rule compliance. The legacy component will be removed in a future release whereupon the strict rules will be unconditionally enforced.

Below are examples of how device and pathnames are interpreted using legacy and strict rules.

Path Example	Legacy Rules Interpretation	Strict Rules Interpretation
mirror:/dir	Interpreted as absolute if a device named **mirror:** exists.	A relative path describing a file **dir** in a directory **mirror:**.

WNDRVR

Path Example	Legacy Rules Interpretation	Strict Rules Interpretation
		In most cases, the device **mirror:** would have been named **/host.mirror** in strict mode.
paul/foo/bar	A relative path, assuming no device name matches a prefix of **paul/foo/bar**. If there were a device named **paul**, this would be an absolute path.	A relative path.
/foo/bar	An absolute path, if there exists a device named [**/**] or **/f** or **/foo**, and if the CWD path begins with [**/**]. If the CWD path does not begin with **/**, for example **host:/dir**, then **/foo/bar** may refer to **host:/foo/bar**.	Always an absolute path. Would match a device named [**/**] or **/foo**, but would not match **/f**.
/dev/zero	Likely a device named **/dev/zero**, but the directory **/dev** might not exist.	Same as the legacy interpretation.
/tyCo/0	The old name for a serial port. The new name is likely **/ttyS0**, and **/tyCo/0** is probably a symbolic link pointing at **/ttyS0**. For more information, see Setting Up Backwards Compatibility using Symlinks on page 13.	Same as the legacy interpretation.

Device Naming Conventions

The following rules apply to both the strict pathname resolution (unconditionally) and the legacy pathname resolution configurations (with the default **DEV_NAME_POLICY** parameter value):

- The whole device name must not be NULL or empty.
- The whole device name must be less than **PATH_MAX** characters in length (including the terminating null character).
- The device name cannot contain any backslash [****] character.
- The device name cannot end with a forward slash [**/**] except for the root path.
- The device name may not contain consecutive [**/**] characters or any dot [**.**] or dot-dot [**..**] components.
- Pathname components of the device name cannot be longer than **NAME_MAX** in length.

The only difference between the two models is that when the legacy component is enabled, the device name does not have to begin with a forward slash [**/**] (which is not allowed when strict rules are enforced).

Enabling Legacy Rules During the Migration Period

The **INCLUDE_PATHNAME_RESOLUTION_LEGACY** component is the default setting and allows for the legacy handling of path and device names.

> Note: The legacy component will be removed in a future release whereupon the strict rules are enforced permanently.

When **INCLUDE_PATHNAME_RESOLUTION_LEGACY** is enabled, another component called **INCLUDE_LEGACY_DEV_NAME_LINKS** is enabled by default. The **INCLUDE_LEGACY_DEV_NAME_LINKS** component requires

VRFS (Virtual Root File System), and uses it to create symbolic links from old device names (for example, **/tyCo/0**) to the corresponding new names (for example, **/ttyS0**). For more information, see Linking to Legacy Device Names on page 14.

The **INCLUDE_PATHNAME_RESOLUTION_LEGACY** component includes a parameter called **DEV_NAME_POLICY**. This parameter allows you to modify the device name rules in legacy mode. You can select any combination of these bit flags (the first three in the list are the default settings):

- **IOS_DEVNAMEPOL_NO_BACKSLASHES** (default)
- **IOS_DEVNAMEPOL_NO_TRAILING_SEP** (default)
- **IOS_DEVNAMEPOL_GENERIC** (default)
- **IOS_DEVNAMEPOL_STARTS_WITH_SEP**
- **IOS_DEVNAMEPOL_NO_MULTICOMP**
- **IOS_DEVNAMEPOL_GENERIC**

Enforcing Strict Rules

Enable the **INCLUDE_PATHNAME_RESOLUTION_STRICT** component to ensure the following requirements are met:

- all device names start with [**/**]
- device names do not match partial pathname components
- absolute paths are identified based on only the first character of the pathname
- simpler rules are used for combining the *CWD* with a relative pathname

Note: It is strongly recommended you enable the **INCLUDE_VRFS** component if you enable the **INCLUDE_PATHNAME_RESOLUTION_STRICT** component.

Setting Up Backwards Compatibility using Symlinks

Enable the **INCLUDE_LEGACY_DEV_NAME_LINKS** component under the **I/O System** folder to automatically create symbolic links so that legacy device names are renamed using the strict naming convention. This is useful for programs that have been hard-coded using the legacy device naming convention. For example, a symbolic link **/tyCo/0** pointing at the new serial device name **/ttyS0** is automatically created when the legacy links component is enabled.

Note: At some point, you must modify your software to use the new device naming convention because this component will be removed in a future release. In the meantime, the legacy links component may be used with either **INCLUDE_PATHNAME_RESOLUTION_LEGACY** or **INCLUDE_PATHNAME RESOLUTION_STRICT**.

The symlinks are created by selected device drivers where the device names have recently changed to be compatible with Linux. In most cases, the devices are serial devices and file-system or disk partition devices. A symlink is not created for some drivers such as **netDrv** and **passfs** because the old device name is not allowed according to the strict device naming rules. For example, if **INCLUDE_PATHNAME_RESOLUTION_STRICT** is enabled, a symlink is not created for a netDrv device named **ark:**; instead, the device is renamed to **/host.ark**. Any additional desired symlinks may be created by calling the symlink( ) function.

Symbolic links created by VRFS appear as devices in the I/O system device list (see figure below). However, this is an implementation detail that may change in the future.

WNDRVR

```
Terminal   Build Console
VxWorks 7   VxWorks Simulator
  \7777777\      /7777777/
   \7777777\    /7777777/
    \7777777\   \7777777/
     \7777777\   \77777/              VxWorks 7 SMP 64-bit
      \7777777\   \777/
       \7777777\   \7/     Core Kernel version: 3.1.0.0
        \7777777\   -      Build date: Jun 25 2019 19:17:54
         \7777777\
          \777777/         Copyright Wind River Systems, Inc.
           \77777/   -                 1984-2019
            \777/   /7\
             \7/   /777\
              -   -------

                 Board: SIMNT board
              CPU Count: 2
         OS Memory Size: 508MB
       ED&R Policy Mode: Deployed
    Debug Agent: Started (always)
        Stop Mode Agent: Started (always)

Loading symbol table from host:C:/WindRiver/vxworks7-r2-20190627_ARCH/vxworks-7/
/default/vxWorks.sym ...done

-> devs
drv refs name
  1 [ 3] /
 13 [ 3] /hostfs
  0 [ 3] /null
  5 [ 3] /ram0
  3 [ 3] /ttyS0
  2 [ 3] /tyCo/0  ==>  /ttyS0
 10 [ 3] host:
value = 0 = 0x0
-> 
```

For more information on the relationship between device names and access to devices and file systems, see VRFS Example on page 17. A symlink may point to anything such as a file, directory, device, another symlink, or even a non-existent path but it must adhere to the **PATH_MAX** length restrictions.

> **Note:** Links in VRFS are not persistent and must be created after each boot in the same way the VRFS tree is created.

> **Note:** The iosDevFind( ) device look-up function does not follow symlinks to locate the named device.

5. Linking to Legacy Device Names

If there are compatibility issues in your applications because the legacy device naming convention was used, you can create a symbolic link between the old name and the new name.

For the current block device naming convention, see I/O System and Device Naming Conventions on page 4.

In the legacy name for a VxWorks block device, the partition table was identified with **:***n*, where *n* is the partition number. For example, **/ata1:1** indicated the first partition in the **/ata1** device.

If you need to use an old device name (for example, **/ata1:1**), you must enable the **INCLUDE_LEGACY_DEV_NAME_LINKS** component to create a symbolic link between the old name and the new name. You must also enable the **INCLUDE_VRFS** component. For more information, see Enforcing Strict I/O Path and Device Naming Conventions on page 11.

The following is an example of device name linking.

```
-> devs
drv refs name
  1 [ 3] /
  2 [ 3] /ata1:1  ==>  /ata1a
  2 [ 3] /ata1:2  ==>  /ata1b
  2 [ 3] /ata1:3  ==>  /ata1c
  7 [ 3] /ata1a
  7 [ 3] /ata1b
```

```
  7 [ 3] /ata1c
  2 [ 3] /bd0:1  ==>  /bd0a
  6 [ 3] /bd0a
  0 [ 3] /null
  7 [ 3] /ram0
  4 [ 3] /shm
  3 [ 3] /ttyS0
  3 [ 3] /ttyUSB0
  2 [ 3] /tyCo/0  ==>  /ttyS0
  2 [ 3] /tyCo/1  ==>  /ttyUSB0
 12 [ 3] host:
```

With this link, you can the use old device name to access the device with an IO API or shell command; for example, **ls "/bd0:1"**.

WNDRVR

3. CONFIGURING FILE SYSTEMS

1. VRFS

VxWorks provides a Virtual Root File System (VRFS) so you can manage device trees and pathnames for applications that run on a POSIX root file system.

The root of the VRFS is identified by a forward slash [**/**] and may contain:

- I/O system devices such as file system devices
- symbolic links
- other VRFS directories

VRFS is not a physical file system, therefore, you cannot use it to store files or use it for temporary storage. However, devices that provide file storage may be embedded within a VRFS directory structure.

You must include a slash [**/**] before a device name for it to appear in the VRFS tree. When VRFS is enabled and a device with a multi-component name such as **/dev/random** is created, a VRFS directory named **/dev** is automatically created if it does not already exist. The VRFS directory is a child of the root VRFS directory [**/**].

VRFS directories may also contain symlinks. For example, you can create a symlink named **/dev/null** targeting the **/null** device. Similarly, if there is a device /**hrfs** containing executable files in a sub-directory named **bin**, you can create a symlink named **/bin** targeting **/hrfs/bin**. For more information, see Enforcing Strict I/O Path and Device Naming Conventions on page 11.

Configuring VxWorks with VRFS on page 16 You may need to enable VRFS if it gets automatically disabled by other configurations.
VRFS Example on page 17 This shell session illustrates the relationship between device names and access to devices and file systems with the VRFS.

1.1. Configuring VxWorks with VRFS

You may need to enable VRFS if it gets automatically disabled by other configurations.

About This Task

VRFS is enabled by default because the **INCLUDE_LEGACY_DEV_NAME_LINKS** and **INCLUDE_PATHNAME_RESOLUTION_LEGACY** are enabled by default and depend on VRFS. For more information, see Enforcing Strict I/O Path and Device Naming Conventions on page 11. However, if you enable **INCLUDE_PATHNAME_RESOLUTION_STRICT** in the VIP and remove **INCLUDE_PATHNAME_RESOLUTION_LEGACY**, the following components are both disabled:

- **INCLUDE_LEGACY_DEV_NAME_LINKS**
- **INCLUDE_VRFS**

In the above case, you can enable VRFS by enabling **INCLUDE_LEGACY_DEV_NAME_LINKS** or follow the procedure below.

Before You Begin

You must know how to use Workbench or wrtool to create and configure VSB and VIP projects. For more information, see the *VxWorks 7 Configuration and Build Guide*.

WNDRVR

Procedure

1. If **FS_VRFS** is not already enabled in your VSB, enable it and rebuild your VSB. Otherwise, create a VSB project, select the **FS_VRFS** layer, and build the source code.
2. Create a VIP project based on the VSB project, select the **INCLUDE_VRFS** component, and build VxWorks.

   At run-time, an instance of the VRFS file system is created and mounted automatically.

Parent topic: VRFS on page 16

1.2. VRFS Example

This shell session illustrates the relationship between device names and access to devices and file systems with the VRFS.

```
-> devs
drv refs  name
1   [ 3]  /
13  [ 3]  /fifos
6   [ 3]  /fsroot
18  [ 3]  /ioSystemTest
0   [ 3]  /null
12  [ 3]  /ram0
11  [ 3]  /romfs
4   [ 3]  /shm
3   [ 3]  /ttyS0
2   [ 3]  /tyCo/0  ==>  /ttyS0
14  [ 3]  mirror:
value = 0 = 0x00
-> cmd
[vxWorks *]# cd/
[vxWorks *]# ls -l
dr-xr-xr-x  9   0       0                 152 Jan  1  1970  ./
dr-xr-xr-x  9   0       0                 152 Jan  1  1970  ../
d---------  9   0       0                 152 Jan  1  1970  ioSystemTest/
?rw-rw-rw-  0   0       0                   0 Jan  1  1970  null
drwxrwxrwx  1   0       0                   0 Jan  1  1970  shm/
?rw-rw-rw-  1   0       0                   0 Jan  1  1970  ttyS0
dr-xr-xr-x  3   0       0                  16 Jan  1  1970  tyCo/
drwxr-xr-x  0   1000    985               1448 Jan  1  1970  romfs/
drwxrwxrwx  2   0       0                 8192 Jun 24 17:09  ram0/
drwxrwxrwx  1   0       0                   0 Jan  1  1970  fifos/
drwxrwxrwx  1   0       0                 512 Jun 24 17:09  fsroot/
[vxWorks *]# ls -l tyCo

Listing  Directory tyCo:
dr-xr-xr-x  3   0      0                   16 Jan  1  1970  ./
dr-xr-xr-x  9   0      0                  152 Jan  1  1970  ../
lrwxrwxrwx  0   0      0                    6 Jan  1  1970  0 -> /ttyS0
[vxWorks *]#
```

Entries that the ls -l command can identify as directories have a trailing [**/**] character.

WNDRVR

Note: **mirror:** does not show up in the listing for [**/**], since only devices starting with [**/**] are treated as children of the VRFS root. Device **/ttyS0** shows up in the root directory listing as a device **ttyS0**. The symlink device **/tyCo/0** shown by the **devs** command appears in the root directory listing as the directory **tyCo/,** and in the listing of the **tyCo** sub-directory as **0**. **/tyCo** is in fact a VRFS directory device, automatically created when the symlink device **/tyCo/0** was created, but hidden in the **devs** output. VRFS symlinks and VRFS sub-directories are currently represented as devices; this is an implementation that may change in future VxWorks releases.

Note: Configuring VxWorks with support for POSIX PSE52 conformance (using **BUNDLE_RTP_POSIX_PSE52**) provides the **/dev/null** device. Listing the VRFS root directory would list the directory **dev/** and listing /**dev** would list **null** under **/dev**. For information about null devices, see *Null Devices Overview on page 195*. For more information about POSIX PSE52, see the *VxWorks 7 POSIX Programmer's Guide: POSIX PSE52 Configuration*.

Enabling VRFS provides a root directory device **/** that alters how VxWorks handles paths beginning with [**/**] as shown in the examples below.

Example	Results	Notes
There is no [**/**] device, and a pathname **/home/panloki** needs to be resolved. The current working directory is **host:/opt/bin** (which starts with a device name that does not begin with [**/**].	It is possible no device name matches a prefix of the pathname **/home/panloki** The legacy rules for resolving pathnames combine the current working directory device name **host:** with the path **/home/panloki** to create **host:/home/panloki**.	The path **/home/panloki** is treated as relative to the CWD device **host:**.
There is a VRFS root directory and the path **/home/panloki** starts with a device name (the [**/**]). The current working directory is **host:/opt/bin** (which starts with a device name that does not begin with **/**).	**/home/panlok** is resolved as absolute and the CWD path **host:/opt/bin** is ignored.	If **/home/panloki** was meant to be a path that is subordinate to the device **host:**, then the look-up will fail in most cases. However, the look-up will succeed if there is file system device named **/home** where **/home/panloki** exists as a file. To access **host:/home/panloki**, you must specify the whole path including the **host:** device name.

For more information, see *VxWorks Devices Overview on page 167*.

Parent topic: VRFS on page 16

2. HRFS

The Highly Reliable File System (HRFS) is a transactional file system for real-time systems.

The primary features of the HRFS file system are:

- Fault tolerance. The file system is never in an inconsistent state, and is able to recover quickly from unexpected loses of power.
- Configurable commit policies.

WNDRVR

> **Note:** HRFS uses a commit task to write data. The default priority of the commit task is 2, which can impact real-time requirements. To avoid this and be strict with the real-time requirement, set the parameter **INCLUDE_HRFS_DIRECT_COMMIT** to TRUE.

- Hierarchical file and directory system, allows for efficient organization of files on a volume.
- Compatible with widely available storage devices.

> **Note:** HRFS reliability depends on hardware support. HRFS should be paired with either an eMMC storage device that supports atomic write operations or an SSD device with battery backup or power-storing capacitors that ensure data integrity during power failures. If HRFS is used without hardware support, the file system reduces the chance of data loss in the case of power failure but cannot fully prevent it.

- POSIX compliance.

For more information about the HRFS libraries see the VxWorks API references for **hrfsFormatLib**, **hrFsLib**, and **hrfsChkDskLib**.

HRFS and Flash Memory

For information about using HRFS with flash memory, see True Flash File System (TrueFFS) on page 106 and the implementation instructions.

VxWorks Configuration for HRFS on page 20 To configure VxWorks with support for Highly Reliable File System (*HRFS*), you must select the required options and components for your VSB and VIP projects. *HRFS* has numerous optional components that may be used as well.
HRFS and POSIX PSE52 on page 21 If an RTP application requires a PSE52 file system, then VxWorks must be configured with an extended set of HRFS components.
Creating an HRFS File System on page 22 Creating an HRFS file system involves creating an XBD device wrapper (if required), and then creating partitions and formatting the volume.
HRFS, ATA, and RAM Disk Examples on page 23 These examples provide generic illustrations of HRFS use.
Create HRFS in Partitions on an ATA Disk on page 23 This example demonstrates how to initialize an ATA disk with HRFS on two partitions from the shell.
Creating and Partitioning a Disk and Creating Volumes on page 25 This code takes the name of a block device that you have already instantiated, creates three partitions, creates the partition handler for these partitions, and creates the HRFS device handler for them. Then it formats the partitions using hrfsFormat( ).
Creating and Formatting a RAM Disk Volume and Performing File I/O on page 26 This code creates a RAM disk, formats it for use with the HRFS file system, and performs file system operations.
Optimizing HRFS Performance on page 28 The performance of HRFS can be optimized with a few VxWorks configuration and HRFS formatting steps.
Transactional Operations and Commit Policies on page 28HRFS is a transactional file system. Transaction or commit points are set to make disk changes permanent. Commit policies define specific conditions under which commit points are set.
File Access Time Stamps on page 31 Access time stamps can be enabled by configuring VxWorks with the **INCLUDE_HRFS_ACCESS_TIMESTAMP** component.
Maximum Number of Files and Directories on page 31HRFS files and directories are stored on disk in data structures called inodes, the number of which can be specified during formatting.
Creating Subdirectories on page 31 Directories can be created with open( ) or mkdir( ).
Removing Subdirectories on page 32 Subdiriectories can be removed with ioctl( ), remove( ), or rmdir( ).
Reading Directory Entries on page 32 Directory entries can be read with standard **dirLib** APIs.
Working with Files and Volumes in HRFS on page 33HRFS supports standard file I/O, file linking and unlinking, and file permission management functions. You can also retrieve the volume size by using files system utilities.
I/O Control Functions Supported by HRFS on page 35 The HRFS file system supports a set of ioctl( ) functions.
Crash Recovery and Volume Consistency on page 36HRFS is a transactional based file system that is designed to be consistent at all times.
File Management and Full Devices on page 37HRFS is designed to manage file operations on a full volume in a way that is consistent with its design as a highly reliable file system.

WNDRVR

2.1. VxWorks Configuration for HRFS

To configure VxWorks with support for Highly Reliable File System (*HRFS*), you must select the required options and components for your VSB and VIP projects. *HRFS* has numerous optional components that may be used as well.

VSB Project

Select **FS_HRFS** and build the project.

VIP Required Components

Either the **INCLUDE_HRFS** or the **INCLUDE_HRFS_READONLY** component is required. As its name indicates, the latter is a read-only version of the main HRFS component. The libraries it provides are smaller as there are no facilities for disk modifications. The configuration parameters for these components are as follows:

HRFS_DEFAULT_MAX_BUFFERS

Defines how many buffers HRFS uses for its caching mechanism. HRFS needs a minimum of 6 buffers. The default setting is 16. This parameter applies to all HRFS volumes. Note that while increasing the number of buffers can increase performance, it does so at the expense of heap memory. For information about using this parameter in configuration for performance, see Optimizing HRFS Performance on page 28.

HRFS_DEFAULT_MAX_FILES

Defines how many files can be open simultaneously on an HRFS volume. The minimum is 1. The default setting is 10. Note that this is not the same as the maximum number of file descriptors.

HRFS_DEFAULT_ERROR_HANDLE_POLICY

Defines how errors are handled on the HRFS. You can specify one of these options: HRFS_ERR_HANDLE_STOP_IO, HRFS_ERR_HANDLE_STOP_IO_SILENT, HRFS_ERR_HANDLE_TO_EDR, HRFS_ERR_HANDLE_TO_USER_FUNC, HRFS_ERR_HANDLE_IGNORE.

HRFS_DEFAULT_ERROR_HANDLE_USER_FUNC

Enable this component to use your own routine when handling HRFS errors. To activate this component, you must first specify

the HRFS_ERR_HANDLE_TO_USER_FUNC option for the **HRFS_DEFAULT_ERROR_HANDLE_POLICY** component (see above).

HRFS_UID_GID_ENABLE

Set this component to TRUE to enable user ID and group ID field support on the HRFS. When enabled, the HRFS checks the user ID (UID) or group ID (GID) before accessing files. You can use chown( ) or fchown( ) with the UID or GID number to change the user or group ownership of a file.

HRFS_LOSTFOUND_DIR_DELETABLE

Set this component to TRUE (default setting) to allow the user to delete the **.lost+found** directory.

HRFS_LOSTFOUND_AUTOCLEAN

Set this component to TRUE (default setting) to have all **~DelMe** files in the **.lost+found** directory automatically deleted when HRFS is mounted.

INCLUDE_HRFS_DIRECT_COMMIT

Set this parameter to TRUE to enable direct commit mode. The default value is FALSE. By default, HRFS uses the commit task to write data. The commit task priority is 2, which can impact real-time requirements. To avoid this impact set this parameter to TRUE to disable the commit task, and the file-write request will be directly committed to the storage driver in the application's task context.

WNDRVR

This parameter is valid only when you include the **INCLUDE_HRFS_DEFAULT_WRITE_MODE** component.

The **INCLUDE_HRFS_DEFAULT_WRITE_MODE** component is included automatically, providing the default write-mode, which performs a commit with each write. (For the alternative, see **INCLUDE_HRFS_HISPEED_WRITE_MODE**, below.)

In addition to either **INCLUDE_HRFS** or **INCLUDE_HRFS_READONLY**, HRFS requires the appropriate component for your block device.

And if you are using a device driver that is not designed for use with the XBD facility, you must use the **INCLUDE_XBD_BLK_DEV** wrapper component in addition to **INCLUDE_XBD**.

VIP Optional Components

The optional components for HRFS are as follows:

INCLUDE_HRFS_HISPEED_WRITE_MODE

The high-speed write-mode, which performs a commit for the current transaction when either of the following are true: cache usage is over 40%, or five seconds have elapsed (if the current transaction is still active). This component provides an alternative to the default **INCLUDE_HRFS_DEFAULT_WRITE_MODE** component, which performs a commit with each write.

For information about using **INCLUDE_HRFS_HISPEED_WRITE_MODE** in configuration for performance, see Optimizing HRFS Performance on page 28. For information about transactions and commit policies, see Transactional Operations and Commit Policies on page 28.

INCLUDE_HRFS_FORMAT

Formatter for HRFS media.

INCLUDE_HRFS_CHKDSK

File system consistency checker.

INCLUDE_HRFS_ACCESS_TIMESTAMP

Access time stamp for HRFS volumes. Note that this component is included in the **BUNDLE_RTP_POSIX_PSE52** component bundle.

Optional XBD Components

The **INCLUDE_XBD_PART_LIB** (disk partitioning) and **INCLUDE_XBD_RAMDRV** (RAM disk) components are optional.

Optional SATA Component

The **INCLUDE_SATA_DISK_NAME_CFG** component is optional. It can be used to change the default name of the device. The assignment of an empty string means that the default name will be used (**/ata0**).

Parent topic: HRFS on page 18

2.2. HRFS and POSIX PSE52

If an RTP application requires a PSE52 file system, then VxWorks must be configured with an extended set of HRFS components.

The INCLUDE_HRFS( ) component for the highly reliable file system.

The **INCLUDE_HRFS_ACCESS_TIMESTAMP** component for updating file and directory access timestamps on an HRFS formatted volume must be included. This component is part of the **BUNDLE_RTP_POSIX_PSE52** bundle.

WNDRVR

CAUTION: **The use of the INCLUDE_HRFS_ACCESS_TIMESTAMP component can cause significant degradation in file system performance. For more information, see** File Access Time Stamps on page 31**.**

The appropriate device driver component (for example **INCLUDE_XBD_RAMDRV**) and **INCLUDE_XBD_BLK_DEV** if the device driver requires it (see Overview of VxWorks File Systems and I/O Devices on page 1).

The **INCLUDE_VRFS** component for the VRFS file system, which is included in the **BUNDLE_RTP_POSIX_PSE52** component bundle

In addition, a **/tmp** directory must be created at run-time. It must appear on the VRFS file system, and must be formatted as an HRFS file system. The following examples illustrate creation of a **/tmp** RAM disk device and formatting the directory as an HRFS file system:

```
xbdRamDiskDevCreate (512, 0x10000, 0, "/tmp")
hrfsDiskFormat "/tmp", 1000
```

For more information, see VRFS on page 16

Parent topic: HRFS on page 18

2.3. Creating an HRFS File System

Creating an HRFS file system involves creating an XBD device wrapper (if required), and then creating partitions and formatting the volume.

About This Task

Before any other operations can be performed, the HRFS file system library, **hrFsLib**, must be initialized.This happens automatically at boot time, triggered by the required HRFS components that were included in the system.

Initializing HRFS involves the creation of a vnode layer. HRFS installs an number of internal vnode operators into the this layer. The vnode layer invokes iosDrvInstall( ) when media is detected, which adds the driver to the I/O driver table. The driver number assigned to vnodes—and therefore HRFS—is recorded in a global variable, **vnodeAffDriverNumber**. The table specifies entry points for the vnode file operations that are accessing devices using HRFS.

Note: Enabling the vnode layer starts the **vnSyncer** task. Altbough this task is enabled, it is not currently used by HRFS.

The block device is created automatically by the block device driver.

Procedure

1. Create an XBD Device Wrapper.

   If you are using a device driver that is not XBD-compliant, it requires an XBD device wrapper.

   The wrapper is created automatically if you have configured VxWorks with the **INCLUDE_XBD_BLK_DEV** wrapper component (see VxWorks Devices and Driver Model Overview on page 167). Otherwise, create a wrapper for each block device using xbdBlkDevCreate( ).

   After the XBD device wrapper is created, the physical device is automatically probed for a file system and partitions. If a disk is already formatted, the disk is mounted. If a file system is found, it is mounted as well. If file system is not HRFS, it must be formatted (see below).

2. Create partitions.

WNDRVR

If you have included the **INCLUDE_XBD_PART_LIB** component in your system, you can create partitions on a disk and mount volumes atop the partitions. Use the xbdCreatePartition( ) function to create partitions.

This step should only be performed once, when the disk is first initialized. If partitions are already written to the disk, this step should not be performed as it destroys data.

3. Format the volume.

   If you are using unformatted disk or wish to replace the current file system on the disk, format the disk by calling hrfsFormat( ). For more information, see the VxWorks API reference for this function.

   CAUTION: **Reformatting a disk destroys any data that may be on it.**

Parent topic: HRFS on page 18

2.4. HRFS, ATA, and RAM Disk Examples

These examples provide generic illustrations of HRFS use.

They illustrate the steps discussed in the preceding topics. They are meant to be relatively generic, and illustrate the following:

- Creating and working with an HRFS file system on an ATA disk with commands from the shell.
- Code that creates and formats partitions.
- Code that creates and formats a RAM disk volume.

CAUTION: **Because device names are recognized by the I/O system using simple substring matching, file systems should not use a slash (/) alone as a name; unexpected results may otherwise occur.**

Parent topic: HRFS on page 18

2.5. Create HRFS in Partitions on an ATA Disk

This example demonstrates how to initialize an ATA disk with HRFS on two partitions from the shell.

About This Task

While these steps use an ATA device, they are applicable to other block devices.

The block device is created automatically by the block device driver.

Procedure

1. Display information about devices.

```
-> devs
drv name
0 /null
1 /ttyS0
1 /ttyS1
8 yow-grand:
9 /vio
```


WNDRVR

```
4 /ata0
value = 25 = 0x19
```

The new ATA device **/ata0** is listed and no partitions were detected. Note that if no file system is detected on the device, the rawFs file system is instantiated automatically and appears in the device list.

2. Prepare the disk for use.

   Create two partitions on this disk device, specifying 50% of the disk space for the second partition, leaving 50% for the first partition. This step should only be performed once, when the disk is first initialized. If partitions are already written to the disk, this step should not be performed since it destroys any data on the disk.

```
-> xbdCreatePartition ("/ata0", 2, 50, 0, 0)
value = 0 = 0x0
```

   Note that the second parameter defines the number of partitions, and that the parameters that follow define the percentage of space for the second, third, and fourth partitions—with the remainder being used for the first partition.

3. List the devices to display information about the new partitions.

```
-> devs
drv name
0 /null
1 /ttyS0
1 /ttyS1
8 yow-grand:
9 /vio
3 /ata0a
3 /ata0b
```

   Note that **/ata0** does not appear in the list because the disk has been partitioned, and two new devices, **/ata0a** and **/ata0b**, have been added to represent the new partitions. The rawfs file system is automatically instantiated in each volume, as they are new and unformatted.

4. Format the volumes for HRFS.

   This step need only be done once, when the volumes are first created. If the volumes have already been formatted, then omit this step. This example formats the file system volumes with default options.

```
-> hrfsFormat ("/ata0a", 0ll, 0, 0)
Formatting /ata0a for HRFS
Instantiating /ata0a as rawFs
Formatting...OK.
value = 0 = 0x0

-> hrfsFormat ("/ata0b", 0ll, 0, 0)
Formatting /ata0b for HRFS
Instantiating /ata0b as rawFs
Formatting...OK.
value = 0 = 0x0
```

   Note that in the hrfsFormat( ) call, the ll (two lower-case L letters) used with the second parameter is required to indicate to the shell that the data type is long long.

   For more information, see the API reference for **hrfsFormatLib**.

WNDRVR

5. Display information about the HRFS volumes.

```
-> ll "/ata0a"
Listing Directory /ata0a:
drwxrwxrwx 1 0 0 8192 Jan 1 00:13 ./
drwxrwxrwx 1 0 0 8192 Jan 1 00:13 ../
value = 0 = 0x0

-> ll "/ata0b"
Listing Directory /ata0b:
drwxrwxrwx 1 0 0 8192 Jan 1 00:13 ./
drwxrwxrwx 1 0 0 8192 Jan 1 00:13 ../
value = 0 = 0x0
```

If you are working with an ATA hard disk or a CD-ROM file system from an ATAPI CD-ROM drive, you can, alternatively, use usrAtaConfig( ). This function processes several steps at once. For more information, see the API reference.

Parent topic: HRFS on page 18

2.6. Creating and Partitioning a Disk and Creating Volumes

This code takes the name of a block device that you have already instantiated, creates three partitions, creates the partition handler for these partitions, and creates the HRFS device handler for them. Then it formats the partitions using hrfsFormat( ).

```
STATUS usrPartDiskFsInit
  (
    char * xbdName /* device name used during creation of XBD */
  )
  {
    const char * devNames[] = { "/sd0a", "/sd0b", "/sd0c" };
    devname_t xbdPartName;
    int i;

  /* Map partition names */
    for (i = 1; i <= 3; i++)
      {
      sprintf (xbdPartName, "%s:%d", xbdName,i);
      fsmNameInstall (devNames[i-1], xbdPartName);
      }

   /* create partitions */
    if((xbdCreatePartition (xbdName,3,50,45,0)) == ERROR)
      return ERROR;

   /* Formatting the first partition */
    if(hrfsFormat (devNames[0], 0ll,0, 0) == ERROR)
      return ERROR;

   /* Formatting the second partition */
    if(hrfsFormat (devNames[1], 0ll, 0, 0) == ERROR)
      return ERROR;

   /* Formatting the third partition */
    if(hrfsFormat (devNames[2], 0ll, 0, 0) == ERROR)
      return ERROR;
```


WNDRVR

```
    return OK;
    }
```

Note that in most cases you would be likely to format the different partitions for different file systems.

Parent topic: HRFS on page 18

2.7. Creating and Formatting a RAM Disk Volume and Performing File I/O

This code creates a RAM disk, formats it for use with the HRFS file system, and performs file system operations.

```
#include <vxWorks.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <hrFsLib.h>
#include <xbdPartition.h>
#include <xbdRamDisk.h>

#define  DEVNAME "/myram" /* name of the RAM disk */
#define  BLOCKSIZE  512
#define  DISKSIZE  (BLOCKSIZE * 2000)

STATUS hrfsSetup
  (
  void
  )
  {
  STATUS error;
  device_t xbd;
  /* Create a RAM disk. Don't support partitions */
  xbd = xbdRamDiskDevCreate (BLOCKSIZE, DISKSIZE, 0, DEVNAME);
  if (xbd == NULLDEV)
    {
    printf("Failed to create RAM disk. errno = 0x%x\n", errno);
    return (ERROR);
    }
  /*
  * Format the RAM disk for HRFS. Allow for upto a 1000 files/directories
  * and let HRFS determine the logical block size.
  */
  error = hrfsFormat (DEVNAME, 0ll, 0, 1000);
  if (error != OK)
    {
    printf("Failed to format RAM disk. errno = 0x%x\n", errno);
    return (ERROR);
    }
  printf ("%s now ready for use.\n", DEVNAME);
  return (OK);
}

STATUS hrfsFileExample
  (
  void
  )
  {
```

```
  int fd;
  char path[PATH_MAX];
  char *testString = "hello world";
  int size = strlen (testString) + 1; /* size of test string including EOS */
  int len;
  /* Create a file under the root directory */
  /* First build the path */
  sprintf (path, "%s/myfile", DEVNAME);
  fd = open (path, O_RDWR | O_CREAT, 0777);
  if (fd < 0)
  {
    printf ("Couldn't create file %s. errno = 0x%x\n", path, errno);
    return (ERROR);
  }

  /* Write to the file */
  printf("Writing %d bytes to file.\n", size);
    len = write (fd, testString, size);
  if (len != size)
  {
    printf ("Couldn't write to file %s. errno = 0x%x\n", path, errno);
    close (fd);
    return (ERROR);
  }

/* Close and re-open file */
close (fd);
fd = open (path, O_RDWR, 0777);
if (fd < 0)
  {
  printf ("Couldn't re-open file %s. errno = 0x%x\n", path, errno);
  return (ERROR);
  }

/* Now read back what we wrote */
printf("Reading %d bytes from file.\n", size);
  len = read (fd, path, size);
if (len != 12)
{
  printf ("Couldn't read from file %s. errno = 0x%x\n", path, errno);
  close (fd);
  return (ERROR);
}

/* Make sure we read what we wrote */
if ((len = strcmp (path, testString)) != 0)
{
  printf ("Read data different from written data. errno = 0x%x, %d\n", errno, len);
  close (fd);
  return (ERROR);
}
close (fd);
return (OK);
}
```

Note that to use this code, you must configure VxWorks with the **INCLUDE_HRFS_FORMAT**, **INCLUDE_XBD_RAMDRV** and **INCLUDE_XBD_PART_LIB** components.

WNDRVR

The following illustrates running the example from the shell.

```
-> hrfsSetup
Instantiating /myram as rawFs
Formatting /myram for HRFS
Instantiating /myram as rawFs
Formatting...OK.
/myram now ready for use.
value = 0 = 0x0
-> hrfsFileExample
Writing 12 bytes to file.
Reading 12 bytes from file.
value = 0 = 0x0
-> ll "/myram"
Listing Directory /myram:
drwxrwxrwx 1 0 0 2048 Jan 1 00:00 ./
drwxrwxrwx 1 0 0 2048 Jan 1 00:00 ../
-rwxrwxrwx 1 0 0 12 Jan 1 00:00 myfile
value = 0 = 0x0
->
```

Parent topic: HRFS on page 18

2.8. Optimizing HRFS Performance

The performance of HRFS can be optimized with a few VxWorks configuration and HRFS formatting steps.

About This Task

Procedure

1. Configure VxWorks with **INCLUDE_HRFS_HISPEED_WRITE_MODE** (instead of **INCLUDE_HRFS_DEFAULT_WRITE_MODE**).

   The high-speed write mode component performs a commit for the current transaction when either cache usage is over 40%, or when five seconds have elapsed (if the current transaction is still active). The default write mode, on the other hand, performs a commit with each write.

   Note: **INCLUDE_HRFS_HISPEED_WRITE_MODE** can break the real-time requirement because it uses a commit task to write data.

2. Set the **HRFS_DEFAULT_MAX_BUFFER** component parameter to 1024.
3. Use a larger block size—16 or 32 KB— with hrfsFormat( ) when formatting HRFS.

Parent topic: HRFS on page 18

2.9. Transactional Operations and Commit Policies

HRFS is a transactional file system. Transaction or commit points are set to make disk changes permanent. Commit policies define specific conditions under which commit points are set.

HRFS can be configured to automatically commit with each write (the default commit policy), or to perform commits based on cache usage and elapsed time (the high-speed commit policy). Some disk operations trigger commits regardless of the configuration, and

WNDRVR

under certain circumstances, HRFS rollbacks undo disk changes made since the last commit, in order to protect the integrity of the file system.

In addition, applications can perform commits independently of the configured commit policy.

Automatic Commit Policies on page 29 By default, HRFS automatically performs a commit with each write operation.
High-Speed Commit Policy on page 29 The high-speed commit policy performs a commit for the current transaction
Mandatory Commits on page 30 Regardless of commit policy, there are circumstances under which a commit is always performed.
Rollbacks on page 30A rollback undoes any disk changes since the last commit.
Programmatically Initiating Commits on page 30 Applications can be coded to initiate commit operations as required with the commit( ) function.
Commit Task Priorities on page 30

Parent topic: HRFS on page 18

Automatic Commit Policies

By default, HRFS automatically performs a commit with each write operation.

This functionality is provided by the **INCLUDE_HRFS_DEFAULT_WRITE_MODE** component.

Any operation that changes data on the disk results in a transaction point being set. This is the safest policy in terms of the potential for data loss. It is also the slowest in terms of performance, as every write to disk cause a commit. The following functions, for example, cause modifications to disk and result in a commit when the automatic commit policy is used:

- write( )
- remove( )
- delete( )
- mkdir( )
- rmdir( )
- link( )
- unlink( )
- truncate( )
- ftruncate( )
- ioctl( ) when used with a control function that requires modifying the disk.

Parent topic: Transactional Operations and Commit Policies on page 28

High-Speed Commit Policy

The high-speed commit policy performs a commit for the current transaction

The high-speed commit policy performs a commit for the current transaction when either of the following are true:

- When cache usage is over 40%.
- When five seconds have elapsed, if the current transaction is still active. This functionality is provided by the **INCLUDE_HRFS_HISPEED_WRITE_MODE** component.

For information about using this component in configuration for performance, see Optimizing HRFS Performance on page 28.

Parent topic: Transactional Operations and Commit Policies on page 28

WNDRVR

Mandatory Commits

Regardless of commit policy, there are circumstances under which a commit is always performed.

Mandatory commits occur under the following circumstances:

- Creation of a file or directory
- Deletion of a file or directory.
- Renaming/moving a file or directory.
- Space in the inode journal is exhausted.

Note that mandatory commits are a subset of automatic commits—they do not, for example, include write( ) and truncate( ).

Parent topic: Transactional Operations and Commit Policies on page 28

Rollbacks

A rollback undoes any disk changes since the last commit.

Rollbacks usually occur when the system is unexpectedly powered down or reset. Rollbacks can also occur when the file system encounters errors (for example, the lack of disk space to complete a write( ), or an error is reported by the underlying device driver). Rollbacks of this nature only happen on operations that modify the media. Errors on read operations do not force a rollback.

A rollback involves HRFS returning to the state of the disk at the last transaction point, which thereby preserves the integrity of the file system, but at the expense of losing file data that has changed since the last transaction point. If the manual or periodic commit policy is specified, there is the potential for losing a lot of data—although the integrity of the file system is preserved.

Parent topic: Transactional Operations and Commit Policies on page 28

Programmatically Initiating Commits

Applications can be coded to initiate commit operations as required with the commit( ) function.

The application can decide when critical data has been written and needs to be committed. This is fast in terms of performance, but has greater potential for data loss. The commit( ) function is provided by the **INCLUDE_DISK_UTILS** component.

Parent topic: Transactional Operations and Commit Policies on page 28

Commit Task Priorities

The following commit priorities should be considered for HRFS tasks.

In normal usage, there are three types of HRFS tasks:

1. Users tasks:
   - call file write
   - wait for the HRFS commit task
   - done and return
2. HRFS commit tasks:
   - wait for a transaction
   - call xbdStrategy to write data

WNDRVR

- wait for xbdStrategy to be done
- wait for the next transaction

3. Storage device XBD service tasks:
   - wait for BIO (xbdStrategy)
   - start to write BIO
   - done and wait for the next BIO

In non-HRFS scenarios, the priority for the BIO is the task priority of the user; however, for HRFS scenarios, the priotity for the BIO is the HRFS commit task priority. As a result, for HRFS, the BIO does not receive the users task priority and therefore, the HRFS needs to set the commit tasks fairly high to ensure the users request is not blocked.

The following details should be considered:

- The HRFS volume consistency will not be affected by a change of the HRFS_DEFAULT_COMMIT_TASK_PRIORITY parameter. The HRFS volume consistency is only affected when the hardware's atomic-writing feature is missing.
- When a system has critical tasks that have a higher priority than HRFS commit task, the risks are:
  - the pending data waiting to be written to disk can be lost if there is a power-failure during the run of the other priority task.
  - the application can be temporarily blocked if the HRFS commit task did not complete the pending transaction. If the HRFS commit task has a very low priority, it may pend a user task until the waiting transaction is done. This may create a dead-lock, or cause a user task with higher priority to be delayed.

Parent topic: Transactional Operations and Commit Policies on page 28

2.10. File Access Time Stamps

Access time stamps can be enabled by configuring VxWorks with the **INCLUDE_HRFS_ACCESS_TIMESTAMP** component.

The component is include in the **BUNDLE_RTP_POSIX_PSE52** component bundle.

When the access timestamp component is included, and the appropriate disk format version is used, reading from a file or directory causes its access time stamp to be updated. This can cause significant performance loss, as a write to disk occurs even on a read operation and a transaction point is set. Only use access time stamps if the application requires it for POSIX compliance.

Parent topic: HRFS on page 18

2.11. Maximum Number of Files and Directories

HRFS files and directories are stored on disk in data structures called inodes, the number of which can be specified during formatting.

The maximum number of inodes is specified as a parameter to hrfsFormat( ). The total number of files and directories can never exceed the number inodes. Attempting to create a file or directory when all inodes are in use generates an error. Deleting a file or directory frees the corresponding inode.

The default maximum number of files is set with the **HRFS_DEFAULT_MAX_FILES** configuration parameter.

For more information see, VxWorks I/O Global Configuration Parameters on page 8.

Parent topic: HRFS on page 18

2.12. Creating Subdirectories

Directories can be created with open( ) or mkdir( ).

WNDRVR

About This Task

You can create as many subdirectories as there are inodes.

Procedure

Create a directory in one of the following ways.

- Use open( ).

  To create a directory, the O_CREAT option must be set in the flags parameter and the S_IFDIR or FSTAT_DIR option must be set in the mode parameter. The open( ) calls returns a file descriptor that describes the new directory. The file descriptor can only be used for reading only and should be closed when it no longer needed.
- Use mkdir( ) from **usrFsLib**.

When creating a directory using either of the above methods, the new directory name must be specified. This name can be either a full pathname or a pathname relative to the current working directory.

Parent topic: HRFS on page 18

2.13. Removing Subdirectories

Subdiriectories can be removed with ioctl( ), remove( ), or rmdir( ).

About This Task

A directory that is to be deleted must be empty (except for the "." and ".." entries). The root directory can never be deleted.

Procedure

Subdirectories can be removed in the following ways:

- Use ioctl( ) with the **FIORMDIR** function and specifying the name of the directory.

  The file descriptor used can refer to any file or directory on the volume, or to the entire volume itself.
- Use the remove( ), specifying the name of the directory.
- Use rmdir( ) from **usrFsLib**.

Parent topic: HRFS on page 18

2.14. Reading Directory Entries

Directory entries can be read with standard **dirLib** APIs.

You can programmatically search directories on HRFS volumes using the opendir( ), readdir( ), rewinddir( ), and closedir( ) functions.

To obtain more detailed information about a specific file, use the fstat( ) or stat( ) function. Along with standard file information, the structure used by these functions also provides the file-attribute byte from a directory entry.

For more information, see the API reference for **dirLib**.

Parent topic: HRFS on page 18

WNDRVR

2.15. Working with Files and Volumes in HRFS

HRFS supports standard file I/O, file linking and unlinking, and file permission management functions. You can also retrieve the volume size by using files system utilities.

File I/O Functions

Files on an HRFS file system device are created, deleted, written, and read using the standard VxWorks I/O functions: creat( ), remove( ), write( ), and read( ). For more information, see Overview of VxWorks File Systems and I/O Devices on page 1 and the **ioLib** API references.

Note that and remove( ) is synonymous with unlink( ) for HRFS.

File Linking and Unlinking

When a link is created an inode is not used. Another directory entry is created at the location specified by the parameter to link( ). In addition, a reference count to the linked file is stored in the file's corresponding inode. When unlinking a file, this reference count is decremented. If the reference count is zero when unlink( ) is called, the file is deleted except if there are open file descriptors open on the file. In this case the directory entry is removed but the file still exists on the disk. This prevents tasks and processes (RTPs) from opening the file. When the final open file descriptor is closed the file is fully deleted freeing its inode.

Note that you cannot create a link to a subdirectory, only to a regular file.

File Permissions

HRFS files have POSIX-style permission bits (unlike dosFs files, which have attributes). The bits can be changed using the chmod( ) and fchmod( ) functions. See the API references for more information.

Maximum File Size

The effective maximum size of a file in HRFS is less than 2 TB. The size of files depends on the disk size and block count. The absolute limit is 2^64-1, but since the maximum block count is 2^32-1, a file must be less than 2 TB. The maximum size of a file in any single partition should be about 90% of the partition size.

Maximum File Name Length

The maximum characters in a file name is 251 excluding the end of string marker (EOS).

Getting the Volume Size

You can get size of the volume several ways as described below.

Use the deviceFsInfoShow( ) API. For more information, see File System Utility on page 82.

```
-> deviceFsInfoShow "/bd0a"
    Device Name [/bd0a]
     File System     [MS-DOS Compatible File System]
     Block Size      [2048]
     Total Blocks    [39212]
     Free Blocks     [37748]
     Available Block [37748]
     Total Files     [-1]
```


WNDRVR

```
Free Files       [-1]
File System ID   [0x199b0000,0x0]
value = 0 = 0x0
```

The volume size is Block Size (2048) * Total Blocks (39212), therefore, the total volume size is 80306176 bytes.

2048 * 39212 = 80306176

Use the xpPartShow( ) APIs. For more information, see File System Utility on page 82.

```
-> xpPartShow "/bd0a"
   Request name [/bd0a], Real Device name [/bd0a], XBD Base name [/bd0]
    Device geometry:
    Total Blocks [15728640] Bytes per sector [512]
    CHS: Cylinder [249660] Head [1] Sector [63]

    Mount state:
    Idx.  IO Path     XBD Name        Start Sector       End Sector     Sector Count
    -------------------------------------------------------------------------------
    Total Not Valid   /bd0:0                     0         15728639         15728640
    1     /bd0a       /bd0a                    128           157311           157184
    2     /bd0b       /bd0b                 157312         15728511         15571200

    MBR Info:
    Idx. Boot  Type  Start Sector   End Sector  Sector Count  CHS(Cyl. Head Sec.)
    -------------------------------------------------------------------------------
    1   80    06           128       157311        157184    0,  2, 3    9,193, 1
    2   00    0C        157312     15728511      15571200    9,193, 2  975, 58,58
    Empty entry
    Empty entry
    value = 0 = 0x0
```

The **/bd0a** Sector Count (157184) * Sector Size (512), therefore, the total volume size is 80478208 bytes.

157184 * 512 = 80478208

> **Note:** The total volume size values are different in the above examples because the file system may reserve areas to store meta-data or other data.

Use the VolumeSizeShow( ) routine below. The routine is based on statfs( ) to show the volume size where you can use either statfs( ) or statfs64( ) (for larger volumes).

```
#include "sys/stat.h"

     STATUS VolumeSizeShow
     (
     char *devName      /* device name */
     )
     {
     STATUS status = -1;
     struct statfs64 fs;

     if (devName == NULL)
     return(-1);

     /* get statistic of device */
```


WNDRVR

```
    if ((status = statfs64 ((char*)devName, &fs)) == 0)
    {
    printf ("%s: Total File System Size: %fMB, Free space: %fMB.\n",
    devName,
    ((float)(fs.f_bsize * fs.f_blocks) /(1024.0*1024.0)),
    ((float)(fs.f_bsize * fs.f_bavail) /(1024.0*1024.0)));
    }

    return (status);
    }
```

Parent topic: HRFS on page 18

2.16. I/O Control Functions Supported by HRFS

The HRFS file system supports a set of ioctl( ) functions.

These functions are defined in the header file **ioLib.h** along with their associated constants.

Function	Description
FIOCHMOD	Change or assign the mode of a file.
FIOCHOWN	Change the owner of a file.
FIOFLUSH	Flushes the output buffer.
FIOFSTATGET	Gets file status information (directory entry data).
FIOMOVE	Moves a file (does not rename the file).
FIONFREE	Gets the number of free bytes on the volume.
FIONREAD	Gets the number of unread bytes in a file.
FIOREADDIR	Reads the next directory entry.
FIORENAME	Renames a file or directory.
FIORMDIR	Removes a directory.
FIOSEEK	Sets the current byte offset in a file.
FIOSYNC	Same as **FIOFLUSH**.
FIOTIMESET	Update the time on a file.
FIOTRUNC	Truncates a file to a specified length.
FIOUNMOUNT	Un-mounts a disk volume.

Function	Description
FIOWHERE	Returns the current byte position in a file.
FIONFREE64	Gets the number of free bytes into a 64-bit integer.
FIONREAD64	Gets the number of unread bytes in a file into a 64-bit integer.
FIOSEEK64	Sets the current byte offset in a file from a 64-bit integer.
FIOWHERE64	Gets the current byte position in a file into a 64-bit integer.
FIOTRUNC64	Set the file's size from a 64-bit integer.

For more information, see the API reference entry for ioctl( ) in **ioLib**.

Parent topic: HRFS on page 18

2.17. Crash Recovery and Volume Consistency

HRFS is a transactional based file system that is designed to be consistent at all times.

HRFS reliability depends on the storage media having the following features:

- Performance of atomic sector writes even when there is a power failure. An atomic sector write is a write operation to the media that either succeeds with all data written correctly, or that fails with no new data written and any old data left intact.
- If the storage media can re-order sector writes, it must provide a way to enforce an order (cache flush and so on).
- Guarantees that sectors are written atomically when flushed from the cache (if present), even on power failure.

If the storage media does not meet the above recommendations, Wind River recommends configuring HRFS for default write mode rather than high speed write mode to lessen the possibility of corruption if a power failure occurs. A storage media without the features listed above may cause HRFS lose data or to be corruption and unable to recover.

CAUTION: **Wind River recommends that you do not use HRFS with TrueFFS, as TrueFFS does not support atomic sector writes.**

Crash Recovery

If a system unexpectedly loses power or crashes, HRFS rolls back to the last transaction point when the system reboots. The rollback occurs automatically when the file system is mounted. Any changes made after the last complete transaction are lost, but the disk remains in a consistent state.

Consistency Checking

The hrfsChkDsk( ) function can be used to check for inconsistencies in the file system. The execution of the disk checker is not automatic; it must be done programmatically. The hrfsChkDsk( ) function is also useful in relation to hardware problems, and is usually run at boot time for this purpose.

Parent topic: HRFS on page 18

2.18. File Management and Full Devices

HRFS is designed to manage file operations on a full volume in a way that is consistent with its design as a highly reliable file system.

When an HRFS device is full, attempts to delete or write to files with standard commands will fail (it is, however, unlikely that a device will become full). This behavior is not a defect, but is inherent in highly reliable file systems that use copy-on-write (COW) to implement their transaction mechanism.

During a write operation, the file system writes modified data to a temporary block during a transaction. For example, in order to write block A, block B is first allocated, and the data is written to B. When the transaction is complete, block B is re-mapped to replace block A, and block A is freed. When HRFS removes a file, it first renames the file. It then allocates a new block, and copies the modified directory entry block to it.

If the device is full, the allocation operation fails. It is, however, unlikely that a device will become full, because when a write operation fails due to a lack of free blocks, HRFS rolls back the failed transaction, and the blocks allocated for the transaction are freed. It is then possible to delete files. If a transaction does happen to write the device full, there is no way to free blocks for further operations. Wind River therefore recommends that you do not allow an HRFS device to become full. Leaving a margin of five percent of device space free is a useful guideline.

Automatically Removing Temporary Files to Regain Storage Space

If the system shuts down unexpectedly, temporary files (for example, **~DelMe23896**) may take up storage space in the **.lost+found** directory. To resolve this issue, these VSB components are enabled by default: **HRFS_LOSTFOUND_DIR_DELETABLE** and **HRFS_LOSTFOUND_AUTOCLEAN**. For more information, see VxWorks Configuration for HRFS on page 20.

Parent topic: HRFS on page 18

3. ROMFS

ROMFS is a simple, read-only file system that stores and represents files and directories in a hierarchical manner. The name ROMFS stands for Read-Only Memory File System (it does not imply any particular relationship to ROM media). No local disk or network connection to a remote disk is required.

ROMFS can be used to store any type of file, including RTP executables and kernel application modules. Applications that are stored in ROMFS can be started automatically at boot time with the VxWorks application startup facility. They can also be executed programmatically or interactively from the kernel shell, which can be useful for interactive diagnostic applications or applications that are started by other applications under specific circumstances (in response to errors, and so on) .

ROMFS can be implemented in two ways:

- Statically configured and built with the VxWorks image, including the contents of the file system.
- Programmatically created at runtime, when the contents are also added to the file system.

In both cases, VxWorks must be configured with support for ROMFS.

The static and programmatic methods can be used together in the same system; they are not mutually exclusive. The static method can only be used to create as single instance of ROMFS. The programmatic method allows for creating multiple instances; and it allows you to update the contents of the file system without reconfiguring and rebuilding the VxWorks image.

For certified systems, static configuration allows for certification of a VxWorks image with its ROMFS contents, while programmatic instantiation allows for adding ROMFS and its contents to a system without modifying the certified image.

Note: The ROMFS file system can have a capacity of up to 1 GB of memory. If the contents of ROMFS is too large, the VxWorks image will not link or the system will not boot.

Creating a ROMFS File System Statically on page 38 To create a ROMFS file system statically—that is, entirely at system configuration time—you must configure VxWorks with support for ROMFS, create a ROMFS project that is subordinate to your VIP project, and add the contents for the file system to the ROMFS project.
Creating a ROMFS File System Programmatically on page 38 To create a ROMFS file system programmatically, you must configure VxWorks with support for ROMFS, create a "payload" file containing the contents for the file system, and write an application that loads the payload file into memory and then instantiates the file system.
Accessing Files in ROMFS on page 39 At run-time, the ROMFS file can be accessed like other local file systems.

3.1. Creating a ROMFS File System Statically

To create a ROMFS file system statically—that is, entirely at system configuration time—you must configure VxWorks with support for ROMFS, create a ROMFS project that is subordinate to your VIP project, and add the contents for the file system to the ROMFS project.

Before You Begin

You must know how to use Workbench or wrtool to create and configure VSB, VIP, and ROMFS projects. For information in this regard, see the *VxWorks 7 Configuration and Build Guide* and the *VxWorks 7 Kernel Application and RTP Application Projects Guide*.

Procedure

1. Create a VSB project, select the **FS** and **FS_ROMFS** layers, and build the source code.
2. Create a VIP project based on the VSB project, and select the **INCLUDE_ROMFS** component.

   This automatically adds the **INCLUDE_ROMFS_DRV** component as well, which initializes the ROMFS file system driver at runtime.

3. Optionally, change the default ROMFS device name, which is set with the **ROMFS_DEV_NAME** configuration parameter.

   ROMFS device names must be unique to the system.

4. Create a ROMFS project as a sub-project of your VIP project.
5. Add the files for the ROMFS file sytem to the ROMFS project.
6. Build VxWorks.

Parent topic: ROMFS on page 37

3.2. Creating a ROMFS File System Programmatically

To create a ROMFS file system programmatically, you must configure VxWorks with support for ROMFS, create a "payload" file containing the contents for the file system, and write an application that loads the payload file into memory and then instantiates the file system.

Before You Begin

You must know how to use Workbench or wrtool to create and configure VSB, VIP, and (optionally) ROMFS projects. For information in this regard, see the *VxWorks 7 Configuration and Build Guide* and the *VxWorks 7 Kernel Application and RTP Application Projects Guide*.

WNDRVR

Procedure

1. Create a VSB project, select the **FS** and **FS_ROMFS** layers, and build the source code.
2. Create a VIP project based on the VSB project, and select the **INCLUDE_ROMFS_DRV** component, which automatically initializes the ROMFS file system driver at runtime.

   If you want to use both the static and programmatic methods in the same system, add **INCLUDE_ROMFS**, which automatically adds **INCLUDE_ROMFS_DRV**.

3. Optionally, change the default ROMFS device name, which is set with the **ROMFS_DEV_NAME** configuration parameter.

   ROMFS device names must be unique within the system.

4. Build VxWorks.
5. Create the ROMFS payload file.

   The ROMFS payload file. It can be created with a ROMFS project using Workbench or wrtool, in which case it is called **content.romfs**. The payload file can also be created with the mkromfs command-line utility.

   If you want to use Workbench or wrtool for a ROMFS project, it must be standalone project. That is, it must not be subordinate to your VIP project.

   If you want to use mkromfs, you can display usage information by executing the command without any options. The basic options specify the output file name, and the directory containing the files you want to add to the ROMFS file system. For example:

   ```
   % mkromfs -o romfscontent ./myfiles
   ```

6. Write an application to load the payload file into memory.

   The file can be loaded into RAM with a memory-mapped device, or some other means (for example mmap( ) or a network load/store operation), depending on system requirements and hardware.

7. In your application, call romfsVolumeCreate( ).

   The arguments for the function are a pointer to the ROMFS device name (which must be unique within the system), the starting address for the ROMFS image, the size of the image, and the bytes offset value of the image. The signature for the function is:

   ```
   STATUS romfsVolumeCreate(char *pDevName, char *pAddr, size_t romfsImageLen, off_t offset)
   ```

   For more information, see the *VxWorks 7 Kernel API Reference*.

Parent topic: ROMFS on page 37

3.3. Accessing Files in ROMFS

At run-time, the ROMFS file can be accessed like other local file systems.

The content of the ROMFS directory can be browsed using shell commands like ls and cd, and accessed programmatically with standard file system functions, such as open( ) and read( ).

For example, if the file **foo.vxe** is in the directory **/romfs**, then using cd and ls from the kernel shell (with the command interpreter) looks like this:

WNDRVR

```
[vxWorks *]# cd /romfs
[vxWorks *]# ls
.
..
foo.vxe
[vxWorks *]#
```

foo.vxe can also be accessed at run-time as **/romfs/foo** by any applications running on the target.

Parent topic: ROMFS on page 37

4. dosFs

The dosFs file system is an MS-DOS-compatible file system that offers considerable flexibility appropriate to the multiple demands of real-time applications.

The primary features of dosFs are:

- Hierarchical files and directories, allowing efficient organization and an arbitrary number of files to be created on a volume.
- Maximum path length of 1024 characters and maximum file name length of 255 characters.
- A choice of contiguous or non-contiguous files on a per-file basis.
- Compatible with widely available storage and retrieval media (diskettes, hard drives, and so on).
- The ability to boot VxWorks from a dosFs file system.
- Support for VFAT (Microsoft VFAT long file names)
- Support for FAT12, FAT16, and FAT32 file allocation table types.

CAUTION: **While dosFs supports path names of 1024 characters, using a path name of more than 259 characters makes it incompatible with Windows (that is, the file cannot be used on a Windows host).**

For information about dosFs libraries, see the VxWorks API references for dosFsLib and dosFsFmtLib.

For information about the MS-DOS file system, please see the Microsoft documentation.

dosFs and Flash Memory

For information about using dosFs with flash memory, see True Flash File System (TrueFFS) on page 106 and the implementation instructions.

VxWorks Configuration for dosFs on page 41 To configure VxWorks with support for dosFs, you must select the required layers and components for your VSB and VIP projects. dosFs has numerous optional components that may be used as well.
dosFs Configuration Options on page 43 Several dosFs component configuration parameters can be used to define how the file system behaves when a dosFs volume is mounted.
dosFsDevCreate on page 44 Normally dosFsDevCreate( ) function is called automatiaclly by the operting system when it detects the presence of a FAT formated disk, either at boot or when removable media is inserted.
Creating a dosFs File System on page 45 To create a dosFs file system, VxWorks must be configured with the appropriate support, and your code must minimally create partitions (if required), and format the volume.
dosFs, ATA Disk, and RAM Disk Examples on page 47 These examples provide generic illustrations of dosFs use.
Creating dosFs for an ATA Disk on page 47 This example demonstrates how to initialize an ATA disk with dosFs from the shell.

WNDRVR

Creating and Partitioning a Disk and Creating Volumes on page 49 This code example takes a pointer to a block device, creates three partitions, creates the partition handler for these partitions, and creates the dosFs device handler for them. Then, it formats the partitions using dosFsVolFormat( )
Creating and Formatting a RAM Disk Volume on page 50 This code creates a RAM disk and formats it for use with the dosFs file system.
Optimizing dosFs Performance on page 51dosFs performance can be optimized by changing configuration parameters, and with write( ) usage at runtime.
Working with Volumes and Disks on page 52dosFs provides APIs for accessing volume configuration information and for synchronizing volumes.
Directory Separators on page 54dosfs supports both back-slash and forward-slash directory separators.
Creating Subdirectories on page 54 Subdirectories can be created using ioctl( ), open( ), or mkdir( ).
Removing Subdirectories on page 55 Subdiriectories can be removed with ioctl( ), remove( ), or rmdir( ).
Reading Directory Entries on page 55 Directory entries can be read with standard **dirLib** APIs.
Working with Files on page 56 Files on a dosFs file system device are created, deleted, written, and read using the standard VxWorks I/O functions. File attributes are changed with ioctl( ). You can also retrieve the volume size.
Code Example for Setting DosFs File Attributes on page 59 This example makes a dosFs file read-only, and leaves other attributes intact..
Disk Space Allocation Options on page 59 The dosFs file system allocates disk space using one of three methods. Two are automatic, based on the size of the write operation. The third must be implemented with your code.
Choosing an Allocation Method on page 60 Under most circumstances, cluster group allocation is preferred to absolutely contiguous file access.
Cluster Group Allocation on page 60 Cluster group allocation occurs when the cluster group size is considered sufficiently large so that the seek time is negligible compared to the read/write time.
Using Absolutely Contiguous Allocation on page 61 Absolutely contiguous allocation is intended to allocate contiguous space to a specified file (or directory) and, by so doing, optimize access to that file.
Crash Recovery and Volume Consistency on page 63dosFs provides an optional consistency-checking mechanism to detect and recover from data structure inconsistencies.
dosFsLib API Overview on page 64 Describes expanded use cases for dosFsLib and descriptions for how to use different calls and functions.
I/O Control Functions Supported by dosFsLib on page 68 The dosFs file system supports a set of ioctl( ) functions.

4.1. VxWorks Configuration for dosFs

To configure VxWorks with support for dosFs, you must select the required layers and components for your VSB and VIP projects. dosFs has numerous optional components that may be used as well.

VSB Project

Select **FS_DOSFS** and build the source code.

VIP Required Components

The following components are required:

INCLUDE_DOSFS_MAIN	**dosFsLib**
INCLUDE_DOSFS_FAT	dosFs FAT12/16/32 FAT handler
INCLUDE_XBD	XBD component.

And, either one or both of the following components are required:

WNDRVR

INCLUDE_DOSFS_DIR_VFAT	Microsoft VFAT direct handler
INCLUDE_DOSFS_DIR_FIXED	Strict 8.3 & VxLongNames directory handler

In addition, you must include the appropriate component for your block device.

If you are using a device driver that is not designed for use with the XBD facility, you must use the **INCLUDE_XBD_BLK_DEV** wrapper component in addition to **INCLUDE_XBD**. For information about the XBD facility, see Extended Block Device Facility: XBD Overview on page 79.

Note that you can use **INCLUDE_DOSFS** to automatically include the following components:

- **INCLUDE_DOSFS_MAIN**
- **INCLUDE_DOSFS_DIR_VFAT**
- **INCLUDE_DOSFS_DIR_FIXED**
- **INCLUDE_DOSFS_FAT**
- **INCLUDE_DOSFS_CHKDSK**
- **INCLUDE_DOSFS_FMT**

VIP Optional Components

The optional dosFs components are the following:

INCLUDE_DOSFS_PRTMSG_LEVEL	Message output level. To suppress messages, set the **DOSFS_PRINTMSG_LEVEL** configuration parameter to 0 (zero). The default (with or without this component) is to output messages.
INCLUDE_DOSFS_CACHE	Disk cache facility.
INCLUDE_DOSFS_FMT	dosFs file system formatting module.
INCLUDE_DOSFS_CHKDSK	File system integrity checking.
INCLUDE_DISK_UTIL	Standard file system operations, such as ls, cd, mkdir, xcopy, and so on.
INCLUDE_TAR	The tar utility.

Optional XBD Components

Optional XBD components are:

INCLUDE_XBD_PART_LIB	Disk partitioning facilities.
INCLUDE_XBD_RAMDRV	RAM disk facility.

WNDRVR

Optional SATA Component

The **INCLUDE_SATA_DISK_NAME_CFG** component is optional. It can be used to change the default name of the device. The assignment of an empty string means that the default name will be used (**/ata0**).

Parent topic: dosFs on page 40

4.2. dosFs Configuration Options

Several dosFs component configuration parameters can be used to define how the file system behaves when a dosFs volume is mounted.

DOSFS_CHK_ONLY

When a dosfs volume is mounted, the media is analyzed for errors, but no repairs are made.

DOSFS_CHK_REPAIR

Similar to **DOSFS_CHK_ONLY**, but an attempt to repair the media is made if errors are found.

DOSFS_CHK_NONE

Media is not checked for errors on mount.

DOSFS_CHK_FORCE

Used in conjunction with **DOSFS_CHK_ONLY** and **DOSFS_CHK_REPAIR** to force a consistency check even if the disk has been marked clean.

DOS_CHK_VERB_SILENT or **DOS_CHK_VERB_0**

dosFs does not to produce any output to the terminal when mounting.

DOS_CHK_VERB_1

dosFs produces a minimal amount of output to the terminal when mounting.

DOS_CHK_VERB_2

dosFs produces a maximum amount of output to the terminal when mounting.

Other parameters can be used to configure physical attributes of the file system. They are as follows:

DOSFS_DEFAULT_CREATE_OPTIONS

The default parameter for the dosFsLib component. It specifies the action to be taken when a dosFs file system is instantiated. Its default is **DOSFS_CHK_NONE**.

DOSFS_DEFAULT_MAX_FILES

The maximum number of files. The default is 20.

DOSFS_DEFAULT_FAT_CACHE_SIZE

The size of the FAT group cache (for each dosFs volume).

DOSFS_DEFAULT_DATA_DIR_CACHE_SIZE

The disk-data and directory-group cache size (for each dosFs volume).

WNDRVR

DOSFS_CACHE_BACKGROUND_FLUSH_TASK_ENABLE

The cache-flushing mode. Set to FALSE (default), cache-flushing takes place in the context of the user's read/write task. Set to TRUE, a separate high-priority task is spawned to carry out cache flushing. Use of a separate task speeds up write operations (especially when dosFs is configured with a larger cache size), but using a separate high-priority task might not be suitable for systems with strict real-time requirements.

For information about the use of some of these component parameters in optimizing dosFs performance, see Optimizing dosFs Performance on page 51.

Caches can be tuned dynamically for individual instances of the file system using the dosFsCacheInfo( ) and dosFsCacheTune( ) functions.

The functions dosFsCacheDelete( ) and dosFsCacheCreate( ) can be used to delete and changes the size of caches. To change the size, first use the delete function, and then the create function.

Parent topic: dosFs on page 40

4.3. dosFsDevCreate

Normally dosFsDevCreate( ) function is called automatiaclly by the operting system when it detects the presence of a FAT formated disk, either at boot or when removable media is inserted.

Using The dosFsDevCreate API

The parmeters used are determined by the kernel configuration. See VxWorks Configuration for dosFs on page 41 for more information. The dosFsDevCreate() function associates a device with the dosFsLib functions. The function expects four parameters:

- A pointer to a name string. The pointer becomes part of the pathname for I/O operations that operate on the device. This name appears in the I/O system device table, which can be displayed using the iosDevShow( ) function.
- **device_t** - a XBD for the device on which to create the file system. Examples include: a partition XBD, an XBD block wrapper, or an ATA device XBD.
- A maximum number of files that can be simultaneously opened on a particular device.
- Flags for volume checking, metadata integrity, and file name interpretation. Because the volume, integrity, and check utility can be automatically invoked every time a device is mounted, this parameter indicates whether the consistency check needs to be performed automatically on a given device, and on what level of verbosity is required. You can invoke the consistency check at a later time by calling chkdsk( ). See description for **FIOCHKDSK** ioctl command for more information.

For example:

```
dosFsDevCreate
    (
    "/sd0",     /* name to be used for volume   */
    device,     /* underlying XBD device */
    10,         /* max no. of simultaneously open files */
    DOS_CHK_REPAIR | DOS_CHK_VERB_1
                /* check volume during mounting and repair */
                /* errors, and display volume statistics */
    )
```

Once you call dosFsDevCreate(), you can access the device using **ioLib** generic I/O routines: open( )), read( ), write( ), close( ), ioctl( ) and remove( ). In addition, you can use the user-level utility functions to access the device at a higher level (See **usrFsLib** reference page for more details).

WNDRVR

Device and Path Names

On true MS-DOS machines, disk device names are typically of the form "A:", a single letter designator followed by a colon. Such names can be used with the VxWorks dosFs file system. However, Wind River recommends using longer, more mnemonic device names, such as "DOS1:", or "/floppy0". The name is specified during the dosFsDevCreate() call. Typically the File System Monitor module automatically calls this function. Thus you can call fsmNameInstall( ) to specify the desired name for the device. For more details about the File System Monitor, refer to File System Monitor on page 80.

The pathnames used to specify dosFs files and directories can use either forward slashes (**"/"**) or backslashes (**"\"**) as separators. These can be freely mixed. The choice of forward slashes or backslashes does not effect the directory data written to the disk.

Wind River recommends the use of forward slashes (**"/"**) at all times.

Note: The leading slash of a dosFs pathname following the device name is optional. For example, both "DOS1:newfile.new" and "DOS1:/newfile.new" refer to the same file.

Parent topic: dosFs on page 40

4.4. Creating a dosFs File System

To create a dosFs file system, VxWorks must be configured with the appropriate support, and your code must minimally create partitions (if required), and format the volume.

Before You Begin

For basic operating system configuration, see VxWorks Configuration for dosFs on page 41.

About This Task

The dosFs file system library is initialized automatically at boot time. Initializing the file system invokes **iosDrvInstall**, which adds the driver to the I/O system driver table. The driver number assigned to the dosFs file system is recorded in a global variable, *dosFsDrvNum*. The table specifies the entry points for the dosFs file operations that are accessed by the devices using dosFs. The block device is created automatically by the block device driver.

This topic describes the process of creating a dosFs file system. It first provides a summary overview and then a detailed description of each step. See dosFs, ATA Disk, and RAM Disk Examples on page 47 for examples of the steps and code examples.

Procedure

1. Create an XBD device wrapper.

   If you are using a device driver that is not XBD-compliant, it requires an XBD device wrapper.

   The wrapper is created automatically if you have configured VxWorks with the **INCLUDE_XBD_BLK_DEV** wrapper component. Otherwise, create a wrapper for each block device using xbdBlkDevCreate( ).

   After the XBD device wrapper is created the physical device is automatically probed for a file system and partitions. If a disk is already formatted, the disk is mounted. If a a file system is found, it is mounted. If the file system is not dosFs, it must be formatted (see below).

2. Create partitions.

WNDRVR

If you have included the **INCLUDE_XBD_PART_LIB** component in your system, you can create partitions on a disk and mount volumes atop the partitions. Use the xbdCreatePartition( ) function to create partitions.

This step should only be performed once, when the disk is first initialized. If partitions are already written to the disk, this step should not be performed as it destroys data.

3. Format the volume

   If you are using unformatted disk or wish to replace the current file system on the disk, format the disk by calling dosFsVolFormat( ). For more information, see the VxWorks API reference for this function.

   The MS-DOS and dosFs file systems provide options for the format of the File Allocation Table (FAT) and the format of the directory. These options, described below, are completely independent.

   CAUTION: **Reformatting a disk destroys any data that may be on it.**

   File Allocation Table (FAT) Formats

Format	FAT Table Entry Size	Usage	Size
FAT12	12 bits per cluster number	Appropriate for very small devices with up to 4,084 clusters.	Typically, each cluster is two sectors large.
FAT16	16 bits per cluster number	Appropriate for small disks of up to 65,524 clusters.	Typically, used for volumes up to 2 GB; can support up to 8 GB.
FAT32	32 bits (only 28 used) per cluster number	Appropriate for medium and larger disk drives.	By convention, used for volumes larger than 2 GB.

   The options for the directory format are:

   - MSFT Long Names (VFAT)
     Uses case-insensitive long filenames, with up to 254 characters. This format accepts disks created with short names. MSFT Long Names is the default directory format.

     Note: The MSFT Long Names (VFAT) format supports 32-bit file size fields, limiting the file size to a 4 GB maximum.

   - Short Names (8.3)
     Case-insensitive MS-DOS-style filenames (8.3), with eight uppercase characters for the name itself and three for the extension.
   - Sector Size
     Hardware sector size must be 128, 256, 512, 1024, 2048 or 4096 bytes. Any other sector size value may cause format or mount failures with dosFs.

4. Change the disk cache size.

   If you have included the optional **INCLUDE_DOSFS_CACHE** component, disk caches are automatically created. Two component parameters define the size of the data, directory entry, and FAT caches: **DOSFS_DEFAULT_DATA_DIR_CACHE_SIZE** and **DOSFS_DEFAULT_FAT_CACHE_SIZE**.

   In addition, the caches can be tuned dynamically using the dosFsCacheInfo( ) and dosFsCacheTune( ) functions.

WNDRVR

You can change the size of the cache for a particular instantiation of the file system by first deleting the cache with dosFsCacheDelete( ) and then re-creating the cache with dosFsCacheCreate( ). For more information see Creating dosFs for an ATA Disk on page 47 and the VxWorks API references for these functions.

A disk cache is intended to reduce the number of accesses to the media. It is not intended for use with RAM disks or TrueFFS. If the cache component is included with TrueFFS, the cache should be removed programmatically with dosFsCacheDelete( ).

5. Check disk volume integrity.

   Optionally, check the disk for volume integrity using dosFsChkDsk( ). Disk checking large disks can be time-consuming.

   The parameter **DOSFS_DEFAULT_CREATE_OPTIONS** (of the **INCLUDE_DOSFS_MAIN** component) provides an option for checking the disk, which takes place automatically when the dosFs file system is mounted. This can, however, be a time consuming process and makes the file system inaccessible until the check is complete. Alternatively, the check can be performed programmatically by calling dosFsChkDsk( ).

Parent topic: dosFs on page 40

4.5. dosFs, ATA Disk, and RAM Disk Examples

These examples provide generic illustrations of dosFs use.

They illustrate the steps discussed in the preceding topics. They also use a variety of configurations and device types. They are meant to be relatively generic and applicable to most block devices.

The examples illustrate the following:

- Creating and working with a dosFs file system on an ATA disk with commands from the shell.
- Code that creates and formats partitions.
- Code that creates and formats a RAM disk volume.

The examples in this section require that VxWorks be configured with the **INCLUDE_DOSFS_FMT** component. One example also relies on the optional **INCLUDE_DOSFS_CACHE** component.

CAUTION: **Because device names are recognized by the I/O system using simple substring matching, file systems should not use a slash (/) alone as a name; unexpected results may occur.**

Parent topic: dosFs on page 40

4.6. Creating dosFs for an ATA Disk

This example demonstrates how to initialize an ATA disk with dosFs from the shell.

About This Task

While these steps use an XBD-compatible ATA block device, they are applicable to any XBD-compatible block device.

The block device is created automatically by the block device driver.

Procedure

1. Display information about devices.

WNDRVR

```
-> devs
drv name
  0 /null
  1 /ttyS0
  1 /ttyS1
  8 yow-grand:
  9 /vio
  4 /ata0
value = 25 = 0x19
```

The new ATA device **/ata0** is listed and no partitions were detected. Note that if no file system is detected on the device, the rawFs file system is instantiated automatically and appears the device list.

2. Create two partitions on this disk device, specifying 50% of the disk space for the second partition, leaving 50% for the first partition.

   This step should only be performed once, when the disk is first initialized. If partitions are already written to the disk, this step should not be performed since it destroys data.

```
-> xbdCreatePartition ("/ata0", 2, 50, 0, 0)
value = 0 = 0x0
```

   The second parameter defines the number of partitions, and that the parameters that follow define the percentage of space for the second, third, and fourth partitions—with the remainder being used for the first partition.

3. List the devices to display information about the new partitions.

```
-> devs
drv name
  0 /null
  1 /ttyS0
  1 /ttyS1
  8 yow-grand:
  9 /vio
  3 /ata0a
  3 /ata0b
```

   /ata0 does not appear in the list because the disk has been partitioned, and two new devices, **/ata0a** and **/ata0b**, have been added to represent the new partitions. Each volume has rawfs instantiated in it as they are new and unformatted.

4. Format the volumes for dosFs.

   This step need only be done once, when the volumes are first initialized. If the volumes have already been initialized (formatted), then omit this step. This example formats the file system volumes with default options.

```
-> dosFsVolFormat ("/ata0a", 0, 0)
Formatting /ata0a for DOSFS
Instantiating /ata0a as rawFs
Formatting...Retrieved old volume params with %100 confidence:
Volume Parameters: FAT type: FAT32, sectors per cluster 8
  2 FAT copies, 0 clusters, 38425 sectors per FAT
  Sectors reserved 32, hidden 0, FAT sectors 76850
  Root dir entries 0, sysId (null) , serial number 3a80000
  Label:" " ...
Disk with 40149184 sectors of 512 bytes will be formatted with:
Volume Parameters: FAT type: FAT32, sectors per cluster 8
```


WNDRVR

```
  2 FAT copies, 5008841 clusters, 39209 sectors per FAT
  Sectors reserved 32, hidden 0, FAT sectors 78418
  Root dir entries 0, sysId VX5DOS32, serial number 3a80000
  Label:" " ...
OK.
value = 0 = 0x0

-> dosFsVolFormat ("/ata0b", 0, 0)
Formatting /ata0b for DOSFS
Instantiating /ata0b as rawFs
Formatting...Retrieved old volume params with %100 confidence:
Volume Parameters: FAT type: FAT32, sectors per cluster 8
  2 FAT copies, 0 clusters, 19602 sectors per FAT
  Sectors reserved 32, hidden 0, FAT sectors 39204
  Root dir entries 0, sysId (null) , serial number c78ff000
  Label:" " ...
Disk with 40144000 sectors of 512 bytes will be formatted with:
Volume Parameters: FAT type: FAT32, sectors per cluster 8
  2 FAT copies, 5008195 clusters, 39204 sectors per FAT
  Sectors reserved 32, hidden 0, FAT sectors 78408
  Root dir entries 0, sysId VX5DOS32, serial number c78ff000
  Label:" " ...
OK.
value = 0 = 0x0
```

5. Optionally, create or modify a cache for a dosFs volume.
   - If you included the optional **INCLUDE_DOSFS_CACHE** component, a cache is automatically created (as defined by its parameters). To change the data-directory cache size and the FAT cache size, you must first delete the cache with dosFsCacheDelete( ) and then create a new one with dosFsCacheCreate( ). For example:

     ```
     -> dosFsCacheDelete "/ata0a"
     value = 0 = 0x0
     -> dosFsCacheCreate "/ata0a", 0, 512 * 1024, 0, 128 * 1024
     value = 0 = 0x0
     ```

   - If a cache does not exist, use dosFsCacheCreate( ) to create one.

Parent topic: dosFs on page 40

4.7. Creating and Partitioning a Disk and Creating Volumes

This code example takes a pointer to a block device, creates three partitions, creates the partition handler for these partitions, and creates the dosFs device handler for them. Then, it formats the partitions using dosFsVolFormat( )

```
STATUS usrPartDiskFsInit
    (
    char * xbdName /* device name used during creation of XBD */
    )
    {
    const char * devNames[] = { "/sd0a", "/sd0b", "/sd0c" };
    devname_t xbdPartName;
    u_int newDataDirCacheSize = 0x800000
    u_int newFatCacheSize = 0x100000
    DOSFS_CACHE_INFO cacheParams;
    int i;
```


WNDRVR

```
  /* Map partition names */
  for (i = 1; i <= 3; i++)
      {
      sprintf (xbdPartName, "%s:%d", devNames[i-1],i);
      fsmNameInstall (devNames[i], xbdPartName);
      }

  /* create partitions */

  if((xbdCreatePartition (xbdName,3,50,45,0)) == ERROR)
    return ERROR;

  /* Formatting the first partition */

 if(dosFsVolFormat (devNames[0], 2,0) == ERROR)
   return ERROR;

  /* Re-configure the cache for the first partition */

  if (dosFsCacheCreate (devNames[0], NULL, newDataDirCacheSize,
                                        NULL, newFatCacheSize) == ERROR)
  return ERROR;

  /* Retrieve the current data cache tuning parameters and double them */

  if (dosFsCacheInfoGet (devNames[0], DOS_DATA_CACHE, &cacheParams) == ERROR)
     return ERROR;

  cacheParams.bypass = cacheParams.bypass * 2;
  cacheParams.readAhead = cacheParams.readAhead * 2;

  if (dosFsCacheTune (devNames[0], DOS_DATA_CACHE, &cacheParams) == ERROR)
      return ERROR;

  /* Formatting the second partition */

  if(dosFsVolFormat (devNames[1], 2,0) == ERROR)
     return ERROR;

  /* Formatting the third partition */

  if(dosFsVolFormat (devNames[2], 2,0) == ERROR)
    return ERROR;

  return OK;
  }
```

Note that in most cases you would be likely to format the different partitions for different file systems.

Parent topic: dosFs on page 40

4.8. Creating and Formatting a RAM Disk Volume

This code creates a RAM disk and formats it for use with the dosFs file system.

```
#include "vxWorks.h"
#include "xbdRamDisk.h"
#include "dosFsLib.h"

STATUS usrRamDiskInit
    (
    void /* no argument */
    )
    {
    int ramDiskSize = 512 * 1024 ; /* 512KB, 512 bytes per sector */
    char *ramDiskDevName = "/ram0" ;
    device_t xbd;

    /* 512 byte/sec, no partition support */

    xbd = xbdRamDiskDevCreate (512, ramDiskSize, 0, ramDiskDevName);

    if( xbd == NULL )
    return ERROR ;

    /* format the RAM disk, ignore memory contents */

    dosFsVolFormat( ramDiskDevName, DOS_OPT_BLANK | DOS_OPT_QUIET, NULL );
    return OK;
    }
```

Parent topic: dosFs on page 40

4.9. Optimizing dosFs Performance

dosFs performance can be optimized by changing configuration parameters, and with write( ) usage at runtime.

About This Task

The dosFS cache includes two parts: the FAT cache and the data cache. If all data is kept in cache, there will be a good hit ratio. A larger cache size can improve the hit ratio, but only up to a certain point. For specific types of target hardware, performance can also be improved by adjusting the cluster and block sizes of the caches.

Note: For more information about the dosFs component parameters, see dosFs Configuration Options on page 43.

The following method may be used to improve dosFs performance.

Procedure

- Set the **DOSFS_CACHE_BACKGROUND_FLUSH_TASK_ENABLE** configuration parameter to TRUE. This creates a separate high priority task to carry out cache flushing.
- Set **DOSFS_DEFAULT_DATA_DIR_CACHE_SIZE** to 32 MB (the default is 8 MB).
- Set **DOSFS_DEFAULT_FAT_CACHE_SIZE** to 1 MB (the default is 1 MB).
- At runtime, after calling dosFsVolFormat( ), call dosFsCacheTune( ) to re-set the block group size for the data and directory cache, and the FAT cache, to get better read/write performance. Try using 16 for the clusters argument and 32 for the blocks arguments.

WNDRVR

- After you have determined which values provide the best performance by using dosFsCacheTune( ), you can set them permanently for your system with the configuration parameters **DOSFS_ BLOCKS_PER_DATA_DIR_CACHE_GROUP** and **DOSFS_BLOCKS_PER_FAT_CACHE_GROUP**.

  When setting configuration parameters for **DOSFS_BLOCKS_PER_FAT_CACHE_GROUP**, the following demonstrates the link between **DOSFS_DEFAULT_DATA_DIR_CACHE_SIZE** and **DOSFS_BLOCKS_PER_FAT_CACHE_GROUP**:

  - blksPerGrp: **DOSFS_BLOCKS_PER_DATA_DIR_CACHE_GROUP** (512)
  - cacheSize: **DOSFS_DEFAULT_DATA_DIR_CACHE_SIZE** (1M)
  - blockSize: Sector size 512B
  - CacheHeaderSize: size of (**DOS_CACHE_DESC**) + size of (void *)

  The number of cashe blocks is determined by cacheSize / (blksPerGrp * blockSize + CacheHeaderSize)

  Since the **CacheHeaderSize** is not zero, the real cache block will not reach the full cache size (**DOSFS_DEFAULT_DATA_DIR_CACHE_SIZE** / (**DOSFS_BLOCKS_PER_DATA_DIR_CACHE_GROUP** * 512)).

  For example, to create a 32-bit OS image sizeof(**DOS_CACHE_DESC**)+size of(void*) will be 2164. If you want to use a 32KB block cache node (32768B), one cache node is 32768+ 2164. 1MB cache will be divided into 1024*1024/(32768+2164) to create 30.0176 cache nodes. Therefore, the **DOSFS_BLOCKS_PER_DATA_DIR_CACHE_GROUP** should be set to 64 (64 * sector_size = 32KB). For a 8KB block cache node, **DOSFS_BLOCKS_PER_DATA_DIR_CACHE_GROUP** should be set to 16 (16 * sector_size = 8KB).

> **Note:** When calling write( ) in your applications, try using 512 KB for the bytes argument.
>
> If you cannot include the dosFs cache component due to some project/hardware requirements, increasing **DOSFS_DEFAULT_VOL_BIO_BUFFER_SIZE** can increase performance.

Parent topic: dosFs on page 40

4.10. Working with Volumes and Disks

dosFs provides APIs for accessing volume configuration information and for synchronizing volumes.

For information about ioctl( ) support functions, see I/O Control Functions Supported by dosFsLib on page 68.

Accessing Volume Configuration Information

The dosFsShow( ) function can be used to display volume configuration information from the shell. The dosFsVolDescGet( ) function can be used programmatically to obtain or verify a pointer to the **DOS_VOLUME_DESC** structure. For more information, see the API references for these functions.

Getting the Volume Size

You can get the size of the volume several ways as described below.

Use the deviceFsInfoShow( ) API. For more information, see File System Utility on page 82.

```
-> deviceFsInfoShow "/bd0a"
Device Name [/bd0a]
File System     [MS-DOS Compatible File System]
Block Size      [2048]
Total Blocks    [39212]
Free Blocks     [37748]
Available Block [37748]
```


WNDRVR

```
Total Files     [-1]
Free Files      [-1]
File System ID  [0x199b0000,0x0]
value = 0 = 0x0
```

The volume size is Block Size (2048) * Total Blocks (39212), therefore, the total volume size is 80306176 bytes.

2048 * 39212 = 80306176

Use the xpPartShow( ) APIs. For more information, see File System Utility on page 82.

```
-> xpPartShow "/bd0a"
Request name [/bd0a], Real Device name [/bd0a], XBD Base name [/bd0]
Device geometry:
Total Blocks [15728640] Bytes per sector [512]
CHS: Cylinder [249660] Head [1] Sector [63]

Mount state:
Idx.  IO Path     XBD Name        Start Sector       End Sector     Sector Count
--------------------------------------------------------------------------------
Total Not Valid   /bd0:0                     0         15728639         15728640
1     /bd0a       /bd0a                    128           157311           157184
2     /bd0b       /bd0b                 157312         15728511         15571200

MBR Info:
Idx. Boot  Type  Start Sector   End Sector  Sector Count  CHS(Cyl. Head Sec.)
--------------------------------------------------------------------------------
   1   80    06           128       157311        157184    0,  2, 3    9,193, 1
   2   00    0C        157312     15728511      15571200    9,193, 2  975, 58,58
Empty entry
Empty entry
value = 0 = 0x0
```

The **/bd0a** Sector Count (157184) * Sector Size (512), therefore, the total volume size is 80478208 bytes.

157184 * 512 = 80478208

> **Note:** The total volume size values are different in the above examples because the file system may reserve areas to store meta-data or other data.

Use the VolumeSizeShow( ) routine below. The routine is based on statfs( ) to show the volume size where you can use either statfs( ) or statfs64( ) (for larger volumes).

```
#include "sys/stat.h"

     STATUS VolumeSizeShow
     (
     char *devName      /* device name */
     )
     {
     STATUS status = -1;
     struct statfs64 fs;

     if (devName == NULL)
     return(-1);
```


WNDRVR

```
    /* get statistic of device */
    if ((status = statfs64 ((char*)devName, &fs)) == 0)
    {
    printf ("%s: Total File System Size: %fMB, Free space: %fMB.\n",
    devName,
    ((float)(fs.f_bsize * fs.f_blocks) /(1024.0*1024.0)),
    ((float)(fs.f_bsize * fs.f_bavail) /(1024.0*1024.0)));
    }

    return (status);
    }
```

Synchronizing Volumes

When a disk is synchronized, all modified buffered data is physically written to the disk, so that the disk is up to date. This includes data written to files, updated directory information, and the FAT. To avoid loss of data, a disk should be synchronized before it is removed. For more information, see the API reference for close( ).

Changing, Unmounting and Synchronizing Disks

The disk cache handles the buffering of disk data in RAM and synchronization of these buffers. See reference manual on dosFsCacheLib for more details.

Detection of removable disk replacement is done by the File System Monitor subsystem. For more details about the File System Monitor, refer to File System Monitor on page 80. If a disk is physically removed, the File System Monitor subsystem will delete the filesystem entry from coreIO and free all its allocated resources, including disk cache buffers.

If a new dosFs formatted disk is inserted, the File System Monitor will detect it and will automatically create a dosFs filesystem with the name previously registered through a call to fsmNameInstall( ) (or a default name will be assigned). Also global parameters **DOSFS_DEFAULT_MAX_FILES**, **DOSFS_DEFAULT_CREATE_OPTIONS** and if disk cache is supported, **DOSFS_DEFAULT_CACHE_SIZE** will be enabled.

Parent topic: dosFs on page 40

4.11. Directory Separators

dosfs supports both back-slash and forward-slash directory separators.

Filenames and directory names on dosFs devices are often separated by backslashes (\). These can be used interchangeably with forward slashes (/).

Parent topic: dosFs on page 40

4.12. Creating Subdirectories

Subdirectories can be created using ioctl( ), open( ), or mkdir( ).

About This Task

For FAT32, subdirectories can be created in any directory at any time. For FAT12 and FAT16, subdirectories can be created in any directory at any time, except in the root directory once it reaches its maximum entry count.

WNDRVR

Procedure

Create a subdirectory in the following ways:

- Use ioctl( ) with the **FIOMKDIR** function. The name of the directory to be created is passed as a parameter to ioctl( ).
- Use open( ).

  To create a directory, the O_CREAT option must be set in the flags parameter and the FSTAT_DIR option must be set in the mode parameter. The open( ) call returns a file descriptor that describes the new directory. Use this file descriptor for reading only, and close it when it is no longer needed.
- Use mkdir( ) from **usrFsLib**.

When creating a directory using any of the above methods, the new directory name must be specified. This name can be either a full pathname or a pathname relative to the current working directory.

Parent topic: dosFs on page 40

4.13. Removing Subdirectories

Subdiriectories can be removed with ioctl( ), remove( ), or rmdir( ).

About This Task

A directory that is to be deleted must be empty (except for the "." and ".." entries). The root directory can never be deleted.

Procedure

Create a subdirectory in one of the following ways:

- Use ioctl( ) with the **FIORMDIR** function, specifying the name of the directory. The file descriptor used can refer to any file or directory on the volume, or to the entire volume itself.
- Use the remove( ) function, specifying the name of the directory.
- Use rmdir( ) from **usrFsLib**.

Parent topic: dosFs on page 40

4.14. Reading Directory Entries

Directory entries can be read with standard **dirLib** APIs.

About This Task

Procedure

Programmatically search directories on dosFs volumes using one of the following functions:

- opendir( )
- readdir( )
- rewinddir( )
- closedir( )

WNDRVR

To obtain more detailed information about a specific file, use the fstat( ) or stat( ) function. Along with standard file information, the structure used by these functions also returns the file-attribute byte from a directory entry. For more information, see the API reference for **dirLib**.

Parent topic: dosFs on page 40

4.15. Working with Files

Files on a dosFs file system device are created, deleted, written, and read using the standard VxWorks I/O functions. File attributes are changed with ioctl( ). You can also retrieve the volume size.

CAUTION: **If you are going to work with a large number of simultaneously open files on an ATA drive (for example, over 1,000) do not put them all in a single directory. Doing so has a negative impact on I/O performance. This issue can be avoided by storing the files in multiple directories (for example, 500 files per directory).**

File I/O Functions

The standard VxWorks I/O functions: creat( ), remove( ), write( ), and read( ). For more information, see Overview of VxWorks File Systems and I/O Devices on page 1 and the **ioLib** API references.

File Attributes

The file-attribute byte in a dosFs directory entry consists of a set of flag bits, each indicating a particular file characteristic. The characteristics described by the file-attribute byte are shown below.

DOS_ATTR_RDONLY

Hex Value: 0x01

Read-only file. If this flag is set, files accessed with open( ) cannot be written to. If the O_WRONLY or O_RDWR flags are set, open( ) returns **ERROR**, setting errno to **S_dosFsLib_READ_ONLY**.

DOS_ATTR_HIDDEN

Hex Value: 0x02

Hidden file. This flag is ignored by **dosFsLib** and produces no special handling. For example, entries with this flag are reported when searching directories.

DOS_ATTR_SYSTEM

Hex Value: 0x04

System file.This flag is ignored by **dosFsLib** and produces no special handling. For example, entries with this flag are reported when searching directories.

DOS_ATTR_VOL_LABEL

Hex Value: 0x08

Volume label. This flag indicates that a directory entry contains the dosFs volume label for the disk. A label is not required. If used, there can be only one volume label entry per volume, in the root directory. The volume label entry is not reported when reading the contents of a directory (using readdir( )). It can only be determined using the ioctl( ) function **FIOLABELGET**. The volume label can be set (or reset) to any string of 11 or fewer characters, using the ioctl( ) function **FIOLABELSET**. Any file descriptor open to the volume can be used during these ioctl( ) calls.

WNDRVR

DOS_ATTR_DIRECTORY

Hex Value: 0x10

Sub-directory. This flag indicates that this entry is a subdirectory, and not a regular file.

DOS_ATTR_ARCHIVE

Hex Value: 0x20

File is subject to archiving. This flag is set when a file is created or modified. This flag is intended for use by other programs that search a volume for modified files and selectively archive them. Such a program must clear the archive flag, since VxWorks does not.

All the flags in the attribute byte, except the directory and volume label flags, can be set or cleared using the ioctl( ) function **FIOATTRIBSET**. This function is called after the opening of the specific file with the attributes to be changed. The attribute-byte value specified in the **FIOATTRIBSET** call is copied directly; to preserve existing flag settings, determine the current attributes using stat( ) or fstat( ), then change them using bitwise AND and OR operators.

Filenames

dosFs filenames are limited to 255 characters, not including the trailing NUL. The total path length cannot exceed 1024 characters, including the trailing NUL. While dosFs supports path names of 1024 characters, using a path name of more than 259 characters makes it incompatible with Windows (that is, the file cannot be used on a Windows host).

VxWorks dosFs has two options for handling longer filenames (VFAT and VXLONGNAMES). Both are backwards compatible to the original DOS4.0 short file name format. Known as the 8.3 format, but commonly referred to as the short file name format, the format limits filenames to 8-characters and a 3-character file extension. Both long name directory formats support displaying the 8.3 style filenames.

The pointer to where the file begins on physical media is stored with a file name. The width of this pointer limits the supported maximum disk size.

Name	Notes	Maximum File Size	Maximum Disk Size
VFAT	Currenlty widely compatible format.	255 Bytes	4GB
VXLONGNAMES	Deprecated and proprietary to VxWorks. Formating of new disks is not supported. Updating of existing disks is supported.	40 bytes	1TB

Windows style VFAT standard long file names that are stored mixed cased on disk are case insensitive when searched and matched (such as during open( ) call). The VFAT long file name is stored in a variable number of consecutive directory entries.The file size is restricted to 4 GB (32 bit value).

To provide additional flexibility, this implementation of the DOS file system provides proprietary long file name format (VXLONGNAMES). This implementation uses a simpler directory structure: the directory entry is of fixed size. When using this option, file names can consist of any sequence of up to 40 ASCII characters. No case conversion is performed, and file name match is case-sensitive. With this directory format the file maximum size is expanded to 1 Terabyte (40 bit value).

WNDRVR

Note: This option only supports read-only access to files in the VxWorks 6.2 version.

Because special directory entries are used on the disk, disks which use the extended names are not compatible with other implementation of the MS-DOS systems, and cannot be read on MS-DOS or Windows machines.

To enable the extended file names, set the **DOS_OPT_VXLONGNAMES** flag when calling the function dosFsVolFormat( ).

The kernel configuration includes a Boolean parameter **DOSFS_COMPAT_NT** that the kernel disables by default. This Boolean parameter enables an extension to the 8.3 short filenames used by Windows NT and XT, but not compatible with Windows 95, 98, 98 SE, and ME. The **DOSFS_COMPAT_NT** directory entry is a flag used to indicate compatibility with Windows. It is only used for FAT32. If the flag is set to TRUE, VxWorks will interpret the field (offset 0x0C of the directory entry) the same way that Windows does.

This field is always 0x00 for the VFAT LFN (long file name), for special usage of bits 4 and 3 in short file names. Windows NT and later versions of Windows use bits 3 and 4 of this byte to encode case information.

A VFAT entry is not created in Windows NT or in later versions of Windows if a filename:

- contains only lowercase letters or is a combination of a lowercase basename with an uppercase extension (or vice versa)
- has no special characters, and
- fits within the 8.3 limits

For such filenames, two bits in byte 0x0C of the **DOSFS_COMPAT_NT** directory entry are used to indicate that the filename should be considered as entirely or partially lowercase. Specifically, bit 3 indicates a lowercase basename, and bit 4 indicates a lowercase extension. This allows for combinations such as **example.TXT** or **HELLO.txt**, but not **Mixed.txt**. Few other operating systems support this approach. This creates a backwards-compatibility problem with older Windows versions (Windows 95/98/98 SE/ME) that see all uppercase filenames if this extension has been used, and therefore can change the name of a file when it is transported between operating systems, such as on a USB flash drive.

File Date and Time

Directory entries on dosFs volumes contain creation, last modification time and date, and the last access date for each file or subdirectory. The fields **directory last modification time** and **date fields** are set only when a new entry is created, but not when any directory entries are deleted. The filed **last access date** indicates the date of the last read or write. The **last access date** field is an optional field, per Microsoft. By default, file open-read-close operations do not update the **last access date** field. This default avoids media writes (writing out the date field) during read only operations. To enable the updating of the optional last access date field for open-read-close operations, you must call dosFsLastAccessDateEnable(), passing the function the volumes **DOS_VOLUME_DESC_ID** and TRUE.

The dosFs file system uses the ANSI time( ) function, that returns system clock value to obtain date and time. Wind River recommends that you set the target system's time during system initialization to the time from a network server or from an embedded Calendar / Clock hardware component. Setting up the time in this manner allows all files on the file system to have the correct date and time.

The file system consistency checker sets system clock to value following the latest date-time field stored on the disk. If it discovers, that function time( ) returns a date earlier then Jan 1, 1998, it means that the target system does not have a source of valid date and time to synchronize with.

See also the reference manual entry for ansiTime.

Files Synchronization

Files can be opened with the **O_SYNC** flag, indicating that each write must be immediately written to the backing media. This includes synchronizing the FAT and the directory entries.

WNDRVR

Using Unicode Characters

When Unicode characters are in use, they are encoded in UTF-8 through the open( ) and readdir( ) interface, and in Windows-compatible UTF-16 format on disk. The translation between external (UTF-8) and internal (UTF-16) encodings is automatic, avoiding all the byte-order problems associated with UTF-16 encodings.

Existing VxWorks file systems that use "high bit" characters (such as ISO Latin 1 character sets) are not compatible with Unicode encodings. For this reason, Unicode file names must be enabled explicitly using the **DOS_FILENAMES_UNICODE** flag.

Unicode is only supported on VFAT (variable-length file name) volumes.

Parent topic: dosFs on page 40

4.16. Code Example for Setting DosFs File Attributes

This example makes a dosFs file read-only, and leaves other attributes intact..

```
STATUS changeAttributes
    (
    void
    )
    {
    int fd;
    struct stat statStruct;

    /* open file */

    if ((fd = open ("file", O_RDONLY, 0)) == ERROR)
       return (ERROR);

    /* get directory entry data */

    if (fstat (fd, &statStruct) == ERROR)
       return (ERROR);

    /* set read-only flag on file */

    if (ioctl (fd, FIOATTRIBSET, (statStruct.st_attrib | DOS_ATTR_RDONLY)) == ERROR)
      return (ERROR);

    /* close file */
    close (fd);
    return (OK);
    }
```

Note: You can also use the attrib( ) function to change file attributes. For more information, see the entry in **usrFsLib**.

Parent topic: dosFs on page 40

4.17. Disk Space Allocation Options

The dosFs file system allocates disk space using one of three methods. Two are automatic, based on the size of the write operation. The third must be implemented with your code.

WNDRVR

single cluster allocation

Single cluster allocation uses a single cluster, which is the minimum allocation unit. This method is automatically used when the write operation is smaller than the size of a single cluster.

cluster group allocation (nearly contiguous)

Cluster group allocation uses adjacent (contiguous) groups of clusters, called extents. Cluster group allocation is nearly contiguous allocation and is the default method used when files are written in units larger than the size of a disk's cluster.

absolutely contiguous allocation

Absolutely contiguous allocation uses only absolutely contiguous clusters. Because this type of allocation is dependent upon the existence of such space, it is specified under only two conditions: immediately after a new file is created and when reading from a file assumed to have been allocated to a contiguous space. Using this method risks disk fragmentation. Use ioctl( ) for this method.

For any allocation method, you can deallocate unused reserved bytes by using the POSIX-compliant function ftruncate( ) or the ioctl( ) function **FIOTRUNC**.

Parent topic: dosFs on page 40

4.18. Choosing an Allocation Method

Under most circumstances, cluster group allocation is preferred to absolutely contiguous file access.

Because it is nearly contiguous file access, it achieves a nearly optimal access speed. Cluster group allocation also significantly minimizes the risk of fragmentation posed by absolutely contiguous allocation.

Absolutely contiguous allocation attains raw disk throughput levels, however this speed is only slightly faster than nearly contiguous file access. Moreover, fragmentation is likely to occur over time. This is because after a disk has been in use for some period of time, it becomes impossible to allocate contiguous space. Thus, there is no guarantee that new data, appended to a file created or opened with absolutely continuous allocation, will be contiguous to the initially written data segment.

It is recommended that for a performance-sensitive operation, the application regulate disk space utilization, limiting it to 90% of the total disk space. Fragmentation is unavoidable when filling in the last free space on a disk, which has a serious impact on performance.

Parent topic: dosFs on page 40

4.19. Cluster Group Allocation

Cluster group allocation occurs when the cluster group size is considered sufficiently large so that the seek time is negligible compared to the read/write time.

The dosFs file system defines the size of a cluster group based on the media's physical characteristics. That size is fixed for each particular media. Since seek operations are an overhead that reduces performance, it is desirable to arrange files so that sequential portions of a file are located in physically contiguous disk clusters. Cluster group allocation occurs when the cluster group size is considered sufficiently large so that the seek time is negligible compared to the read/write time. This technique is sometimes referred to as nearly contiguous file access because seek time between consecutive cluster groups is significantly reduced.

Because all large files on a volume are expected to have been written as a group of extents, removing them frees a number of extents to be used for new files subsequently created. Therefore, as long as free space is available for subsequent file storage, there are always extents available for use. Thus, cluster group allocation effectively prevents fragmentation (where a file is allocated in small units spread across distant locations on the disk). Access to fragmented files can be extremely slow, depending upon the degree of fragmentation.

WNDRVR

Parent topic: dosFs on page 40

4.20. Using Absolutely Contiguous Allocation

Absolutely contiguous allocation is intended to allocate contiguous space to a specified file (or directory) and, by so doing, optimize access to that file.

You can specify absolutely contiguous allocation using the ioctl( ) functions when creating a file, or when opening a file previously created in this manner. The VxWorks dosFs file system provides efficient files storage. dosFs allocates space in groups of clusters (also termed extents). These clusters compose a file of relatively large contiguous units. This nearly contiguous allocation technique is designed to effectively eliminate the effects of disk space fragmentation, keeping throughput very close to the maximum of which the hardware is capable.

dosFs provides a mechanism to allocate truly contiguous files, meaning files that are made up of a consecutive series of disk sectors. This support includes both the ability to allocate contiguous space to a file and optimize access to such a file when it is used. Typically, this will slightly improve performance when compared to Nearly Contiguous allocation, at the price of disk space fragmentation.

To allocate a contiguous area to a file, create the file in the normal fashion, using open( ) or creat( ). The file descriptor returned during the creation of the file is then used to make an ioctl( ) call, specifying the **FIOCONTIG** or **FIOCONTIG64** function. The last parameter to the **FIOCONTIG** function is the size of the requested contiguous area in bytes. If using the **FIOCONTIG64**, the last parameter is a pointer to a 64-bit integer variable that contains the required file size. It is also possible to request to obtain the largest contiguous free area on the disk (must be less than **CONTIG_MAX**). This case uses the size value **CONTIG_MAX** instead of an actual size. These ioctl( ) codes are not supported for directories. The volume is searched for a contiguous area of free space, that is assigned to the file. If a segment of contiguous free space large enough for the request was not found, **ERROR** is returned, with errno set to **ENOSPC**.

The file remains empty when contiguous space is allocated to a file and while the newly allocated space has not been initialized. The data must then be written to the file, and when all data has been written, the file is closed. When a file is closed, its space is truncated to reflect the amount of data actually written to the file. This file can be opened again and used for further I/O operations read( ) or write( ), but it cannot be guaranteed that appended data will be contiguous to the initially written data segment.

For more information, see I/O Control Functions Supported by dosFsLib on page 68.

Allocating Contiguous Space for a File on page 61
Allocating Space for Subdirectories on page 62
Opening and Using a Contiguous File on page 63 Fragmented files require following cluster chains in the FAT. However, if a file is recognized as contiguous, the system can use an enhanced method that improves performance.

Parent topic: dosFs on page 40

Allocating Contiguous Space for a File

About This Task

To allocate a contiguous area to a newly created file, follow these steps:

Procedure

1. Create the file in the normal fashion using open( ) or creat( ).
2. Call ioctl( ).

   Use the file descriptor returned from open( ) or creat( ) as the file descriptor argument. Specify **FIOCONTIG** as the function code argument and the size of the requested contiguous area, in bytes, as the third argument.

WNDRVR

To request the largest available contiguous space, use **CONTIG_MAX** for the size of the contiguous area.

The following example creates a file and allocates the largest contiguous area on the disk to it:

```
fd = creat ("file", O_RDWR, 0);             /* open file             */
status = ioctl (fd, FIOCONTIG, CONTIG_MAX); /* get contiguous area   */
if (status != OK)
   ...                                      /* do error handling     */
close (fd);                                 /* close file            */
```

The following example creates a file and allocates 85 MB of contiguous space:

```
fd = creat ("file", O_RDWR, 0);               /* open file */
status = ioctl (fd, FIOCONTIG, 85*0x100000);  /* get contiguous area   */
if (status != OK)
     ...                                      /* do error handling     */
close (fd);                                   /* close file            */
```

The FAT is searched for a suitable section of the disk. If found, this space is assigned to the new file. The file can then be closed, or it can be used for further I/O operations. The file descriptor used for calling ioctl( ) should be the only descriptor open to the file. Always perform the ioctl **FIOCONTIG** operation before writing any data to the file.

Directories cannot be allocated a contiguous disk area. To determine the actual amount of contiguous space obtained when **CONTIG_MAX** is specified as the size, use fstat( ) to examine the number of blocks and block size for the file.

When any file is opened, it can be checked for contiguity. Use the extended flag **DOS_O_CONTIG_CHK** when calling open( ) to access an existing file that has been allocated contiguous space. If a file is detected as contiguous, all subsequent operations on the file will not require access to the file allocation table, thus eliminating any disk seek operations. However, if this option is used, open( ) will take an amount of time which is linearly proportional to the file size.

Parent topic: Using Absolutely Contiguous Allocation on page 61

Allocating Space for Subdirectories

About This Task

Subdirectories can be allocated a contiguous disk area in the same manner as discussed in Allocating Contiguous Space for a File on page 61:

Procedure

Do one of the following.

- If the directory is created using the ioctl( ) function **FIOMKDIR**, it must be subsequently opened to obtain a file descriptor to it.
- If the directory is created using options to open( ), the returned file descriptor from that call can be used.

A directory must be empty (except for the "." and ".." entries) when it has contiguous space allocated to it.

WNDRVR

Parent topic: Using Absolutely Contiguous Allocation on page 61

Opening and Using a Contiguous File

Fragmented files require following cluster chains in the FAT. However, if a file is recognized as contiguous, the system can use an enhanced method that improves performance.

This applies to all contiguous files, whether or not they were explicitly created using **FIOCONTIG**. Whenever a file is opened, it is checked for contiguity. If it is found to be contiguous, the file system registers the necessary information about that file to avoid the need for subsequent access to the FAT table. This enhances performance when working with the file by eliminating seek operations.

When you are opening a contiguous file, you can explicitly indicate that the file is contiguous by specifying the **DOS_O_CONTIG_CHK** flag with open( ). This prompts the file system to retrieve the section of contiguous space, allocated for this file, from the FAT table.

To find the maximum contiguous area on a device, you can use the ioctl( ) function **FIONCONTIG**. This information can also be displayed by dosFsConfigShow( ).

Finding the Maximum Contiguous Area on a DosFs Device

In this example, the size (in bytes) of the largest contiguous area is copied to the integer pointed to by the third parameter to ioctl( ) (count).

```
STATUS contigTest
    (
    void /* no argument */
    )
    {
    int count; /* size of maximum contiguous area in bytes */
    int fd; /* file descriptor */

    /* open device in raw mode */

    if ((fd = open ("/DEV1/", O_RDONLY, 0)) == ERROR)
       return (ERROR);

    /* find max contiguous area */

    ioctl (fd, FIONCONTIG, &count);

    /* close device and display size of largest contiguous area */

    close (fd);
    printf ("largest contiguous area = %d\n", count);
    return (OK);
}
```

Parent topic: Using Absolutely Contiguous Allocation on page 61

4.21. Crash Recovery and Volume Consistency

dosFs provides an optional consistency-checking mechanism to detect and recover from data structure inconsistencies.

The DOS file system is inherently susceptible to data structure inconsistencies that result from interruptions during certain types of disk updates. These types of interruptions include power failures, system crashes (for fixed disks), and the manual removal of a disk.

WNDRVR

Inconsistencies occur because the file system data for a single file is stored in three separate regions of the disk. The data stored in these regions are:

- The file chain in the File Allocation Table (FAT), located in a region near the beginning of the disk.
- The directory entry, located in a region that could be anywhere on the disk.
- File clusters containing file data, that could be located anywhere on the disk.

Since all three regions are not always updated before an interruption, dosFs includes an optional integrated consistency-checking mechanism to detect and recover from inconsistencies. For example, if a disk is removed when a file is being deleted, a consistency check completes the file deletion operation. Or, if a file is being created when an interruption occurs, then the file is un-created. In other words, the consistency checker either rolls forward or rolls back the operation that experienced the inconsistency, making whichever correction is possible.

Parent topic: dosFs on page 40

4.22. dosFsLib API Overview

Describes expanded use cases for dosFsLib and descriptions for how to use different calls and functions.

Using The dosFsDevCreate API

Normally this API is called automatiaclly by the operting system when it detects the presence of a FAT formated disk, either at boot or when removable media is inserted. The parmeters used are determined by your kernels configuration.The dosFsDevCreate() function associates a device with the dosFsLib functions. the function expects four parameters:

- A pointer to a name string. The pointer becomes part of the pathname for I/O operations that operate on the device. This name appears in the I/O system device table, which can be displayed using the iosDevShow( ) function.
- **device_t** - a XBD for the device on which to create the file system. Examples include: a partition XBD, an XBD block wrapper, or an ATA device XBD.
- A maximum number of files can be simultaneously opened on a particular device.
- Flags for volume checking, metadata integrity, and file name interpretation. Because the volume, integrity, and check utility can be automatically invoked every time a device is mounted, this parameter indicates whether the consistency check needs to be performed automatically on a given device, and on what level of verbosity is required. You can invoke the consistency check at a later time by calling chkdsk( ). See description for **FIOCHKDSK** ioctl command for more information.

For example:

```
dosFsDevCreate
    (
    "/sd0",     /* name to be used for volume   */
    device,     /* underlying XBD device */
    10,         /* max no. of simultaneously open files */
    DOS_CHK_REPAIR | DOS_CHK_VERB_1
                /* check volume during mounting and repair */
                /* errors, and display volume statistics */
    )
```

Once you call dosFsDevCreate(), you can access the device using ioLib generic I/O routines: open( )), read( ), write( ), close( ), ioctl( ), remove( ). In addition, you can use the user-level utility functions to access the device at a higher level (See **usrFsLib** reference page for more details).

WNDRVR

DEVICE AND PATH NAMES

On true MS-DOS machines, disk device names are typically of the form "A:", a single letter designator followed by a colon. Such names can be used with the VxWorks dosFs file system. However, Wind River recommends using longer, more mnemonic device names, such as "DOS1:", or "/floppy0". The name is specified during the dosFsDevCreate() call. Typically the File System Monitor module automatically calls this function. Thus you can call fsmNameInstall( ) to specify the desired name for the device. Please refer to the fsMonitor documentation for further details.

The pathnames used to specify dosFs files and directories can use either forward slashes ("/") or backslashes ("\") as separators. These can be freely mixed. The choice of forward slashes or backslashes does not effect the directory data written to the disk.

Wind River recommends the use of forward slashes ("/") at all times.

> Note: Forward slashes are not allowed in VxWorks dosFs filenames, although they are normally legal for pure MS-DOS implementations.

The leading slash of a dosFs pathname following the device name is optional. For example, both "DOS1:newfile.new" and "DOS1:/newfile.new" refer to the same file.

USING EXTENDED DIRECTORY STRUCTURE

This library supports DOS4.0 standard file names that fit the restrictions of eight uppercase characters optionally followed by a three-character extension. In addition to Windows style VFAT standard long file names that are stored mixed cased on disk, but are case insensitive when searched and matched (such as during open( ) call). The VFAT long file name is stored in a variable number of consecutive directory entries. Both standards restrict file size to 4 GB (32 bit value).

To provide additional flexibility, this implementation of the DOS file system provides proprietary long file name format (VXLONGNAMES). This implementation uses a simpler directory structure: the directory entry is of fixed size. When using this option, file names can consist of any sequence of up to 40 ASCII characters. No case conversion is performed, and file name match is case-sensitive. With this directory format the file maximum size is expanded to 1 Terabyte (40 bit value).

> Note: This option only supports read-only access to files in the VxWorks 6.2 version.

> Note: Because special directory entries are used on the disk, disks which use the extended names are not compatible with other implementation of the MS-DOS systems, and cannot be read on MS-DOS or Windows machines.

To enable the extended file names, set the **DOS_OPT_VXLONGNAMES** flag when calling dosFsVolFormat().

USING UNICODE CHARACTERS

When Unicode characters are in use, they are encoded in UTF-8 through the open( ) and readdir( ) interface, and in Windows-compatible UTF-16 format on-disk. The translation between external (UTF-8) and internal (UTF-16) encodings is automatic, avoiding all the byte-order problems associated with UTF-16 encodings.

Existing VxWorks file systems that use "high bit" characters (such as ISO Latin 1 character sets) are not compatible with Unicode encodings. For this reason, Unicode file names must currently be enabled explicitly using the **DOS_FILENAMES_UNICODE** flag.

Unicode is only supported on VFAT (variable-length file name) volumes.

READING DIRECTORY ENTRIES

You can search directories on VxWorks dosFs volumes using the opendir( ), readdir( ), rewinddir( ), and closedir( ) routines. These calls allow the names of files and subdirectories to be determined.

WNDRVR

To obtain more detailed information about a specific file, use the fstat( ) or stat( ) function. Along with standard file information, the function's structure also returns the file attribute byte from a dosFs directory entry.

For more information, see the manual entry for **dirLib**

SYNCHRONOUS FILES

Files can be opened with the **O_SYNC** flag, indicating that each write must be immediately written to the backing media. This includes synchronizing the FAT and the directory entries.

FILE DATE AND TIME

Directory entries on dosFs volumes contain creation, last modification time and date, and the last access date for each file or subdirectory. The fields "directory last modification time" and "date fields" are set only when a new entry is created, but not when any directory entries are deleted. The "last access date" field indicates the date of the last read or write. The "last access date" field is an optional field, per Microsoft. By default, file open-read-close operations do not update the "last access date" field. This default avoids media writes (writing out the date field) during read only operations. To enable the updating of the optional last access date field for open-read-close operations, you must call dosFsLastAccessDateEnable(), passing the function the volumes **DOS_VOLUME_DESC_ID** and **TRUE**.

The dosFs file system uses the ANSI time( ) function, that returns system clock value to obtain date and time. Wind River recommends that the you set the target system's time during system initialization to the time from a network server or from an embedded Calendar / Clock hardware component. Setting up the time in this manner allows all files on the file system to have the correct date and time.

The file system consistency checker sets system clock to value following the latest date-time field stored on the disk. If it discovers, that function time( ) returns a date earlier then Jan 1, 1998, it means that the target system does not have a source of valid date and time to synchronize with.

See also the reference manual entry for ansiTime.

CONTIGUOUS FILE SUPPORT

The VxWorks dosFs file system provides efficient files storage. dosFs allocates space in groups of clusters (also termed extents). These clusters compose a file of relatively large contiguous units. This nearly contiguous allocation technique is designed to effectively eliminate the effects of disk space fragmentation, keeping throughput very close to the maximum of which the hardware is capable.

dosFs provides a mechanism to allocate truly contiguous files, meaning files that are made up of a consecutive series of disk sectors. This support includes both the ability to allocate contiguous space to a file and optimized access to such a file when it is used. Typically, this will slightly improve performance when compared to Nearly Contiguous allocation, at the price of disk space fragmentation.

To allocate a contiguous area to a file, create the file in the normal fashion, using open( ) or creat( ). The file descriptor returned during the creation of the file is then used to make an ioctl( ) call, specifying the FIOCONTIG or FIOCONTIG64 function. The last parameter to the FIOCONTIG function is the size of the requested contiguous area in bytes. If using the FIOCONTIG64, the last parameter is a pointer to a 64-bit integer variable that contains the required file size. It is also possible to request to obtain the largest contiguous free area on the disk (must be less than **CONTIG_MAX**). This case uses the size value **CONTIG_MAX** instead of an actual size. These ioctl( ) codes are not supported for directories. The volume is searched for a contiguous area of free space, that is assigned to the file. If a segment of contiguous free space large enough for the request was not found, **ERROR** is returned, with errno set to **ENOSPC**.

The file remains empty when contiguous space is allocated to a file and while the newly allocated space has not been initialized. The data must then be written to the file, and when all data has been written, the file is closed. When a file is closed, its space is truncated to reflect the amount of data actually written to the file. This file can be opened again and used for further I/O operations read( ) or write( ), but it cannot be guaranteed that appended data will be contiguous to the initially written data segment.

For example, the following creates a file and allocates 85 MB of contiguous space:

```
fd = creat ("file", O_RDWR, 0);                /* open file */
status = ioctl (fd, FIOCONTIG, 85*0x100000);  /* get contiguous area   */
if (status != OK)
     ...                                       /* do error handling     */
close (fd);                                    /* close file            */
```

In contrast, the following example creates a file and allocates the largest contiguous area on the disk to it:

```
fd = creat ("file", O_RDWR, 0);              /* open file             */
status = ioctl (fd, FIOCONTIG, CONTIG_MAX); /* get contiguous area   */
if (status != OK)
   ...                                       /* do error handling     */
close (fd);                                  /* close file            */
```

The FIOCONTIG operation must take place right after the file has been created, before any data is written to the file. Directories cannot be allocated a contiguous disk area.

To determine the actual amount of contiguous space obtained when **CONTIG_MAX** is specified as the size, use fstat( ) to examine the number of blocks and block size for the file.

When any file is opened, it can be checked for contiguity. Use the extended flag **DOS_O_CONTIG_CHK** when calling open( ) to access an existing file that has been allocated contiguous space. If a file is detected as contiguous, all subsequent operations on the file will not require access to the File Allocation Table, thus eliminating any disk Seek operations. However, the negative is that if this option is used, open( ) will take an amount of time which is linearly proportional of the file size.

CHANGING, UNMOUNTING, AND SYNCHRONIZING DISKS

Buffering of disk data in RAM, and synchronization of these buffers with the disk are handled by the disk cache. See reference manual on dosFsCacheLib for more details. Detection of removable disk replacement is done by the File System Monitor subsystem.

If a disk is physically removed, the File System Monitor subsystem will delete the filesystem entry from coreIO and free all its allocated resources, including disk cache buffers.

If a new dosFs formatted disk is inserted, the File System Monitor will detect it and will automatically create a dosFs filesystem with the name previously registered through a call to fsmNameInstall( ) (or a default name will be assigned), and with the global parameters **DOSFS_DEFAULT_MAX_FILES**, **DOSFS_DEFAULT_CREATE_OPTIONS**, and if disk cache is supported (see **dosFsCacheLib** for details), with a **DOSFS_DEFAULT_CACHE_SIZE** cache.

VXWORKS DOSFS LONG FILENAMES

VxWorks dosFs has two options for handling longer filenames. Both are backward compatible to the original DOS4.0 short file name format. Known as the 8.3 format, but commonly referred to as the short file name format. The format limits filenames to 8-characters and a 3-character file extension. Both long name directory formats support displaying the 8.3 style filenames.

The pointer to where the file begins on physical media is stored with a file name. The width of this pointer limits the supported maximum disk size.

WNDRVR

Name	Notes	Maximum File Size	Maximum Disk Size
VFAT	Currenlty widely compatible format.	255 Bytes	4GB
VXLONGNAMES	Deprecated and proprietary to VxWorks. Formating of new disks is not supported. Updating of existing disks is supported.	40 bytes	1TB

OPTIONAL EXTENSION TO SHORT FILENAMES

The kernel configuration includes a Boolean parameter **DOSFS_COMPAT_NT** that the kernel disables by default. This Boolean parameter enables an extension to the 8.3 short filenames used by Windows NT and XT, but not compatible with Windows 95, 98, 98 SE, and ME. When enabled, two bits in the directory structure in the field following the file attributes have special meaning. Theses bits indicate the file name and extension are lower case instead of the normal uppercase representation. Thus, **DOSFS_COMPAT_NT** allows for combinations such as "example.TXT" or "HELLO.txt". Yet, "Mixed.txt" is not possible. These bits are unset if the longer, mixed case VFAT filenames are used.

If you cannot include the DosFs cache component due to some project/hardware requirements, increasing **DOSFS_DEFAULT_VOL_BIO_BUFFER_SIZE** can increase performance. Also use dosFsVolFormat( ) with the **optional _func_FormatPrompt** parameter to create a disk with a larger *secPerClust* (sectors per cluster) to avoid write buffer fragmentation.

Parent topic: dosFs on page 40

4.23. I/O Control Functions Supported by dosFsLib

The dosFs file system supports a set of ioctl( ) functions.

The functions listed below are defined in the header file **ioLib.h** along with their associated constants. For more information, see the API references for **dosFsLib** and for ioctl( ) in **ioLib**.

Function	Description
FIOATTRIBSET	Sets the file-attribute byte in the dosFs directory entry.
FIOCONTIG	Allocates contiguous disk space for a file or directory
FIOFLUSH	Flushes the file output buffer.
FIOFSTATGET	Gets file status information (directory entry data).
FIOGETNAME	Gets the filename of the fd.
FIOLABELGET	Gets the volume label.
FIOLABELSET	Sets the volume label.
FIOMKDIR	Creates a new directory.

WNDRVR

Function	Description
FIOMOVE	Moves a file (does not rename the file).
FIONCONTIG	Gets the size of the maximum contiguous area on a device.
FIONFREE	Gets the number of free bytes on the volume.
FIONREAD	Gets the number of unread bytes in a file.
FIOREADDIR	Reads the next directory entry.
FIORENAME	Renames a file or directory.
FIORMDIR	Removes a directory.
FIOSEEK	Sets the current byte offset in a file.
FIOSYNC	Same as **FIOFLUSH**.
FIOTRUNC	Truncates a file to a specified length.
FIOUNMOUNT	Un-mounts a disk volume.
FIOWHERE	Returns the current byte position in a file.
FIOCONTIG64	Allocates contiguous disk space using a 64-bit size.
FIONCONTIG64	Gets the maximum contiguous disk space into a 64-bit integer.
FIONFREE64	Gets the number of free bytes into a 64-bit integer.
FIONREAD64	Gets the number of unread bytes in a file into a 64-bit integer.
FIOSEEK64	Sets the current byte offset in a file from a 64-bit integer.
FIOWHERE64	Gets the current byte position in a file into a 64-bit integer.
FIOTRUNK64	Set the file's size from a 64-bit integer.

Parent topic: dosFs on page 40

5. rawFs

VxWorks provides a raw file system (rawFs) for use in systems that require only the most basic disk I/O functions.

WNDRVR

The rawFs file system, implemented with **rawFsLib**, treats the entire disk volume much like a single large file. Although the dosFs file system provides this ability to varying degrees, the rawFs file system offers advantages in size and performance if more complex functions are not required.

The rawFs file system imposes no organization of the data on the disk. It maintains no directory information; and there is therefore no division of the disk area into specific files. All open( ) operations on rawFs devices specify only the device name; no additional filenames are possible.

The entire disk area is treated as a single file and is available to any file descriptor that is open for the device. All read and write operations to the disk use a byte-offset relative to the start of the first block on the disk.

A rawFs file system is created by default if inserted media does not contain a recognizable file system.

Configuring VxWorks for rawFs on page 70 To configure VxWorks with support for rawFs, you must select the required components for your VIP project, and set the parameter for the maximum number of open files.
Creating a rawFS File System on page 70 The rawFs file system is the default file system. It is created automatically when VxWorks cannot instantiate a known file system such as dosFs, HRFS, or cdromFs.
Mounting rawFs Volumes on page 71rawFs disk volumes are mounted automatically.
rawFs File I/O on page 72 Standard file I/O operations are used with rawFs.
I/O Control Functions Supported by rawFsLib on page 72 The rawFs file system supports a set of ioctl( ) functions..

5.1. Configuring VxWorks for rawFs

To configure VxWorks with support for rawFs, you must select the required components for your VIP project, and set the parameter for the maximum number of open files.

Before You Begin

You must know how to use Workbench or wrtool to create and configure VIP projects. For information in this regard, see the *VxWorks 7 Configuration and Build Guide*.

Procedure

1. Create a VSB project, select whatever other options you require, and build the project.

   Support for rawFs is included in the VxWorks source code by default. You do not need to select an option for rawFs.

2. If you are using a device driver that is not designed for use with the XBD facility, you must use the **INCLUDE_XBD_BLK_DEV** wrapper component in addition to **INCLUDE_XBD**.

   For information about XBD, see Extended Block Device Facility: XBD Overview on page 79.

3. Set the **NUM_RAWFS_FILES** parameter of the **INCLUDE_RAWFS** component to the desired maximum open file descriptor count.

   For information about using multiple file descriptors with what is essentially a single large file, see rawFs File I/O on page 72.

Parent topic: rawFs on page 69

5.2. Creating a rawFS File System

The rawFs file system is the default file system. It is created automatically when VxWorks cannot instantiate a known file system such as dosFs, HRFS, or cdromFs.

Unlike dosFs and HRFS, rawFs does not have a formatter. There are no particular data structures on the media that signify the disk as being raw. To create a rawFs file system manually, the current file system must be un-instantiated and replaced with rawFs. Having two or more file systems on the same media can produce instabilities in the VxWorks system. Hence, when instantiating a new file system the previous one must be removed.

See File System Monitor on page 80 for information about default creation of rawFs.

The rawFs library **rawFsLib** is initialized automatically at boot time. The rawFsInit( ) function is called by the usrRoot( ) task after starting the VxWorks system.

The rawFsInit( ) function takes two parameters, the maximum number of rawFs file descriptors that can be open at one time, and the maximum number of blocks per request.

The maximum number of rawFs file descriptors is used to allocate a set of descriptors; a descriptor is used each time a rawFs device is opened. The parameter can be set with the **NUM_RAWFS_FILES** configuration parameter of the **INCLUDE_RAWFS** component.

The maximum number of blocks is used for determining how many blocks rawFs will read or write per request. A large value may be helpful for large block continuous read or write.

The rawFsInit( ) function also makes an entry for the rawFs file system in the I/O system driver table (with iosDrvInstall( )). This entry specifies the entry points for rawFs file operations, for all devices that use the rawFs file system. The driver number assigned to the rawFs file system is placed in a global variable, *rawFsDrvNum* .

After the rawFs file system is initialized, one or more devices must be created. Devices are created with the device driver's device creation function (*xx*DevCreate( )). The driver function returns a pointer to a block device descriptor structure (**BLK_DEV**). The **BLK_DEV** structure describes the physical aspects of the device and specifies the functions in the device driver that a file system can call.

Immediately after its creation, the block device has neither a name nor a file system associated with it. To initialize a block device for use with rawFs, the already-created block device must be associated with rawFs and a name must be assigned to it. This is done with the rawFsDevInit( ) function. Its parameters are the name to be used to identify the device and a pointer to the block device descriptor structure (**BLK_DEV**):

```
RAW_VOL_DESC *pVolDesc;
BLK_DEV *pBlkDev;
pVolDesc = rawFsDevInit ("/DEV1:", pBlkDev);
```

The rawFsDevInit( ) call assigns the specified name to the device and enters the device in the I/O system device table (with iosDevAdd( )). It also allocates and initializes the file system's volume descriptor for the device. It returns a pointer to the volume descriptor to the caller; this pointer is used to identify the volume during certain file system calls.

In normal usage, user should not call rawFsDevInit( ) directly, rawFsDevInit( ) will be called by File System Monitor on page 80.

Note that initializing the device for use with rawFs does not format the disk. That is done using an ioctl( ) call with the **FIODISKFORMAT** function.

Note: No disk initialization (**FIODISKINIT**) is required, because there are no file system structures on the disk. Note, however, that rawFs accepts that ioctl( ) function code for compatibility with other file systems; in such cases, it performs no action and always returns OK.

Parent topic: rawFs on page 69

5.3. Mounting rawFs Volumes

rawFs disk volumes are mounted automatically.

WNDRVR

A volume is mounted generally during the first open( ) or creat( ) operation. Certain ioctl( ) functions also cause the disk to be mounted. The volume is again mounted automatically on the first disk access following a ready-change operation.

CAUTION: **Because device names are recognized by the I/O system using simple substring matching, file systems should not use a slash (/) alone as a name or unexpected results may occur.**

Parent topic: rawFs on page 69

5.4. rawFs File I/O

Standard file I/O operations are used with rawFs.

To begin I/O operations upon a rawFs device, first open the device using the standard open( ) function (or the creat( ) function). Data on the rawFs device is written and read using the standard I/O functions write( ) and read( ). For more information, see Overview of VxWorks File Systems and I/O Devices on page 1.

The character pointer associated with a file descriptor (that is, the byte offset where the read and write operations take place) can be set by using ioctl( ) with the **FIOSEEK** function.

Multiple file descriptors can be open simultaneously for a single device. These must be carefully managed to avoid modifying data that is also being used by another file descriptor. In most cases, such multiple open descriptors use **FIOSEEK** to set their character pointers to separate disk areas.

Parent topic: rawFs on page 69

5.5. I/O Control Functions Supported by rawFsLib

The rawFs file system supports a set of ioctl( ) functions..

The rawFs file system supports the ioctl( ) functions shown below. The functions listed are defined in the header file **ioLib.h**. For more information, see the API references for **rawFsLib** and for ioctl( ) in ioLib( ).

Function	Description
FIOFLUSH	Flushes the file output buffer.
FIOGETNAME	Gets the filename of the fd.
FIONREAD	Gets the number of unread bytes in a file.
FIOSEEK	Sets the current byte offset in a file.
FIOSYNC	Same as **FIOFLUSH**.
FIOUNMOUNT	Un-mounts a disk volume.
FIOWHERE	Returns the current byte position in a file.
FIONREAD64	Gets the number of unread bytes in a file into a 64-bit integer.
FIOSEEK64	Sets the current byte offset in a file from a 64-bit integer.

WNDRVR

Function	Description
FIOWHERE64	Gets the current byte position in a file into a 64-bit integer.

Parent topic: rawFs on page 69

6. cdromFs

The VxWorks CD-ROM file system, cdromFs, allows applications to read data from CDs and mounted images formatted according to the ISO 9660 standard file system, with or without the Joliet extensions.

The cdromFs library, **cdromFsLib**, lets applications read any CD-ROMs, CD-Rs, CD-RWs (collectively called CDs), or mounted images that are formatted in accordance with the ISO 9660 file system standard, with or without the Joliet extensions. ISO 9660 interchange level 3, implementation level 2, is supported. Note that multi-extent files, interleaved files, and files with extended attribute records are supported.

Note: Wind River does not provide VxBus-compatible ATAPI device drivers. You must implement your own extended block device (XBD) driver to use cdromFs with a CD drive.

The cdromFs file system does not support the following CD and ISO 9660 features:

- Multi-volume sets
- Record format files
- CDs with a sector size that is not a power of two; therefore, mode 2/form 2 sectors are not supported, as they have 2324 bytes of user data per sector. Both mode 1/form 1 and mode 2/form 1 sectors are supported, as they have 2048 bytes of user data per sector.
- Multi-session CD-R or CD-RW. The first session (that is, the earliest session) is always read. The most commonly desired behavior is to read the last session (that is, the latest session).

After initializing cdromFs and mounting it on a CD-ROM block device, you can access data on that device using the standard POSIX I/O calls: open( ), close( ), read( ), ioctl( ), readdir( ), and stat( ). The write( ) call always returns an error.

The cdromFs utility supports multiple drives, multiple open files, and concurrent file access. When you specify a pathname, cdromFS accepts both forward slashes (/) and back slashes (\) as path delimiters; however, the backslash is not recommended because it might not be supported in future releases.

The initialization sequence for the cdromFs file system is similar to installing a dosFs file system on other devices.

After you have created the CD file system device (see Creating and Using cdromFs on page 75), use the ioctl( ) function to set file system options. The files system options are described below:

CDROMFS_DIR_MODE_SET/GET

These options set and get the directory mode. The directory mode controls whether a file is opened with the Joliet extensions or without them. The directory mode can be set to any of the following:

MODE_ISO9660

Do not use the Joliet extensions.

MODE_JOLIET

Use the Joliet extensions.

MODE_AUTO

Try opening the directory first without Joliet, and then with Joliet.

> **CAUTION:** **Changing the directory mode unmounts the file system; thus, any open file descriptors are marked as obsolete.**

CDROMFS_STRIP_SEMICOLON

This option sets the readdir( ) strip semicolon setting to FALSE if arg is **0** and to TRUE otherwise. If TRUE, the readdir( ) function removes the semicolon and following version number from the directory entries retrieved.

CDROMFS_GET_VOL_DESC

This option returns, in arg, the primary or supplementary volume descriptor by which the volume is mounted. arg must be of type **T_ISO_PVD_SVD_ID**, as defined in the **cdromFsLib.h** header file. The result is the volume descriptor, adjusted for the endianness of the processor (not the raw volume descriptor from the CD). This result can be used directly by the processor. The result also includes some information not in the volume descriptor, such as which volume descriptor is in use.

For information on using the cdromFs( ) function, see the API reference for the **cdromFsLib** library.

Configuring VxWorks for cdromFs on page 74 To configure VxWorks with support for cdromFs, you must select the required options and components for your VSB and VIP projects.
Creating and Using cdromFs on page 75 To use cdromFs with an ISO 9660 image, you must first mount the image with loopFs.
I/O Control Functions Supported by cdromFsLib on page 77 The cdromFs file system supports the ioctl( ) functions. These functions, and their associated constants, are defined in the header files **ioLib.h** and **cdromFsLib.h**.

6.1. Configuring VxWorks for cdromFs

To configure VxWorks with support for cdromFs, you must select the required options and components for your VSB and VIP projects.

About This Task

To use cdromFs with a CD drive, you must implement your own extended block device (XBD) driver. For information using custom device drivers with VxWorks, see *VxWorks 7 BSP and Driver Guide* .

Before You Begin

You must know how to use Workbench or wrtool to create and configure VSB and VIP projects. For information in this regard, see *VxWorks 7 Configuration and Build Guide*.

Procedure

1. Create a VSB project, enable the **FS_CDROMFS** layer, and build the VSB project.
2. Create a VIP project based on the VSB project, add the **INCLUDE_CDROMFS** component, and ensure that the **INCLUDE_XBD** component is added.
3. Add support for using cdromFs with either an ISO image or a CD drive:
   - To use cdromFs with an ISO image that you will mount with the loopFs file system, add the **INCLUDE_LOOPFS** component.
   - To use cdromFs with a CD drive (for which you have written an XBD-compatible driver), add the component for your custom device driver.
4. Build VxWorks.

Parent topic: cdromFs on page 73

6.2. Creating and Using cdromFs

To use cdromFs with an ISO 9660 image, you must first mount the image with loopFs.

About This Task

Note: If you have implemented a custom XBD driver for a CD drive, the **/cdrom** device is instantiated automatically if a CD is present in the drive. The operations described in this procedure are relevant, with the exception of those related to loopFs.

Procedure

1. Execute loopFsMountImgFile to mount the image as a virtual block device with loopFs.

   For example:

   ```
   -> loopFsMountImgFile "/ata0/cdrom.iso", "/cdrom"
   ```

   The first parameter is the image file path and the second parameter is the name that you specify for the mount path.

2. Execute the devs command to verify that the ISO image is a valid CD-ROM image and has successfully been mounted by loopFs.

   For example:

   ```
   -> devs
   drv name
    0 /null
    1 /ttyS0
    4 /ram0
    9 host:
   12 /hostfs
    3 /cdrom
   ```

   The **/cdrom** device name is the one assigned with the loopFsMountImgFile( ) function.

3. If you want to omit version numbers from the readdir( ) output or specify which volume descriptors to use, first get the file descriptor and then set the file system options:
   1. Open the file to get the file descriptor.

      For example:

      ```
      -> fd = open ("/cdrom", 0, 0)
      New symbol "fd" added to kernel symbol table.
      fd = 0x18cef98: value = 4 = 0x4
      ```

      In the output above, the first **0** is the value of **O_RDONLY** in the **fcntl.h** header file.

      2. To omit version numbers from readdir( ) output, call ioctl( ) with the strip semicolon option.

      For example:

WNDRVR

```
-> ioctl (fd, 0x740002, 1)
```

The strip semicolon mode controls whether the readdir( ) function returns version numbers. After mounting, the strip semicolon mode defaults to FALSE, meaning that version numbers will be returned. It must be set to TRUE to omit version numbers from readdir( ) output.

In the command above, **0x740002** is the value of **CDROMFS_STRIP_SEMICOLON** in the **cdromFsLib.h** header file.

3. To specify which volume descriptor to use, call ioctl( ) with the directory mode option.

For example:

```
-> ioctl (fd, 0x740000, 0)
```

The directory mode controls which volume descriptor is used to open a file or directory. After mounting, the directory mode defaults to **MODE_AUTO**, meaning that all volume descriptors will be tried. The directory mode can be changed to **MODE_ISO9660**, to use only the ISO 9660 volume descriptors, or **MODE_JOLIET**, to use only the Joliet volume descriptors. If either of these modes is selected, and if the CD does not contain the selected volume descriptor, an **S_cdromFsLib_UNKNOWN_FILE_SYSTEM** error is recorded.

In the command above, **0x740000** is the value of **CDROMFS_DIR_MODE_SET** and **0** is the value of **MODE_ISO9660**. Both are located in the **cdromFsLib.h** header file.

4. Close the file.

```
-> close (fd)
```

4. Call cdromFsVolConfigShow( ) to display information about the mounted ISO 9660 image configuration.

For example:

```
-> cdromFsVolConfigShow "/cdrom:0"

device config structure ptr   0x38b0f630
device name            /cdrom
bytes per physical sector   512

Primary directory hierarchy:

volume descriptor number    :1
descriptor logical sector   :16
descriptor offset in sector :0
standard ID                 :CD001
volume descriptor version   :1
UCS unicode level  (0=ISO9660)  :0
system ID                   :
volume ID                   :UEFI+BIOS
volume size                 :400451584 = 381 MB
number of logical blocks        :195533 = 0x2fbcd
volume set size             :1
volume sequence number      :1
logical block size          :2048
path table memory size (bytes)  :194
path table size on CD (bytes)   :194
path table entries          :15
volume set ID               :UEFI+BIOS
```


WNDRVR

```
volume publisher ID         :Tester
volume data preparer ID     :Tester
volume application ID       :OSCDIMG 2.56 (01/01/2005 TM)
copyright file name         :none
abstract file name          :none
bibliographic file name     :none
creation date           :01.08.2014  10:27:45:00
modification date       :01.08.2014  10:27:45:00
expiration date         :00.00.0000  00:00:00:00
effective date          :00.00.0000  00:00:00:00
value = 0 = 0x0
```

Parent topic: cdromFs on page 73

6.3. I/O Control Functions Supported by cdromFsLib

The cdromFs file system supports the ioctl( ) functions. These functions, and their associated constants, are defined in the header files **ioLib.h** and **cdromFsLib.h**.

The table below describes the ioctl( ) functions that **cdromFsLib** supports. For more information, see the API references for **cdromFsLib** and for ioctl( ) in **ioLib**.

Function	Description
CDROMFS_DIR_MODE_GET	Gets the volume descriptor(s) used to open files.
CDROMFS_DIR_MODE_SET	Sets the volume descriptor(s) used to open files.
CDROMFS_GET_VOL_DESC	Gets the volume descriptor that is currently in use.
CDROMFS_STRIP_SEMICOLON	Sets the readdir( ) strip version number setting.
FIOFSTATGET	Gets file status information (directory entry data).
FIOGETNAME	Gets the filename of the fd.
FIOLABELGET	Gets the volume label.
FIONREAD	Gets the number of unread bytes in a file.
FIOREADDIR	Reads the next directory entry.
FIOSEEK	Sets the current byte offset in a file.
FIOUNMOUNT	Un-mounts a disk volume.
FIOWHERE	Returns the current byte position in a file.
FIONREAD64	Gets the number of unread bytes in a file into a 64-bit integer.

WNDRVR

Function	Description
FIOSEEK64	Sets the current byte offset in a file from a 64-bit integer.
FIOWHERE64	Gets the current byte position in a file into a 64-bit integer.

Parent topic: cdromFs on page 73

4. VXWORKS I/O DEVICE AND FILE SYSTEM FACILITIES

1. Extended Block Device Facility: XBD Overview

The extended block device (XBD) facility mediates I/O activity between file systems and block devices. It provides a standard interface between file systems and block drivers.

The XBD facility also provides support for removable file systems, automatic file system detection, and multiple file systems.

> Note: The XBD facility is required for some file systems, such as HRFS, dosFs, cdromFs, and rawFs, but not others, such as ROMFS. For more information, see the XBD column in the table in I/O System and Device Naming Conventions on page 4.

For detailed information on developing XBD-compatible device drivers, see the relevant file system topics under Configuring File Systems in this guide.

The basic XBD facility is proved with the **INCLUDE_XBD** component, which also provides generic service for following optional components:

INCLUDE_XBD_RAMDRV

Provides support for RAM disks.

INCLUDE_XBD_PART_LIB

Provides disk partitioning facilities.

INCLUDE_XBD_BLK_DEV

Provides support for legacy block device drivers that were designed to work with the predecessor to XBD: the cache block I/O (CBIO) facility. These devices include floppy drives, and TrueFFS (the disk-access emulator for flash).

XBD Disk Partition Manager

VxWorks provides support for PC-style disk partitioning with the **INCLUDE_XBD_PART_LIB** component, which facilitates sharing fixed disks and removable cartridges between VxWorks target systems and PCs running Windows. This component includes two modules: **xbdPartition**, which manages partitions; and **partLib**, which provides partitioning facilities.

The **xbdPartition** facility creates a device for each partition that is detected on media. Each partition that is detected is probed by the file system monitor and an I/O device is added to VxWorks. This device is an instantiation of the file system found by the file system monitor, or rawFs if the file system is not recognized or detected. If no partitions are detected, a single device is created to represent the entire media. There can be up to four partitions on a single media. For information about the file system monitor, see File System Monitor on page 80.

The partition facility also names the partitions. The names are derived from the device name and the partition number. To change the default device name, use the **INCLUDE_SATA_DISK_NAME_CFG** component.

For example, the name of an XBD-compatible device for an ATA hard disk without partitions would be **/ata0**.

If four partitions were required, you could name them as follows:

- **/ata0a**
- **/ata0b**
- **/ata0c**

WNDRVR

- **/ata0d**

The **partLib** library provides facilities for creating PC-style partitions on media. It can create up to four primary partitions. When partitions are created, any existing information on the media is lost. For more information see the VxWorks API reference for **partLib**.

XBD Block Device Wrapper

The **INCLUDE_XBD_BLK_DEV** component provides support for legacy block devices that were designed to work the predecessor to XBD: the cache block I/O (CBIO) facility. It provides a wrapper XBD facility that converts the block I/O driver interface based on the **BLK_DEV** logical block device structure into an XBD API-compliant interface.

Note: The Wind River devices that require the **INCLUDE_XBD_BLK_DEV** component in addition to **INCLUDE_XBD** include floppy and TrueFFS drivers. Any third-party device drivers based on the **BLK_DEV** interface also require **INCLUDE_XBD_BLK_DEV**. The Wind River drivers that do not require the **INCLUDE_XBD_BLK_DEV** component are USB block storage, ATA, and the XBD RAM disk.

CAUTION: **Depending on the implementation of the driver, the INCLUDE_XBD_BLK_DEV component may not properly detect media insertion and removal. Therefore, it may not remove the file system when the media is removed or instantiate a file system when media is inserted.** .

Direct BIO Handling Mode

The parameter **XBD_BLK_DEV_DIRECT_BIO_HANDLING** is used to enable direct BIO handling mode. If the value is set to TRUE, then the XBD service task (tXbdService) will not be created. The data will be read or written, with direct BIO mode, by the user task. The scenario where the XBD service task schedules a lot of other tasks and pending high user tasks, will not happen. The side effect of using this parameter is that the amount of writing looks like it is decreasing as there is no background task (tXbdService), all the work is being done by the user task.

XBD Scheduler

The XBD scheduler is the manager of BIOs. It decides the request sequence and when to send the request to the device. How to order the request sequence depends on the assigned scheduler policy that the device driver chooses.

The XBD BIO scheduler provides the BIO schedule and merge feature in the XBD interface. The BIO scheduler may reorder like an elevator, and merge several small BIOs into one BIO, to improve storage driver performance.

The storage driver should provide an **xf_request** function when a device is attaching to XBD.

The SATA storage driver has been enabled as an XBD scheduler.

2. File System Monitor

The file system monitor provides for automatic detection of device insertion, and instantiation of the appropriate file system on the device.

The monitor is required for all file systems that are used with the extended block device (XBD) facility (for more information, see Extended Block Device Facility: XBD Overview on page 79). It is provided with the **INCLUDE_FS_MONITOR** component.

The file systems that require both the XBD and the file system monitor components are HRFS, dosFs, rawFs, and cdromFs.

The process by which devices are detected, and file systems created, is as follows:

WNDRVR

1. When file systems are initialized (at boot time), they register probe functions and instantiation functions with the file system monitor.
2. When a device is detected or inserted (for example, when a driver is initialized, or media is inserted into an existing device—such as a diskette disk into a diskette drive) the block device associated with it generates a primary insertion event.
3. In response to the primary insertion event, the file system monitor creates an XBD partition manager if the device can support partitions. (For information about the partition manager, see the *VxWorks 7 Platform Programmer's Guide: VxWorks Kernel Heap and Memory Partition Management*.)
4. If the partition manager finds partitions on the physical device, it creates a device for each partition; and whether or not partitions are found, the manager generates a secondary insertion event.
5. When the file system monitor receives a secondary event, all the registered file system probe functions are run.
6. When a file system's probe function returns success, that file system's instantiation function is executed. If none of the probes are successful, or if the file system instantiation function fails, a rawFs file system is created on the device by default.

When a device is removed, the following occurs:

1. The block device detects the removal of the hardware device associated with it and generates a removal event.
2. The block device removes itself, freeing all its resources.
3. The file system associated with the block device removes itself from core I/O, invalidating its file handles.
4. The file system removes itself, freeing all its resources.

Device Insertion Events

The types of device insertion events to which the file system monitor responds are described in more detail below.

XBD Primary Insertion Event

An XBD-compliant block device generates a primary insertion event when media that can support partitions is inserted (that is, if a partition table is found). In response, the file system monitor creates a partition manager, which in turn generates secondary insertion events for each partition that it finds on the media (see below).

Note that for block devices used with the XBD wrapper component (**INCLUDE_XBD_BLK_DEV**), a primary insertion event is always generated, regardless of the media. The wrapper element is essentially hardware agnostic; it cannot know if the device might include partitions. For example, the device could be a hard disk—for which partitions are expected—or it could be a diskette device.

Note also that a RAM disk device can generate a primary insertion event, depending on the parameters used when it was created (see the *VxWorks 7 Application Programmer's Guide: RTP Application Control Features* and the API reference for **XbdRamDisk**).

Note: **XBD_HARD_EJECT** with XBD_BASE will trigger a primary insertion event for the specific device.

XBD Secondary Insertion Event

A secondary insertion event is generated by either by a block device whose media does not support partitions, or by an XBD partition manager. The secondary event signals the file system manager to run the probe functions that identify the file system on the device. If a probe function returns OK, the associated file system creation function is executed. If none of the probe functions identifies a file system, or if a file system creation function fails, the rawFs file system is created by default.

Note: **XBD_HARD_EJECT** with XBD_TOP or XBD_PART will trigger a secondary insertion event for the specific partition.

WNDRVR

XBD Soft Insert Event

Unlike the other events, an XBD soft insert event is produced by application directive rather than by physical media being swapped. When **ioctl( )** is called with the **XBD_SOFT_EJECT** control function it tells the file system manager that the current file system has been removed, and that a rawFs file system should be created. This call causes the system to bypass the usual file system detection operations, and ensures that rawFs is instantiated in place of the current file system.

Note: **XBD_SOFT_EJECT** with **XBD_TOP** and XBD_PART will swap the target partition to a rawFs file system.

XBD_SOFT_EJECT with XBD_BASE will swap the whole device to a rawFs file system as a single device.

XBD Name Mapping Facility

The file system monitor name mapping facility allows XBD names to be mapped to a more suitable name. Its primary use is for the partition manager which appends *:x* to the base **xbd** name when it detects a partition. By using the **fsm** name facility, you can map the partition names to something more useful. For example, the diskette drive configlette uses the name component to map the supplied diskette name plus the *:0* the partition manager will add to **/fd***x*, where *x* represents the diskette drive number. If this was not done, one would see the default device names in the list generated by the devs shell command. For more information, see the API references for fsmNameInstall( ), fsmNameMap( ), and fsmNameUninstall( ); also see HRFS, ATA, and RAM Disk Examples on page 23.

Note that the default device name length is 10 characters. If additional characters are used, they are truncated. For more information (including the method for changing the default), see the discussion of device name length in VxWorks Devices and Driver Model Overview on page 167.

Partition Table Support

VxWorks supports master boot record (MBR) and GUID partition table (GPT) partition table. MBR and GPT support are enabled by default. GPT support could be disabled by removing the VSB option **_WRS_CONFIG_XBD_GPT_SUPPORT**.

For MBR-only mode (with GPT support disabled), VxWorks supports up to 24 partitions, including a maximum of four primary partitions and extended partitions.

VxWorks only uses the LBA address field in an MBR partition record and will ignore the CHS address.

If GPT support is enabled, VxWorks supports up to 127 partitions. For both GPT and MBR partition tables, VxWorks ignores the partition table record's type field. VxWorks will try to probe the partition even if it does not set a correct type value in partition table.

3. File System Utility

The file system utility allows a user to rename a single file system IOS device and/or delay VxWorks initialization until that device is available. The utility also enables a user to verify if a device is mounted and to partition the file system.

File system utilities can only work with a file system device. If there is no file system device, the file system utility may fail (for example, by calling device rename for a TTY device or a **netDrv** device)

File System Device Rename

To enable changes to the default name of a VxWorks FS device, include the **INCLUDE_FS_DEVICE_RENAME** component.

File System Mount Validate

To enable the validation of the state of a block device, include the **INCLUDE_FS_DEVICE_MOUNT_VALIDATE** component.

WNDRVR

File System Information Show

The file system information show function could display a file system status and information. To enable the file system information show function, include the **INCLUDE_FSINFO_SHOW** component.

The information may include the mounted file system name, total blocks, free blocks, etc.

Note that some information may not be valid because the mounted file system may not support it.

For example:

```
-> deviceFsInfoShow "/ata1a"
Device Name [/ata1a]
File System     [MS-DOS Compatible File System]
Block Size      [32768]
Total Blocks    [6103341]
Free Blocks     [6103338]
Available Block [6103338]
Total Files     [-1]
Free Files      [-1]
File System ID [0xbfa5ffff,0x0]
value = 0 = 0x0
```

Partition Table Information Display

xpPartShow( ) can displaythe partition table and mount information for a device:

```
-> xpPartShow "/ata1a"
Request name [/ata1a], Real Device name [/ata1a], XBD Base name [/ata1]
Device geometry:
Total Blocks [976773168] Bytes per sector [512]
CHS: Cylinder [16383] Head [16] Sector [63]

Mount state:
Idx.  IO Path     XBD Name        Start Sector       End Sector     Sector Count
--------------------------------------------------------------------------------
Total Not Valid   /ata1:0                    0        976773167        976773168
1     /ata1a      /ata1a                   128        390709247        390709120
2     /ata1b      /ata1b             390709248        683741055        293031808
3     /ata1c      /ata1c             683741056        976772863        293031808

MBR Info:
Idx. Boot  Type  Start Sector   End Sector  Sector Count  CHS(Cyl. Head Sec.)
--------------------------------------------------------------------------------
   1   80    0C           128    390709247     390709120    0,  2, 3  378,134, 6
   2   00    0C     390709248    683741055     293031808  378,134, 7  662,168,40
   3   00    0C     683741056    976772863     293031808  662,168,41  946,203,11
```

MBR Partition Table Utilities

MBR partition table utilities are used to create, delete, and modify type and activity fields in partition tables.

By default, any MBR partition will make alignment with 128 sectors and use partLibSetAlignment to change the alignment value.

Create partition:

The following example shows how to create three MBR primary partitions in the device **/ata1**. Two of the partitions are sized at 20% each of the total. The remaining partition is sized at 60% of the total.

WNDRVR

```
-> xbdCreatePartition "/ata1a", 3, 20, 20
Instantiating /ata1 as rawFs, device = 0x10001
Instantiating /ata1a as rawFs,  device = 0x100001
Instantiating /ata1b as rawFs,  device = 0x110001
Instantiating /ata1c as rawFs,  device = 0x120001
```

Note: The device name is any valid device or partition on the target device. The size of the first partition is determined by subtracting the other partition sizes from the total.

xbdCreatePartitionSz can create a partition with specific size in MB.

xbdCreatePartitionEx can create a partition of a specific size in percent and reserve some blocks before the first partition.

Clean Partition:

xbdCleanPartition will clean a whole partition table on disk; if the resetMbr parameter is set to **TRUE**, this function will write a new MBR for the device.

For more information about MBR partition management, see the API reference for **partLib**.

To enable MBR partition table utilities, include the **INCLUDE_XBD_PART_LIB** component.

GPT Partition Table Utilities

GPT partition table utilities provide a group of APIs to create GPT partitions.

By default, VxWorks will create GPT partitions with type **GPT_ENT_TYPE_MS_BASIC_DATA**. This value could be changed by modifying **gptDefaultPartType** at run time or by using **xbdGptPartAddWithType**.

GPT functions use the XBD device name for a parameter. Use **gptSetApiUseIoName** to change the parameter name from the XBD name to the core I/O device name.

The following example shows how to create GPT partitions:

Firstly, start partition creation:

```
-> xbdGptPartStart  "/ata1"
Instantiating /ata1 as rawFs, device = 0x40001
```

Then, add a partition of a specific location and size:

```
-> xbdGptPartAdd  "/ata1", 10000, 10000
```

Prepare to add a partition offset **0x2710**, size **0x2710**:

```
-> xbdGptPartAdd  "/ata1", 30000, 20000
```

Prepare to add a partition offset **0x7530**, size **0x4e20**.

When you are finished adding file partitions, flush the entire file system to disk:

```
-> xbdGptPartFin  "/ata1"
Instantiating /ata1a as rawFs,  device = 0x190001
Instantiating /ata1b as rawFs,  device = 0x1a0001
```

Use display functions to check the result:

```
  -> xpPartShow "/ata1a"
Request path [/ata1a], XBD Base name [/ata1]
Device geometry:
```

```
Total Blocks [1953525168] Bytes per sector [512]
CHS: Cylinder [16383] Head [16] Sector [63]

Mount state:
Idx.  IO Path     XBD Name        Start Sector       End Sector     Sector Count
--------------------------------------------------------------------------------
Total Not Valid   /ata1:0                    0       1953525167       1953525168
1     /ata1:1     /ata1a                10000            19999            10000
2     /ata1:2     /ata1b                30000            49999            20000

MBR Info:
Idx. Boot  Type  Start Sector   End Sector  Sector Count  CHS(Cyl. Head Sec.)
--------------------------------------------------------------------------------
GPT  00    EE             1   1953525167    1953525167    0,  0, 2 1023,254,63
GPT1   ebd0a0a2-b9e5-4433-87c0-68b6b72699c7 90a65040-43cd-11b2-a453-9fe70648dd03
       LBA From:             10000          LBA To:            19999
GPT2   ebd0a0a2-b9e5-4433-87c0-68b6b72699c7 90a65041-43cd-11b2-af18-fdf824df3ad7
       LBA From:             30000          LBA To:            49999
```

Note: For more information about GPT partition management, see the API reference for **gpt**.

4. RAM Drives

VxWorks provides facilities for several types of RAM drives (also called RAM disks), which treat a block of memory as if it were hardware storage media. Some types of RAM drives can be created using volatile as well a non-volatile RAM.

memDrv

A **memDrv** drive allows memory to be treated as a flat file with no block device. It provides a high-level method of reading and writing bytes in absolute memory locations through I/O calls. It can also be used to implement a simple, essentially read-only file system (existing files can be rewritten within their existing sizes); directory searches and a limited set of IOCTL calls are supported. It cannot, however, be used to implement a file system.

Memory location and size are specified when the device is created. This feature is useful when data must be preserved between warm reboots, or when sharing data between CPUs.

For more information, see the **memDrv** API reference entry.

xbdRamDisk

An **xbdRamDisk** drive allows the use of a file system in RAM memory. The RAM drive can be used with the HRFS, dosFs, and rawFs file systems. It links into the file system monitor and event framework.

Note that this XBD-compatible RAM drive facility supersedes the legacy **ramDrv** facility (which also supports file systems). However, you cannot specify the location of an **xbdRamDisk** drive in memory, whereas you can specify the location of a **ramDrv** drive.

For more the API reference for **XbdRamDisk**, as well as implementation examples in this guide.

ramDrv

A **ramDrv** drive uses a simple block device in RAM addressable by the I/O system. It can be formatted directly with the dosFs file system, or wrapped in an XBD block device and formatted with the HRFS file system.

While **ramDrv** is a legacy technology superseded by **xbdRamDisk**, it does allow you to specify the location of the drive in memory, which is useful in allowing recovery of RAM drive contents after a warm reboot.

For more information, see the **ramDrv** API reference entry.

romfsDrv

A **romfsDrv** drive can be created in RAM or ROM memory. It must be associated with the ROMFS file system, which is a small portable hierarchical read-only file system. ROMFS is particularly useful for incorporating read-only files into a small-footprint system.

For more information, see the **romfsDrv** and **romfsLib** API reference entries and Creating a ROMFS File System Programmatically on page 38. For an integrated workflow using Workbench, that requires no coding, to create a ROMFS file system, see *VxWorks 7 Kernel Application and RTP Application Projects Guide: ROMFS File System Projects* .

5. NFS and Remote File Systems

Network File System (NFS) devices allow files on remote systems to be accessed with the NFS protocol.

The NFS protocol specifies both client software, to read files from remote machines, and server software, to export files to remote machines. VxWorks provides an implementation of the Network File System (NFS) application protocol, versions 2 and 3.

NFS Version 2 Implementation

The **nfs2dLib** library implements the NFS version 2 procedures as specified in the Network File System Protocol Specification (RFC 1094). The mount1Lib library implements a mount version one server to support mounting VxWorks file systems remotely. The mount server is an implementation of version 1 of the mount protocol as defined in RFC 1094.

NFS Version 3 Implementation

The **nfs3dLib** library provides an implementation of the NFS version 3 procedures as specified in the RFC 1813 (Network File System Protocol Specification). The mount3Lib library implements a mount server. The mount server is an implementation of version 3 mount protocol as defined in RFC 1813. NFS version 3 requires mount versions 1 and 3.

Note: VxWorks does not normally provide authentication services for NFS requests. If you need the NFS server to authenticate incoming requests, see the nfsdInit( ) and mountdInit( ) reference entries for information on authorization hooks.

Accessing Remote File Systems from VxWorks on page 87 VxWorks lets you access remote file systems, including network file systems (NFS) and non-NFS file systems accessed over a network. Accessing remote file systems from VxWorks (either programmatically or from the shell) requires special consideration, depending on the connection protocol.
Non-NFS Network Device File Systems on page 88 VxWorks supports network access to files on a remote host through the Remote Shell protocol (RSH) or the File Transfer Protocol (FTP).
Network File System (NFS) Devices Overview on page 89 Network File System (NFS) devices allow files on remote hosts to be accessed with the NFS protocol. The NFS protocol specifies both *client* software, to read files from remote machines, and *server* software, to export files to remote machines.
VSB Project for NFS on page 91 The primary VSB layer for NFS, which is FS_NFS, is enabled by default.
Configuring VxWorks With an NFS Client on page 91 VxWorks supports several NFS client configuration components, which you can use to include the modules needed to create an NFS client
Setting Up an NFS Client on page 95 Set up an NFS client and use it to access a remote NFS system.
Setting the NFS Client Name, User ID, and Group ID on page 96 If you have not set the user name, user ID, and group name required for authentication with **INCLUDE_CORE_NFS_CLIENT** component parameters, you must call nfsAuthUnixSet( ) to do so.
Adding Remote NFS System to Host Table on page 96 Call hostAdd( ) to add remote NFS systems to the VxWorks' host table.
Mounting a Remote File System on page 96 Call nfsMount( ) or nfsMountAll( ) to mount a remote file systems.

Configuring VxWorks With an NFS Server on page 97 VxWorks supports several NFS server configuration components, which you can use to include the modules needed to create an NFS server.
Exporting NFS File Systems from a VxWorks System on page 101 To make a file system on a VxWorks system accessible to NFS clients on a network, you must call nfsExport( ).
Exporting NFS File Systems From a UNIX System on page 102 For a UNIX NFS server, the **/etc/exports** file specifies which of the server's file systems are exported for mounting by remote NFS clients.
Exporting NFS File Systems from a Windows System on page 102 Windows systems support NFS. It is therefore possible to configure a directory on a Windows system so that it is exported over NFS.
Configuring VxWorks for NFS Debugging on page 103 Use the NFS debugging component with NFS mount, NFS client, or NFS server, as required. The component provides for five debug levels.

5.1. Accessing Remote File Systems from VxWorks

VxWorks lets you access remote file systems, including network file systems (NFS) and non-NFS file systems accessed over a network. Accessing remote file systems from VxWorks (either programmatically or from the shell) requires special consideration, depending on the connection protocol.

NFS File System Access from VxWorks

By convention, NFS-based network devices are *mounted* with names that begin with a slash. To access directories and files from VxWorks, use the path from the mount point.

If the mount point for **/usr/xeno/foo** on VxWorks is **/foo**, then **/usr/xeno/foo/myFile** is accessed from VxWorks as **/foo/myFile**.

For example:

```
fd=open("/foo/myFile", O_CREAT | O_RDWR, 0);
```

Non-NFS Network File System Access from VxWorks With FTP or RSH

To access non-NFS file systems over a network with FTP or RSH, the name of the remote machine must be specified by prefixing the file system path with the remote machine name followed by a colon.

The exception to this rule is if the remote machine is a Linux host system, the host and colon prefix is not required (but may still be used). By default VxWorks uses the host system as the default device if it cannot find a matching device name, which works for Linux hosts. It does not work, however, with a Windows machine because VxWorks interprets Windows drive letters as device names, and they must therefore be disambiguated with the host name and colon prefix.

Note that for the files of a remote host to be accessible with RSH or FTP, permissions and user identification must be established on both the remote and local systems.

If the file **/usr/xeno/foo/myfile** on the Linux machine **mars** and the file **C:\bar\myotherfile** on Windows machine **jupiter** would be identified as follows:

```
mars:/usr/xeno/foo/myfile

jupiter:C:\bar\myotherfile
```

And the paths used accordingly:

```
fd=open("mars:/usr/xeno/foo/myfile", O_CREAT | O_RDWR, 0);

fd=open("jupiter:C:\bar\myotherfile", O_CREAT | O_RDWR, 0);
```


WNDRVR

If **mars** is the host machine (for Linux), however, the following usage (without the machine name and colon) would also work:

```
fd=open("/usr/xeno/foo/myfile", O_CREAT | O_RDWR, 0);
```

Remote System Name Definition

For the host system, the name is defined with the host name boot parameter (for which the default is simply host). For other remote machines, a network device must first be created by calling the kernel function netDevCreate( ).

Host File System Access from the VxWorks Simulator

The host file system is accessed from the VxWorks simulator by way of the pass-through file system (passFs). For more information about the VxWorks simulator, see the *VxWorks 7 VxSim User's Guide*.

For a Linux host, the path must be prefixed with name of this host on which the simulator is running, followed by a colon. For example:

```
mars:/usr/xeno/foo/myfile
```

For a Windows hosts, the path must be prefixed with literal string **host** followed by a colon. For example:

```
host:/C/bar/myotherfile
```

A colon may be used after the Windows drive letter, but this use is deprecated and is retained only for backward-compatibility reasons.

Note: Do not confuse the **host:** prefix used with the VxWorks simulator on a Windows host with the default host device name. The default host name is specified with the host name boot loader parameter, and it appears as **host:** in the list generated by the VxWorks shell devs command.

Note: The VxWorks simulator's pass-through file system requires forward-slash path delimiters, even on a Windows host.

Parent topic: NFS and Remote File Systems on page 86

5.2. Non-NFS Network Device File Systems

VxWorks supports network access to files on a remote host through the Remote Shell protocol (RSH) or the File Transfer Protocol (FTP).

These implementations of net-work devices use the driver **netDrv**, which is included in the Wind River Network Stack. Using this driver, you can open, read, write, and close files located on remote systems without needing to manage the details of the underlying protocol used to effect the transfer of information.

When a remote file is opened using RSH or FTP, the entire file is copied into local memory. As a result, the largest file that can be opened is restricted by the available memory. Read and write operations are performed on the memory-resident copy of the file. When closed, the file is copied back to the original remote file if it was modified.

In general, NFS devices are preferable to RSH and FTP devices for performance and flexibility, because NFS does not copy the entire file into local memory. However, NFS is not supported by all host systems.

Note: Within processes, there are limitations on RSH and FTP usage: directories cannot be created and the contents of file systems cannot be listed.

WNDRVR

Creating Network Devices

To access files on a remote host using either RSH or FTP, a network device must first be created by calling the kernel function netDevCreate( ).

The arguments to netDevCreate( ) are (1) the name of the device, (2) the name of the host the device accesses, and (3) which protocol to use: 0 (RSH) or 1 (FTP). By convention, the name for a network device is the remote machine's name followed by a colon (for example **mars:**).

Files on a network device can be created, opened, and manipulated as if on a -local disk. Note that creating a network device allows access to any file or device on the remote system, while mounting an NFS file system allows access only to a specified file system.

For the files of a remote host to be accessible with RSH or FTP, permissions and user identification must be established on both the remote and local systems. Creating and configuring network devices is discussed in detail in *Wind River Network Stack Programmer's Guide: File Access Applications* and in the API reference entry for **netDrv**.

For example, the following call creates an RSH device called **mars:** that accesses the host **mars**. Opening the file **mars:/usr/foo** actually opens **/usr/foo** on host **mars**.

```
netDevCreate ("mars:", "mars", 0);
```

I/O Control Functions

RSH and FTP devices respond to the same ioctl( ) functions as NFS devices except for **FIOSYNC** and **FIOREADDIR**. The functions are defined in the header file **ioLib.h**. For more information, see the API reference entries for **netDrv** and ioctl( ).

Parent topic: NFS and Remote File Systems on page 86

5.3. Network File System (NFS) Devices Overview

Network File System (NFS) devices allow files on remote hosts to be accessed with the NFS protocol. The NFS protocol specifies both *client* software, to read files from remote machines, and *server* software, to export files to remote machines.

The driver **nfsDrv** acts as a VxWorks NFS client to access files on any NFS server on the network. VxWorks also allows you to run an NFS server to export files to other systems.

Using NFS devices, you can create, open, and access remote files exactly as though they were on a file system on a local disk. This is called *network transparency*. For more information, see *Overview of NFS on page 86*.

I/O Control Functions for NFS Clients

NFS client devices respond to the ioctl( ) functions described in the table below. The functions listed are defined in **ioLib.h**. For more information, see the reference entries for **nfsDrv**, **ioLib**, and ioctl( ).

IOCTL	Description
FIOGETNAME	Gets the file name of **fd** and copies it to the buffer referenced by **nameBuf**:

WNDRVR

IOCTL	Description
	`status = ioctl (fd, FIOGETNAME, &nameBuf);`
FIONREAD	Copies to **nBytesUnread** the number of bytes remaining in the file specified by **fd**: `status = ioctl (fd, FIONREAD, &nBytesUnread);`
FIOSEEK	Sets the current byte offset in the file to the position specified by **newOffset**. If the seek goes beyond the end-of-file, the file grows. The end-of-file pointer gets moved to the new position, and the new space is filled with zeros: `status = ioctl (fd, FIOSEEK, newOffset);`
FIOSYNC	Flush data to the remote NFS file. It takes no additional argument: `status = ioctl (fd, FIOSYNC, 0);`
FIOWHERE	Returns the current byte position in the file. This is the byte offset of the next byte to be read or written. It takes no additional argument: `position = ioctl (fd, FIOWHERE, 0);`

FIOREADDIR	Reads the next directory entry. Use the third argument in the ioctl( )call to supply a pointer to a directory descriptor of type **DIR**.
	`DIR dirStruct;`
	`fd = open ("directory", O_RDONLY);`
	`status = ioctl (fd, FIOREADDIR, &dirStruct);`
	Normally, you do not use the **FIOREADDIR** functionality directly. Instead, you would call readdir( ). See the reference entry for **dirLib**.

FIOFSTATGET	Gets file status information (directory entry data). Use the third argument in the ioctl( )call to supply a pointer to a **stat** structure that is filled with data describing the specified file. For example:
	`struct stat statStruct;`
	`fd = open ("file", O_RDONLY);`
	`status = ioctl (fd, FIOFSTATGET, &statStruct);`
	Normally, you do not use the **FIOFSTATGET** functionality directly. Instead, you would stat( ) or fstat( ) functions get file information. See the manual entry for **dirLib**.

IOCTL	Description

FIOFSTATFSGET	Gets the file system parameters for and open file descriptor. Use the third argument in the ioctl( )call to supply a pointer to a **statfs** structure that is filled with data describing the underlying file system.
	`statfs statfsStruct;`
	`fd = open ("directory", O_RDONLY);`
	`status = ioctl (fd, FIOFSTATFSGET, &statfsStruct);`
	Normally, you do not use the **FIOFSTATFSGET** functionality directly. Instead, you would stat( ) or fstat( ) functions get file information. See the manual entry for **dirLib**.

Parent topic: NFS and Remote File Systems on page 86

5.4. VSB Project for NFS

The primary VSB layer for NFS, which is FS_NFS, is enabled by default.

The NFS3_RWOPT option provides the ability to optimize read/write operations in NFS3 server; however, note that some NFS3 client versions on Linux or UNIX may not work if it is turned on.

The NFS3_BLKSIZE_FACTOR option provides a factor to configure the maximum block size that can be transferred by a NFS3 server. The block size is equal to the factor multiplied by 8192 bytes. A larger factor takes more memory.

Note: The VSB option **FILE_CREATION_MASK** changes the default permission from 0777 to 0755. This may cause permission checking failure on the NFS client side. Disable this option or set the VIP parameter **UMASK_DEFAULT**to 000 when you need to enable the NFS client component.

Parent topic: NFS and Remote File Systems on page 86

5.5. Configuring VxWorks With an NFS Client

VxWorks supports several NFS client configuration components, which you can use to include the modules needed to create an NFS client

- Core NFS Client
- NFS Client All
- NFS v2 Client
- NFS v3 Client
- NFS Mount All

These components are described in detail below.

Core NFS Client

The **INCLUDE_CORE_NFS_CLIENT** component provides **nfsCommon**, a library that provides core functionality for NFS clients. This library also provides parameters that you can use to specify the NFS user ID, group identifier, and the maximum length of the pathname for a file.

WNDRVR

This component contains the following configuration parameters:

NFS_USER_ID

Synopsis: User identifier for NFS access.

Default: 2001

NFS_GROUP_ID

Synopsis: Group identifier for NFS access.

Default: 100

NFS_MAXPATH

Synopsis: Maximum file path length.

Default: 255

NFS_CLIENT_AUTO_CLOSE

Synopsis: When set to TRUE, close NFS client automatically after file close. Setting it to FALSE can help improve NFS performance by saving time to re-setup the NFS client.

Default: FALSE

NFS Client All

Including the **INCLUDE_NFS_CLIENT_ALL** component is equivalent to selecting both **INCLUDE_NFS2_CLIENT** and **INCLUDE_NFS3_CLIENT**, the components that pull in client support for NFS version 2 and NFS version 3 respectively.

This component requires the **INCLUDE_NFS2_CLIENT** and **INCLUDE_NFS3_CLIENT** components.

NFS v2 Client

The **INCLUDE_NFS2_CLIENT** component provides **nfs2Lib** and **nfs2Drv**, which provide support of an NFS version two client. Using this client, you can mount exported NFS (version two) directories to your VxWorks target.

This component requires the **INCLUDE_CORE_NFS_CLIENT** component.

This component initializes the NFS v2 client at boot time with a call to nfs2Drv( ), as well as a call to nfsAuthUnixSet( ) as follows:

```
nfsAuthUnixSet ( sysBootParams.hostName, NFS_USER_ID, NFS_GROUP_ID, 0,
(int *) 0);
```

Values for the NFS_USER_ID and NFS_GROUP_ID parameters are taken from the corresponding **INCLUDE_CORE_NFS_CLIENT** component parameters. See Core NFS Client on page 91.

NFS v2 Client Configuration Parameters

This component contains the following configuration parameters:

NFS2_CLIENT_CACHE_DEFAULT_NUM_LINES

Synopsis: Default number of cache lines.

WNDRVR

Default: 16

The NFS client implementation uses a persistent cache, which is structured as a multi-line buffer cache. This parameter configures the default number of lines in the NFS v2 client cache. The value can be decreased to as low as 1, or increased.

You can modify the number of cache lines, either at build time or at run-time. To configure the cache at run time, call the function usrNfs2CacheInit( ):

```
usrNfs2CacheInit (UINT32 numLines, UINT32 lineSize, UINT32 options);
```

NFS2_CLIENT_CACHE_DEFAULT_LINE_SIZE

Synopsis: Default number of bytes in cache line.

Default: 16384

The NFS client implementation uses a persistent cache, which is structured as a multi-line buffer cache. The cache line size must be a power of two.

This parameter configures the default size of the NFS v2 client cache line. The default value for this is 16384 (16 kB). When combined with the default number of cache lines, this yields a default cache size of 256 kB.

You can modify the number of cache line size, either at build time or at run-time. To configure the cache at run time, call the function usrNfs2CacheInit( ):

```
usrNfs2CacheInit (UINT32 numLines, UINT32 lineSize, UINT32 options);
```

Changing the size of the cache will not affect any existing cache. It will only impact future caches.

NFS2_CLIENT_CACHE_DEFAULT_OPTIONS

Synopsis: Default cache options.

Default: 0

This parameter configures the default options for the NFS v2 client cache. The two valid settings for this parameter are:

0

The default value of zero (0) means that the cache will collect the written data and only send it to the server when the cache line is full, or it needs to be flushed (no options), which is the default value

1, NFS_CACHE_WRITE_THROUGH

A value of one means that the cache will be write-through.

You can modify the cache options, either at build time or at run-time. To configure the cache at run time, call the function usrNfs2CacheInit( ):

```
usrNfs2CacheInit (UINT32 numLines, UINT32 lineSize, UINT32 options);
```

NFS v3 Client

The **INCLUDE_NFS3_CLIENT** component provides the **nfs3Lib** and **nfs3Drv** libraries, which provide support of an NFS version three client. Using this client, you can mount exported NFS (version three) directories to your VxWorks target.

This component requires the **INCLUDE_CORE_NFS_CLIENT** component.

This component initializes the NFS v3 client at boot time with a call to nfs3Drv( ), as well as a call to nfsAuthUnixSet( ) as follows:

WNDRVR

```
nfsAuthUnixSet ( sysBootParams.hostName, NFS_USER_ID, NFS_GROUP_ID, 0,
(int *) 0);
```

Values for the **NFS_USER_ID** and **NFS_GROUP_ID** parameters are taken from the corresponding **INCLUDE_CORE_NFS_CLIENT** component parameters. See Core NFS Client on page 91.

NFS v3 Client Parameters

This component contains the following configuration parameters:

NFS3_CLIENT_CACHE_DEFAULT_NUM_LINES

Synopsis: Default number of cache lines.

Default: 16

The NFS client implementation uses a persistent cache, which is structured as a multi-line buffer cache. This parameter configures the default number of lines in the NFS v3 client cache. The value can be decreased to as low as 1, or increased.

You can modify the number of cache lines, either at build time or at run-time. To configure the cache at run time, call the function usrNfs3CacheInit( ):

```
usrNfs3CacheInit (UINT32 numLines, UINT32 lineSize, UINT32 options);
```

NFS3_CLIENT_CACHE_DEFAULT_LINE_SIZE

Synopsis: Default number of bytes in cache line.

Default: 16384

The NFS client implementation uses a persistent cache, which is structured as a multi-line buffer cache. The cache line size must be a power of two.

This parameter configures the default size of the NFS v3 client cache line. The default value for this is 16384 (16 kB). When combined with the default number of cache lines, this yields a default cache size of 256 kB.

You can modify the number of cache line size, either at build time or at run-time. To configure the cache at run time, call the function usrNfs3CacheInit( ):

```
usrNfs3CacheInit (UINT32 numLines, UINT32 lineSize, UINT32 options);
```

Changing the size of the cache will not affect any existing cache. It will only impact future caches.

NFS3_CLIENT_CACHE_DEFAULT_OPTIONS

Synopsis: Default cache options.

Default: 0

This parameter configures the default options for the NFS v3 client cache. The two valid settings for this parameter are:

0

The default value of zero (0) means that the cache will collect the written data and only send it to the server when the cache line is full, or it needs to be flushed (no options), which is the default value

1, NFS_CACHE_WRITE_THROUGH

A value of one means that the cache will be write-through.

WNDRVR

You can modify the cache options, either at build time or at run-time. To configure the cache at run time, call the function usrNfs3CacheInit( ):

```
usrNfs3CacheInit (UINT32 numLines, UINT32 lineSize, UINT32 options);
```

Configuring NFS3 Client Writes

The NFS v3 client has one additional configurable parameter that is not available on NFS v2. According to the RFC, the NFS v3 client can dictate to an NFS v3 server how it should perform the write operations. At runtime, the NFS v3 client can be set to inform the server that it should perform writes one of the following styles:

- **UNSTABLE**
- **FILE_SYNC**
- **DATA_SYNC**

The default setting is **UNSTABLE**.

You can use the nfs3StableWriteGet( ) and nfs3StableWriteSet( ) functions to configure these options at run-time.

NFS Mount All

The **INCLUDE_NFS_MOUNT_ALL** component configures a VxWorks target to make a boot-time call to nfsMountAll( ). This function automatically mounts all file systems exported by the host from which VxWorks is booted. This component requires the **INCLUDE_NFS_CLIENT_ALL** component. This component contains the following configuration parameter:

GROUP_EXPORTS

Synopsis: Remove check for GroupName during MountAll.

Default: FALSE

An NFS server can specify that an exported file system is intended for particular group of clients. Before mounting an exported file system, a client typically checks that it is a group member. If the client is not a group member, it does not mount the file system. To configure the client to skip this check, set this parameter to TRUE. The default of FALSE ensures that the check occurs.

Parent topic: NFS and Remote File Systems on page 86

5.6. Setting Up an NFS Client

Set up an NFS client and use it to access a remote NFS system.

Overview of Steps

To set up an NFS client and use it to access a remote NFS system, you must perform the following steps, which are described in detail below:

1. If you have not set the user name, user ID, and group name required for authentication with the **INCLUDE_CORE_NFS_CLIENT** configuration parameters, you must call nfsAuthUnixSet( ) to do so.
2. Call hostAdd( ) to add remote NFS systems to the VxWorks' host table.
3. Call nfsMount( ) or nfsMountAll( ) to mount the remote file systems.

WNDRVR

Parent topic: NFS and Remote File Systems on page 86

5.7. Setting the NFS Client Name, User ID, and Group ID

If you have not set the user name, user ID, and group name required for authentication with **INCLUDE_CORE_NFS_CLIENT** component parameters, you must call nfsAuthUnixSet( ) to do so.

Internally, NFS depends upon RPC to handle the remote execution of the commands (open, read, write, and others) that access the data in the remote file system. Associated with the RPC protocol is an authentication system known as AUTH_UNIX. This authentication system requires RPC peers to provide a user name, a user ID, and a group name. The recipient of an RPC message uses this information to decide whether to honor or ignore the RPC request.

For VxWorks, you can set the NFS user name, user ID, and group name using the NFS_GROUP_ID and NFS_USER_ID parameters of the **INCLUDE_CORE_NFS_CLIENT** component. VxWorks calls nfsAuthUnixSet( ) automatically at boot time using these values.

You can also set these values at runtime by calling nfsAuthUnixSet( ) or nfsAuthUnixPrompt( ). For example, to use nfsAuthUnixSet( ) to set the NFS user ID to 1000 and the NFS group ID to 200 for the machine mars, you would call nfsAuthUnixSet( ) as follows:

```
-> nfsAuthUnixSet "mars", 1000, 200, 0
```

The nfsAuthUnixPrompt( ) function provides a more interactive way of setting the NFS authentication parameters from the shell.

Parent topic: NFS and Remote File Systems on page 86

5.8. Adding Remote NFS System to Host Table

Call hostAdd( ) to add remote NFS systems to the VxWorks' host table.

VxWorks automatically calls hostAdd( ) for the host from which it is booted. To mount file systems from other remote NFS server systems, you must make explicit hostAdd( ) calls to add them to the list of hosts known to the VxWorks system. Use the following syntax:

```
hostAdd ("host", "IPaddress" )
```

For example, if you wanted to mount a file system exported by a system called mars, which add the address 192.168.10.1, you would first need to add it to the host table by calling hostAdd( ) as follows:

```
hostAdd ("mars", "192.168.10.1" )
```

Parent topic: NFS and Remote File Systems on page 86

5.9. Mounting a Remote File System

Call nfsMount( ) or nfsMountAll( ) to mount a remote file systems.

Access to a remote NFS file system is established by mounting that file system locally and creating an I/O device for it by calling nfsMount( ), or by calling nfsMountAll( ) to mount all file systems that are exported from the remote system (but without the option of changing their names).

If you have included the **INCLUDE_NFS_MOUNT_ALL** component, VxWorks automatically calls nfsMountAll( ) to mount all file systems exported by the host from which VxWorks is booted.

To unmount a file system, use nfsUnmount( ). Use nfsDevShow( ) to display a list of the mounted NFS devices.

WNDRVR

Using nfsMount( )

The syntax of an nfsMount( ) call is as follows:

```
nfsMount ("hostName", "hostFileSys", "localName")
```

hostName

The host name of the NFS system that exports the file system you want to mount.

hostFileSys

The name of the host file system or subdirectory as it is known on the exporting NFS system.

localName

The local name to assign to the file system.

For example, the following call mounts the remote file system **/usr** from the NFS system mars locally as **/vxusr**:

```
nfsMount ("mars", "/usr", "/vxusr");
```

This creates a VxWorks I/O device with the specified local name (**/vxusr**, in this example). After a remote file system is mounted, the files can be accessed as though the file system were local.

Thus, for the preceding example, opening the file **/vxusr/foo** opens the file /usr/foo on the NFS system mars.

Using nfsMountAll( )

If you do not need to mount the remote file system under a new name, consider using nfsMountAll( ) instead of nfsMount( ). A call to nfsMountAll( ) mounts all file systems that are exported from the remote NFS system and that are accessible to the specified client.

The syntax of nfsMountAll( ) is as follows:

```
nfsMountAll( "hostName", "clientName", quietFlag )
```

hostName

The name of the NFS system from which you want to mount all exported file systems.

clientName

The name of a client specified in an access list, if any. A NULL clientName mounts only those file systems that are accessible to any client.

quietFlag

A boolean value that tells nfsMountAll( ) whether to execute in verbose or silent mode. FALSE indicates verbose mode, and TRUE indicates quiet mode.

Parent topic: NFS and Remote File Systems on page 86

5.10. Configuring VxWorks With an NFS Server

VxWorks supports several NFS server configuration components, which you can use to include the modules needed to create an NFS server.

WNDRVR

The modules provide:

- NFS Server
- NFS server All
- NFS server V2
- NFS server V3

NFS Server

The **INCLUDE_CORE_NFS_SERVER** component provides **nfsd**, the library that supplies the NFS server daemon initialization functions.

For more information on these functions, see the nfsd reference entry. This component also supplies configuration parameters that set basic configuration values for the server.

Note:

The NFS server maintains a reply cache of up to 64 recently transmitted reply messages. The purpose of this cache is to deal with RPC retransmissions. If the XID of an incoming NFS request matches that of a previously replied message, the cached reply is sent back instead of actually executing the NFS request.

Each entry in the server reply cache requires 8800 bytes for the entry itself and 48 bytes for each cache node. Thus, if there were three entries in the cache, the memory usage would be 26,544 bytes, which is (3 * 8800) + (3 * 48) bytes. If there were the full 64 entries in the cache, the memory usage would be 566,272 bytes.

If you think you have noticed a memory leak in NFS, please consider whether this memory use accounts for the discrepancy.

This component contains the following configuration parameters:

NFS_MAXPATH

Synopsis: Maximum length of a file path string (excludes file name).

Default: 255

Maximum: 1024

See Using Larger Values with NFS_MAXPATH and NFS_MAXFILENAME on page 99.

NFS_USER_ID

Synopsis: User identifier for NFS access.

Default: 2001

NFS_MAXFILENAME

Synopsis: Maximum file name length.

Default: 40

Maximum: 255

This parameter specifies the maximum length of a filename. If a filename is longer than this value then it is truncated before returning it to the client. This parameter should be set carefully since it affects the amount of memory allocated by the server. See Using Larger Values with NFS_MAXPATH and NFS_MAXFILENAME on page 99.

NFS_GROUP_ID

Synopsis: Group identifier for NFS access.

WNDRVR

Default: 100

Using Larger Values with NFS_MAXPATH and NFS_MAXFILENAME

If you use values larger than the defaults with the NFS_MAXPATH and NFS_MAXFILENAME parameters, you should increase the stack size for NFS server and client accordingly.

For example, if you increase both to their maximum values, you should also do the following:

- For the NFS server, increase the global integer variable nfsdStackSize from 14000 to 32000 before the call to nfsdInit( ), which is called from usrNfsdInitInternal( ) in **usrNfsd.c**.

  This can be done be modifying **nfsd.c** or **usrNfsd.c**. Alternatively, it can be accomplished by adding the following to sysHwInit( ) in sysLib.c( ):

  ```
  extern int nfsdStackSize;
  ...
  nfsdStackSize = 32000;
  ```

- For the NFS client, make sure that the task that calls nfsMount( ), read( ), and write( ) has an adequate stack size (for example, about four times of normal stack size).

NFS server All

Including the **INCLUDE_NFS_SERVER_ALL** component is equivalent to selecting both **INCLUDE_NFS3_SERVER** and **INCLUDE_NFS2_SERVER**.

This component requires the **INCLUDE_NFS3_SERVER**, **INCLUDE_NFS2_SERVER,** and **INCLUDE_NFS_SERVER_INSTALL** components.

NFS server V2

The **INCLUDE_NFS2_SERVER** component provides **nfs2dLib** and **mount1Lib**.

The **INCLUDE_NFS2_SERVER** component requires the **INCLUDE_CORE_NFS_SERVER** and **INCLUDE_NFS_SERVER_INSTALL** components.

Note: VxWorks does not normally provide authentication services for NFS requests. If you need to authenticate incoming requests, see the documentation for nfsdInit( ) and mountdInit( ) for information about authorization hooks.

To actually export a file system, you must call nfsExport( ).

Table 1 on page 100 lists the requests that RFC 1813 accepted from clients. For details of their use, see *RFC 1094, NFS: Network File System Protocol Specification*.

NFS server V3

The **INCLUDE_NFS3_SERVER** component provides **nfs3dLib** and **mount3Lib**. The **INCLUDE_NFS3_SERVER** component requires the **INCLUDE_CORE_NFS_SERVER** and **INCLUDE_NFS_SERVER_INSTALL** components.

To export a file system, you must call nfsExport( ). See Exporting NFS File Systems from a VxWorks System on page 101.

Note: VxWorks does not normally provide authentication services for NFS requests. If you need to authenticate incoming requests, see the documentation for nfsdInit( ) and mountdInit( ) for information about authorization hooks.

WNDRVR

The following requests are accepted from clients. For details of their use, see *RFC 1813: NFS: Network File System Protocol Specification*.

Table 1. RFC 1813 Supported Client Requests

Procedure Name	Procedure Number
NFSPROC_NULL	0
NFSPROC_GETATTR	1
NFSPROC_SETATTR	2
NFSPROC_LOOKUP	3
NFSPROC_ACCESS	4
NFSPROC_READLINK	5 – not supported, limitation in DOSFS
NFSPROC_READ	6
NFSPROC_WRITE	7
NFSPROC_CREATE	8
NFSPROC_MKDIR	9
NFSPROC_SYMLINK	10 – not supported
NFSPROC_MKNOD	11 – not supported
NFSPROC_REMOVE	12
NFSPROC_RMDIR	13
NFSPROC_RENAME	14
NFSPROC_LINK	15 – not supported
NFSPROC_READDIR	16
NFSPROC_READDIRPLUS	17
NFSPROC_FSSTAT	18
NFSPROC_FSINFO	19
NFSPROC_PATHCONF	20
NFSPROC_COMMIT	21

Parent topic: NFS and Remote File Systems on page 86

5.11. Exporting NFS File Systems from a VxWorks System

To make a file system on a VxWorks system accessible to NFS clients on a network, you must call nfsExport( ).

For example, the following exports the VxWorks file system **/foo**:

```
nfsExport ("/foo", 0, FALSE, 0);
```

The first three arguments specify the name of the file system to export; the VxWorks NFS export ID (0 means to assign one automatically); and whether to export the file system as read-only. The last argument is a placeholder for future extensions.

The following code fragment creates a RAM drive, initializes it for dosFs, and exports the file system for NFS clients on the network:

```
unsigned myBlockSize; /* block size in bytes */
unsigned myTotalSize; /* disk size in bytes */

myBlockSize = 512;
myTotalSize = 16777216; /* 16Mb */

xbdRamDiskDevCreate (myBlockSize, myTotalSize, FALSE, "/ramDrv");
dosFsVolFormat ("/ramDrv", 2, 0);
nfsExport ("/ramDrv", 0, FALSE, 0);
```

CAUTION: **For NFS-exportable file systems, the device name must not end in a slash.**

CAUTION: **The NFS server cannot export a VxWorks host file system (that is, the system from which VxWorks is booted). NFS servers can export only local file systems. Use the appropriate command on the server to see which file systems are local.**

For information about mounting a file system from a VxWorks client, see Setting Up an NFS Client on page 95. For other operating systems, see the appropriate system documentation. For example, on a typical UNIX system, the following command (executed with root privilege) mounts the /export file system from the VxWorks system vxTarget, using the name **/mnt** for it on UNIX:

```
-> /etc/mount vxTarget:/export /mnt
```

Parent topic: NFS and Remote File Systems on page 86

5.12. Exporting NFS File Systems From a UNIX System

For a UNIX NFS server, the **/etc/exports** file specifies which of the server's file systems are exported for mounting by remote NFS clients.

About This Task

If a file system on a UNIX NFS server is not listed in **/etc/exports**, the file system is not exported, which means other machines cannot use NFS to mount it. For example, consider an **/etc/exports** file that contains the line:

```
/usr
```


WNDRVR

The server exports **/usr** without restriction. If you want to limit access to this directory, you can include additional parameters on the line. For example:

Procedure

1. On the UNIX box, login as root (super user).
2. Edit **/etc/exports**.
3. Specify the path and permission for the file system that you would export.

   For example:

   ```
   /usr * (rw)
   ```

   For more information on these parameters, consult your UNIX system documentation.

4. Export the file system with the following command:

   ```
   # exportfs -ra
   ```

5. On the UNIX target, run the NFS daemon:

   ```
   # rpc.nfsd
   ```

6. Run: rpc.rquotad
7. To mount the daemon, run:

   ```
   # rpc.mountd
   ```

8. To check whether NFS is running, use:

   ```
   # rpcinfo -p.
   ```

Parent topic: NFS and Remote File Systems on page 86

5.13. Exporting NFS File Systems from a Windows System

Windows systems support NFS. It is therefore possible to configure a directory on a Windows system so that it is exported over NFS.

The exact procedures for doing so depend upon the particular network package you purchased. For more information, consult the documentation included with your Windows networking package.

Parent topic: NFS and Remote File Systems on page 86

5.14. Exporting NFS File Systems From a UNIX System

For a UNIX NFS server, the **/etc/exports** file specifies which of the server's file systems are exported for mounting by remote NFS clients.

About This Task

If a file system on a UNIX NFS server is not listed in **/etc/exports**, the file system is not exported, which means other machines cannot use NFS to mount it. For example, consider an **/etc/exports** file that contains the line:

```
/usr
```

The server exports **/usr** without restriction. If you want to limit access to this directory, you can include additional parameters on the line. For example:

Procedure

1. On the UNIX box, login as root (super user).
2. Edit **/etc/exports**.
3. Specify the path and permission for the file system that you would export.

   For example:

   ```
   /usr * (rw)
   ```

   For more information on these parameters, consult your UNIX system documentation.

4. Export the file system with the following command:

   ```
   # exportfs -ra
   ```

5. On the UNIX target, run the NFS daemon:

   ```
   # rpc.nfsd
   ```

6. Run: rpc.rquotad
7. To mount the daemon, run:

   ```
   # rpc.mountd
   ```

8. To check whether NFS is running, use:

   ```
   # rpcinfo -p.
   ```

Parent topic: NFS and Remote File Systems on page 86

5.15. Configuring VxWorks for NFS Debugging

Use the NFS debugging component with NFS mount, NFS client, or NFS server, as required. The component provides for five debug levels.

Before You Begin

You must know how to use Workbench or wrtool to create and configure VSB and VIP projects. For information in this regard, see the *VxWorks 7 Configuration and Build Guide*.

Procedure

1. As part of your NFS VIP project configuration, add the **NFS_DEBUG_LEVEL** component.
2. Set the **NFS_DEBUG_LEVEL** configuration parameter to the appropriate level.

WNDRVR

Macro	Value
NFS_NONE_DEBUG	0
NFS_ERROR_DEBUG	1
NFS_NORMAL_DEBUG	2
NFS_WARNING_DEBUG	3
NFS_VERBOSE_DEBUG	4

The debug output includes all levels with a lesser value. For example, if this value is set to **NFS_ERROR_DEBUG** (1), then debug output lists errors. If it is set to **NFS_WARNING_DEBUG** (2), then the debug output lists errors and warnings. The default value is 0, or **NFS_NONE_DEBUG**.

At runtime, debug information is output according to the level you have set wit this configuration.

Parent topic: NFS and Remote File Systems on page 86

6. HostFs

The debug agent host file system (HostFS) is designed for development and diagnostic purposes. It is a full-featured VxWorks file system, but the files are actually located on the host system.

HostFS provides all of the I/O features of the network driver for remote file access (see the discussion of non-NFS network devices in VxWorks Devices and Driver Model Overview on page 167), without requiring any target resources—except those required for communication between the target system and Workbench tools on the host. HostFS is an I/O driver that communicates with Workbench tools through the debug agent. The Workbench tools make the host file system visible. When you open a file with HostFS, the file being opened is actually on the host. Subsequent read( ) and write( ) calls on the file descriptor obtained from the open( ) call read from and write to the opened host file.

The HostFS driver provides typical I/O features like open( ), close( ), read( ), write( ), and ioctl( ). Not all ioctl functions are supported: it is not possible to create or delete a directory, and it is not possible to rename a file. HostFS does not require any target resource beyond what is already configured to support communication between target system and Workbench.

It is possible to access host files randomly without copying the entire file to the target, to load an object module from a virtual file source, and to supply the filename to functions such as ld( ) and copy( ).

Each I/O request, including open( ), is synchronous; the calling target task is blocked until the operation is complete.

Error Handling on page 104 HostFs errors are reported back to the VxWorks target.
HostFs Implementation and Use on page 105 To implement HostFs, you must configure VxWorks with the debug agent, start the agent on the target, and create a target connection from the host.
Security Considerations on page 106 While HostFS has much in common with common network file systems, the security considerations are different.

6.1. Error Handling

HostFs errors are reported back to the VxWorks target.

Errors can arise at various points within HostFS and are reported back to the original caller on the target, along with an appropriate error code. The error code returned is the VxWorks errno which most closely matches the error experienced on the host.

Parent topic: HostFs on page 104

6.2. HostFs Implementation and Use

To implement HostFs, you must configure VxWorks with the debug agent, start the agent on the target, and create a target connection from the host.

VxWorks Configuration

Configure VxWorks with **DEBUG_AGENT** in your VSB project and **INCLUDE_DEBUG_AGENT** in your VIP project.

If you want to have the debug agent start automatically at boot time, add **INCLUDE_DEBUG_AGENT_START** or **INCLUDE_STOP_MODE_AGENT_START**.

CAUTION: **For security reasons, Wind River recommends that you only use the automatic start feature for development, and not for deployed systems. Furthermore, you should in any case shut down the debug agent when you are not using HostFs.**

Starting the Debug Agent

If you have not configured VxWorks to start the debug agent automatically, you can do so with either of the following methods:

- Use the the 0x400 VxWorks boot flag. The boot flag can be set in the VxWorks **DEFAULT_BOOT_LINE** configuration parameter, or the U-Boot **bootargs** variable (as f=0x400), or in the flattened device tree (FTD).
- Use debugAgentStart( ) or stopModeAgentStart( ), from the kernel shell or programmatically.

Creating a Workbench Connection to the Target

From the Workbench tool bar, select **New Connection** from the drop-down list, and then use the VxWorks connection dialog to configure the host-target connection. If **Connect on finish** is selected, the connection is started after you click on **Finish**. Otherwise, select the name of the configuration you have created from the drop-down list, and click on the **Connect** button to the right of it.

HostFs File System Contents

On the VxWorks system, HostFs is created as **/hostfs**, with the **ws** and **root** subdirectories.

Workbench exports the following directories to HostFS:

- The current Workbench workspace, under **/hostfs/ws**, with read/write access.
- The host root file system, under **/hostfs/root**, with read-only access.

For Linux, the root **/** tree appears directly under **/hostfs/root**.

For Windows, all drives are exported. For example, **c:** is visible under **/hostfs/root/C**.

Stopping the Debug Agent

If you have not configured VxWorks to start the debug agent automatically, reboot the target to stop it.

WNDRVR

If you have configured VxWorks to start the debug agent automatically, you must reconfigure VxWorks and remove **INCLUDE_DEBUG_AGENT_START** and **INCLUDE_STOP_MODE_AGENT_START** to be able to stop the agent.

Parent topic: HostFs on page 104

6.3. Security Considerations

While HostFS has much in common with common network file systems, the security considerations are different.

With HostFS, the host file operations are done on behalf of the user that launched Workbench, with the restriction that only the workspace is fully accessible, and the rest of the host file system is read only. The user name given to the target as a boot parameter has no effect. In fact, none of the boot parameters have any effect on the access privileges of HostFS.

For information on non-NFS network devices, see VxWorks Devices and Driver Model Overview on page 167.

Parent topic: HostFs on page 104

7. True Flash File System (TrueFFS)

TrueFFS is a flash management facility that provides access to flash memory by emulating disk access.

TrueFFS is not really a file system; it is a translation layer between the Flash media and a file system such as HRFS or DosFs. Flash translation layers must do wear leveling and bad block management functions that are similar to a filesystem, however, a filesystem is always required on the top. VxWorks provides two file translation layers for Flash: NAND FTL is for NAND flash (see NAND Flash Translation Layer (NFTL) on page 121) and the original TrueFFS FTL is used for NOR and SPI flash.

TrueFFS provides VxWorks with block device functionality, which allows the dosFs file system to be used with flash memory in the same manner as a disk. For information about dosFs, see dosFs on page 40.

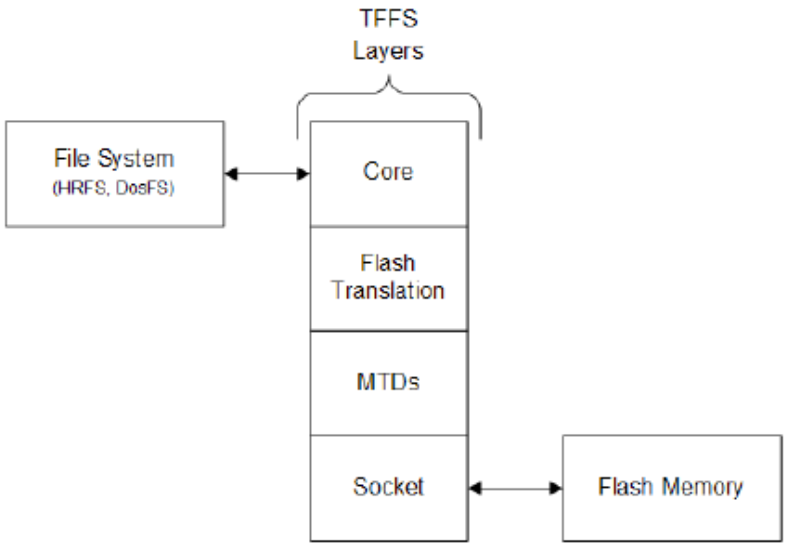
In addition, TrueFFS provides full flash media management capabilities.

TrueFFS is a VxWorks-compatible implementation of M-Systems FLite, version 2.0. This system is reentrant, thread-safe, and supported on all CPU architectures that host VxWorks. TrueFFS consists of the following four layers:

- The core layer, which connects the other layers and handles global facilities, such as back-grounding, garbage collection, timers, and other system resources. This layer provides the block device interface for a file system.
- The flash translation layer, which maintains the block allocation map that associates the file system's view of the storage medium with erase blocks in flash.
- The Memory Technology Device (MTD) layer, which implements the low-level programming of the flash medium (map, read, write, and erase functions).
- The socket layer, which provides an interface between TrueFFS and the board hardware with board-specific hardware access functions.

The following figure illustrates the relationship between the file system, TrueFFS layers, and the flash medium itself.

Figure 7. File System, TrueFFS Layers, and Flash



The topics in this section provide instructions for using TrueFFS with the MTDs and drivers. Also provided in this section is quick-start material for configuring TrueFFS and formatting TrueFFS drives, and thus presents the basic steps required to use the default TrueFFS facilities with your application. It also provides information about creating a boot image region that excludes TrueFFS, and about writing the boot image to that region.

For information on customizing or creating new socket drivers or MTDs, see the *VxWorks 7 BSP and Driver's Guide:FLASH (NOR) Devices*.

Note: This version of the TrueFFS product is a block device driver to VxWorks that, although intended to be file system neutral, is provided for use with the dosFs file.

Overview of Implementation Steps on page 108 The basic steps for implementing TrueFFS include selecting the MTD component and socket driver, configuring and building VSB and VIP projects, formatting and mounting the drives, and instantiating a file system.
Selecting an MTD on page 108 Use a MTD provided with the release, or modify one to better suit the needs of your system.
Identifying the Socket Driver on page 109 The socket driver that you include in your system must be appropriate for your BSP.
VSB Configuration on page 109BDM_FLASH is required (other TrueFFS options are optional). In addition, the options required for the file system you wish to use must be selected.
VIP Configuration on page 109 To configure VxWorks for TrueFFS with a file system, include TrueFFS components for basic support and each of the TrueFFS layers, the XBD block device wrapper component, and dosFs file system support.
Formatting the Flash on page 113 To format the flash for TrueFFS, boot the system and use either sysTffsFormat( ) or the tffsDevFormat( ).
Formatting With sysTffsFormat( ) on page 114 If your BSP provides the sysTffsFormat( ) function, you can use it from the kernel shell to format the flash.
Formatting With tffsDevFormat( ) on page 114 If your BSP does not provides the sysTffsFormat( ) function, use tffsDevFormat( ) from the kernel shell to format the flash.
Reserving a Region in Flash for a Boot Image on page 116 In order to use flash media for a VxWorks boot image as well as a file system, a portion of the flash memory (a fallow region) must be reserved so that it is excluded from the area subject to formatting and the run-time operations of TrueFFS.
Mounting the Drive on page 118 Use the usrTffsConfig( ) function to mount the a TrueFFS flash drive, which means that a new TFFS device is attached, and the TFFS layer is enabled for the NOR flash device.
Testing the Drive on page 119 You can test a drive by copying a file to it, then from it to another location, and then comparing the copy with the original.
Using TrueFFS With dosFs on page 119 Implementing TrueFFS with dosFs can be done programmatically, as well as interactively from the kernel shell.
TrueFFS With dosFs Code Example on page 120 This code fragment creates a TrueFFS block device, creates an XBD block device wrapper, and then formats the TrueFFS media for an dosFs file system.

TrueFFS With dosFs Shell Command Example on page 120 This example illustrate the procedure for implementing TrueFFS with dosFs from the kernel shell.

7.1. Overview of Implementation Steps

The basic steps for implementing TrueFFS include selecting the MTD component and socket driver, configuring and building VSB and VIP projects, formatting and mounting the drives, and instantiating a file system.

To determine if your BSP provides TrueFFS support, see the appropriate BSP Supplement Guide for your BSP in the Wind River Knowledge Library. You may need to write certain sub-components for your application. This is most often the case for the MTD layer. To determine if this release provides an MTD suitable for your flash hardware, see Selecting an MTD on page 108 and Including the MTD Component on page 110.

1. Select an MTD Component
   Choose an MTD, appropriate for your hardware, from those provided with the TrueFFS product. See Selecting an MTD on page 108.
2. Identify the Socket Driver
   Ensure that you have a working socket driver. The socket driver is a source code component, implemented in the file **sysTffs.c**. For VxWorks 6.9 legacy BSPs that already support TrueFFS, the socket driver is fully defined and located in the BSP directory. See Identifying the Socket Driver on page 109.
3. Configure the VSB Project
   Configure the VxWorks source code with the required facilities. See VSB Configuration on page 109.
4. Configure the VIP Project
   Configure your system for TrueFFS by adding the appropriate VxWorks components. Minimum support requires components for a file system and the four TrueFFS layers. See VIP Configuration on page 109.
5. Boot the Target and Format the Drives
   See Formatting the Flash on page 113.
6. Create a Flash Region for Boot Image
   Optionally, create a boot image region that excludes TrueFFS, and write the boot image to that region (this region can alternatively be used for a system startup log or configuration data). See Reserving a Region in Flash for a Boot Image on page 116.
7. Mount the TrueFFS flash drive
   See Mounting the Drive on page 118
8. Test the drive(s)
   See Testing the Drive on page 119.
9. Create a File System.
   The rawFs file system is instantiated by default. Create a dosFs file system (see Using TrueFFS With dosFs on page 119).

Parent topic: True Flash File System (TrueFFS) on page 106

7.2. Selecting an MTD

Use a MTD provided with the release, or modify one to better suit the needs of your system.

Determine whether any of the MTDs provided with this release support the device that you intend to use for TrueFFS. Devices are usually identified by their JEDEC IDs. If you find an MTD appropriate to your flash device, you can use that MTD. These drivers are also provided in binary form; so you do not need to compile the MTD source code unless you have modified it.

The *installDir* **/vxworks-7/pkgs_v2/storage/bdm/flash/tffs_drv/src** directory contains the source code for the following types of MTD components:

- MTDs that work with several of the devices provided by Intel, AMD, Fujitsu, and Sharp.
- Two generic MTDs that can be used for devices complying with CFI.

To better support the out-of-box experience, these MTDs attempt to cover the widest possible range of devices (in their class) and of bus architectures. Consequently, the drivers may not provide the performance you may need for the run-time environment that you want to target. Also note that your hardware configuration may require some changes to the generic drivers in order to work. Consider such requirements as word or byte access limitations, or unusual array configurations.

Note: For the list of the MTD components and details about adding the MTD component to your system, see Including the MTD Component on page 110.

Parent topic: True Flash File System (TrueFFS) on page 106

7.3. Identifying the Socket Driver

The socket driver that you include in your system must be appropriate for your BSP.

Some BSPs include socket drivers, others do not. The socket driver file is *installDir* **/vxworks-7/pkgs_v2/storage/bdm/flash/tffs/cfg/sysTffs.c**. If it is in a 6.9 compatible BSP, the socket driver is located in your BSP directory. For more information, see Including the Socket Driver on page 112.

Parent topic: True Flash File System (TrueFFS) on page 106

7.4. VSB Configuration

BDM_FLASH is required (other TrueFFS options are optional). In addition, the options required for the file system you wish to use must be selected.

Optional VSB Configuration

TFFS_HIGH_FAULT_TOLERANCE

Enables high fault tolerance. As the NOR flash is used, it becomes more error prone. When the macro is enabled, TrueFFS performs a checksum of its metadata to check for any inconsistencies caused by bit-flips or other flash errors. Of course, the checksum operation has the cost of minor performance degradation.

TFFS_TRUE_STATIC_WL

Enables true static wear-leveling. TrueFFS no longer chooses erase units randomly, instead it relies on the erase count saved in each unit. This makes the flash wear-leveling more even.

TFFS_BACKGROUND_TASK

Enables background de-fragmentation. A background task is created for each TrueFFS device that is mounted, which performs de-fragmentation at system idle time. It attempts to prepare a reasonable amount free space when it detects a shortage of free space. If applications are not I/O intensive, this can produce significant performance benefits.

Parent topic: True Flash File System (TrueFFS) on page 106

7.5. VIP Configuration

To configure VxWorks for TrueFFS with a file system, include TrueFFS components for basic support and each of the TrueFFS layers, the XBD block device wrapper component, and dosFs file system support.

You must include the following:

WNDRVR

- The core TrueFFS component, **INCLUDE_TFFS** (see Including the Core TrueFFS Component on page 110).
- At least one component or software module from each of the other three TrueFFS layers: translation, MTD, and socket (see Including the MTD Component on page 110, Including the Translation Layer Component on page 111, and Including the Socket Driver on page 112).
- The **INCLUDE_XBD_BLK_DEV** block device wrapper component (see Including the XBD Wrapper Component on page 113).
- Components to support the dosFs file system (see Including the File System Component on page 113).
- Optionally, various TrueFFS utility components (See Including Utility Components on page 113.)

You can configure and build your system either from the command line or with Workbench. For general information on configuration procedures, see the *Wind River Workbench by Example guide* and your Platform user's guide.

For either configuration and build method, special consideration must be given to cases in which either the socket driver or the MTD, or both, are not provided. The drivers must be registered and MTDs need appropriate component descriptions. For supported BSPs this is usually taken care of automatically by including components. It is more of a concern when adding TrueFFS support to a BSP.

Note: Included with TrueFFS are sources for several MTDs and socket drivers. The MTDs are in the *installDir* **/vxworks-7/pkgs_v2/storage/bdm/flash/mtd** directory. The socket drivers are defined in the **sysTffs.c** file provided in the *installDir* **/vxworks-7/pkgs_v2/storage/bdm/flash/tffs/cfg** directory.

Including the Core TrueFFS Component

VxWorks must be configured with the TrueFFS core component, **INCLUDE_TFFS**. Including this component triggers the correct sequence of events, at boot time, for initializing this product. It also ensures that the socket driver is included in your system (see Including the Socket Driver on page 112).

Some 6.9 compatible BSPs have a custom MTD in sysTffs.c that is added to the configuration of VxWorks automatically when **INCLUDE_TFFS** is added. For this BSP you must identify the flash in which TrueFFS will reside with either **INCLUDE_ON_BOARD_TFFS_FLASH** or **INCLUDE_SIMM_TFFS_FLASH**. Other BSPs may use different components and parameters.

Including the MTD Component

Add the MTD component appropriate to your flash device (see Selecting an MTD on page 108) to your system. The MTD components for flash devices from Intel, AMD, Fujitsu, and Sharp, are described in the table below.

Table 1.

Component	Device
INCLUDE_MTD_CFISCS	CFI/SCS device.
INCLUDE_MTD_CFIAMD	CFI-compliant AMD and Fujitsu devices.
INCLUDE_MTD_I28F016	Intel 28f016 device.
INCLUDE_MTD_I28F008	Intel 28f008 device.
INCLUDE_MTD_AMD	AMD, Fujitsu: 29F0{40,80,16} 8-bit devices.
INCLUDE_MTD_VXBFLASH	Stub for the VxBus NOR/SPI flash driver.

VxBus NOR/SPI Flash Driver use With INCLUDE_MTD_VXBFLASH

If you want to use TrueFSS flash media with the VxBus NOR/SPI flash driver, add the **INCLUDE_MTD_VXBFLASH** component.

To configure the VxBus driver based TFFS partitions, set the VXBFLASH_CFG_STR parameter to the appropriate flash partitions string, which is provided in the VxWorks 7 BSP Reference entry for your BSB. By default the string is set to:

```
$norflash0#0:0x0,0x400000,0x0,rfa0
```

The flash partitions string is composed of flash chip strings, using the following syntax:

```
flashChip0string;flashChip1string;...
```

In cases where there is more than one flash chip, the strings are separated by a semi-colon, as in the preceding syntax example. Each flash chip string is composed of the following:

```
$flashInstanceName#index:startAddress,length,bootSize,partitionName
```

Custom MTDs

If you have written your own MTD, you must be sure that it is correctly defined for inclusion in the system, and that it explicitly requires the transition layer.

Including the Translation Layer Component

Choose the translation layer appropriate to the technology used by your flash medium. The main variants of flash devices are NOR and NAND. TrueFFS provides support for:

- NOR devices.
- NAND devices that conform to the SSFDC specification.

The translation layer is provided in binary form only.

Table 2.

Component	Description
INCLUDE_TL_FTL	The translation layer for NOR flash devices. If you can execute code in flash, your device uses NOR logic.
INCLUDE_TL_SSFDC	The translation layer for devices that conform to Toshiba Solid State Floppy Disk Controller Specifications. TrueFFS supports only those NAND devices that comply with the SSFDC specification.

The component descriptor files specify the dependency between the translation layers and the MTDs; therefore, when configuring through Workbench or the command-line wrtool facility, you do not need to explicitly select a translation layer.

The build process handles it for you. For more information about the translation layer, see the *VxWorks Device Driver Developer's Guide: Flash File System Support with TrueFFS*.

WNDRVR

Including the Socket Driver

Inclusion of the socket driver is automatic for BSPs that provide the driver. By including the core TrueFFS component, **INCLUDE_TFFS**, in VxWorks, the build process checks for a socket driver, **sysTffs.c**.

Including the NOR Flash Driver

There are two types of interface to access the NOR flash: the parallel method and the SPI/QSPI method. The components for the parallel method are as follows:

Table 3.

Component	Description
DRV_VXB_NORFLASH	NOR flash VxBus driver.
DRV_VXB_NORFLASH_CMDSET_AMD	AMD/Spansion NOR flash command set library
DRV_VXB_NORFLASH_CMDSET_ITL	Intel NOR flash command set library.

The command set library depends on the NOR flash VxBus driver. If you include one of the command set library components, then the NOR flash VxBus driver is included.

The components for the flash serial SPI/QSPI interface are as follows:

Table 4.

Component	Description
DRV_SPIFLASH_SP25	The Spansion S25XX serial SPI flash chip driver, which enables access to most current model SPI flash chip series including Atmel AT25, Spansion S25, ST M25P, EN25, and Winbond W25X. See the driver source for the current list.
DRV_FDT_ALT5QSPIF LASH	QSPI chip and controller driver for Altera SoC Generation 5
DRV_FSL_ESPI	Freescale eSPI host controller driver
DRV_QSPI_FDT_ZYN Q7K	XILINX ZYNQ7K QSPI host controller driver
DRV_FDT_FSLQSPI	Freescale QSPI host controller driver

To enable the SPI Flash, components for both the SPI host controller driver and flash chip driver need to be included. The Altera CycloneV BSP is a special case.

Its chip driver and controller driver are merged into one driver, so you can choose only one driver for both the chip and controller.

WNDRVR

Including the XBD Wrapper Component

VxWorks must be configured with the **INCLUDE_XBD_BLK_DEV** component for the TrueFFS block device driver (which is not XBD-compatible).

Including the File System Component

File system support for dosFs must be included in your system for TrueFFS to be useful.

Including Utility Components

TrueFFS provides optional utility components for automatic drive detection, show functions, and writing a boot image to flash.

INCLUDE_TFFS_MOUNT

Including this component adds automatic detection (on booting) and mounting of existing formatted TrueFFS drives.

INCLUDE_TFFS_SHOW

Including this component adds two TrueFFS configuration display utilities, tffsShow( ) and tffsShowAll( ) for use from the shell.

The tffsShow( ) function prints device information for a specified socket interface. It is particularly useful when trying to determine the number of erase units required to write a boot image (see Reserving a Region in Flash for a Boot Image on page 116). The tffsShowAll( ) function provides the same information for all socket interfaces registered with VxWorks. The tffsShowAll( ) function can be used from the shell to list the drives in the system. The drives are listed in the order in which they were registered. This component is not included by default.

INCLUDE_TFFS_BOOT_IMAGE

Including this component provides the tffsBootImagePut( ) function (in **sysTffs.o**). The function is used to write a boot image to flash memory (see Writing the Boot Image to Flash on page 117)

Note: **INCLUDE_TFFS_BOOT_IMAGE** is included by default for some BSPs.

Note: tffsBootImagePut( ) is only available with vxWorks 6.9 legacy BSP.

Parent topic: True Flash File System (TrueFFS) on page 106

7.6. Formatting the Flash

To format the flash for TrueFFS, boot the system and use either sysTffsFormat( ) or the tffsDevFormat( ).

CAUTION: **If the flash array for your system is used for a boot image (a boot loader or self-booting VxWorks system) as well as file system media, space must be reserved for the boot image before you format the flash for TrueFFS. The areas cannot overlap. For more information, see** Reserving a Region in Flash for a Boot Image on page 116**.**

First, boot your target system. The boot process registers socket driver(s). Then use the kernel shell to format the flash memory for use with TrueFFS using either the sysTffsFormat( ) or the tffsDevFormat( ) function. For more information about using these functions, see Formatting With sysTffsFormat( ) on page 114 and Formatting With tffsDevFormat( ) on page 114.

CAUTION: **Do not interrupt the formatting process, which may take several minutes to complete. Doing so may corrupt the boot loader in flash.**

WNDRVR

When formatting is complete—a return value of zero indicates success—reboot the target to mount the flash drive (this automatic mount depends on VxWorks being configured with the **INCLUDE_TFFS_MOUNT** component). By default, a rawFs file system is instantiated on the drive. For information about the rawFs file system, see rawFs on page 69.

Note: You can format the flash medium even if though block device driver has not yet been associated with the flash.

Note: The size of the flash media available for use is reduced by one sector for TrueFFS internal use.

Parent topic: True Flash File System (TrueFFS) on page 106

7.7. Formatting With sysTffsFormat( )

If your BSP provides the sysTffsFormat( ) function, you can use it from the kernel shell to format the flash.

About This Task

The sysTffsFormat( ) function performs some setup operations and then calls the tffsDevFormat( ) function with the appropriate default values for the BSP in question.

Procedure

Run sysTffsFormat( ) as shown. Output will vary according to your set-up:

```
-> sysTffsFormat
Formatted 128 of 128 units = 100.0%
value = 0 = 0x0
```

For more information, see Using sysTffsFormat( ) to Reserve a Fallow Region on page 116.

Parent topic: True Flash File System (TrueFFS) on page 106

7.8. Formatting With tffsDevFormat( )

If your BSP does not provides the sysTffsFormat( ) function, use tffsDevFormat( ) from the kernel shell to format the flash.

The tffsDevFormat( ) Function

The tffsDevFormat( ) function takes two arguments, a drive number and a format argument:

```
tffsDevFormat: (int tffsDriveNo, int formatArg);
```

The first argument for tffsDevFormat( ), tffsDriveNo, is the drive number (socket driver number), which identifies the flash medium to be formatted. Most systems have a single flash drive, but TrueFFS supports up to five.

The socket registration process determines the drive number. Drive numbers are assigned to the flash devices on the basis of the order in which the socket drivers are registered in sysTffsInit( ) at boot time. The first registered is drive 0, the second is drive 1, and so on up to 4.

The second argument for tffsDevFormat( ), formatArg, is a pointer to a **tffsDevFormatParams** structure (cast to an int); however, zero or a null pointer can be used to specify default values.

WNDRVR

Using the TFFS_STD_FORMAT_PARAMS Macro

To facilitate calling tffsDevFormat( ) from the shell, you can simply use zero (or a NULL pointer) for the second tffsDevFormat( ) argument, formatArg. Doing so makes use of the **TFFS_STD_FORMAT_PARAMS** macro, which defines default values for the **tffsDevFormatParams** structure.

> Note: If you want to reserve a region in flash for a boot image, do not make use of the **TFFS_STD_FORMAT_PARAMS** macro, because the entire flash media will be formatted for use with TrueFFS. You must modify the **tffsDevFormatParams** structure instead (see tffsDevFormatParams Structure on page 115).

The **TFFS_STD_FORMAT_PARAMS** macro is defined in **tffsDrv.h** as follows:

```
#define TFFS_STD_FORMAT_PARAMS {{0, 99, 1, 0x100001, NULL, {0,0,0,0}, NULL,
2, 0, NULL}, FTL_FORMAT_IF_NEEDED}
```

The macro passes values for the first and second members of the **tffsDevFormatParams** structure, which are as follows:

```
formatParams = {0, 99, 1, 0x100001, NULL, {0,0,0,0}, NULL, 2, 0, NULL}
formatFlags = FTL_FORMAT_IF_NEEDED
```

For information about structure, the meaning of the default values, and other possible arguments for the members of this structure, see tffsDevFormatParams Structure on page 115.

The tffsDevFormatParams Structure

The **tffsDevFormatParams** structure describes how the volume should be formatted.

If you use the **TFFS_STD_FORMAT_PARAMS** macro (see Using the TFFS_STD_FORMAT_PARAMS Macro on page 115), the default values are used format the entire flash medium for use with TrueFFS.

If you want to store a boot image in flash, you must change the value of the **bootImageLen** member of the **tffsFormatParams** structure to reserve a region flash for the image, which is separate from that used by TrueFFS (for more information, see Reserving a Region in Flash for a Boot Image on page 116).

The **tffsDevFormatParams** structure is defined in *installDir* **/vxworks-7/pkgs_v2/storage/bdm/flash/tffs/h/tffs/tffsDrv.h** as follows:

```
typedef struct
    { tffsFormatParams formatParams;
      unsigned formatFlags;
    } tffsDevFormatParams;
```

Also see the commentary in **tffsDrv.h**.

The **formatParams** member is structure that includes the following definitions (as well as others):

- The size of the space to reserve for a boot image at the beginning of the flash media.
- The percentage of the flash media to use.
- The number of spare erase units.
- The size of the part of the FTL virtual map that resides in RAM.

The **formatFlags** member determines the option used to format the drive. There are several possible values for **formatFlags**:

WNDRVR

Table 1. Options for formatFlags

Macro	Value	Meaning
FTL_FORMAT	1	FTL formatting.
FTL_FORMAT_IF_NEEDED	2	FTL formatting if needed.
NO_FTL_FORMAT	0	No formatting.

By default the macro **TFFS_STD_FORMAT_PARAMS** passes **FTL_FORMAT_IF_NEEDED** as the value for this argument.

Performance Tuning Configuration

By default, tffsDevFormat( ) formats 99% of the flash media. However, write performance for NOR flash devices depends on how full the media is, and it gets slower as usage approaches 100%. This effect can be mediated by formatting a smaller percent of the media, which of course means that some capacity is sacrificed in the process.

To format a smaller percent of the media, assign a value of less than 99 to the **percentUse** element of the **tffsFormatParams** structure.

Also consider changing how much of the FTL virtual map can reside in RAM. The RAM part is used to address the part of the media below the VM addressing limit. Reading and writing to this part is usually somewhat faster. However, the larger the limit, the more RAM is needed. The default value is 0x10000 (first 64 KB).

To define a higher limit, set the **vmAddressingLimit** element of the **tffsFormatParams** structure to a higher value.

Parent topic: True Flash File System (TrueFFS) on page 106

7.9. Reserving a Region in Flash for a Boot Image

In order to use flash media for a VxWorks boot image as well as a file system, a portion of the flash memory (a fallow region) must be reserved so that it is excluded from the area subject to formatting and the run-time operations of TrueFFS.

The two areas cannot overlap. Note that the fallow region can be used for purposes other than boot loader code (for example, a system startup log or configuration data).

If TrueFFS is used with the **vxWorks_romResident** VxWorks image type, TrueFFS must be read-only.

Reserving a Fallow Region

To create a boot image region, TrueFSS must be configured so that it is excluded from an area of flash that is large enough to accommodate the boot image. If your BSP provides sysTffsFormat( ), that function does so automatically. Otherwise you must modify the tffsFormatParams structure (for more information, see Formatting With tffsDevFormat( ).

Using sysTffsFormat( ) to Reserve a Fallow Region

Some BSPs provide an optional, BSP-specific, helper function sysTffsFormat( ), which preserves a boot image region when it formats the flash for TrueFFS.

This function first sets up a pointer to a tffsFormatParams structure that has been initialized with a value for bootImageLen (which defines the boot image region); then it calls tffsDevFormat( ).

The implementation is BSP-specific, and is not always designed to reserve space for a boot loader (which may in any case be in a different flash device).

If your BSP does not provide a sysTffsFormat( ) function, then create a similar function, or pass the appropriate argument to tffsDevFormat( ).

Using tffsDevFormat( ) to Reserve a Fallow Region

If your BSP does not provide sysTffsFormat( ), you must modify the *tffsFormatParams* structure to reserve a fallow region before you call tffsDevFormat( ).

Change the *bootImageLen* member of the *tffsFormatParams* structure to a value that is at least as large as the boot image. The area defined by bootImageLen is excluded from TrueFFS activity (formatting and wear-leveling).

For more information about bootImageLen and other members of the structure, see Formatting With tffsDevFormat( ) on page 114.

Writing the Boot Image to Flash

Once you have created a boot image region, you can write the boot image to the flash device using tffsBootImagePut( ). This function bypasses TrueFFS (and its translation layer) and writes directly into any location in flash memory. However, because tffsBootImagePut( ) relies on a call to tffsRawio( ), do not use this function once the TrueFFS volume is mounted.

Note: tffsBootImagePut( ) is only available with vxWorks 6.9 legacy BSP.

The arguments to tffsBootImagePut( ) are the following:

driveNo

The same drive number as the one used as input to the format function.

offset

Offset from the start of flash at which the image is written (most often specified as zero). The offset is largely BSP-specific, but can usually be computed by subtracting the flash base address (usually called **FLASH_BASE_ADRS**, the address of the first byte of flash), from **ROM_LINK_ADRS** (or **ROM_TEXT_ADRS** - 0x100). For example, if the flash starts at 0xff000000, and **ROM_TEXT_ADRS** (in the BSP makefile) is 0xfff00100, then the offset would be 0x00f00000.

filename

Pointer to the boot image. A **.bin** image must be used.

CAUTION: **Because tffsBootImagePut( ) lets you write directly to any area of flash, it is possible to accidentally overwrite and corrupt the TrueFFS-managed area of flash if you do not specify the parameters correctly. For more information about how to use this utility, see the reference entry for tffsBootImagePut( ) in the VxWorks API reference.**

Example

To write the boot image to flash, you can simply change to the host directory that contains the boot image and execute a command like the following:

```
-> tffsBootImagePut(0,0,"vxWorks_rom.bin")
```

Or, you could first copy the image from the host to the TrueFFS file system, and then execute the following command:

```
-> tffsBootImagePut(0,0,"/tffs0/vxWorks_rom.bin")
```

The commands used here assume that the boot image should be installed at the beginning of flash. However, the offset differs from BSP to BSP.

Note that the **INCLUDE_DISK_UTILS** component must be included in the VxWorks image that is running on the target to provide support for the commands cd, ls, and so on. For more information about tffsBootImagePut( ), see the *VxWorks Driver API Reference*.

Parent topic: True Flash File System (TrueFFS) on page 106

7.10. Mounting the Drive

Use the usrTffsConfig( ) function to mount the a TrueFFS flash drive, which means that a new TFFS device is attached, and the TFFS layer is enabled for the NOR flash device.

The parameters for usrTffsConfig( ) are as follows:

drive

Specifies the drive number of the TFFS flash drive; valid values are 0 through N-1, where N is the number of socket interfaces provided by the BSP.

removable

Specifies whether the media is removable. Use 0 for non-removable, 1 for removable.

fileName

Specifies the mount point, for example, **/tffs0/**.

The following example runs usrTffsConfig( ) to attach a drive to the file system, and then runs devs to list all devices:

```
-> usrTffsConfig 0,0,"/tffs0/"
Instantiating /tffs0/ as rawFs, device = 0x50001
value = 0 = 0x0
-> devs
drv name
  0 /null
  1 /ttyS0
  1 /ttyS1
  3 /tffs0
  7 mars:
  8 /vio
  9 /tgtsvr
value = 25 = 0x19
```

Internally, usrTffsConfig( ) calls other functions, passing the parameters it is given. Among these functions is tffsDevCreate( ), which creates a TrueFFS block device on top of the socket driver. This function takes, as input, a number (0 through 4) that identifies the socket driver on top of which to construct the TrueFFS block device. The tffsDevCreate( ) call uses this number as an index into the array of *FLSocket* structures. This number is visible later to the file system as the driver number. The xbdBlkDevCreateSync( ) function is then called, which sends a message the file system handler. If the TFFS device has a valid file system installed (dosFS) the file system handler mounts the underlying file system. If there is no valid file system on the device, the rawfs file system is instantiated on the device. The user can then format the device with the file system of choice.

The **INCLUDE_TFFS_MOUNT** component can be used to mount the TrueFFS drive automatically.

Parent topic: True Flash File System (TrueFFS) on page 106

7.11. Testing the Drive

You can test a drive by copying a file to it, then from it to another location, and then comparing the copy with the original.

About This Task

The following examples assume that the host is a Windows system called mars.

Procedure

1. Copy a text file from the host (or from another storage medium) to the flash file system on the target.

```
-> copy "mars:c:/scratch/junk.txt" , "/tffs0/junkOnTarget.txt"
value = 0 = 0x0
```

2. List the contents of **/tffs0**.

```
-> ll "/tffs0"
Listing Directory /tffs0:
drwxrwxrwx 1 0 0 1024 Jan 1 00:17 ./
drwxrwxrwx 1 0 0 1024 Jan 1 00:17 ../
-rwxrwxrwx 1 0 0 51 Jan 1 00:00 junkOnTarget.txt
value = 0 = 0x0
```

3. Verify the contents of the file by copying it to the console or to a temporary file for comparison.

```
-> copy "/tffs0/junkOnTarget.txt"
this text is the contents of the file
value = 0 = 0x0
->
```

Note: If the file is not stored on the target system, the path must be valid from the point of view of the target itself.

Note: The copy command (among others) is provided by the **INCLUDE_DISK_UTIL** component.

Parent topic: True Flash File System (TrueFFS) on page 106

7.12. Using TrueFFS With dosFs

Implementing TrueFFS with dosFs can be done programmatically, as well as interactively from the kernel shell.

For information about file system support, see Including the File System Component on page 113.

As noted in Mounting the Drive on page 118, if the TrueFFS device already has a valid file system installed (dosFS) the file system handler mounts the file system. If there is no valid file system on the device, the rawfs file system is instantiated by default.

Parent topic: True Flash File System (TrueFFS) on page 106

7.13. TrueFFS With dosFs Code Example

This code fragment creates a TrueFFS block device, creates an XBD block device wrapper, and then formats the TrueFFS media for an dosFs file system.

The example assumes that you have already formatted the flash (for information in this regard, see Formatting the Flash on page 113).

```
/* create block device for the entire disk, */

if ((pBlkDev = tffsDevCreate (0, 0)) == NULL)
{
printErr ("tffsDevCreate failed.");
return (ERROR);
}

/* wrap this block device into an XBD */

xbd = xbdBlkDevCreateSync(pBlkDev, "/tffs0");
printf("Wrap XBD=0x%x\n", (u_int)xbd);

/*
* If the dosFs file system exists already, it will be
* automatically instantiated and we are done.
*/

/*
* But if a file system does not exist or it isn't dosFs
* format it.
*/

dosFsVolFormat ("/tffs0", 0, 0);
```

This code requires the following include statements:

```
#include <vxWorks.h>
#include <stdio.h>
#include <tffs/tffsDrv.h>
#include <xbdBlkDev.h>
#include <dosFsLib.h>
```

Parent topic: True Flash File System (TrueFFS) on page 106

7.14. TrueFFS With dosFs Shell Command Example

This example illustrate the procedure for implementing TrueFFS with dosFs from the kernel shell.

About This Task

Procedure

1. Perform a low-level format of the flash device.

WNDRVR

This step must be performed before you use the flash device for the first time.

```
-> sysTffsFormat
```

2. Create the TrueFFS block device.

```
-> dev = tffsDevCreate (0,0)
```

3. Create an XBD device wrapper.

```
-> xbdBlkDevCreateSync dev, "/tffs0"
```

4. Format the TrueFFS device for dosFs.

```
-> dosFsVolFormat "/tffs0"
```

5. The TFFS device is now ready for use.

   After the media and file system and been prepared, for subsequent reboots the procedure is slightly simpler, as the flash media does not have to be formatted for TrueFFS and dosFs. You only need to create the TrueFFS block device with tffsDevCreate( ) and then create the XBD block device wrapper with xbdBlkDevCreateSync( ).

Parent topic: True Flash File System (TrueFFS) on page 106

8. NAND Flash Translation Layer (NFTL)

The NAND Flash Translation Layer (NFTL) facility provides block device functionality to enable file systems to access NAND flash memory in the same way as accessing a disk drive.

VxWorks provides two file translation layers for Flash: NFTL is for NAND flash and the original TrueFFS FTL is used for NOR and SPI flash (see True Flash File System (TrueFFS) on page 106).

The rawFs file system, dosFs file system, and Highly Reliable File System (HRFS) can be used with NFTL. NFTL provides VxWorks 7 with full flash memory management capabilities.

Note: Symmetric multiprocessing (SMP) is not supported in NFTL. Calls to FTL layers should not be done across cores.

The flash translation layer provides the NAND flash option interface for the NTFL system and connects all the other layers. The flash translation layer handles global facilities, such as block management, garbage collection, timers, and memory technology devices (MTDs). The flash translation layer maintains the block allocation map that associates the file system's view of the storage medium with the erase blocks in flash. It also provides the NAND flash controller driver (which operates below the NAND flash controller layer).

NFTL consists of the following layers:

- The MTD layer implements the low-level programming of the flash medium (map, read, write, and erase functions).
- The NAND flash controller layer provides an interface function and the board hardware with board-specific hardware access functions.

You may need to write certain sub-components for your application. This is most often the case for the MTD layer. To determine if this release provides an MTD suitable for your flash hardware, see Selecting an MTD on page 108.

Configuring VxWorks with NFTL and a File System on page 122 To configure VxWorks with support for NFTL and a file system, you must select the required options and components for your VSB and VIP projects.
MTD_CFG_STR Syntax on page 123 The MTD_CFG_STR parameter for the **INCLUDE_MTD** component is used to specify your flash configuration.
Formatting Flash and Creating the NFTL Device on page 123 The NAND flash must be formatted before it displays any device information.
Displaying Information about the NFTL Device on page 124 You can use the mtdDevShow( ) function from the kernel shell to display information about the device you have created.
Using NFTL with HRFS or dosFs on page 125 To use NFTL with Highly Reliable File System (HRFS) or dosFs, call the file system's format function with the NFTL device name.

8.1. Configuring VxWorks with NFTL and a File System

To configure VxWorks with support for NFTL and a file system, you must select the required options and components for your VSB and VIP projects.

About This Task

Procedure

1. Create a VSB project, select the BDM_FLASH option, and build the project.
2. Create a VIP project based on the VSB project, and to include garbage collection, wear-leveling, bad block management, data caching, mapping, and formatting facilities, add the following components:
   - **INCLUDE_FTL**
   - **INCLUDE_FTL_FORMAT**
3. Optionally include components for space reclamation, hot data caching, and show functions.

   INCLUDE_FTL_DISCARD

   Reclaims discarded file space.

   INCLUDE_FTL_CACHE

   Caches hold data in main memory without accessing the flash device for each I/O request.

   INCLUDE_FTL_SHOW

   Provides show functions for displaying information about flash devices that are managed by NFTL.

4. To mount the NFTL devices automatically at reboot, set the FTL_AUTO_MOUNT configuration parameter to **TRUE** or 1.

   If FLT_AUTO_MOUNT is set to **FALSE**, ftlMount( ) must be used to mount the NFTL devices

5. Optionally, change the default settings of the configuration parameters for memory and a service background task.

   FTL_MEM_BUDGET_CFG_PROFILE

   Controls the amount of memory available for NFTL.

   FTL_SERVICE_TASK_ENABLE

   Provides a background task for operations such as garbage collection and wear-leveling.

6. Add the **INCLUDE_MTD** component.
7. Set the MTD_CFG_STR component parameter to specify your flash configuration.

WNDRVR

For information about the syntax, see MTD_CFG_STR Syntax on page 123.

8. Optionally, add **INCLUDE_MTD_SHOW** for the mtdShow( ) function.
9. Select the NAND flash controller driver for your BSP.

   For information in this regard, see the VxWorks 7 BSP Reference entry for your BSP.

10. Optionally, add the components that will allow you to use the kernel shell to format flash.

    INCLUDE_SHELL

    Provides the shell interpreter for debugging.

    INCLUDE_DISK_UTIL

    Provides utilities such as cd, copy, ll, mkdir, and so on.

11. Add the components that provide support for the file system that you want to implement (HRFS or dosFs).

    If you do not include support for another file system, rawFs is instantiated by default at runtime.

12. Build VxWorks.

Parent topic: NAND Flash Translation Layer (NFTL) on page 121

8.2. MTD_CFG_STR Syntax

The MTD_CFG_STR parameter for the **INCLUDE_MTD** component is used to specify your flash configuration.

The MTD_CFG_STR parameter must be set to an MTD configuration string that is composed of flash chip specification strings, using the following syntax:

```
flashChip0string;flashChip1string;...
```

In cases where there is more than one flash chip, the strings are separated by a semi-colon, as in the preceding syntax example.

Each flash chip string is composed of flash partition strings:

```
$flashInstanceName#chipIndex:\flashPartition0string;\flashPartition0string;\...
```

And each flash partition string consists of the following elements:

```
startBlockAddress,blockCount,mtdDeviceName#flashGroupIndex,flashUsage
```

For example:

```
$gpmc0#0:0x0,0x100,/nvram#0,1;0x100,0x700,/ftl0#0,0
```

Parent topic: NAND Flash Translation Layer (NFTL) on page 121

8.3. Formatting Flash and Creating the NFTL Device

The NAND flash must be formatted before it displays any device information.

WNDRVR

About This Task

To format the flash use the ftlFormat( ) function.

Procedure

From the kernel shell, for example:

```
-> ftlFormat "/ftl0",3,3,0
ftl formatting...
unmount device first.
Instantiating /ftl0:0 as rawFs, device = 0x70001
OK.
value = 0 = 0x0
```

The parameters used with ftlFormat( ) are as follows:

- The first parameter is the NFTL device name.
- The second parameter is the size (in percentage of the total size) of the flash space reserved by the flash translation layer to facilitate garbage collection and wear leveling.
- The third parameter is the size (in percentage of the total size) of the flash space reserved by NFTL to replace runtime bad blocks.
- The fourth parameter is used for options for additional functions. Use zero.

For additional information, see the **ftlFormat** library API reference entry.

Parent topic: NAND Flash Translation Layer (NFTL) on page 121

8.4. Displaying Information about the NFTL Device

You can use the mtdDevShow( ) function from the kernel shell to display information about the device you have created.

About This Task

Procedure

- Call mtdDevShow().

```
-> mtdDevShow
-----------------------------------
MTD device #0 information:
device name: /ftl0
device name
Usage: FTL
Attached: Yes
supported features(SLC NAND):
Sequential Read
Sequential Write
Scatter-Gather Read
Scatter-Gather Write
Block Erase
Bad Block Check
```


WNDRVR

```
Page Move
Total Size(Byte): 0x40000000
Block Count: 0x2000
Block Size(Byte): 0x20000
Oob Available Size(Byte): 0x32
Page Count per Block: 0x40
partition count per group: 0x1
------------------------------
Flash group #0 information
mapped MTD block address: 0x0
block count per partition: 0x2000
------------------------------
Flash partition list:
start block Flash Chip ID Device Name
0x00000000 0x2023c060 fcm0:0
value = 38 = 0x26 = '&'
```

- Call devs( ) to see the NFTL device in the list of devices:

```
-> devs
drv name
  0 /null
  1 /tyCo/0
  1 /tyCo/1
  8 host:
  3 /ftl0:0
value = 25 = 0x19
```

Parent topic: NAND Flash Translation Layer (NFTL) on page 121

8.5. Using NFTL with HRFS or dosFs

To use NFTL with Highly Reliable File System (HRFS) or dosFs, call the file system's format function with the NFTL device name.

About This Task

Procedure

Use commands similar to the following examples, depending on the file system you are working with.

In this example, hrfsFormat( ) formats the NFTL device, and ll( ) displays the new HRFS directory:

```
-> hrfsFormat "/ftl0:0"
Formatting /ftl0:0 for HRFS v1.2
Instantiating /ftl0:0 as rawFs, device = 0x10001
Formatting...OK.
value = 0 = 0x0
-> ll "/ftl0:0"
Listing Directory /ftl0:0:
drwxrwxrwx 1 0 0 1024 Jan 1 00:07 ./
drwxrwxrwx 1 0 0 1024 Jan 1 00:07 ../
value = 0 = 0x0
```


WNDRVR

Similarly, the dosFs file system uses dosFsVolFormat( ) with the device name to format the NFTL device:

```
-> dosFsVolFormat ("/ftl0:0", 0, 0)
```

Parent topic: NAND Flash Translation Layer (NFTL) on page 121

9. Loop File System and Tcplay

The loop file system (LoopFs) is a facility that mounts an image file as a virtual block device.

With LoopFs, the contents of the file are treated as the virtual block device's sector data. Data is mapped one-to-one. If the image file contains a valid layout of file system format, such as DOSFS, HRFS or CDROMFS (ISO image), the new virtual block device is mounted by the appropriate file system driver.

The tcplay tool creates encrypted containers.

The LoopFs file system can be configured with support for mounting image files that have been encrypted with the VxWorks version of tcplay, which is partially compatible with the FreeBSD version of tcplay, and with the GPL TrueCrypt facility. VxWorks tcplay uses the Advanced Encryption Standard (AES) specification for encryption based on OpenSSL. It currently supports AES-128-XTS and AES-256-XTS for encryption and or decryption, and supports multi-key files and hidden volume, like tcplay for other hosts. The VxWorks API provides functions that can be used programmatically or from the shell (the shell command tcplay is not supported).

The LoopFs file system also provides an extension for mounting an image file with a non-flat layout (that is, a mapping that is not one-to-one). Making use of this extension requires that you write XBD access functions, custom XBD service task functions and virtual block device parameters. The loopFsMountFile( ) function takes additional parameters for virtual block devices to support this functionality (see the API reference for more information).

Mount ISO image with LoopFs

LoopFs can be used for mounting an ISO image (ISO 9660 image). To enable ISO mounting, you must enable **FS_CDROMFS** layer in VSB, and enable **INCLUDE_CDROMFS** and **INCLUDE_LOOPFS** in VIP.

After system start up, use loopFsMountImgFile( ) to mount ISO image.

```
-> loopFsMountImgFile "/bd16a/sysroot.iso", "/cdrom1"
value = -140737482209840 = 0xffff8000005dc5d0 = _sysTableEnd + 0xffff8000001ce5d0
-> ll "/cdrom1"

Listing Directory /cdrom1:
drwxr-xr-x  0 0       0                2048 Jan  2 2020 ./
drwxr-xr-x  0 0       0                2048 Jan  2 2020 ../
drwxr-xr-x  0 0       0                2048 Jan  2 2020 lib/
drwxr-xr-x  0 0       0                2048 Jan  2 2020 usr/
value = 0 = 0x0
```

Configuring VxWorks With loopFs and Encrypted Image Support on page 127 To configure VxWorks with support for loopFs, and for mounting a tcplay-encrypted image file, you must select the required options and components for your VSB and VIP projects.
Mounting and Unmounting an Image File on page 127 To mount and unmount an image file, use APIs from the the **loopFsLib** library.
Working With an Encrypted Image File on page 128 Common operations used in working with an encrypted image file involve mounting the image, displaying information about mounted tcplay device, and un-mounting the tcplay volume.

WNDRVR

Adding Support for Auto-Mounting Encrypted Images on page 130 If you have configured VxWorks with the loopfs file system and tcplay, you can provide additional support for automatically mounting encrypted images.
Adding Auto-Mount Support on page 130 To add auto-mount support for encrypted images, you must configure VxWorks with the required component, set a parameter to the locations of one or more encrypted images, and set another parameter to the mount points for each image.
Using Plain-Text Passwords on page 131 If you choose to use plain-text passwords with auto-mount support for encrypted images, you must set the passwords parameter to plain text passwords, and the password converter parameter to NULL.
Using a Custom Password De-Encryption Function With Encrypted Passwords on page 131 If you choose to use a custom password de-encryption function with auto-mount support for encrypted images, you must write your function, make it accessible to your VIP project, and set VIP project parameters to the name of the function and its parameters (if required), as well as to the encrypted passwords.
Using the IPCOM Key Database Support on page 132 If you choose to use the IPCOM key database with auto-mount support for encrypted images, you must configure VxWorks with the required VSB layer and VIP component, and set the configuration parameters for the database to the required values.

9.1. Configuring VxWorks With loopFs and Encrypted Image Support

To configure VxWorks with support for loopFs, and for mounting a tcplay-encrypted image file, you must select the required options and components for your VSB and VIP projects.

About This Task

Procedure

1. Create a VSB project.
2. For the loopFs file system, select the **BDM** layer.
3. For mounting a tcplay-encrypted image file, also select **OPENSSL** and **FSAPI_TCPLAY**.
4. Build the source code.
5. Create a VIP project based on your VSB project.
6. For loopFs file system support, add **INCLUDE_LOOPFS**.
7. For support for mounting a tcplay-encrypted file system, add **INCLUDE_TC_DRIVER**.
8. To automatically unmount inaccessible images, set the **TC_AUTO_UNMOUNT** parameter to **TRUE**. (If an image is located on a removable device, the image is inaccessible when the device is unplugged.)

Parent topic: Loop File System and Tcplay on page 126

9.2. Mounting and Unmounting an Image File

To mount and unmount an image file, use APIs from the the **loopFsLib** library.

About This Task

Procedure

1. To mount an image file as a virtual block device, use the loopFsMountImgFile( ) function.

   For example:

```
-> ls "/bd0"
/bd0/VXBOOT.IMG
```

```
/bd0/FOO.TC
value = 0 = 0x0
-> loopFsMountImgFile "/bd0/VXBOOT.IMG", "/loop0"
value = 31252640 = 0x1dce0a0
-> devs
drv name
  0 /null
  1 /ttyS0
  1 /ttyS1
  8 host:
  3 /bd0
  3 /loop0
value = 0 = 0x0
-> ls "/loop0"
/loop0/README.TXT
/loop0/BOOTROM.SYS
value = 0 = 0x0
->
```

2. To unmount an image file as a virtual block device, use the loopFsUnmount( ) function.

   For example:

```
-> loopFsUnmount 0x1dce0a0
value = 0 = 0x0
-> devs
drv name
  0 /null
  1 /ttyS0
  1 /ttyS1
  8 host:
  3 /bd0
value = 0 = 0x0
->
```

   For more information, see the **loopFsLib** library entry.

Parent topic: Loop File System and Tcplay on page 126

9.3. Working With an Encrypted Image File

Common operations used in working with an encrypted image file involve mounting the image, displaying information about mounted tcplay device, and un-mounting the tcplay volume.

About This Task

This example illustrates working with an encrypted image created by tcplay on another system.

Procedure

1. Display the contents of **/bd0**.

WNDRVR

```
-> ls "/bd0"
/bd0/VXBOOT.IMG
/bd0/FOO.TC
value = 0 = 0x0
->
```

2. Mount the image with tcVolMount( ), and then display device information.

```
-> tcVolMount "/loop1", "/bd0/foo.tc", "miBadP@$$werd"
value = 0 = 0x0
->
-> devs
drv name
  0 /null
  1 /ttyS0
  1 /ttyS1
  8 host:
  3 /bd0
  3 /loop1
value = 0 = 0x0
->
```

The third argument for tcVolMount( ) is the password.

3. Display the contents of **/loop1**, and then information about mounted tcplay device with tcVolShow( ).

For example:

```
-> ls "/loop1"
/loop1/test
/loop1/2.txt
/loop1/1.txt
value = 0 = 0x0
->
-> tcVolShow "/loop1"
Device: /bd0/foo.tc
Cipher: AES-256-XTS
Key Length: 512 bits
Sector size: 512
Volume size: 1536 sectors
IV offset: 256
Block offset: 256
value = 0 = 0x0
->
```

VxWorks must be configured with **INCLUDE_DOSFS_MAIN** to be able to use the ls command.

4. Un-mount the tcplay volume using the tcVolUnmount( ) function.

For example.

```
-> tcVolUnmount "/loop1"
value = 0 = 0x0
-> devs
drv name
0 /null
```


WNDRVR

```
1 /ttyS0
1 /ttyS1
8 host:
3 /bd0
value = 0 = 0x0
```

Parent topic: Loop File System and Tcplay on page 126

9.4. Adding Support for Auto-Mounting Encrypted Images

If you have configured VxWorks with the loopfs file system and tcplay, you can provide additional support for automatically mounting encrypted images.

To use the auto-mount facility, you must also choose one of the following for password support:

- Plain-text passwords.
- A custom de-encryption function with encrypted passwords.
- The IPCOM key database.

See also

- Configuring VxWorks With loopFs and Encrypted Image Support on page 127
- Working With an Encrypted Image File on page 128

Parent topic: Loop File System and Tcplay on page 126

9.5. Adding Auto-Mount Support

To add auto-mount support for encrypted images, you must configure VxWorks with the required component, set a parameter to the locations of one or more encrypted images, and set another parameter to the mount points for each image.

About This Task

Procedure

1. Add **INCLUDE_TC_AUTOMOUNT** to your VIP project:.
2. Set the **TC_PATHS** parameter to one or more locations of the encrypted images, separating each path string with a comma. You must use NULL as the last entry.

   For example:

   ```
   "/romfs/test1.tc", "/romfs/test2.tc", NULL
   ```

3. Set the **TC_MOUNTPOINTS** to the mount points for the images.

   Each image identified with **TC_PATHS** must have a corresponding mount point.

   For example:

WNDRVR

```
"/loop0", "/loop1"
```

Parent topic: Loop File System and Tcplay on page 126

9.6. Using Plain-Text Passwords

If you choose to use plain-text passwords with auto-mount support for encrypted images, you must set the passwords parameter to plain text passwords, and the password converter parameter to NULL.

About This Task

Procedure

1. In your VIP project, set **TC_PASSPHRASES** to the plain-text passwords used to access each of the encrypted images.

   For example:

   ```
   "123", "Abc123"
   ```

2. Set the **TC_PASSPHRASES_CONVERTER** parameter to NULL.

   The passwords are not encrypted, so no de-encryption function is used.

3. Rebuild VxWorks.

Parent topic: Loop File System and Tcplay on page 126

9.7. Using a Custom Password De-Encryption Function With Encrypted Passwords

If you choose to use a custom password de-encryption function with auto-mount support for encrypted images, you must write your function, make it accessible to your VIP project, and set VIP project parameters to the name of the function and its parameters (if required), as well as to the encrypted passwords.

About This Task

Procedure

1. Write your own function to de-encrypt the passwords that you will assign to the **TC_PASSPHRASES** parameter in your VIP project.

   An example password converter function called usrTcPassphraseConvertDemo( ) is provided in *installDir* **/vxworks-7/pkgs_v2/storage/tcplay/cfg/usrTcplay.c**. The function uses Base64 encoding for the passwords. It is provided for illustrative purposes only; do not use it in deployed systems.

2. Add your decryption function to your VIP, layer code, or library file.

   Your VIP project must be able to find the symbol name of the converter function.

3. Set **TC_PASSPHRASES_CONVERTER** (the type is function pointer) to the name of the converter function.
4. If your converter function takes a parameter, set **TC_PASS_CONVERT_PARAM** (the type is void *) to the parameter name.

WNDRVR

5. Encrypt all the passwords and then assign them to the **TC_PASSPHRASES** configuration parameter. See Configuring VxWorks With loopFs and Encrypted Image Support on page 127.

   If you wish to try the usrTcPassphraseConvertDemo( ) example, use Base64 encoding for the passwords.

6. Rebuild VxWorks.

Results

At runtime, the auto-mount facility passes the encrypted passwords to the converter function, which returns the de-encrypted passwords, which are then used to mount the encrypted image.

Parent topic: Loop File System and Tcplay on page 126

9.8. Using the IPCOM Key Database Support

If you choose to use the IPCOM key database with auto-mount support for encrypted images, you must configure VxWorks with the required VSB layer and VIP component, and set the configuration parameters for the database to the required values.

Before You Begin

Before using the IPCOM key database with the auto-mount feature, you should be thoroughly familiar with how the database itself is used. See VxWorks 7 Cryptography Libraries Programmer's Guide.

About This Task

Procedure

1. Add the **OPENSSL** layer to your VSB project, and rebuild it.
2. Close your VIP project, and then reopen it (to pick up the VSB change).
3. Add **INCLUDE_IPCOM_USE_KEY_DB** to your VIP project.
4. Select and configure a key-encrypting password provider.

   For example, if you are using the password obfuscator KEP provider, select **INCLUDE_SEC_VAULT_KEP_OBFUSCATOR**, and set the **SEC_VAULT_FILE_ROOT** and **SEC_VAULT_KEY_ENCRYPTING_PW** parameters.

   For more information about the password obfuscator KEP provider, as well as information about other KEP providers, see VxWorks 7 Cryptography Libraries Programmer's Guide.

5. Set the **TC_KEYDB_PATHS** parameter to one or more locations of the encrypted images, separating each path string with a comma. You must use NULL as the last entry.

   For example:

   ```
   "/romfs/test1.tc", "/romfs/test2.tc", NULL
   ```

6. Set the **TC_KEYDB_MOUNTPOINTS** to the mount points. Each image identified with **TC_KEYBD_PATHS** must have a corresponding mount point.

   For example:

```
"/loop0", "/loop1"
```

7. Set **TC_KEYDB_IDS** to the key IDs in the key database.

   For example:

```
"KeyID1", "KeyID2"
```

8. Rebuild and boot VxWorks.
9. Import the encrypted image's password as a keydb buffer and store it in the key database.

   For information about this process, see VxWorks 7 Cryptography Libraries Programmer's Guide.

Parent topic: Loop File System and Tcplay on page 126

10. Watchdog I/O Devices Overview

VxWorks provides a software library and an IOS driver framework designed to facilitate the use of watchdog timer devices for VxWorks customers and developers. The software watchdog driver framework provides a hardware-independent POSIX-like open/close/read/write/ioctl interface to applications which allows for potability across VxWorks platforms. This is used to create device drivers for hardware watchdog devices on target boards, for example the Renesas R-car H3 (SiP) Salvator-X board. A virtual watchdog driver is also provided which makes use of the VxWorks wdLib library to approximate the functionality of a hardware watchdog on platforms where one is not available.

Hardware watchdog devices connect to the CPU reset pin and, if the software framework does not periodically re-arm the watchdog, it expires and triggers a hard CPU reset. This mechanism can be used to indicate that the software is running normally, and also to help the system automatically recover if the software malfunctions.

Hardware and virtual watchdog devices should link to the watchdog IOS driver framework and use it to export the watchdog device so that it is accessible from Kernel space or RTP space.

The virtual device, which requires both the watchdog IOS driver framework and the VxWorks wdLib (which replaces a hardware watchdog driver), can be used as a software alternative for BSPs that do not have a compatible hardware watchdog driver. This enables software frameworks like AUTOSAR to be supported on boards that lack a hardware watchdog.

> **Note:** The virtual driver should only be used for testing, development, or demo purposes. It is not a reliable means of recovery from a lockup or system hang, since the CPU can, in some cases, ignore general purpose timer events.

The logical layers of a typical system using a watchdog are:

```
     +--------------------+
     | Application module |
     +--------------------+ <-- read(), write(), ioctl(), close()
               |
     +--------------------+
     | VxWorks I/O System | <-- IOS layer iosWrite, ioctl...(vxbWatchdog.c)
     +--------------------+ <-- (vxbVirtualTimerWatchdog.c)
               |
     +---------------------------------------+
     | Diff interface WATCHDOG driver module |
     +---------------------------------------+ <-- vxbI2cWatchdog.c, vxbSpiWatchdog.c
                      |_vxbFdtRsWatchdog.c
```


WNDRVR

VSB Options

None - all drivers will be built automatically depending on the platform

VIP Components

INCLUDE_WATCHDOGDRV

DRV_VIRTUAL_TIMER_WATCHDOG

DRV_FDT_RS_WATCHDOG - specific to the Renesas BSP

New or Changed APIs

No API changes associated with the watchdog IOS driver.

Note: Both **INCLUDE_WATCHDOGDRV** and **DRV_VIRTUAL_TIMER_WATCHDOG** are required for the virtual watchdog to work. If you include the **DRV_VIRTUAL_TIMER_WATCHDOG** component, **INCLUDE_WATCHDOGDRV** should also be pulled in as a dependency.

To configure and test the watchdog IOS driver, see Configuring the VIP for Watchdog I/0 Devices Overview on page 134.

11. Configuring the VIP for Watchdog I/0 Devices Overview

Choose from a list of options to enable and verify a watchdog I/O device.

About This Task

During the build process, all drivers will be built automatically depending on the platform. For additional information, see Watchdog I/O Devices Overview on page 133.

Before You Begin

You will need an existing VSB and VIP to work with. For information on creating a VSB and VIP, see: VxWorks 7 Getting Started.

Procedure

1. Enable the following component to support Watchdog IOS driver.

```
vxprj vip component add INCLUDE_WATCHDOGDRV
```

2. Choose between using a virtual watchdog or a watchdog for a Renesas R-Car platform.

Option	Description
Enable Virtual Timer Watchdog on all platforms.	`vxprj vip component add DRV_VIRTUAL_TIMER_WATCHDOG`
Enable R-Car Watchdog (on Renesas R-Car platforms).	`vxprj vip componenet add DRV_FDT_RS_WATCHDOG`

3. Choose between disable options.

Option	Description

Disable the reboot when Virtual Watchdog is not fed and times out.	This requires setting the **VIRT_WDT_NO_REBOOT** parameter to TRUE, for example:
	`vxprj vip parameter set myVip VIRT_WDT_NO_REBOOT TRUE`
	If you compile the virtual watchdog driver with DEBUG enabled, the driver uses **kprintf** to emit messages when the timer expires.
	If **VIRT_WDT_NO_REBOOT** is FALSE, then the message is:
	`WATCHDOG: Watchdog triggered. Rebooting.…`
	If **VIRT_WDT_NO_REBOOT** is TRUE, then the message is:
	`WATCHDOG: Watchdog triggered. Ignoring reboot.`
	If you compile the virtual watchdog driver without DEBUG (which is the default), then no message is printed regardless of the action taken. If **VIRT_WDT_NO_REBOOT** is TRUE, this means it looks like nothing happens.

Disable the "deactivate on close" behavior for the Virtual Watchdog device.	When **NO WAY OUT** is set, the watchdog should be pinged continuously and cannot be disabled until reboot. This requires setting the **VIRT_WDT_NO_WAY_OUT** parameter to TRUE, for example: `vxprj vip parameter set myVip VIRT_WDT_NO_WAY_OUT TRUE` The default behavior is that if you do a close() on the device, that turns off the watchdog timer. Using **NO WAY OUT** disables this behavior, so the timer remains active.
Disable the "deactivate on close" behavior for the R-Car Watchdog device.	When **NO WAY OUT** is set the watchdog should be pinged continuously and cannot be disabled until reboot. This requires setting the **RS_WDT_NO_WAY_OUT** parameter to TRUE, for example:

Option	Description
	`vxprj vip parameter set myVip RS_WDT_NO_WAY_OUT TRUE` The default behavior is that if you do a close() on the device, that turns off the watchdog timer. Using **NO WAY OUT** disables this behavior, so the timer remains active.

4. Verify the watchdog I/O device operation.
   1. Boot the board or simulator.
   2. Run the devs command to get the watchdog device ID.

```
-> devs
```

   3. Open the watchdog device to enable read-write access.

```
-> fd=open("/watchdog/1", 2)
```

   4. Write one or multiple characters to device fd to ping the watchdog.

```
-> write(fd, "xyz", 3)
```

   In order to re-arm the watchdog timer you need to perform a write operation. The contents of the write buffer don't matter and will be ignored.

   5. Wait at least 60 seconds for the board or simulator to reboot.

   The wait simulates a target lock up scenario. The target reboot confirms the unpinged watchdog has correctly triggered a target reboot.

5. PROGRAMMING I/O IN YOUR APPLICATIONS

1. I/O Devices, Named Files, and File Systems

VxWorks applications access I/O devices by opening named files.

A named file can refer to either of the following:

- A *logical file* on a structured, random-access device containing a file system.
- An unstructured *raw device* such as a serial communications channel or an inter-task pipe.

I/O can be done to or from either type of named file in the same way. For VxWorks, they are all called *files*, even though they refer to very different physical objects.

Device Naming

Non-block devices are named when they are added to the I/O system, usually at system initialization time. Block devices are named when they are initialized for use with a specific file system. For more information, see I/O System and Device Naming Conventions on page 4. For a graphical representation of how file systems and devices are represented in VxWorks see the figure in VxWorks Devices and Driver Model Overview on page 167.

The VxWorks I/O system has some restrictions on the names given to devices as described in Enforcing Strict I/O Path and Device Naming Conventions on page 11.

Devices are handled by *device drivers*. In general, using the I/O system does not require any further understanding of the implementation of devices and drivers.

> Note: The VxWorks I/O system gives drivers considerable flexibility in the way they handle each specific device. Drivers conform to the conventional user view presented here, but can differ in the specifics.

VxWorks File Path Rendering

In keeping with the *POSIX* standard, VxWorks uses a forward slash (**/**) to separate files and directories in a pathname. The forward slash is the preferred pathname component separator and is fully supported. In special cases such as when a pathname has been condensed, VxWorks allows the backslash character (****) as a pathname component separator. However, this normally applies only after condensing a pathname. Among other effects, condensing a pathname replaces backslash characters with forward slash characters.

> Note: It is highly recommended you avoid using backslashes in pathnames. Future VxWorks versions may interpret a backslash as a file name character rather than a pathname component separator.

The format for device names depends upon your VIP configuration where your choice of strict or legacy rules determines what names are permissible. For more information, see Enforcing Strict I/O Path and Device Naming Conventions on page 11. In most cases, a device name consists of a single pathname component such as **/ram0** for a RAM disk file system. However, some device names may have multiple pathname components (for example, **/pipe/mypipe**). In a system with the *Virtual Root File System* (*VRFS*) enabled, **/** may refer to the root VRFS device, **/pipe** may refer to an automatically created VRFS directory device, and **/pipe/mypipe** may be a VxWorks pipe device created using that name. In VxWorks, the process of resolving a pathname begins with finding the longest prefix of the pathname which is the name of an existing device. A method provided by the device driver managing the longest-prefix-match device is then called to help process the remainder of the pathname.

WNDRVR

Pathnames and Device Names

A pathname is a NULL-terminated string of bytes similar to a **C** string. The byte sequence does not have to conform to any character encoding, however, ASCII or UTF-8 encoding is assumed. An ASCII slash character (**/**, **0x2F**) or in special circumstances, an ASCII backslash (****, **0x5C**), separates the pathname into components (file names) not containing the slash [**/**] or backslash [****] character. Multiple consecutive pathname component separator characters are treated as only one separator. The components[**.**] and [**..**] components are special where the following applies:

[**.**] - Refers to the same directory as the pathname prefix preceding that component.

[**..**] - Refers to the parent directory of the pathname prefix preceding the [**..**] component.

Below are examples:

/pipe/mypipe

Refers to a named pipe. Using a pathname starting with **/pipe** for pipes is a convention not always followed. When VRFS is enabled, and a device named **/pipe/mypipe** is created, a VRFS directory named **/pipe** is automatically created to contain the new pipe (if the directory does not already exist). This allows you to list the **/pipe** directory to see all the files (**pipes**) it contains.

/ttyS0

Refers to a serial channel.

/usr/mydir/../mydir2/myfile

Refers to a file **myfile** in a directory **mydir2** on a file system device **/usr**.

/ata0/purple/.

Refers to the directory **purple** on the**/ata0 disk** (file system) device.

mirror:

This is usually a *netDrv* or *passfs* device for accessing files on the development host named **mirror**. This device name does not start with a [**/]**. This is allowed in the default configuration, but may be forbidden when stricter device names or strict pathname resolution is configured (see Enforcing Strict I/O Path and Device Naming Conventions on page 11). For a device name that does not start with a [**/]**, legacy rules may apply. For example, the pathname **mirror:foo** may refer to a file or directory **foo** on the device **mirror:**, even though there is no [**/**] character separating **mirror:** and **foo**.

Current Working Directory (CWD) and Default I/O Device

You can obtain the current working directory (*CWD*) by calling the getcwd( ) function. You can set the *CWD* by calling the chdir( ) function.

A relative (or non-absolute) pathname is one that does not start with a device name. If the *Virtual Root File System* (*VRFS*) is enabled and there is a root device, all pathnames starting with a [**/**] are absolute. If legacy rules are enabled, a pathname that does not start with [/] or [\ ] may be absolute, but only if it starts with a device name that does not start with [/] or [\ ]. For example, the path **mydir/myfile** may have no device matching a prefix of that path. In this case, the path is relative and it must be combined with the CWD path.

For RTPs, the CWD is a property of the RTP and all tasks belonging to the RTP share the same working directory. In most cases, this applies to kernel tasks as well. However, a kernel task can have its own private CWD if the option **VX_PRIVATE_CWD** is enabled when the task is spawned. For more information, see the **taskLib** API reference for taskSpawn( ).

What is the Default Device?

The default device is the device underlying the CWD. For example, if the CWD is **host:/home/paul**, the default device would be **host:**, or if the CWD was **/ram0/dir**, the default device would be the file-system device **/ram0**. When the result of combining an input pathname with the CWD yields a pathname that does not match any device, the I/O system, if in legacy pathname resolution mode, will finally interpret the pathname as relative to the default device. For example, if there is no root device [**/**], and the current working directory is **/ram0/dir**, and the input path was **/home/paul**, the result of combining the input path with the CWD path in legacy mode is **/home/paul**. But if no device matches **/home/paul**, the system finally tries passing the path **/home/paul** to the driver managing the default device **/ram0**, as if the combined path were **/ram0/home/paul**. This only occurs in legacy pathname resolution mode.

File I/O

VxWorks supports a POSIX derived I/O interface of open( ), close( ), read( ), write( ), ioctl( ), and others. These functions appear as system calls (or system call wrappers) in RTPs, and are normal functions in the kernel. The interface represents an open file as an integer file descriptor number returned from open( ). It is provided by the I/O system facilities in the *CORE* layer.

The standard C library also provides facilities for *buffered* I/O using APIs from stdio.h( ) such as fopen( ), fclose( ), fread( ), fwrite( ), fseek( ), fprintf( ), and several others. These APIs represent an open file as a *FILE* pointer returned from fopen( ) and provides buffering. For example, several short fwrite( ) calls result in a write( ) system call only when enough written bytes have accumulated to fill a buffer (or a complete line, depending on the buffering mode). This buffering improves performance by minimizing the number of system calls.

2. Basic I/O

Basic I/O is the lowest level of I/O in VxWorks.

These are the basic I/O calls.

Function	Description
creat( )	Creates a file.
remove( )	Deletes a file.
open( )	Opens a file (optionally, creates a file if it does not already exist.)
close( )	Closes a file.
read( )	Reads a previously created or opened file.
write( )	Writes to a previously created or opened file.
ioctl( )	Performs special control functions on files.

File Descriptors

At the basic I/O level, files are referred to by a *file descriptor*. A file descriptor is a small integer returned by a call to open( ) or creat( ). The other basic I/O calls take a file descriptor as a parameter to specify a file.

WNDRVR

Note: Some other functions can return new file descriptors such as socket( ), accept( ), dup( ), dup2( ), pipe( ), pipe2( ), and fcntl( ).

File descriptors are not global. The kernel has its own set of file descriptors and each process (RTP) has its own set. Tasks within the kernel, or within a specific process share file descriptors. The only instance in which file descriptors may be shared across these boundaries, is when one process is a child of another process or the kernel and it does not explicitly close a file using the descriptors it inherits from its parent. Processes created by kernel tasks share only the spawning kernel task's standard I/O file descriptors 0, 1 and 2. For example:

- If task **A** and task **B** are running in process **foo**, and they each perform a write( ) on file descriptor 7, they will write to the same file and device.
- If process **bar** is started independently of process **foo** (it is not **foo**'s child) and its tasks **X** and **Y** each perform a write( ) on file descriptor 7, they will likely be writing to a different file than tasks **A** and **B** in process **foo** unless both processes opened the same file getting back the same file descriptor number.
- If process **foobar** is started by process **foo** (it is **foo**'s child) and its tasks **M** and **N** each perform a write( ) on file descriptor 7, they will be writing to the same file as tasks **A** and **B** in process **foo**. However, this is only true as long as the tasks do not close the file. If they close it, and subsequently open file descriptor 7 they will operate on a different file.

  A child RTP does not inherit file descriptors when the **FD_CLOEXEC** flag is set in the parent RTP. The flag may be set on a file descriptor by opening the file descriptor with the **O_CLOEXEC** flag to open( ) or modified after opening the descriptor by using the fcntl( ) F_SETFD command to set (or clear) the **FD_CLOEXEC** bit. Some additional functions creating file descriptors also allow enabling **FD_CLOEXEC**. For example, **O_CLOEXEC** may be passed to pipe2( ) or the F_DUPFD_CLOEXEC command to fcntl( ) may be used to duplicate a file descriptor (setting **FD_CLOEXEC** for the duplicate).

Listing File Descriptors

When a file is opened, a file descriptor is allocated and returned. When the file is closed, the file descriptor is deallocated. Use iosFdShow( )to list file descriptors open in the kernel including its file name and driver number. The file name is an absolute pathname for the file if it was opened using open( ), however, file descriptors obtained in other ways may not be named with an absolute pathname. They might be unnamed or they might have a name assigned to all file descriptors of the same type.

```
-> iosFdShow
 fd name                                    drv
  3 /ttyS0                                     3 in out err
  4 (anonpipe)                                 5
  5 (socket)                                  15
  6 (socket)                                  15
  7 (socket)                                  15
  8 (socket)                                  15
  9 (socket)                                  15
 10 (unknown)                                 13
 11 (unknown)                                 13
value = 50 = 0x32 = '2' = thrdSelf + 0x2
```

In the above example:

- only file descriptor 3 was opened with open( ), therefore, its name is the path name **/ttyS0** (which is also the device name for this serial device)
- file descriptor 4 is an anonymous VxWorks pipe
- descriptors 5-9 are sockets
- the unnamed descriptors 10 and 11 are POSIX pipes created with pipe( )

File Descriptor Table in the Kernel

Specify the number of file descriptors you want available in the kernel by setting the **NUM_FILES** configuration macro (the default is 50). This macro specifies the size of the file descriptor table, which controls how many file descriptors can be used simultaneously in your system.

To avoid running out of file descriptors and to prevent errors on file creation, ensure your applications close any descriptors that are no longer in use.

You can also change the size of the file descriptor table for the kernel programmatically. Use the rtpIoTableSizeGet( ) function to read the size of the file descriptor table and then use the rtpIoTableSizeSet( ) function to change it.

> Note: These functions can be used in both the kernel and processes, The I/O system treats the kernel as a special type of process.

The calling entity (either the kernel or process) can be specified with an rtpIoTableSizeSet( ) call by setting the first parameter to zero. The new size of the file descriptor table is set with the second parameter (you can only increase the size).

File Descriptor Table in RTPs

The file descriptor tables of the kernel and of RTPs have administrative hard (maximum) and soft (current) size limits. If an RTP is spawned by the kernel, its hard limit is set to the usrFdSetFdBound( ) value and its soft limit is set to the value specified in the **RTP_FD_NUM_MAX** component in the VIP.

If an RTP is spawned by another RTP (its parent), the child RTP inherits its hard and soft file descriptor table size limits from the parent RTP. You can use the rtpIoTableSizeSet( ) function to change the current (soft) limit for either the kernel or an RTP. You can also use the setrlimit( ) function in an RTP to modify both the soft and hard limits of the **RLMIT_NOFILE** resource.

Standard Input, Standard Output, and Standard Error

There are three main file descriptors:

- 0 is used for standard input and underlies the **_stdin_** stream.
- 1 is used for standard output and underlies the **_stdout_** stream.
- 2 is used for standard error and underlies the **_stderr_** stream.

In the kernel, the standard file descriptors 0, 1, and 2 are not real file descriptors because these values are never returned by open( ). Instead, the kernel provides two mechanisms for directing I/O on these special descriptors to real kernel file descriptors numbered 3 and above.

The kernel maintains real file descriptor numbers for each of the three standard file descriptors. Each kernel task has its own mapping for descriptors 0, 1, and 2 to three corresponding file descriptor numbers. If a task's descriptor is in the set of {0, 1, 2}, global redirection is used for that descriptor. The global kernel redirections for standard descriptors 0, 1, or 2 may be modified with the **ioGlobalStdSet()** function, A kernel task's standard I/O redirections may be modified with ioTaskStdSet( ) or fetched with ioTaskStdGet( ).

> Note: When a kernel file descriptor **_fd_** = 0, 1, or 2 is used in a call by a kernel task, that call operates on the real descriptor. For example, if a kernel task calls **close (1)**, this closes the real file descriptor underlying descriptor 1 for that task. Furthermore, if the real file descriptor is shared by other tasks, they are closed as well (not a desired result in most cases).

The standard C library streams stdin, stdout, and stderr (of type FILE *) are based upon the (pseudo) file descriptors 0, 1, and 2, respectively and are redirectible on a task-specific basis as described above.

WNDRVR

Standard I/O Redirection in the Kernel

When VxWorks is initialized, the global standard I/O file descriptors, **stdin** (0), **stdout** (1) and **stderr** (2), are set in the kernel to the system console device file descriptor by default, which is usually the serial **tty** device.

Each kernel task uses these global standard I/O file descriptors by default. Therefore, standard I/O operations such as calls to printf( ) and getchar( ) use the global standard I/O file descriptors. However, standard I/O can be redirected either at the individual task level or globally for the kernel.

You can change the standard I/O of a specific task by using the ioTaskStdSet( ) function. The parameters of this function are the task ID of the task for which the change is to be made (0 is used for the calling task itself), the standard file descriptor to be redirected, and the file descriptor to direct it to.

The third argument (fileFd in this case) can be any valid open file descriptor. If it is a file system file, all the task's subsequent standard output, such as that from printf( ), is written to it.

To reset the task standard I/O back to global standard I/O, the third argument can be 0, 1, or 2.

The global standard I/O file descriptors can also be changed from the default setting, which affects all kernel tasks except for the ones that have had task-specific standard I/O file descriptors changed from the global ones.

Global standard I/O file descriptors are changed by calling ioGlobalStdSet( ). The parameters to this function are the standard I/O file descriptor to be redirected, and the file descriptor to direct it to.

You can determine the current settings of the global and any task's task standard I/O by calling ioGlobalStdGet( ) and ioTaskStdGet( ). For more information, see the VxWorks API references for these functions.

To have a task write the standard output to the **fileFd** file descriptor, use the following ioTaskStdSet( ) call:

```
ioTaskStdSet (0, 1, fileFd);
```

Use the ioGlobalStdSet( ) call to set the global standard output to **newFd** (which can be any valid open file descriptor):

```
ioGlobalStdSet (1, newFd);
```

All tasks that do not have their individual task standard output redirected are affected by this redirection, and all their subsequent standard I/O output goes to **newFd**.

Issues with Standard I/O Redirection in the Kernel

Be careful with file descriptors used for task standard I/O redirection to ensure that data corruption does not occur. Before any task's standard I/O file descriptors are closed, they should be replaced with new file descriptors by using a call to ioTaskStdSet( ).

If a task's standard I/O is set with ioTaskStdSet( ), the file descriptor number is stored in that task's memory. In some cases, this file descriptor may be closed, released by some other task or the one that opened it. Once it is released, it may be reused and opened to track a different file. Should the task holding it as a task standard I/O descriptor continue to use it for I/O, data corruption is unavoidable.

For example, when a task is spawned from a telnet or rlogin session, the task inherits the network standard I/O file descriptors. If the session exits, the standard I/O file descriptors of the network session task are closed. However, the spawned task still holds those file descriptors and continues with input and output to them. However, if the closed file descriptors are recycled and re-used by other open( ) calls, data corruption may occur possibly crashing the system. To prevent this from happening, you must redirect all spawned task I/O file descriptors before the network session is terminated.

The following example, redirects a spawned task's standard I/O to the global standard I/O from the shell before logout. The taskspawn( ) call is abbreviated to simplify presentation.

```
-> taskSpawn "someTask",......
Task spawned: id = 0x52a010, name = t4
value = 5414928 = 0x52a010
-> ioTaskStdSet 0x52a010,0,0
value = 0 = 0x0
-> ioTaskStdSet 0x52a010,1,1
value = 0 = 0x0
-> ioTaskStdSet 0x52a010,2,2
value = 0 = 0x0
-> logout
```

The following example illustrates task standard I/O redirection to other file descriptors.

```
-> taskSpawn "someTask",......
Task spawned: id = 0x52a010, name = t4
value = 5414928 = 0x52a010
-> ioTaskStdSet 0x52a010,0,someOtherFdx
value = 0 = 0x0
-> ioTaskStdSet 0x52a010,1,someOtherFdy
value = 0 = 0x0
-> ioTaskStdSet 0x52a010,2,someOtherFdz
value = 0 = 0x0
-> logout
```

Standard I/O Redirection in RTPs

If a process is spawned by a kernel task, the process inherits the standard I/O file descriptor assignments of the spawning kernel task. These may be the same as the global standard I/O file descriptors for the kernel or different task-specific standard I/O file descriptors.

If a process is spawned by another process, it inherits the standard I/O file descriptor assignments of the spawning process.

After a process has been spawned, its standard I/O file descriptors can be changed to any file descriptor that it owns.

The POSIX dup( ) and dup2( ) functions are used for redirecting standard I/O to a different file and then restoring them, if necessary (this is a different process from standard I/O redirection in the kernel).

The first function is used to save the original file descriptors so they can be restored later. The second function assigns a new descriptor for standard I/O, and can also be used to restore the original. Every duplicated file descriptor should be closed explicitly when it is no longer in use.

First use the dup( ) function to duplicate and save the standard I/O file descriptors as follows:

```
/* Temporary fd variables */
int  oldFd0;
int  oldFd1;
int  oldFd2;
int  newFd;

/* Save the original standard file descriptor. */
oldFd0 = dup(0);
oldFd1 = dup(1);
oldFd2 = dup(2);
```


WNDRVR

Then use dup2( ) to change the standard I/O files:

```
/* Open new file for stdin/out/err */
newFd = open ("newstandardoutputfile", O_RDWR, 0);

/* Set newFd to fd 0, 1, 2 */
dup2 (newFd, 0);
dup2 (newFd, 1);
dup2 (newFd, 2);
```

If the process' standard I/O must be redirected again, the preceding step can be repeated with a another new file descriptor.

If the original standard I/O file descriptors must be restored, the following procedure can be performed:

```
/* When complete, restore the original standard IO */
dup2 (oldFd0, 0);
dup2 (oldFd1, 1);
dup2 (oldFd2, 2);

/* Close them after they are duplicated to fd 0, 1, 2 */
close (oldFd0);
close (oldFd1);
close (oldFd2);
```

This redirection only affect the process in which it is done. It does not affect the standard I/O of any other process or the kernel. However, any new processes spawned by this process inherit the current standard I/O file descriptors of the spawning process as their initial standard I/O setting.

For more information, see the VxWorks API references for dup( ) and dup2( ).

Open and Close

Before I/O can be performed on a device, a file descriptor must be opened to the device by invoking the open( ) function, or creat( ). The arguments to open( ) are the filename, the type of access, and the mode (file permissions). The syntax is as follows:

fd = open ("*name* ", *flags* , *mode*);

For open( ) calls made in the kernel, the mode parameter can be set to zero if file permissions do not need to be specified.

For open( ) calls made in processes and the kernel, the mode parameter is optional. However, it must be provided if the O_CREAT open flag is used.

For both user-level and kernel calls, the file-access options that can be used with the flags parameter to open( ) are listed below.

Flag	Description
O_RDONLY	Open for reading only.
O_WRONLY	Open for writing only.
O_RDWR	Open for reading and writing.
O_CREAT	Create a file if it does not already exist.

WNDRVR

Flag	Description
O_EXCL	Error on open if the file exists and **O_CREAT** is also set.
O_SYNC	Write on the file descriptor complete as defined by synchronized I/O file integrity completion.
O_DSYNC	Write on the file descriptor complete as defined by synchronized I/O data integrity completion.
O_RSYNC	Read on the file descriptor complete at the same sync level as **O_DSYNC** and **O_SYNC** flags.
O_APPEND	Set the file offset to the end of the file prior to each write, which guarantees that writes are made at the end of the file. It has no effect on devices other than the regular file system.
O_NONBLOCK	Non-blocking I/O.
O_NOCTTY	If the named file is a terminal device, don't make it the controlling terminal for the process.
O_TRUNC	Open with truncation. If the file exists and is a regular file, and the file is successfully opened, its length is truncated to 0. It has no effect on devices other than the regular file system.
O_CLOEXEC	Sets the FD_CLOEXEC file descriptor flag for the returned file descriptor. In an RTP, any child RTP spawned from this parent will not inherit this file descriptor. There is no effect in the kernel.

CAUTION: **While the third parameter to open( ) which is mode for file permissions, is usually optional for other operating systems, it is required for the VxWorks implementation of open( ) in the kernel (but not in processes). When the mode parameter is not appropriate for a given call, it should be set to zero. Note that this can be an issue when porting software from UNIX to VxWorks.**

Special cases for open( ) file access and mode (file permissions) parameters are listed below:

- In general, you can open only preexisting devices and files with open( ). However, with NFS network, dosFs, and HRFS devices, you can also create files with open( ) by OR'ing **O_CREAT** with one of the other access flags.
- HRFS directories can be opened with the open( ) function, but only using the **O_RDONLY** flag.
- With HRFS, dosFs and NFS devices, you can use the **O_CREAT** flag to create a subdirectory by setting *mode* to **FSTAT_DIR**. Other uses of the mode parameter with dosFs devices are ignored.
- The netDrv remote filesystem device does not support the use of the **F_STAT_DIR** option in conjunction with **O_CREAT** to create a subdirectory. netDrv also does not support the use of **O_EXCL** in conjunction with **O_CREAT**.
- For NFS devices, the third parameter to open( ) is normally used to specify the mode of the file.
- While HRFS supports setting the permission mode for a file, it is not used by the VxWorks operating system. File permissions attributes are recorded in the file properties on the disk, but it is up to an application to read those properties and enforce those attributes. They are not enforced by the I/O system.
- Files can be opened with the **O_SYNC** flag, indicating that each write should be immediately written to the backing media. This flag is currently supported by the dosFs file system, and includes synchronizing the FAT and the directory entries.

WNDRVR

- The **O_SYNC** flag has no effect with HRFS. HRFS default write mode is always synchronous. HRFS high speed write mode is always asynchronous.

> Note: Drivers or file systems may or may not honor the flag values or the mode values. A file opened with **O_RDONLY** mode may in fact be writable if the driver allows it. Consult the driver or file system information for specifics.

See the VxWorks file system API references for more information about the features that each file system supports.

The open( ) function, if successful, returns a file descriptor. This file descriptor is then used in subsequent I/O calls to specify that file. The file descriptor is an identifier that is not task specific; that is, it is shared by all tasks within the memory space. Within a given process or the kernel, therefore, one task can open a file and any other task can then use the file descriptor. The file descriptor remains valid until close( ) is invoked with that file descriptor.

At that point, I/O to the file is flushed (completely written out) and the file descriptor can no longer be used by any task within the process (or kernel). However, the same file descriptor number can again be assigned by the I/O system in any subsequent open( ).

For RTPs (processes), files descriptors are closed automatically only when a process terminates. It is, therefore, recommended that tasks running in processes explicitly close all file descriptors when they are no longer needed. As noted above, there is a limit to the number of files that can be open at one time. Note that a process owns the files, so that when a process is destroyed, its file descriptors are automatically closed.

Since the kernel only terminates when the system shuts down, there is no situation analogous to file descriptors being closed automatically when a process terminates. File descriptors in the kernel can only be closed by direct command.

Possible Race Condition with Close Fixed

A task within an RTP can close a file descriptor that is in active use by other tasks in the same RTP. (This is also possible between two kernel tasks.) This is potentially risky for a couple of reasons:

- If a task within the same RTP opens a new file or device shortly after the descriptor in question is closed, the original file descriptor may be reused for the new file, and the tasks that were actively using that descriptor may unexpectedly find themselves operating on the newly opened file or device.
- POSIX says very little about what happens when one thread of a process closes a file descriptor being actively used by another thread. For instance, in the case that the second thread is blocked in a read or write on the device underlying the descriptor, what actually happens will vary depending on the device and the OS (and maybe on the OS version).

Prior to the SR0640 release, in VxWorks it was possible to encounter a (rare) race condition between one task closing a file descriptor, and another task looking up the descriptor to access the file, possibly leading to the second task accessing and corrupting or operating on kernel memory objects related to the old file, after the objects have been freed and possibly re-used.

This potential race condition is fixed in SR0640. The fix introduces a change in how the I/O system calls an I/O driver's de_close( ) method. This change may (for some drivers) result in a behavior change visible to applications, regarding what happens if the last descriptor open on a device is closed while there are tasks in the same RTP blocked in I/O operations on the device. To understand this, we need to dive into the I/O system.

When a file or device is opened by name, or when an anonymous object like a socket or pipe is created, the I/O system driver that manages such devices provides a value to the I/O system that represents the newly opened file or device. This open file handle is understood by the driver, but is opaque to the I/O system. It is typically a pointer to, or some other identifier for, a driver object that represents the open instance on the file or represents the anonymous object that is created. The I/O system associates the open file handle with a data structure that the I/O system understands, called the **FD_ENTRY**. The **FD_ENTRY** contains the open file handle, pointers to a set of functions that the driver provides to manipulate the open file, and some additional members. The I/O system (or in some cases like sockets, the driver using I/O system routines) allocates a slot in the RTP's file descriptor table and stores a pointer to the **FD_ENTRY** in that slot of the table. The index of the slot is the file descriptor that is returned from the call to open( ) or socket( ) or other APIs that produce file descriptors. On each basic I/O call like read( ) or write( ) which takes a file descriptor argument, I/O system routines are used to look up the file descriptor in the caller's file descriptor table and get the corresponding **FD_ENTRY** and then the open file handle value and the driver function to call to handle the read( ) or write( ).

Although initially only one file descriptor maps to a particular **FD_ENTRY**, file descriptors can be duplicated using APIs like dup( ), dup2( ) or fcntl( ), and file descriptors can be inherited from a parent RTP or the kernel to a spawned child RTP. In these cases you would wind up with multiple file descriptors referring to the same **FD_ENTRY** (and so referring to the same underlying open file handle). Therefore, an **FD_ENTRY** contains a reference count of the number of file descriptors open on the **FD_ENTRY** across all RTPs and the kernel. The count is incremented when a file descriptor is duplicated or inherited by a child RTP. Each time some task calls close( ) on a file descriptor open on the **FD_ENTRY**, the reference count is decremented, and when the count reaches zero, i.e. when the last file descriptor open on the **FD_ENTRY** is closed, the **FD_ENTRY** is no longer accessible (although I/O calls recently started or in progress may still have active references to the **FD_ENTRY**, the open file handle and possibly other underlying driver structures for the file).

Prior to SR0640 release, when the reference count of open file descriptors on the **FD_ENTRY** reached zero, the driver's de_close( ) method was called. The de_close( ) method would typically free the open file handle and perhaps other resources associated with the open file. (Some drivers might first wake up tasks already blocked in I/O on the same file handle.) The I/O system would then return the **FD_ENTRY** to a free pool. Any of these resources might then be reused. The I/O system took no note of the fact that recently started I/O calls might still hold active references to the **FD_ENTRY**, the open file handle from the driver or the driver's function table and might (perhaps after being pre-empted for a while) proceed to operate based upon those now obsolete references.

In SR0640 release, the I/O system maintains an additional atomic reference count in the **FD_ENTRY** of all the active references to the **FD_ENTRY**. Each time a file descriptor is looked up for a call like read( ) or write( ) (or many others), the active reference count on the looked-up **FD_ENTRY** is incremented as part of the lookup. The active reference count is decremented at the end of the read( ) or write( ) (or similar) call. There is additionally a single active reference count that is only removed when the last file descriptor open on the **FD_ENTRY** is closed.

In SR0640 release, the driver's de_close( ) method is not called as soon as there are no remaining open file descriptor references to the **FD_ENTRY**. Instead, the I/O system calls de_close( ) only when the active reference count for the **FD_ENTRY** reaches zero. Waiting until the last active reference is released before calling de_close( ) and freeing the **FD_ENTRY** is what prevents the potential corruption from recently started I/O calls accessing the now obsolete data structures. The I/O system only calls the driver's de_close( ) method when it really has no remaining references.

Now consider what happens if a read( ) call (for example) that went via the **FD_ENTRY** is blocked waiting for data when the last file descriptor open on the **FD_ENTRY** is closed. Such a read( ) call holds an active reference count on the **FD_ENTRY** until it completes. Prior to SR0640, there was at least the possibility that when the last file descriptor on the **FD_ENTRY** is closed, the de_close( ) method gets called and might potentially wake up the blocked read( ) call. However, in SR640, that cannot happen, since the de_close( ) method will not be called as long as the read( ) call has not returned and released its active reference.

While using a close( ) call to wake up blocked read( ), write( ) or similar call on the same open file handle is still not recommended, some code does do this and sometimes there is no easy alternative. To support this, SR0640 also creates a new ioctl code, FIOSHUTDOWN. The I/O system calls the driver's de_ioctl( ) method with this code (and zero argument) at the time the last descriptor open on the **FD_ENTRY** is closed, before releasing the single active reference count corresponding to the last file descriptor reference. The FIOSHUTDOWN ioctl gives the driver a chance to take actions (such as waking up blocked I/O calls and somehow marking the file handle as invalid for subsequent uses, to prevent any further calls using it from blocking) at the time the last file descriptor open on the **FD_ENTRY** is closed. Typically the FIOSHUTDOWN handler invalidates but does not free resources. The driver's de_close( ) method is called later to allow freeing resources.

In the SR0640 release, VxWorks added support for FIOSHUTDOWN to several drivers:

- VxWorks pipe and POSIX pipe drivers can now handle the FIOSHUTDOWN ioctl, to allow the last close( ) on an anonymous VxWorks pipe or POSIX pipe to wake up tasks blocked on read or write on the pipe.
- sockLib now translates the FIOSHUTDOWN ioctl to sockLib back-end's shutdownRtn( ) with the argument SHUT_RDWR, if the back-end provides shutdownRtn( ). Otherwise, the back-end's ioctlRtn( ) method is called with FIOSHUTDOWN.
- Drivers for USB serial, mouse, keyboard devices and the evdev library core also add FIOSHUTDOWN support.

Note:

If your application was using close( ) to terminate outstanding I/O calls blocked on a device for which the driver does not support the new FIOSHUTDOWN ioctl, the close( ) function will no longer terminate such blocked calls. You need to modify the driver to support the FIOSHUTDOWN ioctl or use a different method to terminate the blocked calls.

Create and Remove

File-oriented devices must be able to create and remove files as well as open existing files.

The creat( ) function directs a file-oriented device to make a new file on the device and return a file descriptor for it. The arguments to creat( ) are similar to those of open( ) except that the filename specifies the name of the new file rather than an existing one; the creat( ) function returns a file descriptor identifying the new file.

Note that with the HRFS file system the creat( ) function is POSIX-compliant, and the second parameter is used to specify file permissions; the file is opened in **O_WRONLY** mode.

With dosFs, however, the creat( ) function is not POSIX-compliant and the second parameter is used for open mode flags.

The remove( ) function deletes a named file on a file-system device. For dosFs only, files must be closed before they are removed.

With non-file-system devices, the creat( ) function performs the same function as open( ). The remove( ) function, however has no effect.

Read and Write

After a file descriptor is obtained by invoking open( ) or creat( ), tasks can read bytes from a file with read( ) and write bytes to a file with write( ). The arguments to read( ) are the file descriptor, the address of the buffer to receive input, and the maximum number of bytes to read.

The read( ) function waits for input to be available from the specified file, and returns the number of bytes actually read. For file-system devices, if the number of bytes read is less than the number requested, a subsequent read( ) returns 0 (zero), indicating end-of-file. For non-file-system devices, the number of bytes read can be less than the number requested even if more bytes are available; a subsequent read( ) may or may not return 0. In the case of serial devices and TCP sockets, repeated calls to read( ) are sometimes necessary to read a specific number of bytes. (See the reference entry for fioRead( ) in **fioLib**). A return value of **ERROR** (-1) indicates an unsuccessful read.

The arguments to write( ) are the file descriptor, the address of the buffer that contains the data to be output, and the number of bytes to be written. The syntax is as follows:

actualBytes = write (*fd* , *&buffer* , *nBytes*);

The write( ) function ensures that all specified data is at least queued for output before returning to the caller, though the data may not yet have been written to the device (this is driver dependent). The write( ) function returns the number of bytes written; if the number returned is not equal to the number requested, an error has occurred.

> **Note:** For some device types such as stream sockets, an error may not occur because a (nonzero) smaller number of bytes is returned. For example, the call may have been interrupted by a signal after bytes already have been written (a *short write*).

The user-level (RTP) read( ) and write( )functions are POSIX-compliant.

File Truncation

It is sometimes convenient to discard part of the data in a file. After a file is open for writing, you can use the ftruncate( )function to truncate a file to a specified size. Its arguments are a file descriptor and the desired length of the file in bytes. The syntax is as follows:

status = ftruncate (*fd* , *length*);

If it succeeds in truncating the file, ftruncate( )returns **OK**.

If the file descriptor refers to a device that cannot be truncated, ftruncate( )returns **ERROR**, and sets **errno** to **EINVAL**.

If the size specified is larger than the actual size of the file, the result depends on the file system. For both dosFs and HRFS, the size of the file is extended to the specified size; however, for other file systems, ftruncate( )returns **ERROR**, and sets **errno** to **EINVAL** (just as if the file descriptor referred to a device that cannot be truncated). The NFS client may allow truncating the file size larger than actual size depending on the NFS server implementation. For example, Linux NFS servers allow truncation but Windows servers do not allow truncation.

The ftruncate( ) function is part of the POSIX 1003.1b standard. It is fully supported as such by the HRFS. The dosFs implementation is, however, only partially compliant: creation and modification times are not changed.

Also note that with HRFS the *seek* position is not modified by truncation, but with dosFs the seek position is set to the end of the file.

I/O Control

The ioctl( ) function provides a flexible mechanism for performing I/O functions that are not performed by the other basic I/O calls. Examples include determining how many bytes are currently available for input, setting device-specific options, obtaining information about a file system, and positioning random-access files to specific byte positions.

The arguments to the ioctl( ) function are the file descriptor, a code that identifies the control function -requested, and an optional function-dependent argument. The syntax is as follows:

result = ioctl (*fd* , *function* , *arg*);

In the following example, the call uses the **FIOBAUDRATE** function to set the baud rate of a *tty* device to 9600:

```
status = ioctl (fd, FIOBAUDRATE, 9600);
```

The discussion of each devices (for example, Serial I/O Devices: Terminal and Pseudo-Terminal Devices Overview on page 191) summarizes the ioctl( ) functions available for that device. The ioctl( ) control codes are defined in **ioLib.h**. For more information, see the reference entries for specific device drivers or file systems.

The user-level (RTP) ioctl( ) function is POSIX-compliant. In addition, the *arg* parameter is optional. Both of the following syntax examples are legitimate calls:

```
fd = ioctl (fd, func, arg);
fd = ioctl (fd, func);
```

VxWorks also provides the user-mode posix_devctl( ) function for special devices.

Pending on Multiple File Descriptors with select()

The VxWorks select( ) function provides a UNIX- and Windows-compatible method for pending on multiple file descriptors (allowing tasks to wait for multiple devices to become active). You must add the **INCLUDE_SELECT** component to the VIP to use the **selectLib** library.

The task-level support provided by the select facility not only gives tasks the ability to simultaneously wait for I/O on multiple devices, but it also allows tasks to specify the maximum time to wait for I/O to become available. An example of using the select facility to pend on multiple file descriptors is a client-server model, in which the server is servicing both local and remote clients. The server task uses a pipe to communicate with local clients and a socket to communicate with remote clients. The server task must respond to clients as quickly as possible. If the server blocks waiting for a request on only one of the communication streams, it cannot service requests that come in on the other stream until it gets a request on the first stream. For example, if the server blocks waiting for a request to arrive in the socket, it cannot service requests that arrive in the pipe until a request arrives in the socket to unblock it. This can delay local tasks waiting to get their requests serviced. The select facility solves this problem by giving the server task the ability to monitor both the socket and the pipe and service requests as they come in, regardless of the communication stream used.

Tasks can block until data becomes available or the device is ready for writing. The select( ) function returns when one or more file descriptors are ready or a timeout has occurred. Using the select( ) function, a task specifies the file descriptors on which to wait for

WNDRVR

activity. Bit fields are used in the select( ) call to specify the read and write file descriptors of interest. When select( ) returns, the bit fields are modified to reflect the file descriptors that have become available. The macros for building and manipulating these bit fields are listed below.

Macro	Description
FD_ZERO	Zeroes all bits.
FD_SET	Sets the bit corresponding to a specified file descriptor.
FD_CLR	Clears a specified bit.
FD_ISSET	Returns non-zero if the specified bit is set; otherwise returns 0.

Note: The *bit fields* are fixed-sized **fd_set** structures. They can represent sets of file descriptors in the range 0 to **FD_SETSIZE** - 1 where **FD_SETSIZE** is the maximum of VSB parameters **FD_SETSIZE** and **NET_SOCK_MAX** (default is 2048 bytes). Unless special configuration steps are taken, the I/O system never returns file descriptors larger than **FD_SETSIZE** - 1.

Applications can use select( ) with any character I/O devices that provide support for this facility (for example, pipes, serial devices, and sockets). For more information, see the API reference entry for select( ).

Using select() in RTP Applications

```
/* selServer.c - select example
* In this example, a server task uses two pipes: one for normal-priority
* requests, the other for high-priority requests. The server opens both
* pipes and blocks while waiting for data to be available in at least one
* of the pipes.
*/

#include <vxWorks.h>
#include <selectLib.h>
#include <fcntl.h>

#define MAX_FDS 2
#define MAX_DATA 1024
#define PIPEHI   "/pipe/highPriority"
#define PIPENORM "/pipe/normalPriority"

/*********************************************************************
* selServer - reads data as it becomes available from two different pipes
*
* Opens two pipe fds, reading from whichever becomes available. The
* server code assumes the pipes have been created from either another
* task or the shell. To test this code from the shell do the following:
* -> ld < selServer.o
* -> pipeDevCreate ("/pipe/highPriority", 5, 1024)
* -> pipeDevCreate ("/pipe/normalPriority", 5, 1024)
* -> fdHi = open   ("/pipe/highPriority", 1, 0)
* -> fdNorm = open ("/pipe/normalPriority", 1, 0)
* -> iosFdShow
* -> sp selServer
```

```
* -> i

* At this point you should see selServer's state as pended. You can now
* write to either pipe to make the selServer display your message.
* -> write fdNorm, "Howdy", 6
* -> write fdHi, "Urgent", 7
*/

STATUS selServer (void)
    {
    struct fd_set readFds;      /* bit mask of fds to read from */
    int      fds[MAX_FDS];      /* array of fds on which to pend */
    int      width;             /* number of fds on which to pend */
    int      i;                 /* index for fd array */
    char     buffer[MAX_DATA];  /* buffer for data that is read *

/* open file descriptors */
    if ((fds[0] = open (PIPEHI, O_RDONLY, 0)) == ERROR)
      {
      close (fds[0]);
      return (ERROR);
      }
    if ((fds[1] = open (PIPENORM, O_RDONLY, 0)) == ERROR)
      {
      close (fds[0]);
      close (fds[1]);
      return (ERROR);
      }

/* loop forever reading data and servicing clients */
    FOREVER
       {
       /* clear bits in read bit mask */
       FD_ZERO (&readFds);

/* initialize bit mask */
       FD_SET (fds[0], &readFds);
       FD_SET (fds[1], &readFds);
       width = (fds[0] > fds[1]) ? fds[0] : fds[1];
       width++;

/* pend, waiting for one or more fds to become ready */
    if (select (width, &readFds, NULL, NULL, NULL) == ERROR)
       {
       close (fds[0]);
       close (fds[1]);
       return (ERROR);
       }

/* step through array and read from fds that are ready */
     for (i=0; i< MAX_FDS; i++)
           {
/* check if this fd has data to read */
     if (FD_ISSET (fds[i], &readFds))
           {
/* typically read from fd now that it is ready */
     read (fds[i], buffer, MAX_DATA);
```

```
/* normally service request, for this example print it */
    printf ("SELSERVER Reading from %s: %s\n",
            (i == 0) ? PIPEHI : PIPENORM, buffer);
   }
  }
 }
}
```

POSIX File System Functions

VxWorks provides POSIX I/O and file system functions for various file manipulations, for both kernel and user (RTP) applications. These functions are described below.

Function	Description
access( )	Determine accessibility of a file.
chmod( )	Change the permission mode of a file.
chown( )	Change the owner of a file.
fchown( )	Change the owner of a file.
fchmod( )	Change the permission mode of a file.
fcntl( )	Perform control functions over open files.
fdatasync( )	Synchronize the data of a file.
fpathconf( )	Determine the current value of a configurable limit.
fsync( )	Synchronize a file.
link( )	Link a file.
pathconf( )	Determine the current value of a configurable limit.
rename( )	Change the name of a file.
umask( )	Set and get the file mode creation mask.
unlink( )	Unlink a file. User-level only.

For more information, see the API references for **fsPxLib**, **ioLib**, and **umaskLib**.

3. Standard I/O

VxWorks provides a standard I/O package (**stdio.h**) with full ANSI C support that is compatible with the UNIX and Windows standard I/O packages.

Configuring VxWorks With Standard I/O

For the VxWorks kernel, the majority of the traditional standard I/O functions are provided with one VxWorks component, and small subsets of functions with other components. This modular approach allows for reducing the footprint of systems that only require the most commonly used standard I/O functions, such as printf( ), sprintf( ), and sscanf( ), by only including the component (or components) that provide these functions.

The following components provide the standard I/O functions:

INCLUDE_ANSI_STDIO

Provides ANSI standard I/O functions.

INCLUDE_FORMATTED_IO

Provides the **fioLib** library (unbuffered), which includes non-ANSI standard I/O routines such as oprintf( ) and fdprintf( ).

The functions in **fioLib** do not use the buffered I/O facilities provided by the standard I/O library **ansiStdio**. They can therefore be used even if **ansiStdio** has not been included in the configuration of VxWorks.

About Standard I/O and Buffering

The use of the buffered I/O functions provided with standard I/O can provide a performance advantage over the non-buffered basic I/O functions when an application performs many small read or write operations. (For information about non-buffered I/O, see Basic I/O on page 139).

Although the VxWorks I/O system is efficient, some overhead is associated with each low-level (basic I/O) call. First, the I/O system must dispatch from the device-independent user call (read( ), write( ), and so on) to the driver-specific function for that function. Second, most drivers invoke a mutual exclusion or queuing mechanism to -prevent simultaneous requests by multiple users from interfering with each other.

This overhead is quite small because the VxWorks primitives are fast. However, an application processing a single character at a time from a file incurs that overhead for each character if it reads each character with a separate read( ) call, as with the following syntax:

```
n = read (fd, &char, 1);
```

To make this type of I/O more efficient and flexible, the standard I/O facility implements a buffering scheme in which data is read and written in large chunks and buffered privately. This buffering is transparent to the application; it is handled automatically by the standard I/O functions and macros.

When a file is created or opened with a standard I/O function, it is commonly referred to as associating a *stream* with the file. To open a file, use fopen( ), using the following syntax:

```
fp = fopen ("/path/file", "r");
```

The fopen( ) call returns a pointer to a **FILE** object (declared as **FILE ***), which is referred to as a *file pointer*. The **FILE** object contains information required by the standard I/O library to manage the stream, including a file descriptor that is actually used for I/O, information about the buffer used for the stream, and so on. (By contrast, the low-level I/O functions simply identify a file with a file descriptor, which is a small integer.)

Note: The user-mode (RTP) implementation of fopen( ) has a limit of 20 file descriptors.

WNDRVR

A file descriptor that is already open can be associated subsequently with a **FILE** buffer by calling fdopen( ), with this syntax:

fp = fdopen (*fd*, "r");

After a file is opened with fopen( ), data can be read with fread( ), or a character at a time with getc( ), and data can be written with fwrite( ), or a character at a time with putc( ). The **FILE** buffer is deallocated when fclose( ) is called.

The functions and macros to get data into or out of a file are extremely efficient. They access the buffer with direct pointers that are incremented as data is read or written by the user. They pause to call the low-level read or write functions only when a read buffer is empty or a write buffer is full.

CAUTION: **The standard I/O buffers and pointers are** *private* **to a particular task. They are not in-terlocked with semaphores or any other mutual exclusion mechanism, because this defeats the point of an efficient private buffering scheme. Therefore, multiple tasks must not perform I/O to the same** *stdio* **FILE pointer at the same time.**

About Standard Input, Standard Output, and Standard Error

As discussed in Basic I/O on page 139, there are three special file descriptors (0, 1, and 2) that are reserved for standard input, standard output, and standard error.

Three corresponding *stdio* **FILE** buffers are provided by the standard I/O system. These buffers are shared by all kernel tasks. The read/write operations between **stdin**, **stdout**, and **stderr** are always unbuffered.

4. Other Formatted I/O

VxWorks provides additional kernel formatted I/O facilities.

Output in Serial I/O Polled Mode: kprintf()

The kprintf( ) kernel function performs formatted output in a manner similar to printf( ), except that the output characters are sent to the target's serial port in polled mode. The function is designed to be used primarily as a debug facility. It can be used during the kernel boot sequence (before interrupts are enabled or the I/O system is initialized) and within ISRs, when it is otherwise impossible to print messages to the target console using printf( ).

For more information, see the kprintf( ) API reference entry. The function is provided by the **INCLUDE_DEBUG_KPRINTF** component.

The kputs( ) function provides similar functionality with un-formatted string output. For more information, see the API reference entry. The function is provided by the **INCLUDE_DEBUG_KPUTS** component.

The optional component **INCLUDE_DEBUG_KWRITE_USER** can be used to implement output to storage media.

The kernel kprintf( ) and kputs( ) functions can be made to write to user-defined storage media (such as flash).

The following function, for example, is used to implement writing to user-reserved memory:

```
#include <vxWorks.h>
#include <string.h>
#include <sysLib.h>
STATUS usrRtn
(
char * buffer,
```


WNDRVR

```
size_t len
)
{
static off_t    offset;
char *        pDstAddr;
size_t         reservedSize;

userReservedGet (&pDstAddr, &reservedSize);
pDstAddr += offset;

if (reservedSize >= len)
    {
    bcopy (pBuffer, pDstAddr, len);
    offset += len;
    return OK;
    }
else
    return ERROR;}
```

To make use of this function, perform the following steps:

1. Add the code to a C file in any directory.
2. Create a VxWorks image project (VIP) for the appropriate BSP.
3. Add the C file to the project.
4. Add the **INCLUDE_DEBUG_KPRINTF**, **INCLUDE_DEBUG_KWRITE_USER**, and **INCLUDE_USER_RESERVED_MEMORY** components to the project.
5. Set the **DEBUG_KWRITE_USR_RTN** and **USER_RESERVED_MEM** configuration parameters, as appropriate.
6. Build the project and boot VxWorks.

When calls are made to kprintf( ) and kputs( ), output is written to user-reserved memory. Messages are written one after the other, from the start of user reserved memory. Messages can be read by dumping memory from the start of user reserved memory.

Additional Formatted I/O Functions

The user-level **fioLib** library provides additional formatted output and scanning functions (non-ANSI).

The kernel fioLib( ) library provides additional formatted but unbuffered output and scanning functions (non-ANSI).

In both cases, for example, the function printErr( ) is analogous to printf( ) but outputs formatted strings to the standard error file descriptor (2). The function fdprintf( ) outputs formatted strings to a specified file descriptor.

Message Logging

Another higher-level I/O kernel facility is provided by the kernel **logLib** library, which allows formatted messages to be logged without having to do I/O in the current task's context, or when there is no task context. The message format and parameters are sent on a message queue to a logging task, which then formats and outputs the message. This is useful when messages must be logged from interrupt level, or when it is desirable not to delay the current task for I/O or use the current task's stack for message formatting (which can take up significant stack space). The message is displayed on the console unless otherwise redirected at system startup using logInit( ) or dynamically using logFdSet( ).

The **logLib** facilities are provided by the **INCLUDE_LOGLIB** component.

WNDRVR

5. Asynchronous Input/Output

Asynchronous Input/Output (AIO) is the ability to perform input and output operations concurrently with ordinary internal processing. AIO enables you to de-couple I/O operations from the activities of a particular task when these are logically independent. The VxWorks AIO implementation meets the specification in the POSIX 1003.1b standard.

The benefit of AIO is greater processing efficiency: it permits I/O operations to take place whenever resources are available, rather than making them await arbitrary events such as the completion of independent operations. AIO eliminates some of the unnecessary blocking of tasks that is caused by ordinary synchronous I/O; this decreases contention for resources between input/output and internal processing, and expedites throughput.

VxWorks Configuration

Include AIO in VxWorks with the **INCLUDE_POSIX_AIO** and **INCLUDE_POSIX_AIO_SYSDRV** components. The second component en-ables the auxiliary AIO system driver, required for asynchronous I/O on all current VxWorks devices.

POSIX AIO Functions

The VxWorks library **aioPxLib** provides POSIX AIO functions. To access a file asynchronously, open it with the open( ) function, like any other file. Thereafter, use the file de-scriptor returned by open( ) in calls to the AIO functions. The POSIX AIO functions (and two associated non-POSIX functions) are listed in the table below.

Function	Description
aioPxLibInit( )	Initializes the AIO library (non-POSIX). Kernel only.
aioShow( )	Displays the outstanding AIO requests (non-POSIX). Kernel only.
aio_read( )	Initiates an asynchronous read operation.
aio_write( )	Initiates an asynchronous write operation.
lio_listio( )	Initiates a list of up to **LIO_MAX** asynchronous I/O requests.
aio_error( )	Retrieves the error status of an AIO operation.
aio_return( )	Retrieves the return status of a completed AIO operation.
aio_cancel( )	Cancels a previously submitted AIO operation.
aio_suspend( )	Waits until an AIO operation is done, interrupted, or timed out.
aio_fsync( )	Asynchronously forces file synchronization.

VxWorks kernel initialization code calls aioPxLibInit( ) automatically when VxWorks is configured with **INCLUDE_POSIX_AIO**. The aioPxLibInit( ) function takes **MAX_LIO_CALLS** as its only parameter, which defines the maximum number of lio_listio() ( )calls that can be outstanding at one time. When the **MAX_LIO_CALLS** configuration parameter is set to 0 (the default), the value of the **AIO_CLUST_MAX** macro is used, which is defined as 100 in **private/aioPxLibP.h**.

WNDRVR

The AIO system driver, **aioSysDrv**, is initialized automatically with the kernel function aioSysInit( ) when both **INCLUDE_POSIX_AIO** and **INCLUDE_POSIX_AIO_SYSDRV** are included in VxWorks. The **aioSysDrv** driver provides request queues independent of any particular device driver, so that you can use any VxWorks device driver with AIO. The aioSysInit( ) function takes three parameters: the number of AIO system tasks to spawn, and the priority and stack size for these system tasks. The number of AIO system tasks spawned equals the number of AIO requests that can be handled in parallel. The values of the three aioSysInit( ) parameters are set with the **MAX_AIO_SYS_TASKS**, **AIO_TASK_PRIORITY**, and **AIO_TASK_STACK_SIZE** VxWorks configuration parameters. By default these are set to zero, in which case the values of the corresponding parameters are taken from the **AIO_IO_TASKS_DFLT**, **AIO_IO_PRIO_DFLT**, and **AIO_IO_STACK_DFLT** macros. These macros are defined in **aioSysDrv.h**, with the values of 2, 50, and 0x700 , respectively.

AIO Control Block

Each of the AIO calls takes an AIO control block (**aiocb**) as an argument. The calling function must allocate space for the **aiocb**, and this space must remain available for the duration of the AIO operation. (Thus the **aiocb** must not be created on the task's stack unless the calling function will not return until after the AIO operation is complete and aio_return( ) has been called). Each **aiocb** describes a single AIO operation. Therefore, simultaneous asynchronous I/O operations using the same **aiocb** are not valid and produce undefined results.

The **aiocb** structure is defined in **aio.h**. It contains the following fields:

aio_fildes

The file descriptor for I/O.

aio_offset

The offset from the beginning of the file.

aio_buf

The address of the buffer from/to which AIO is requested.

aio_nbytes

The number of bytes to read or write.

aio_reqprio

The priority reduction for this AIO request.

aio_sigevent

The signal to return on completion of an operation (optional).

aio_lio_opcode

An operation to be performed by a lio_listio( ) call.

aio_sys

The address of VxWorks-specific data (non-POSIX).

For full definitions and important additional information, see the reference entry for **aioPxLib**.

CAUTION: **The aiocb structure and the data buffers referenced by it are used by the system to perform the AIO request. Therefore, once the aiocb has been submitted to the system, the application must not modify the aiocb structure until after a subsequent call to aio_return( ). The aio_return( ) call retrieves the previously submitted AIO data structures from the system. After the aio_return( ) call, the calling application can modify the aiocb, free the memory it occupies, or reuse it for another AIO call. If space for the aiocb is allocated from the stack, the task should not be deleted (or complete running) until the aiocb has been retrieved from the system with an aio_return( ) call.**

WNDRVR

Using AIO

The functions aio_read( ), aio_write( ), or lio_listio( ) initiate AIO operations. The last of these, lio_listio( ), allows you to submit a number of asynchronous requests (read and/or write) at one time. In general, the actual I/O (reads and writes) initiated by these functions ( )does not happen immediately after the AIO request. For this reason, their re-turn values do not reflect the outcome of the actual I/O operation, but only whether a request is successful. That is, whether the AIO function is able to put the operation on a queue for eventual execution.

After the I/O operations themselves execute, they also generate return values that reflect the success or failure of the I/O. There are two functions that you can use to get information about the success or failure of the I/O operation: aio_error( ) and aio_return( ). You can use aio_error( ) to get the status of an AIO operation (success, failure, or in progress), and aio_return( ) to obtain the return values from the individual I/O operations. Until an AIO operation completes, its error status is **EINPROGRESS**. To cancel an AIO operation, call aio_cancel( ). To force all I/O operations to the synchronized I/O completion state, use aio_fsync( ).

AIO with Periodic Checks for Completion

The following kernel code uses a pipe for the asynchronous I/O operations. The example cre-ates the pipe, submits an AIO read request, verifies that the read request is still in progress, and submits an AIO write request. Under normal circumstances, a synchronous read to an empty pipe blocks and the task does not execute the write, but in the case of AIO, we initiate the read request and continue. After the write -request is submitted, the example task loops, checking the status of the AIO requests periodically until both the read and write complete. Because the AIO control blocks are on the stack, we must call aio_return( ) before returning from aioExample( ).

Asynchronous I/O

```
/* aioEx.c - example code for using asynchronous I/O */

/* includes */

#include <vxWorks.h>
#include <stdio.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>
#include <aio.h>

/* defines */

#define BUFFER_SIZE 200

struct aiocb    aiocb_read; /* read aiocb */
struct aiocb    aiocb_write; /* write aiocb */

/*********************************************************************
* aioExample - use AIO library * This example shows the basic functions
* of the AIO library.
* RETURNS: OK if successful, otherwise ERROR.
*/

STATUS aioExample (const char *exFile)
    {
    int            fd;
    char           buffer [BUFFER_SIZE]; /* buffer for read aiocb */
    static char *  test_string = "testing 1 2 3";
    int            error;
```


WNDRVR

```
    if ((fd = open (exFile, O_CREAT | O_TRUNC | O_RDWR, 0666)) ==
         ERROR)
    {
    printf ("aioExample: cannot open %s for writing. errno 0x%x\n", exFile, errno);
    return (ERROR);
    }


    printf ("aioExample: Example file = %s\tFile descriptor = %d\n", exFile, fd);

    /* initialize read and write aiocbs */
    memset ( &aiocb_read, 0, sizeof (struct aiocb));
    memset ( buffer, 0,  sizeof (buffer));
    aiocb_read.aio_fildes = fd;
    aiocb_read.aio_buf = buffer;
    aiocb_read.aio_nbytes = BUFFER_SIZE;
    aiocb_read.aio_reqprio = 0;

    memset ( &aiocb_write, 0, sizeof (struct aiocb));
    aiocb_write.aio_fildes = fd;
    aiocb_write.aio_buf = test_string;
    aiocb_write.aio_nbytes = strlen (test_string);
    aiocb_write.aio_reqprio = 0;

    /* initiate the read */
    if (aio_read (&aiocb_read) == -1)
    printf ("aioExample: aio_read failed\n");

    /* verify that it is in progress */
    if (aio_error (&aiocb_read) == EINPROGRESS)
    printf ("aioExample: read is still in progress\n");

    /* write to pipe - the read should be able to complete */
    printf ("aioExample: getting ready to initiate the write\n");
    if (aio_write (&aiocb_write) == -1)
    printf ("aioExample: aio_write failed\n");

    /* wait til both read and write are complete */
    while ((error = aio_error (&aiocb_read) == EINPROGRESS) ||
    (aio_error (&aiocb_write) == EINPROGRESS))
    sleep (1);

    printf ("aioExample: error = %d\n", error);

    /* print out what was read */
    printf ("aioExample: message = %s\n", buffer);

/* clean up */
    if (aio_return (&aiocb_read) == -1)
    printf ("aioExample: aio_return for aiocb_read failed\n");
    if (aio_return (&aiocb_write) == -1)
    printf ("aioExample: aio_return for aiocb_write failed\n");

    close (fd);
    return (OK);
    }
```

Alternatives for Testing AIO Completion

A task can determine whether an AIO request is complete in any of the following ways:

- Check the result of aio_error( ) periodically, as in the previous example, until the status of an AIO request is no longer **EINPROGRESS**.
- Use aio_suspend( ) to suspend the task until the AIO request is complete.
- Use signals to be informed when the AIO request is complete.

The following example of kernel code is similar to the preceding aioExample( ), except that it uses signals for notification that the write operation has finished. If you test this from the shell, spawn the function to run at a lower priority than the AIO system tasks to assure that the test function does not block completion of the AIO request.

Asynchronous I/O with Signals

```
#include <vxWorks.h>
#include <stdio.h>
#include <aio.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>

/* defines */

#define BUFFER_SIZE   200
#define LIST_SIZE     1
#define WRITE_EXAMPLE_SIG_NO  25 /* signal number */
#define READ_EXAMPLE_SIG_NO  26 /* signal number */

/* forward declarations */

void writeSigHandler (int sig, struct siginfo * info, void * pContext);
void readSigHandler (int sig, struct siginfo * info, void * pContext);

static struct aiocb         aiocb_read;   /* read aiocb  */
static struct aiocb         aiocb_write;  /* write aiocb */
static struct sigaction     write_action; /* signal info */
static struct sigaction     read_action;  /* signal info */
static char                 buffer [BUFFER_SIZE]; /* aiocb read buffer */

/*********************************************************************
* aioExampleSig - use AIO library.
*
* This example shows the basic functions of the AIO library.
* Note if this is run from the shell it must be spawned. Use:
*  -> sp aioExampleSig
*
* RETURNS: OK if successful, otherwise ERROR.
*/

STATUS aioExampleSig (const char *exFile)
    {
    int            fd;
    static char *  test_string = "testing 1 2 3";
```

```
    if ((fd = open (exFile, O_CREAT | O_TRUNC| O_RDWR, 0666)) == ERROR)
    {
    printf ("aioExample: cannot open %s errno 0x%x\n", exFile, errno);
    return (ERROR);
    }

    printf ("aioExampleSig: Example file = %s\tFile descriptor = %d\n",
    exFile, fd);

    /* set up signal handler for WRITE_EXAMPLE_SIG_NO */

    write_action.sa_sigaction = writeSigHandler;
    write_action.sa_flags = SA_SIGINFO;
    sigemptyset (&write_action.sa_mask);
    sigaction (WRITE_EXAMPLE_SIG_NO, &write_action, NULL);

    /* set up signal handler for READ_EXAMPLE_SIG_NO */

    read_action.sa_sigaction = readSigHandler;
    read_action.sa_flags = SA_SIGINFO;
    sigemptyset (&read_action.sa_mask);
    sigaction (READ_EXAMPLE_SIG_NO, &read_action, NULL);

    /* initialize read and write aiocbs */

    memset ( &aiocb_read, 0, sizeof (struct aiocb));
    memset ( buffer, 0, sizeof (buffer));
    aiocb_read.aio_fildes = fd;
    aiocb_read.aio_buf = buffer;
    aiocb_read.aio_nbytes = BUFFER_SIZE;
    aiocb_read.aio_reqprio = 0;

    /* set up signal info */

    aiocb_read.aio_sigevent.sigev_signo = READ_EXAMPLE_SIG_NO;
    aiocb_read.aio_sigevent.sigev_notify = SIGEV_SIGNAL;
    aiocb_read.aio_sigevent.sigev_value.sival_ptr = (void *) &aiocb_read;

    memset ( &aiocb_write, 0, sizeof (struct aiocb));
    aiocb_write.aio_fildes = fd;
    aiocb_write.aio_buf = test_string;
    aiocb_write.aio_nbytes = strlen (test_string);
    aiocb_write.aio_reqprio = 0;

    /* set up signal info */

    aiocb_write.aio_sigevent.sigev_signo = WRITE_EXAMPLE_SIG_NO;
    aiocb_write.aio_sigevent.sigev_notify = SIGEV_SIGNAL;
    aiocb_write.aio_sigevent.sigev_value.sival_ptr =(void *) &aiocb_write;

    /* initiate the read */

    if (aio_read (&aiocb_read) == -1)
    printf ("aioExampleSig: aio_read failed\n");

    /* verify that it is in progress */
```


WNDRVR

```
        if (aio_error (&aiocb_read) == EINPROGRESS)
        printf ("aioExampleSig: read is still in progress\n");

        /* write to pipe - the read should be able to complete */

        printf ("aioExampleSig: getting ready to initiate the write\n");
        if (aio_write (&aiocb_write) == -1)
        printf ("aioExampleSig: aio_write failed\n");

        close (fd);
        return (OK);
        }

    void writeSigHandler
        (
        int                 sig,
        struct siginfo *    info,
        void *              pContext
        )
        {
        /* print out what was written */
        printf ("writeSigHandler: Got signal for aio write\n");

        /* write is complete so let's do cleanup for it here */
        if (aio_return (info->si_value.sival_ptr) == -1)
        {
        printf ("writeSigHandler: aio_return for aiocb_write failed\n");
        }
        }

    void readSigHandler
        (
        int                 sig,
        struct siginfo *    info,
        void *              pContext
        )
        {
        /* print out what was read */
        printf ("readSigHandler: Got signal for aio read\n");

        /* write is complete so let's do cleanup for it here */
        if (aio_return (info->si_value.sival_ptr) == -1)
        {
        printf ("readSigHandler: aio_return for aiocb_read failed\n");
        }
        else
        {
        printf ("aioExample: message = %s\n", buffer);
        }
        }
```

6. User Mode I/O System Overview

VxWorks provides a user mode I/O system (UMIOS) for real-time process (RTP) applications.

WNDRVR

The UMIOS is currently supported for:

- The **sockLib** library. The real-time network stack (RTNET) makes use of UMIOS with **sockLib**.
- The **memDrv** driver, which abstracts access to RTP memory as a user mode I/O system device.
- The **mvarDrv** driver, which is provided as an example or test driver.

While it is possible to access kernel-level files and devices from a UMIOS-enabled RTP as normal (with a few minor restrictions), there is no benefit to enabling the UMIOS if you are not also accessing user mode drivers such as **sockLib**, **memDrv**, **mvarDrv**, or a non-Wind River I/O system driver ported to run in the UMIOS.

Currently, the following I/O system drivers and devices, while accessible from an RTP, are implemented only in the kernel I/O system, and not with UMIOS:

- file systems
- block devices, including **ramDrv**
- TTY devices
- PTY devices
- sockets of address families that are not supported by the user mode RTNET stack
- VxWorks pipes
- POSIX pipes
- POSIX FIFOs

7. Differences Between the User Mode I/O System and the Kernel I/O System

When using the user mode I/O system (UMIOS), there are some operational differences between the UMIOS and the kernel-level I/O system that you should be aware of during your development.

The treatment of pathnames by UMIOS is somewhat different than in the kernel. For routines such as open( ) or remove( ) that specify a pathname, the UMIOS first combines the pathname with the current working directory and looks up the resulting path in the UMIOS device list. If the path matches a UMIOS-level device, the UMIOS treats the file as being UMIOS-level and allows the matching UMIOS-level device driver to handle it.

If the path does not match a UMIOS-level device, the UMIOS implementation assumes that the path corresponds to a kernel-level device, and lets the kernel look up the file by calling the _open( ) (or _remove( )) system call on the original (uncombined) pathname.

First, note that a user mode device can potentially screen a kernel-mode path. That is, a path that starts with the user mode device name is treated as a user mode file or device even if there could be a kernel mode device that also matches. Currently, this is not a significant consideration because the UMIOS supports few drivers creating named devices and no file systems. However, this could change in a future release.

Secondly, the UMIOS attempts to stay closer to POSIX behavior in dealing with pathnames than the kernel does. For instance:

- Only the forward slash character (/), not the backslash (\), is allowed as a pathname component separator.
- A path that starts with **/** is treated as absolute; such a path is not combined with the current working directory, even if the current working directory starts with a device name like **"host:"**.
- A path that does not start with **/** is treated as a relative path and a combined pathname is formed consisting of the current working directory path, followed by **/** (if the current working directory does not already end in **/**), followed by the relative path.

After (potentially) combining the input path with the current working directory, the resulting path is condensed to simplify the path and remove redundant components. As in the kernel path condensation, **//** becomes **/**, **/./** becomes **/**, and paths like **/foo/dir/../bar**

WNDRVR

become **/foo/bar**. Unlike the kernel, the UMIOS version supports only **/** as a pathname component separator, and replaces a rooted **/../** at the start of the path with **/**.

Further, UMIOS device names have to match a full pathname component. For instance, unlike in the kernel, a UMIOS device named **/foo** does not match a path **/foobar/dir/file1**, because **/foo** does not form the whole of the first pathname component **/foobar**. The **/foo** device matches a path such as **/foo/bar/tweet.txt**.

There are also some additional restrictions on UMIOS device names:

- UMIOS device names must start with /.
- UMIOS device names must be invariant under path condensation.
- UMIOS device names must be shorter than **PATH_MAX** characters.

These concerns are not relevant for user mode sockets, which do not correspond to named devices in the UMIOS. However, they may be relevant if named UMIOS-level devices are added or, in the future, if UMIOS-level file systems are added.

For more information on the kernel I/O system, see Overview of VxWorks File Systems and I/O Devices on page 1.

8. User Mode I/O-Level and Kernel-Level Files and Descriptors

When the user mode I/O system (UMIOS) is enabled for a real-time process (RTP), the RTP can access files, sockets, and devices that are managed by the kernel using kernel I/O system drivers, as well as those that are managed internally in the RTP using UMIOS drivers.

For the purposes of this documentation, the files, sockets, and devices available for access are described generically as "files". Both kernel and internally-managed files are referred to in the RTP by file descriptors. A file descriptor (FD) is an index into the RTP file descriptor table. However, when you enable the UMIOS, the RTP has two file descriptor tables: one in the kernel and one internal to the RTP. Both tables are managed by the user mode I/O system. The UMIOS keeps these two file descriptor tables consistent with each other, so that the same file descriptor value (index) is used in both the RTP in-kernel file descriptor table and in the UMIOS file descriptor table.

CAUTION: **If your RTP directly calls some raw I/O system calls like _open( ) or _close( ), rather than the public APIs (open( ) or close( )), the RTP kernel-level and UMIOS-level file descriptor tables can become inconsistent.**

The UMIOS delegates allocation of slots in the RTP file descriptor table(s) to the kernel I/O system implementation. When the RTP opens a kernel-level file, the UMIOS calls the _open( ) system call to open the file in the kernel and obtain from the kernel the file descriptor to use for the file. In the RTP in-kernel file descriptor table, the slot indexed by the file descriptor holds the real, in-kernel **FD_ENTRY** for the file. The UMIOS uses the same descriptor to refer to the file in the UMIOS, and creates a UMIOS-level **FD_ENTRY** held at the same index in the UMIOS FD table to refer to the file. However, this **FD_ENTRY** does not have the real driver information about the file (that is in the kernel). Instead, the entry is used primarily to map I/O calls on the descriptor in the RTP to corresponding system calls affecting the in-kernel file.

Conversely, when the RTP opens a file (or creates a socket, and so forth) that lives in the RTP at the UMIOS level, the UMIOS uses a special system call that causes the kernel to allocate an available file descriptor from the RTP in-kernel file table and associate it with a shared **FD_ENTRY** for an in-kernel pseudo device; this simply holds a place in the kernel-level FD table. The UMIOS uses the same slot index in the UMIOS FD table to hold the real, UMIOS-level **FD_ENTRY** with the driver information for this UMIOS-level file.

When a non-kernel RTP spawns a child RTP, the child RTP inherits all of the kernel-level file descriptors that are open in the parent. Any such inherited files are shared between the parent and the child; the same in-kernel **FD_ENTRY** is referenced in the FD tables of both RTPs.

The UMIOS-level file descriptors open in the parent, being internal to the parent RTP, cannot be inherited by the child RTP. Therefore, the child sees the corresponding file descriptors as closed.

9. User Mode I/O System Usage

VxWorks must be configured with support for the user-mode I/O system. RTP applications that make use of the user-mode I/O system must be based on a VSB that provides the support. At runtime, environment variables can be used to configure the system.

VxWorks Configuration

Support for UMIOS is provided by the USER_MODE_IOS VSB configuration option.

RTP Applications

RTP applications must be based on a VSB that includes UMIOS support.

If you have existing RTP code, you can build that code based on a UMIOS-enabled VSB to add UMIOS support. (This might be desirable when, for example, you are writing a new socket application for use with the RTNET stack.) However, you must consider the following restrictions:

- If the RTP uses the RTNET stack and sockets, it must respect the limitations of RTNET for supported sockets APIs and it must provide an appropriate RTNET stack configuration.
- The RTP must not depend on inheritance of RTNET sockets or other UMIOS-level file descriptors between RTPs.
- The RTP must not be affected by the changes in path handling for UMIOS.
- The RTP must avoid routines such as dup2( ) that are not currently supported in UMIOS.

For information about RTNET, see the *VxWorks 7 Real-Time Network Stack Programmer's Guide*.

Environment Variables

Two environment variables can be used to configure the operation of the user-mode I/O system.

Environment Variable	Description
UMIOS_NUM_FILES	This variable controls the size of the RTP UMIOS file descriptor table. If UMIOS_NUM_FILES is not specified in the RTP environment, the size defaults to 100. Otherwise, if UMIOS_NUM_FILES is at least 4, it specifies the maximum number of open file descriptors in the UMIOS-level file descriptor table for the RTP. However, your should be aware of the following caveats: **If the size of the kernel-level file descriptor table (normally determined by the VxWorks image project (VIP) parameter RTP_FD_NUM_MAX) exceeds the UMIOS file descriptor table size, the UMIOS file descriptor table size is increased to match the kernel-level file descriptor table size.** **If ioInit( ) finds the UMIOS file descriptor table size exceeds the current size of the RTP in-kernel file descriptor table, the in-kernel file descriptor table is grown (once) to match the UMIOS file descriptor table size.**

WNDRVR

Environment Variable	Description
	In either case, the **ioInit( )** code ensures that the RTP in-kernel and UMIOS-level file descriptor tables are the same size, and prevents either from growing subsequently.
UMIOS_NUM_DRIVERS	This variable controls the size of the UMIOS driver table. This is the maximum number of I/O system drivers that can be added to the UMIOS. If UMIOS_NUM_DRIVERS is not specified, this value defaults to 10.

Environment variables can be set with setenv( ) in the parent RTP, or using the *envp* argument to rtpSpawn( ), or with the rtp command -e option when the RTP is launched from the kernel command shell. For example:

```
[vxWorks *]# rtp exec -i -e UMIOS_NUM_FILES=200 myRtpApp.vxe -- 0
```

10. Limitations of User-Mode I/O System

In addition to the known limitations of I/O system driver support (such as there are no user-mode I/O system (UMIOS) existing file systems), the initial UMIOS release has some limitations and known problems for UMIOS-enabled real-time processes (RTPs).

The following issues affect RTP code, but not kernel code, when _WRS_CONFIG_USER_MODE_IOS is enabled in the VxWorks source build (VSB):

- Removable devices (as in **iosRmvLib.c**) in the UMIOS are not supported.
- The dup2( ) function currently always returns ERROR when called from a UMIOS RTP.
- The growth of an RTP's file descriptor table (after the ioInit( ) constructor) is not supported for a UMIOS-enabled RTP. Certain POSIX APIs such as stat( ), that without UMIOS might grow the RTP's file descriptor table by one slot if called when the file descriptor table is completely full, will fail rather than grow the file descriptor table when UMIOS is enabled. The rtpIoTableSizeSet( ) routine always fails to grow a UMIOS RTP's file descriptor table.
- Growing **fd_set** space in a task's select( ) context is not supported for UMIOS. Enough **fd_set** space is allocated for all file descriptors in the RTP's UMIOS file table, which does not grow hereafter.
- The _epoll( ) functions (for example, epoll_create( ), epoll_ctl( ), epoll_wait( )) are not yet supported in any RTP, whether UMIOS is enabled or not.
- Certain functions work only for kernel-level files in a UMIOS RTP. Typically, routines in this category involve functionality that, at the user level, is provided using system calls, and those system calls have not yet been adapted to allow possible implementation in the UMIOS. For example, currently the fpathconf( ) function in user mode directly calls the raw system call _fpathconf( ), resulting in this function being handled in the kernel. Even if a UMIOS-level driver were available supporting the FIOPATHCONF ioctl( ), it would not get called because fpathconf( ) has not yet been converted to allow implementation at the UMIOS level.

6. I/O DEVICE DRIVERS

1. VxWorks Devices and Driver Model Overview

VxWorks provides serial devices (pseudo and pseudo-terminal), pipes, a pseudo I/O device for accessing memory directly, null devices, and block devices. The extended block device (XBD) layer provides an interface between block device drivers and file systems.

VxWorks supports many devices as shown in the figure in Overview of VxWorks File Systems and I/O Devices on page 1.

VxWorks also provides NFS and non-NFS network devices and sockets (for more information, see Network File System (NFS) Devices Overview on page 89 and the *VxWorks 7 Network Stack Programmer's Guide*.

Filesystem Device Names

The VSB option **MAX_DEVNAME** allows you to set the maximum length of names from 5 to 50 characters (the default is 20) for block or filesystem devices managed by the FS layer. If a device name exceeds the length defined by **MAX_DEVNAME**, the extra characters are truncated. The full device pathname must be no longer than **PATH_MAX** bytes (including the trailing **NUL** byte), while pathname components of the full device pathname may be no longer than **NAME_MAX** bytes (not including pathname separators or **NUL** bytes). The **NUL** character is not used to designate the end of the device name string, therefore, use string-safe functions such as strncpy( ), strncat( ), and strncmp( ) with device names (this is true for the filesystem internal handling of device names, however, the I/O system does expect all device names to be NUL-terminated).

CAUTION: **Ensure device names are unique; devices with the same name overwrite each other in core I/O.**

CAUTION: **Because device names are recognized by the I/O system using simple substring matching, a slash (or \) should not be used alone as a device name, nor should a slash be used as any part of a device name itself.**

Note: To be recognized by the virtual root file system, device names must begin with a single leading forward-slash. For more information, see the *Overview of VRFS on page 16*.

Note: Only VxBus-compatible drivers can be used with the symmetric multiprocessing (SMP) configuration of VxWorks. For general information about VxWorks SMP and about migration, see *VxWorks 7 Application Programmer's Guide:VxWorks SMP* and *Code Migration for SMP* application note.

I/O Driver Model Overview

In many systems, the device driver supplies a few functions to perform low-level I/O functions such as reading a sequence of bytes from, or writing them to, character-oriented devices. The higher-level protocols, such as communications protocols on character-oriented devices, are implemented in the device-independent part of the I/O system. The user requests are heavily processed by the I/O system before the driver functions get control.

While this approach is designed to make it easy to implement drivers and to ensure that devices behave as much alike as possible, it has several drawbacks. The driver writer is often seriously hampered in implementing alternative protocols that are not provided by the existing I/O system. In a real-time system, it is sometimes desirable to bypass the standard protocols altogether for certain devices where throughput is critical, or where the device does not fit the standard model.

In the VxWorks I/O system, minimal processing is done on user I/O requests before control is given to the device driver. The VxWorks I/O system acts as a switch to route user requests to appropriate driver-supplied functions. Each driver can then process the raw user requests as appropriate to its devices. In addition, however, several high-level libraries are available to driver writers that implement standard protocols for both character- and block-oriented devices. Thus the VxWorks I/O system provides the best of both worlds:

while it is easy to write a standard driver for most devices with only a few pages of device-specific code, driver writers are free to execute the user requests in nonstandard ways where appropriate.

There are two fundamental types of device: *block* and *character* (or *non-block*; see Block Devices Overview on page 190. Block devices are used for storing file systems. They are random access devices where data is transferred in blocks. Examples of block devices include hard and floppy disks. Character devices are typically of the tty/sio type.

As discussed in earlier sections, the three main elements of the VxWorks I/O system are drivers, devices, and files. The following sections describe these elements in detail. The discussion focuses on character drivers; however, much of it is applicable to block devices. Because block drivers must interact with VxWorks file systems, they use a slightly different organization.

Note: This discussion is designed to clarify the structure of VxWorks I/O facilities and to highlight some considerations relevant to writing I/O drivers for VxWorks. For detailed information about writing device drivers, see the *VxWorks 7 BSP and Driver's Guide*.

The following example shows the abbreviated code for a hypothetical driver that is used as an example throughout the following discussions. This example driver is typical of drivers for character-oriented devices.

In VxWorks, each driver has a short, unique abbreviation, such as **net** or **tty**, which is used as a prefix for each of its functions. The following is an example Driver. The abbreviation for the example driver is *xx*.

```
/*********************************************************************
*
* xxDrv - driver initialization function
*
* xxDrv() initializes the driver. It installs the driver via iosDrvInstall().
* It may allocate data structures, connect ISRs, and initialize hardware
* as appropriate.
*
* Note that although for backwards compatibility reasons, iosDrvInstall()
* takes a first argument of type DRV_CREATE_PTR, this argument is unused
* by VxWorks 7. In order to handle the creat() API, used to create
* a named file within a device (usually a filesystem), the I/O system
* calls the driver xxOpen entry point with open flags of (O_CREAT |
* O_WRONLY | O_TRUNC). This is consistent with POSIX requirements on
* the behavior of creat().
*
* Note that some drivers may choose to omit some of the
* driver entry points, in which case the corresponding I/O APIs will
* report an error if called.
*/

STATUS xxDrv (void)

    {
    ...
    xxDrvNum = iosDrvInstall
                (NUL, /* create routine, unused by I/O system */
                 NUL, /* delete routine, unsupported by this driver */
                 xxOpen, /* driver open routine */
                 xxClose, /* driver close routine */
                 xxRead, /* driver read routine */
                 xxWrite, /* driver write routine */
                 xxIoctl); /* driver ioctl routine */

    ...
    }
```

```
/*********************************************************************
*
* xxDevCreate - device creation function
*
* An application or startup code calls this routine to create a named
* device in the I/O system, managed by this driver. Besides the device
* name, arguments to xxDevCreate() might contain buffer sizes,
* device addresses, or other similar driver-specific parameters
*
* The driver is responsible for mutual exclusion for the device and
* the software object that represents it.
*
* The function adds the device to the I/O system by calling iosDevAdd().
* It may also allocate and initialize data structures for the device,
* initialize semaphores, initialize device hardware, and so on.
*
* Note that this routine is NOT used to create a file within a filesystem
* device. The driver "xxOpen" routine (when passed the O_CREAT flag)
* handles that.
*/

STATUS xxDevCreate
    (
    char * name, /* device name */
    ... /* additional device specific arguments */
    )
    {
    XX_DEV_HDR * xxDev;
    ...
    xxDev = calloc (1, sizeof(*xxDev));
    ...
    status = iosDevAdd (&xxDev.devHdr, name, xxDrvNum);
    ...
    }

/*
 * The following functions implement the basic I/O functions.
 * The xxOpen() return value is meaningful only to this driver,
 * and is passed back as an argument to the other I/O functions
 * (xxClose(), xxRead(), xxWrite(), xxIoctl()).
 */

void * xxOpen
    (
    DEV_HDR * devHdr, /* DEV_HDR for the device */
    char * remainder, /* path subordinate to the device */
    int flags /* open flags (O_RDONLY, O_RDWR, O_CREAT, etc.) */
    )
    {
    /* Assuming DEV_HDR member is at start of XX_DEV_HDR structure */
    XX_DEV_HDR * xxDev = (XX_DEV_HDR *)devHdr;

    /*
     * The open routine for a simple driver that does not support
     * files or sub-units below a device might forbid a non-empty
     * path 'remainder', and simply return a pointer to the device.
```

```
     * A more complicated driver would typically allocate a structure
     * representing a file open on the device; separate mutual
     * exclusion might well be required on such a file object.
     */

    /* serial devices should have no file name part */

    if (remainder[0] != 0)
        {
        errno = EINVAL;
        return (DRV_VAL_ERROR);
        }

    /* ... check for invalid or unsupported flags (omitted) ... */

    /*
     * Whatever xxOpen() returns is passed as the first argument
     * to xxClose(), xxRead(), xxWrite(), or xxIoctl()
     * when the corresponding file descriptor is used.
     */

    return (xxDev);
    }

int xxClose
    (
    XX_DEV_HDR * xxDev
    )
    {
    /*
     * This very simple driver has nothing to do.
     *
     * The close routine would typically flush any un-flushed
     * output to the device, close connections, and so on.
     *
     * A driver that allocated a file-specific object in its
     * xxOpen entry point would free that here, or possibly arrange
     * for the object to be freed after some device-specific
     * actions.
     */
    return OK;
    }

ssize_t xxRead
    (
    XX_DEV_HDR * xxDev,
    char * buffer,
    size_t nBytes
    )
    {
    /*
     * Read up to nBytes of data from the device <xxDev> into
     * buffer.
     * Return the actual number of bytes read, or
     * ERROR if no bytes were read and an error occurred;
     * in that case, set errno appropriately.
     * The driver would need to implement blocking, if supported.
```

```
     */
    ...
    }

ssize_t xxWrite
    (
    XX_DEV_HDR * xxDev,
    const char * buffer,
    size_t nBytes
    )
    {
    /*
     * Write up to nBytes of data to the device <xxDev> from
     * the <buffer>.
     * Return the actual number of bytes written, or return
     * ERROR if no bytes were written and an error occurred;
     * in that case, set errno appropriately.
     * The driver would need to implement blocking, if supported.
     */
    ...
    }

int xxIoctl
    (
    XX_DEV_HDR * xxDev,
    int requestCode,
    _Vx_ioctl_arg_t arg
    )
    {
    int result = OK;
    XX_PHOOEY_ARG * pFoo;
    ...
    /*
     * Handle supported ioctl requeusts.
     * Most ioctls should return OK or ERROR, setting errno
     * if ERROR is returned. A small number of ioctls return
     * other values.
     */

    switch (requestCode)
        {
        ...
        case XXIOC_PHOOEY:
            /* handle XX_PHOOEY operation */
            pFoo = (XX_PHOOEY_ARG *)arg;
            ...
            break;
        ...
        default:
            result = ERROR;
            errno = ENOTSUP;
        }

    return result;
    }
```

The Driver Table and Installing Drivers

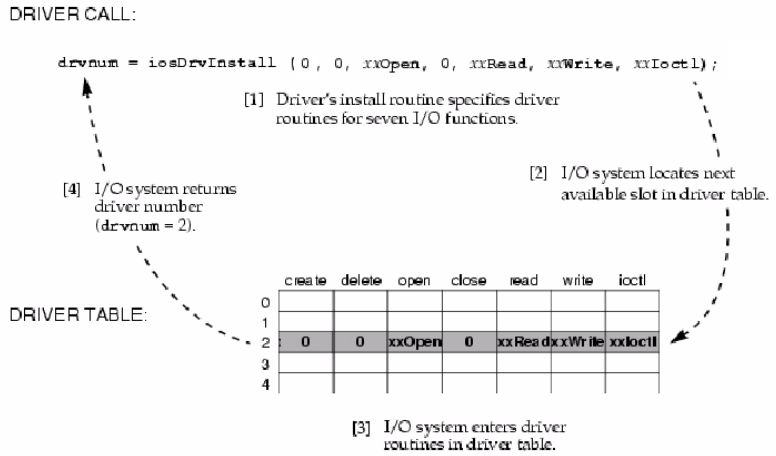
The function of the I/O system is to route user I/O requests to the appropriate function of the appropriate driver. The I/O system does this by maintaining a table that contains the address of each function for each driver. Drivers are installed dynamically by calling the I/O system internal function iosDrvInstall( ). The arguments to this function are the addresses of the seven I/O functions for the new driver. The iosDrvInstall( ) function enters these addresses in a free slot in the driver table and returns the index of this slot. This index is known as the *driver number* and is used subsequently to associate particular devices with the driver.

Nul (0) addresses can be specified for any of the seven basic I/O functions that are not supported by a device. For example, remove( ) is usually not supported for non-file-system devices, and a nul is specified for the driver's remove function.

When a user I/O call matches a nul driver function, the call fails and an **ENOTSUP** error is returned.

VxWorks file systems (such as **dosFsLib**) contain their own entries in the driver table, which are created when the file system library is initialized.

Figure 8. Example - Driver Initialization for Non-Block Devices



Example of Installing a Driver

The figure above shows the actions taken by the example driver and by the I/O system when the initialization function *xx* Drv( ) runs.

The driver calls iosDrvInstall( ), specifying the addresses of the driver's functions for the seven basic I/O functions. Then, the I/O system:

1. Locates the next available slot in the driver table, in this case slot 2.
2. Enters the addresses of the driver functions in the driver table.
3. Returns the slot number as the driver number of the newly installed driver.

Devices

Some drivers are capable of servicing many instances of a particular kind of device. For example, a single driver for a serial communications device can often handle many -separate channels that differ only in a few parameters, such as device address.

In the VxWorks I/O system, devices are defined by a data structure called a *device header* (**DEV_HDR**). This data structure contains the device name string and the driver number for the driver that services this device. The device headers for all the devices in the system are kept in a memory-resident linked list called the *device list*. The device header is the initial part of a larger structure determined by the individual drivers. This larger structure, called a *device descriptor*, contains additional device-specific data such as -device addresses, buffers, and semaphores.

The Device List and Adding Devices

Non-block devices are added to the I/O system dynamically by calling the internal I/O function iosDevAdd( ). The arguments to iosDevAdd( ) are the address of the device descriptor for the new device, the device's name, and the driver number of the driver that services the device. The device descriptor specified by the driver can contain any necessary device-dependent information, as long as it begins with a device header. The driver does not need to fill in the device header, only the device-dependent information. The iosDevAdd( ) function enters the specified device name and the driver number in the de-vice header and adds it to the system device list.

To add a block device to the I/O system, call the device initialization function for the file system required on that device. The device initialization function then calls iosDevAdd( ) automatically.

The function iosDevFind( ) can be used to locate the device structure (by obtaining a pointer to the **DEV_HDR**, which is the first member of that structure) and to verify that a device name exists in the table.

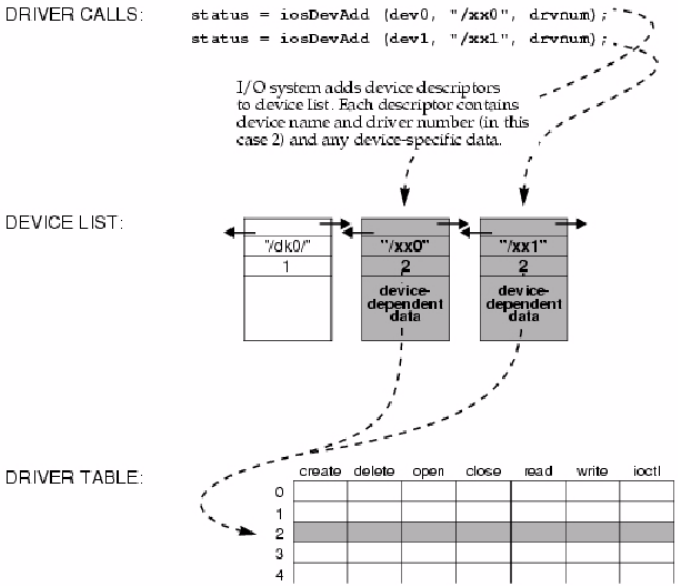
iosDevFind() Example

```
char *  pTail;                  /* pointer to tail of devName */
char devName[6] = "DEV1:";      /* name of device */
DOS_VOLUME_DESC * pDosVolDesc;  /* first member is DEV_HDR */
    ...
    pDosVolDesc = iosDevFind(devName, (char**)&pTail);
    if (NULL == pDosVolDesc)
        {
        /* ERROR: device name does not exist and no default device */
        }
    else
        {
        /*
         * pDosVolDesc is a valid DEV_HDR pointer
         * and pTail points to beginning of devName.
         * Check devName against pTail to determine if it is
         * the default name or the specified devName.
         */
        }
```

Example of Adding Devices

In the figure below, the example driver's device creation function *xx* DevCreate( ) adds devices to the I/O system by calling iosDevAdd( ).

Figure 9. Example - Addition of Devices to I/O System



Deleting Devices

A device can be deleted with iosDevDelete( ) and the associated driver removed with iosDrvRemove( ).

Note that a device-deletion operation causes the file descriptors that are open on the device to be invalidated , but not closed. The file descriptors can only be closed by an explicit act on the part of an application. If this were not the case, and file descriptors were closed automatically by the I/O system, the descriptors could be reassigned to new files while they were still being used by an application that was unaware of the deletion of the device. The new files could then be accessed unintentionally by an application attempting to use the files associated with the deleted device, as well as by an application that was correctly using the new files. This would result in I/O errors and possible device data corruption.

Because the file descriptors of a device that has been deleted are invalid, any subsequent I/O calls that use them, except close( ), will fail. The behavior of the I/O functions in this regard is as follows:

- close( ) releases the file descriptor at I/O system level and the driver close function is not called.
- read( ), write( ), and ioctl( ) fail with error **ENXIO** (no such device or address).
- While open( ), remove( ), and create( ) do not take an open file descriptor as input, they fail because the device name is no longer in the device list.

Note that even if a device is deleted and immediately added again with the same device name, the file descriptors that were invalidated with the deletion are not restored to valid status. The behavior of the I/O calls on the associated file descriptors is the same as if the device had not been added again.

Applications that are likely to encounter device deletion should be sure to check for **ENXIO** errors from read( ), write( ), and ioctl( ) calls, and to then close the relevant file descriptors.

Using Callback Functions to Manage Device Deletion

For situations in which devices are dynamically installed and deleted, the iosDevDelCallback( ) function provides the means for calling a post-deletion handler after all driver invocations are exited.

A common use of a device deletion callback is to prevent a race condition that would result from a device descriptor being deleted in one thread of execution while it was still being used in another.

A device descriptor belongs to an application, and the I/O system cannot control its creation and release. It is a user data structure with **DEV_HDR** data structure embedded at the front of it, followed by any specific member of the device. Its pointer can be used to pass into any iosDev( )*Xyz*( ) function as a **DEV_HDR** pointer, or used as the device descriptor for user device handling.

When a device is deleted, an application should not immediately release the device descriptor memory after iosDevDelete( ) and iosDrvRemove( ) calls because a driver invocation of the deleted device might still be in process. If the device descriptor is deleted while it is still in use by a driver function, serious errors could occur.

For example, the following would produce a race condition: task A invokes the driver function *xyz*Open( ) by a call to open( ) and the *xyz*Open( ) call does not return before task B deletes the device and releases the device descriptor.

However, if descriptor release is not performed by task B, but by a callback function installed with iosDevDelCallback( ), then the release occurs only after task A's invocation of the driver function has finished.

A device callback function is called immediately when a device is deleted with iosDevDelete( ) or iosDrvRemove( ) as long as no invocations of the associated driver are operative (that is, the device driver reference counter is zero). Otherwise, the callback function is not executed until the last driver call exits (and the device driver reference counter reaches zero).

A device deletion callback function should be called with only one parameter, the pointer to the **DEV_HDR** data structure of the device in question.

The callback should be installed with iosDevDelCallback( ) after the iosDevAdd( ) call.

The following code fragments illustrate callback use. The file system device descriptor **pVolDesc** is installed into the I/O device list. Its device deletion callback, fsVolDescRelease( ) performs the post-deletion processing, including releasing memory allocated for the device volume descriptor.

Callback Code

```
void fsVolDescRelease
   (
   FS_VOLUME_DESC * pVolDesc
   )
   {
   ...
   free (pVolDesc->pFsemList);
   free (pVolDesc->pFhdlList);
   free (pVolDesc->pFdList);
   ...
   }

STATUS fsDevCreate
   (
   char *  pDevName,   /* device name */
   device_t  device,   /* underlying block device */
   u_int   maxFiles,   /* max no. of simultaneously open files */
   u_int   devCreateOptions /* write option & volume integrity */
   )
   {
   FS_VOLUME_DESC  *pVolDesc = NULL; /* volume descriptor ptr */
   ...
   pVolDesc = (FS_VOLUME_DESC *) malloc (sizeof(*pVolDesc));
   pVolDesc->device = device;
   ...
   if (iosDevAdd((void *)pVolDesc, pDevName, fsDrvNum ) == ERROR)
      {
      pVolDesc->magic = NONE;
```


WNDRVR

```
      goto error_iosadd;
      }
   /* Device deletion callback installed to release memory resource. */
   iosDevDelCallback((DEV_HDR *) pVolDesc,(FUNCPTR) fsVolDescRelease);
   ...
   }

STATUS fsDevDelete
   (
   FS_VOLUME_DESC *pVolDesc    /* pointer to volume descriptor */
   )
   {
   ...
   /*
    * Delete the file system device from I/O device list. Callback
    * fsVolDescRelease will be called from now on at a
    * safe time by I/O system.
    */
   iosDevDelete((DEV_HDR *) pVolDesc);
   ...
   }

/* The application should check the error returned by a deleted device, as follows. */
if (write (fd, (char *)buffer, nbytes) == ERROR)
   {
   if (errno == ENXIO)
       {
       /* Device is deleted. fd must be closed by application. */
       close(fd);
       }
   else
       {
       /* write failure due to other reason. Do some error dealing. */
       ...
       }
       }
```

File Descriptors

Several file descriptors can be open to a single device at one time. A device driver can maintain additional information associated with a file descriptor beyond the I/O system's device information. In particular, devices on which multiple files can be open at one time have file-specific information (for example, file offset) associated with each file descriptor. You can also have several file descriptors open to a non-block device, such as a *tty*; typically there is no additional information, and thus writing on any of the file descriptors produces identical results.

File Descriptor Table

Files are opened with open( ) or creat( ). The I/O system searches the device list for a device name that matches the filename (or an initial substring) specified by the caller. If a match is found, the I/O system uses the driver number contained in the corresponding device header to locate and call the driver's open function in the driver table.

The I/O system must establish an association between the file descriptor used by the caller in subsequent I/O calls, and the driver that services it. Additionally, the driver must associate some data structure per descriptor. In the case of non-block devices, this is usually the device descriptor that was located by the I/O -system.

WNDRVR

The I/O system maintains these associations in a table called the *file descriptor table*. This table contains the driver number and an additional driver-determined 4-byte value. The driver value is the internal descriptor returned by the driver's open function, and can be any value the driver requires to identify the file. In subsequent calls to the driver's other I/O functions (read( ), write( ), ioctl( ), and close( )), this value is supplied to the driver in place of the file descriptor in the application-level I/O call.

Example of Opening a File

In the following two figures, a user calls open( ) to open the file **/***xx***0**. The I/O system takes the following series of actions:

1. It searches the device list for a device name that matches the specified filename (or an initial substring). In this case, a complete device name matches.
2. It reserves a slot in the file descriptor table and creates a new file descriptor object, which is used if the open is successful.
3. It then looks up the address of the driver's open function, *xx* Open( ), and calls that function. Note that the arguments to *xx* Open( ) are transformed by the I/O system from the user's original arguments to open( ). The first argument to *xx* Open( ) is a pointer to the device descriptor the I/O system located in the full filename search. The next parameter is the remainder of the filename specified by the user, after removing the initial substring that matched the device name. In this case, because the device name matched the entire filename, the remainder passed to the driver is a null string. The driver is free to interpret this remainder in any way it wants. In the case of block devices, this remainder is the name of a file on the device. In the case of non-block devices like this one, it is usually an error for the remainder to be anything but the null string. The third parameter is the file access flag, in this case **O_RDONLY**; that is, the file is opened for reading only. The last parameter is the mode, as passed to the original open( ) function.
4. It executes *xx* Open( ), which returns a value that subsequently identifies the newly opened file. In this case, the value is the pointer to the device descriptor. This value is supplied to the driver in subsequent I/O calls that refer to the file being opened. Note that if the driver returns only the device descriptor, the driver cannot distinguish multiple files opened to the same device. In the case of non-block device drivers, this is usually appropriate.
5. The I/O system then enters the driver number and the value returned by *xx* Open( ) in the new file descriptor object. Again, the value entered in the file descriptor object has meaning only for the driver, and is arbitrary as far as the I/O system is concerned.
6. Finally, it returns to the user the index of the slot in the file descriptor table, in this case 3.

Figure 10. Example: Call to I/O Function open() [Part 1]

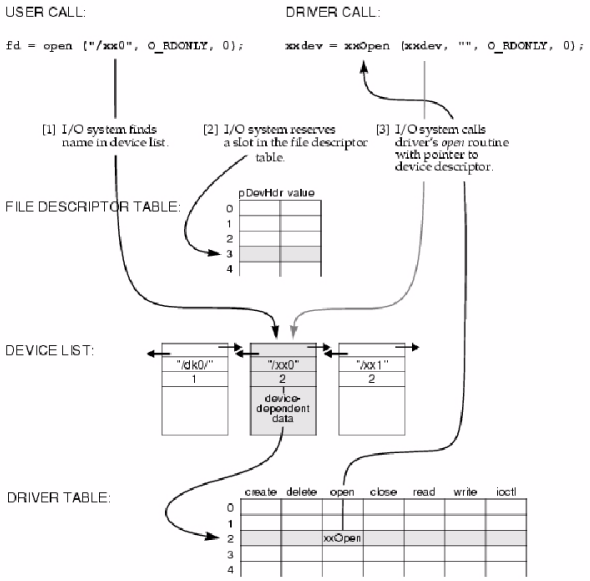
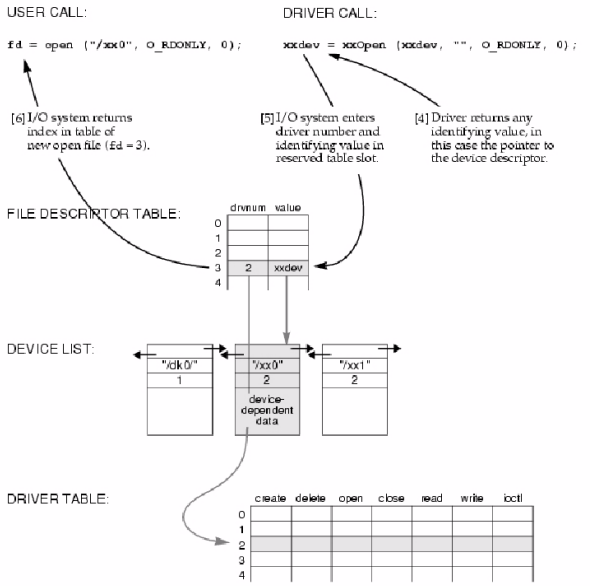


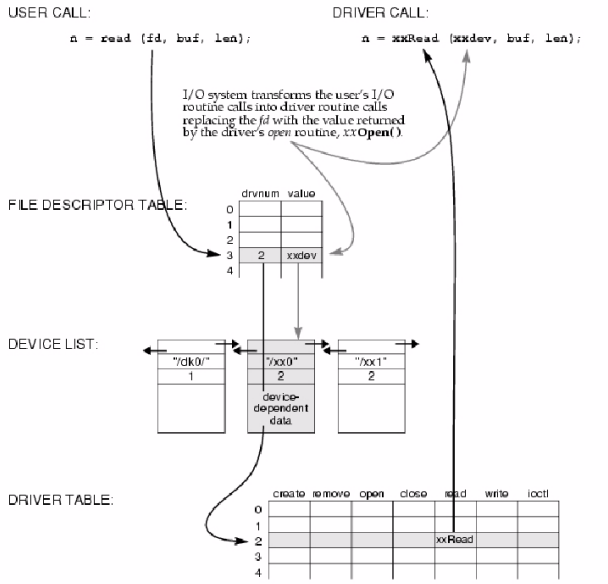
Figure 11. Example: Call to I/O Function open() [Part 2]



Example of Reading Data from the File

In the figure below, the user calls read( ) to obtain input data from the file. The specified file descriptor is the index into the file descriptor table for this file. The I/O system uses the driver number contained in the table to locate the driver's read function, *xx* Read( ). The I/O system calls *xx* Read( ), passing it the identifying value in the file descriptor table that was returned by the driver's open function, *xx* Open( ). Again, in this case the value is the pointer to the device descriptor. The driver's read function then does whatever is necessary to read data from the device. The process for user calls to write( ) and ioctl( ) follow the same procedure.

Figure 12. Example: Call to I/O Function read()



Example of Closing a File

The user terminates the use of a file by calling close( ). As in the case of read( ), the I/O system uses the driver number contained in the file descriptor table to locate the driver's close function. In the example driver, no close function is specified; thus no driver functions are called. Instead, the I/O system marks the slot in the file descriptor table as being available. Any sub-sequent references to that file descriptor cause an error. Subsequent calls to open( ) can reuse that slot.

Implementing select()

Supporting select( ) in your driver allows tasks to wait for input from multiple devices or to specify a maximum time to wait for the device to become ready for I/O. Writing a driver that supports select( ) is simple, because most of the functionality is provided in -**selectLib**. You might want your driver to support select( ) if any of the following is ap-propriate for the device:

- The tasks want to specify a timeout to wait for I/O from the device. For -example, a task might want to time out on a UDP socket if the packet never arrives.
- The driver supports multiple devices, and the tasks want to wait simultaneously for any number of them. For example, multiple pipes might be used for different data priorities.
- The tasks want to wait for I/O from the device while also waiting for I/O from an-other device. For example, a server task might use both pipes and sockets.

To implement select( ), the driver must keep a list of tasks waiting for device activity. When the device becomes ready, the driver unblocks all the tasks waiting on the device.

For a device driver to support select( ), it must declare a **SEL_WAKEUP_LIST** structure (typically declared as part of the device descriptor structure) and initialize it by calling selWakeupListInit( ). This is done in the driver's *xx* DevCreate( ) function. When a task calls select( ), **selectLib** calls the driver's ioctl( ) function with the function **FIOSELECT** or **FIOUNSELECT**. If ioctl( ) is called with **FIOSELECT**, the driver must do the following:

1. Add the **SEL_WAKEUP_NODE** (provided as the third argument of ioctl( )) to the **SEL_WAKEUP_LIST** by calling selNodeAdd( ).
2. Use the function selWakeupType( ) to check whether the task is waiting for data to read from the device (**SELREAD**) or if the device is ready to be written (**SELWRITE**).
3. If the device is ready (for reading or writing as determined by selWakeupType( )), the driver calls the function selWakeup( ) to make sure that the select( ) call in the task does not pend. This avoids the situation where the task is blocked but the device is ready.

If ioctl( ) is called with **FIOUNSELECT**, the driver calls selNodeDelete( ) to remove the provided **SEL_WAKEUP_NODE** from the wakeup list.

When the device becomes available, selWakeupAll( ) is used to unblock all the tasks waiting on this device. Although this typically occurs in the driver's ISR, it can also occur elsewhere. For example, a pipe driver might call selWakeupAll( ) from its ( ) *xx* Read( ) function to unblock all the tasks waiting to write, now that there is room in the pipe to store the data. Similarly the pipe's *xx* Write( ) function might call selWakeupAll( ) to unblock all the tasks waiting to read, now that there is data in the pipe.

Driver Code Using the Select Facility

```
/* This code fragment shows how a driver might support select(). In this
 * example, the driver unblocks tasks waiting for the device to become ready
 * in its interrupt service function.
 */

/* myDrvLib.h -
header file for driver */
```

```
typedef struct    /* MY_DEV */
    {
    DEV_HDR     devHdr;                /* device header */
    BOOL       myDrvDataAvailable;     /* data is available to read */
    BOOL       myDrvRdyForWriting;     /* device is ready to write */
    SEL_WAKEUP_LIST selWakeupList;     /* list of tasks pended in select */
    } MY_DEV;

/* myDrv.c - code fragments for supporting select() in a driver */

#include <vxWorks.h>
#include <selectLib.h>

/* First create and initialize the device */

STATUS myDrvDevCreate
    (
    char *  name,                      /* name of device to create */
    )

    {
    MY_DEV * pMyDrvDev;               /* pointer to device descriptor*/
    /* additional driver code */

    /* allocate memory for MY_DEV */
    pMyDrvDev = (MY_DEV *) malloc (sizeof MY_DEV);
    /* additional driver code */

    /* initialize MY_DEV */
    pMyDrvDev->myDrvDataAvailable=FALSE
    pMyDrvDev->myDrvRdyForWriting=FALSE

    /* initialize wakeup list */
    selWakeupListInit (&pMyDrvDev->selWakeupList);
    /* additional driver code */
    }

/* ioctl function to request reading or writing */

STATUS myDrvIoctl
    (
     MY_DEV * pMyDrvDev,                /* pointer to device descriptor */
     int      request,                  /* ioctl function */
     int      arg                       /* where to send answer */
     )
     {
     /* additional driver code */

    switch (request)
        {
        /* additional driver code */

        case FIOSELECT:

          /* add node to wakeup list */
```

```
          selNodeAdd (&pMyDrvDev->selWakeupList, (SEL_WAKEUP_NODE *) arg);

          if (selWakeupType ((SEL_WAKEUP_NODE *) arg) == SELREAD
              && pMyDrvDev->myDrvDataAvailable)
              {
              /* data available, make sure task does not pend */
              selWakeup ((SEL_WAKEUP_NODE *) arg);
              }
            if (selWakeupType ((SEL_WAKEUP_NODE *) arg) == SELWRITE
              && pMyDrvDev->myDrvRdyForWriting)
              {
              /* device ready for writing, make sure task does not pend */
              selWakeup ((SEL_WAKEUP_NODE *) arg);
              }
            break;

        case FIOUNSELECT:

          /* delete node from wakeup list */
          selNodeDelete (&pMyDrvDev->selWakeupList, (SEL_WAKEUP_NODE *) arg);
          break;

          /* additional driver code */
        }
    }

/* code that actually uses the select() function to read or write */

void myDrvIsr
    (
     MY_DEV * pMyDrvDev;
     )
     {
     /* additional driver code */

  /* if there is data available to read, wake up all pending tasks */


  if (pMyDrvDev->myDrvDataAvailable)
        selWakeupAll (&pMyDrvDev->selWakeupList, SELREAD);

  /* if the device is ready to write, wake up all pending tasks */

  if (pMyDrvDev->myDrvRdyForWriting)
        selWakeupAll (&pMyDrvDev->selWakeupList, SELWRITE);

  }
```

Cache -Coherency in Uniprocessor VxWorks

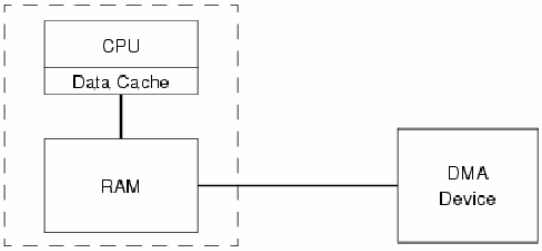
Drivers written for boards with caches must guarantee *cache coherency*.

Note: The kernel cache facilities described in this section are provided for the uniprocessor (UP) configuration of VxWorks, some of which are not appropriate (and not provided) for the symmetric multiprocessor (SMP) configuration. For more information in this regard, see *Code Migration for SMP* application note.

Cache coherency means data in the cache must be in sync, or coherent, with data in RAM. The data cache and RAM can get out of sync any time there is asynchronous access to RAM (for example, DMA device access or VMEbus access). Data caches are used to increase performance by reducing the number of memory accesses. The figure below shows the relationships between the CPU, data cache, RAM, and a DMA device.

Data caches can operate in one of two modes: *writethrough* and *copyback*. Write-through mode writes data to both the cache and RAM; this guarantees cache coherency on output but not input. Copyback mode writes the data only to the cache; this makes cache -coherency an issue for both input and output of data.

Figure 13. Cache Coherency



If a CPU writes data to RAM that is destined for a DMA device, the data can first be written to the data cache. When the DMA device transfers the data from RAM, there is no guarantee that the data in RAM was updated with the data in the cache. Thus, the data output to the device may not be the most recent as the new data may still be sitting in the cache. This data incoherence can be solved by making sure the data cache is flushed to RAM before the data is transferred to the DMA device.

If a CPU reads data from RAM that originated from a DMA device, the data read can be from the cache buffer (if the cache buffer for this data is not marked invalid) and not the data just transferred from the device to RAM. The solution to this data incoherence is to make sure that the cache buffer is marked invalid so that the data is read from RAM and not from the cache.

Drivers can solve the cache coherency problem either by allocating cache-safe buffers (buffers that are marked non-cacheable) or flushing and invalidating cache entries any time the data is written to or read from the device. Allocating cache-safe buffers is useful for static buffers; however, this typically requires MMU support. Non-cacheable buffers that are allocated and freed frequently (dynamic buffers) can result in large amounts of memory being marked non-cacheable. An alternative to using non-cacheable buffers is to flush and invalidate cache entries manually; this allows dynamic buffers to be kept -coherent.

The functions cacheFlush( ) and cacheInvalidate( ) are used to manually flush and invalidate cache buffers. Before a device reads the data, flush the data from the cache to RAM using cacheFlush( ) to ensure the device reads current data. After the device has written the data into RAM, invalidate the cache entry with cacheInvalidate( ). This guarantees that when the data is read by the CPU, the cache is updated with the new data in RAM.

DMA Transfer Function

```
/* This a sample DMA transfer function. Before programming the device
 * to output the data to the device, it flushes the cache by calling
 * cacheFlush(). On a read, after the device has transferred the data,
 * the cache entry must be invalidated using cacheInvalidate().
 */

#include <vxWorks.h>
#include <cacheLib.h>
#include <fcntl.h>
#include "example.h"

void exampleDmaTransfer  /* 1 = READ, 0 = WRITE */
    (
    UINT8 *pExampleBuf,
    int exampleBufLen,
```


WNDRVR

```
    int xferDirection
    )
    {
    if (xferDirection == 1)
        {
        myDevToBuf (pExampleBuf);
        cacheInvalidate (DATA_CACHE, pExampleBuf, exampleBufLen);
        }

    else
        {
        cacheFlush (DATA_CACHE, pExampleBuf, exampleBufLen);
        myBufToDev (pExampleBuf);
        }

}
```

It is possible to make a driver more efficient by combining cache-safe buffer allocation and cache-entry flushing or invalidation. The idea is to flush or invalidate a cache entry only when absolutely necessary. To address issues of cache coherency for static buffers, use cacheDmaMalloc( ). This function initializes a **CACHE_FUNCS** structure (defined in **cacheLib.h**) to point to flush and invalidate functions that can be used to keep the cache coherent.

The macros **CACHE_DMA_FLUSH** and **CACHE_DMA_INVALIDATE** use this structure to optimize the calling of the flush and invalidate functions. If the corresponding function pointer in the **CACHE_FUNCS** structure is NULL, no unnecessary flush/invalidate functions are called because it is assumed that the buffer is cache coherent (hence it is not necessary to flush/invalidate the cache entry manually).

The driver code uses a virtual address and the device uses a physical address. Whenever a device is given an address, it must be a physical address. Whenever the driver accesses the memory, it must use the virtual address.

The device driver should use **CACHE_DMA_VIRT_TO_PHYS** to translate a virtual address to a physical address before passing it to the device. It may also use **CACHE_DMA_PHYS_TO_VIRT** to translate a physical address to a virtual one, but this process is time-consuming and non-deterministic, and should be avoided whenever possible.

Address-Translation Driver

```
/* The following code is an example of a driver that performs address
 * translations. It attempts to allocate a cache-safe buffer, fill it, and
 * then write it out to the device. It uses CACHE_DMA_FLUSH to make sure
 * the data is current. The driver then reads in new data and uses
 * CACHE_DMA_INVALIDATE to guarantee cache coherency.
 */

#include <vxWorks.h>
#include <cacheLib.h>
#include "myExample.h"
STATUS myDmaExample (void)

   {
   void * pMyBuf;
   void * pPhysAddr;

   /* allocate cache safe buffers if possible */
   if ((pMyBuf = cacheDmaMalloc (MY_BUF_SIZE)) == NULL)
   return (ERROR);
```

```
/* add code to fill buffer with useful information */
...

/* flush cache entry before data is written to device */
CACHE_DMA_FLUSH (pMyBuf, MY_BUF_SIZE);

/* convert virtual address to physical */
pPhysAddr = CACHE_DMA_VIRT_TO_PHYS (pMyBuf);

 /* program device to read data from RAM */
 myBufToDev (pPhysAddr);

 /* add code to wait for DMA to complete */
 ...

 /* ready to read new data */
 ...

 /* program device to write data to RAM */
 myDevToBuf (pPhysAddr);

 /* add code to wait for transfer to complete */
 ...

 /* convert physical to virtual address */
 pMyBuf = CACHE_DMA_PHYS_TO_VIRT (pPhysAddr);

 /* invalidate buffer */
 CACHE_DMA_INVALIDATE (pMyBuf, MY_BUF_SIZE);

 /* add code to use data */
 ...

 /* when done free memory */
 if (cacheDmaFree (pMyBuf) == ERROR)
     return (ERROR);

 return (OK);
 }
```

2. Examining the /dev Directory and Getting the Device Name Prefix

You can use the pathIsAbsolute( ) API or the ioctl( ) command **FIOGETDEVNAME** to get the device name prefix.

To examine the **/dev** directory, you must first prepare your project by selecting the **INCLUDE_DEV_DRV** component in your VIP. Use the **/dev** directory to discover mounted file systems and devices that are not part of a file system, such as pipes.

In some cases, it can be difficult to determine which portion of an absolute path is the device name prefix, because VxWorks allows mount points or devices to contain a forward slash (**/**), which make a path look like it contains multiple directories.

POSIX-conforming systems begin absolute paths with a forward slash (**/**). However, when not in PSE52 mode, VxWorks allows I/O devices to begin with any alphanumeric character.

Use the pathIsAbsolute( ) or the ioctl( ) command **FIOGETDEVNAME** to determine if a path begins with a VxWorks device name. You can call pathIsAbsolute( ) in either the kernel or in user space.

Calling pathIsAbsolute( ) in the Kernel

In the kernel, pathIsAbsolute( ) calls iosDevMatch( ) to find the kernel device whose name matches the beginning of the file path.

If iosDevMatch( ) finds an existing kernel device, pathIsAbsolute( ) returns **TRUE**. If the second argument of pathIsAbsolute( ) is not a **NULL** pointer, pathIsAbsolute( ) sets a pointer, which points to the portion of the file path after the matched device name, to the address passed as the second argument.

If iosDevMatch( ) does not find any existing kernel device, pathIsAbsolute( ) returns **FALSE**.

Calling pathIsAbsolute( ) in User Space

If **USER_MODE_IOS** is enabled in your VSB, pathIsAbsolute( ) calls iosDevMatch( ) to find an existing device in user space whose name matches the beginning of the file path first and returns **TRUE** if iosDevMatch( ) finds one.

If the second argument of pathIsAbsolute( ) is not a **NULL** pointer, pathIsAbsolute( ) sets a pointer, which points to the portion of the file path after the matched device name, to the address passed as the second argument.

If the iosDevMatch( ) does not find any existing device, pathIsAbsolute( ) calls the **_pathIsAbsolute** system call (unless arguments passed to pathIsAbsolute( ) are invalid) and returns the return value from the system call.

USER_MODE_IOS is enabled by default in a VSB project. If it is disabled, however, pathIsAbsolute( ) simply calls the **_pathIsAbsolute** system call.

Interpreting the Return Values

If a relative file path (one that starts without a leading **/**, **./** or **../**) is passed to pathIsAbsolute( ) and there is a device whose name is *x*-byte length and the first *x*-byte string of the file path matches the device name, pathIsAbsolute( ) function treats the file path as an absolute path.

For example, assume there is a directory whose name is **dirA** and a device whose name is **dirA** on different paths. The relative path **dirA** or **dirA/***file* will be treated as absolute path by pathIsAbsolute( ), and the return value of pathIsAbsolute(( )"dirA/file", &pNameTail) will be **TRUE** (and the pNameTail will point to the forward slash (**/**) in the **dirA/***file* file path), even if a **dirA** directory resides on the current path.

If there is no device whose name is *x*-byte length and the first *x*-byte string of the file path matches to the device name, pathIsAbsolute( ) treats it as a relative path, whether the file path starts from the leading forward slash (**/**) or not.

The component **INCLUDE_VRFS** provides the **/** device, and pathIsAbsolute( ) treats the file path that start from a leading forward slash (**/**) as an absolute path when **INCLUDE_VRFS** is included. Note that **INCLUDE_VRFS** requires the **FS_VRFS** VSB option, which is disabled by default.

For example, if a path does not contain a forward slash (**/**) or a device named **/devB**, the result of pathIsAbsolute(( )"/", …) or pathIsAbsolute(( )"/devB", …) will be **FALSE**. If the path does contain a **/** device (which is created by the **INCLUDE_VRFS** component), pathIsAbsolute(( )"/devB", &pNameTail) will be **TRUE** (and the pNameTail will point to the **d** in the **/devB** file path) regardless of the presence of the **/devB** device.

Example

The following example uses pathIsAbsolute( ) to distinguish an absolute path from a relative path:

```
#include <limits.h>
#include <stdio.h>
#include <string.h>
```

```
#include <stdlib.h>
#include <ioLib.h>
#include <pathLib.h>

int testAbsolutePath (const char * pFilePath)
    {
    char * pAbsolutePath = NULL;
    char   pathBuf [PATH_MAX];
    int    fd;
    int    retVal = -1;         /* default,  wrong path */

    if (pFilePath == NULL)
        {
        return (retVal);
        }

    /* test if the file path is absolute path */
    if (pathIsAbsolute (pFilePath, NULL) == TRUE)
        {
        printf ("%s is absolute path", pFilePath);

        if ((fd = open (pFilePath, O_RDONLY, 0777)) != -1)
            {
            /* get device name */
            if (ioctl (fd, FIOGETDEVNAME, pathBuf) == 0)
                {
                printf (" on %s device", pathBuf);
                }
            close (fd);
            fd = -1;
            }
        printf (".\n");

        /*
         * If the file path does not start from /, test if there is
         * a file or directory at the file path from the current path.
         */
        if (*pFilePath != '/')
            {
            strcpy (pathBuf, "./");
            strlcpy (&pathBuf[2], pFilePath, PATH_MAX - 2);
            fd = open ((const char *) pathBuf, O_RDONLY, 0777);
            }

            if (fd == -1)
            {
            retVal = 1;         /* file path is absolute path */
            }
            else
            {
            getcwd (pathBuf, PATH_MAX);
            printf ("There is also %s file or directory exists on "
                    "the current path %s.\n", pFilePath, pathBuf);
            retVal = 2;         /* there is a file or dir at the file path */
            close (fd);
            }
        }
```

```
    else
        {
        /* get absolute path from relative path */
        pAbsolutePath = realpath (pFilePath, pathBuf);

        if (pAbsolutePath != NULL)
            {
            printf ("%s is relative path.\n", pFilePath);
            printf ("Absolute path is %s.\n", pAbsolutePath);
            retVal = 3;         /* file path is relative path */
            }
        }

    return (retVal);
    }
```

Getting the Device Name Prefix with FIOGETDEVNAME

Alternately, you can use the ioctl( )command **FIOGETDEVNAME** to get the device name prefix. Use this method when you have a file open, which gives you a file descriptor for it. The **FIOGETDEVNAME** command will get the name of the file system device it is on. You can use **FIOGETDEVNAME** in the kernel or in user space.

When you use **FIOGETDEVNAME**, the device name of an open file descriptor is stored in the buffer passed to the ioctl( ) function as the third argument when its second argument is **FIOGETDEVNAME**. The device name of the open file descriptor can be up to a maximum of (*NAME_NAX*+1) bytes.

If the third argument is a **NULL** pointer, the ioctl( ) function returns **-1** with the **EINVAL** errno set.

Using FIOGETDEVNAME in User Space

You can use **FIOGETDEVNAME** in user space either with or without the **USER_MODE_IOS** option in your VSB project.

USER_MODE_IOS is enabled by default in a VSB project. If it is disabled, however, the ioctl( ) function calls the **_ioctl** system call.

If the third argument of the ioctl( ) function is not a **NULL** pointer but the entire buffer, the size must be (*NAME_MAX*+1) bytes or greater. If the string buffer pointed to by the third argument is not write-accessible in user mode, the ioctl( ) function shall return **-1** with the **EINVAL** errno set.

Example

The following example prints the name of the device where *dir/file* resides.

```
#include <limits.h>
#include <stdio.h>
#include <ioLib.h>

        …

const char * pFilePath = "dir/file";
int          fd;
char         devName [NAME_MAX + 1];
    /* open dir/file on a device */
    fd = open (pFilePath, O_RDONLY, 0777);
    if (fd != -1)
```

```
        {
        /* get device name */
        if (ioctl (fd, FIOGETDEVNAME, devName) == 0)
            {
            printf ("%s is on %s device\n", pFilePath, devName);
            }
        close (fd);    /* close the file descriptor */
        }
```

3. Mounting a Device

Mount points are created when a storage device is detected and an I/O path is added to the system.

If a storage device is detected and an I/O path is added to the system, an IOS device instance is created by **iosDevReplaceExt()** when the file system is mounted (**dosFsDevCreate**, **_cdromFsDevCreate**, **mountInsDev**).

If a device does not contain media, the device driver defines the behavior. For example, a USB drive creates a zero-size device where a SD card does not create a device. If a SD card is inserted into an attached USB device, the USB monitor task detects a device has been plugged-in and the SD card uses the VxBus method (for more information, see the *VxWorks 7 BSP and Driver Guide: VxBus Driver Methods*).

If the file system starts at block 0 for a device, the FS monitor calls **gptCreatePartDev/xpInitialize** to probe the partition table. If a partition table does not exist, **xpInitialize()** returns `ENOENT` to **xbdPartitionDevCreate** to mount the whole device. Likewise, if the partition index is 0, **gptCreatePartDev()** returns `ENOENT` to **xbdGptPartDevCreate()**.

In VxWorks, the partition index starts from 1 (not 0). This means:

```
/dev1:1 -> /dev1a
/dev1:2 -> /dev1b
```

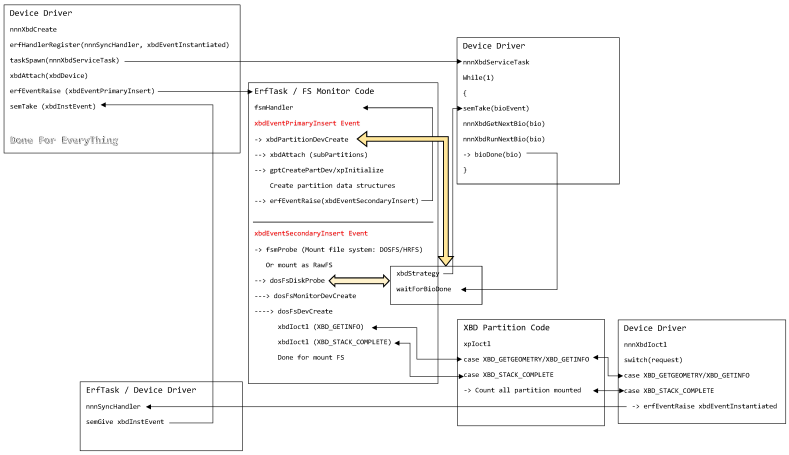
/dev1:0 means the whole device including the partition table. Typically, there is a name map for **/xxx:0** to **/xxx**. Therefore, the devs command results in **/bd0:0** being displayed as **/bd0**. For more information, see Linking to Legacy Device Names on page 14.

Some legacy devices have a partition table such as a hard disk and some do not have a partition table such as floppy disks and CDROMs. However, USB devices are an exception. Some USB disks have a partition table but some older USB disks do not have a partition table. The file system monitor (FSM) tries to probe a partition table for every type of device. If the partition table probe fails, it attempts to mount the whole device as a single device without partitions.

For a devices that do not have a partition table, the internal name (the XBD name, not the I/O name) is used (for example, **/dev1:0**). Meaning, the device does not have partition table so it is considered a single XBD device. The device driver installs a name map for this kind of device: map **/dev1:0** to **/dev1** (removes the partition index) by **fsNameInstall()**.

The following figure shows the workflow for mounting an XBD device.

WNDRVR



For information on how mounting occurs on specific file systems, see the Configuring File Systems section of this guide.

Below shows an example of **xpPartShowAll()** macro where **/ram0** does not have partition table and /**ata1** and /**bd0** have partitions.

```
-> xpPartShowAll
XBD Base name [/ram0] FSM Name [/ram0]
Device geometry:
Total Blocks [2048] Bytes per sector [512]
CHS: Cylinder [128] Head [1] Sector [16]

Mount state:
Idx.  IO Path     XBD Name        Start Sector       End Sector     Sector Count
--------------------------------------------------------------------------------
No partition is mounted

MBR Info:
Idx. Boot  Type  Start Sector   End Sector  Sector Count  CHS(Cyl. Head Sec.)
--------------------------------------------------------------------------------
Invalid entry
Invalid entry
Invalid entry
Invalid entry


XBD Base name [/ata1] FSM Name [/ata1a]
Device geometry:
Total Blocks [312500000] Bytes per sector [512]
CHS: Cylinder [16383] Head [16] Sector [63]

Mount state:
Idx.  IO Path     XBD Name        Start Sector       End Sector     Sector Count
--------------------------------------------------------------------------------
Total Not Valid   /ata1:0                    0        312499999        312500000
1     /ata1a      /ata1a                   128         65625087         65624960
2     /ata1b      /ata1b              65625088        312499839        246874752

MBR Info:
Idx. Boot  Type  Start Sector   End Sector  Sector Count  CHS(Cyl. Head Sec.)
--------------------------------------------------------------------------------
   1   00    0C           128     65625087      65624960    0,  2, 3  127,  4, 4
   2   00    0C      65625088    312499839     246874752  127,  4, 5  605, 58,58
```

```
Empty entry
Empty entry


XBD Base name [/bd0] FSM Name [/bd0a]
Device geometry:
Total Blocks [15728640] Bytes per sector [512]
CHS: Cylinder [249660] Head [1] Sector [63]

Mount state:
Idx.  IO Path     XBD Name        Start Sector       End Sector     Sector Count
--------------------------------------------------------------------------------
Total Not Valid   /bd0:0                     0         15728639         15728640
1     /bd0a       /bd0a                    128           157311           157184
2     /bd0b       /bd0b                 157312         15728511         15571200

MBR Info:
Idx. Boot  Type  Start Sector   End Sector  Sector Count  CHS(Cyl. Head Sec.)
--------------------------------------------------------------------------------
   1   80    06           128       157311        157184    0,  2, 3    9,193, 1
   2   00    0C        157312     15728511      15571200    9,193, 2  975, 58,58
Empty entry
Empty entry
```

4. Block Devices Overview

A physical *block device* is a device that is organized as a sequence of individually accessible blocks of data. The most common type of block device is a disk. In VxWorks, the term *block* refers to the smallest addressable unit on the device. For most disk devices, a VxWorks block corresponds to a *sector*, although terminology varies.

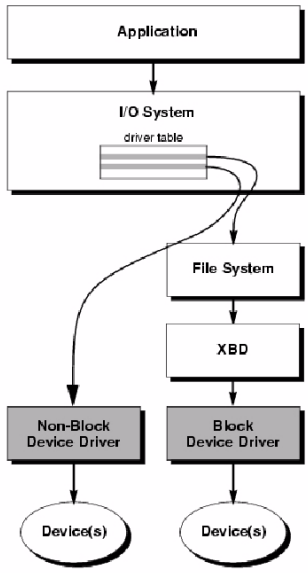
Block devices in VxWorks have a slightly different interface than other I/O devices. Rather than interacting directly with the I/O system, the I/O activity of block device drivers is mediated by the extended block device (XBD) facility and a file system. The XBD facility provides a standard interface for block device drivers on the one hand, and for file systems on the other. For more information, see Extended Block Device Facility: XBD Overview on page 79.

The figure below shows a layered model of I/O for both block and non-block (character) devices. This architecture allows the same block device driver to be used with different file systems, and reduces the number of I/O functions that must be supported in the driver.

For information about block device drivers and how to develop them, see the *VxWorks 7 BSP and Driver's Guide*.

Figure 15. Non-Block Devices and Block Devices

WNDRVR



5. Sockets Overview

In VxWorks, the underlying basis of network communications is *sockets*.

A socket is an endpoint for communication between tasks; data is sent from one socket to another. Sockets are not created or opened using the standard I/O functions. Instead, they are created by calling socket( ), and connected and accessed using other functions in **sockLib**.

The value returned by socket( ) as the socket handle is in fact an I/O system file descriptor. After creation (and for TCP sockets, typically after connection), the socket can be accessed using the normal I/O system APIs read( ), write( ), ioctl( ), and close( ), as well as additional sockets-specific APIs.

VxWorks socket functions are source-compatible with the BSD 4.4 UNIX socket functions and the Windows Sockets (Winsock 1.1) networking standard. Use of these functions is discussed in *VxWorks 7 Sockets Programmer's Guide*.

6. Serial I/O Devices: Terminal and Pseudo-Terminal Devices Overview

VxWorks provides terminal and pseudo-terminal devices (*tty* and *pty*). The *tty* device is for actual terminals; the *pty* device is for processes that simulate terminals. These pseudo terminals are useful in applications such as remote login facilities.

VxWorks serial I/O devices are buffered serial byte streams. Each device has a ring buffer (circular buffer) for both input and output. Reading from a *tty* device extracts bytes from the input ring. Writing to a *tty* device adds bytes to the output ring. The size of each ring buffer is specified when the device is created during system initialization.

Note: For the remainder of this section, the term *tty* is used to indicate both *tty* and *pty* devices.

tty Options

The *tty* devices have a full range of options that affect the behavior of the device. These options are selected by setting bits in the device option word using ioctl( ) with the **FIOSETOPTIONS** function.

For example, to set all of the *tty* options except **OPT_MON_TRAP**, the call would be made like this:

```
status = ioctl (fd, FIOSETOPTIONS, OPT_TERMINAL & ~OPT_MON_TRAP);
```

The options listed in the table below are defined in the **ioLib.h** header file. For more information, see the API reference entry for **tyLib**.

Option	Description
OPT_LINE	Selects *line mode*. (For information about raw mode and line mode , see below.)
OPT_ECHO	Echoes input characters to the output of the same channel.
OPT_CRMOD	Translates input **RETURN** characters into **NEWLINE** (\n); translates output **NEWLINE** into **RETURN-LINEFEED**.
OPT_TANDEM	Responds to software flow control characters **CTRL+Q** and **CTRL+S** (**XON** and **XOFF**).
OPT_7_BIT	Strips the most significant bit from all input bytes.
OPT_MON_TRAP	Enables the special *ROM monitor trap* character, **CTRL+X** by -default.
OPT_ABORT	Enables the special kernel shell abort character, **CTRL+C** by default. (Only useful if the kernel shell is configured into the system)
OPT_TERMINAL	Sets all of the above option bits.
OPT_RAW	Sets none of the above option bits.

Polled Mode

By default, **tyLib** supports the read( ) and write( ) functions in interrupt mode. To provide both interrupt and polled mode support, the VSB project must be configured with the **DUAL_MODE_TTY** option.

Polled mode can then be enabled by using the ioctl( ) **SIO_MODE_SET** command with the **SIO_MODE_POLL** option. For example:

```
status = ioctl (fd, SIO_MODE_SET, SIO_MODE_POLL);
```

In polled mode, read and write operations time out if a character is not sent or received in the expected amount of time. The function that calls read( ) or write( ) must be coded to retry after a timeout, if required.

The calculation of the polled-mode timeout is based on the baud rate. For an accurate timeout measurement, the high resolution clock support must be enabled in the VIP configuration (with the **HIGH_RES_POSIX_CLOCK** configuration parameter set to **TRUE**). Without the high resolution clock, the timeout will be rounded to system tick granularity. For example, if the system clock is running at 1000 ticks per second, the timeout granularity is one millisecond.

Polled mode does not support the **OPT_TANDEM** (automatic handling of X-on/X-off flow control) or **OPT_LINE** (line mode) options. If necessary, support for these options must be implemented at the application level. In addition, polled mode does not support the **FIONREAD**, **FIONWRITE**, or **FIOCANCEL** ioctl( ) functions.

WNDRVR

Raw Mode and Line Mode

A *tty* device operates in one of two modes: *raw mode* (unbuffered) or *line mode*. Raw mode is the default. Line mode is selected by the **OPT_LINE** bit of the device option word.

In *raw mode*, each input character is available to readers as soon as it is input from the device. Reading from a *tty* device in raw mode causes as many characters as possible to be extracted from the input ring, up to the limit of the user's read buffer. Input cannot be modified except as directed by other *tty* option bits.

In *line mode*, all input characters are saved until a **NEWLINE** character is input; then the entire line of characters, including the **NEWLINE**, is made available in the ring at one time. Reading from a *tty* device in line mode causes characters up to the end of the next line to be extracted from the input ring, up to the limit of the user's read buffer. Input can be modified by the special characters CTRL+H (backspace), CTRL+U (line-delete), and CTRL+D (end-of-file).

tty Special Characters

The following special characters are enabled if the *tty* device operates in line mode, that is, with the **OPT_LINE** bit set:

- The backspace character, by default CTRL+H, causes successive previous characters to be deleted from the current line, up to the start of the line. It does this by echoing a backspace followed by a space, and then another backspace.
- The line-delete character, by default CTRL+U, deletes all the characters of the current line.
- The end-of-file (EOF) character, by default CTRL+D, causes the current line to become available in the input ring without a **NEWLINE** and without entering the EOF character itself. Thus if the EOF character is the first character typed on a line, reading that line returns a zero byte count, which is the usual indication of end-of-file.

The following characters have special effects if the *tty* device is operating with the -corresponding option bit set:

- The software flow control characters CTRL+Q and CTRL+S (**XON** and **XOFF**). Receipt of a CTRL+S input character suspends output to that channel. Subsequent receipt of a CTRL+Q resumes the output. Conversely, when the VxWorks input buffer is almost full, a CTRL+S is output to signal the other side to suspend transmission. When the input buffer is empty enough, a CTRL+Q is output to signal the other side to resume transmission. The software flow control characters are enabled by **OPT_TANDEM**.
- The *ROM monitor trap* character, by default CTRL+X. This character traps to the ROM-resident monitor program. Note that this is drastic. All normal VxWorks functioning is suspended, and the computer system is controlled entirely by the monitor. Depending on the particular monitor, it may or may not be possible to restart Vx-Works from the point of interruption. (It is not possible to restart VxWorks if un-handled external interrupts occur during the boot countdown.) The monitor trap character is enabled by **OPT_MON_TRAP**.
- The special *kernel shell abort* character is, by default, CTRL+C. This character restarts the kernel shell if it gets stuck in an unfriendly function, such as one that has taken an unavailable semaphore or is caught in an infinite loop. The kernel shell abort character is enabled by **OPT_ABORT**.

The characters for most of these functions can be changed using the **tyLib** functions shown below.

Character	Description	Modifier
CTRL+H	backspace (character delete)	tyBackspaceSet( )
CTRL+U	line delete	tyDeleteLineSet( )
CTRL+D	EOF (end of file)	tyEOFSet( )
CTRL+C	kernel shell abort	tyAbortSet( )

Character	Description	Modifier
CTRL+X	trap to boot ROMs	tyMonitorTrapSet( )
CTRL+S	output suspend	N/A
CTRL+Q	output resume	N/A

Kernel I/O Control Functions

The *tty* devices respond to the ioctl( ) functions in the following table, defined in **ioLib.h**. For more information, see the reference entries for **tyLib**, **ttyDrv**, and ioctl( ).

Function	Description
FIOBAUDRATE	Sets the baud rate to the specified argument.
FIOCANCEL	Cancels a read or write.
FIODATASYNC	Discards all bytes in the input and output buffers.
FIOFLUSH	Same as **FIODATASYNC**.
FIOGETNAME	Gets the filename of the file descriptor.
FIOGETOPTIONS	Returns the current device option word.
FIONREAD	Gets the number of unread bytes in the input buffer.
FIONWRITE	Gets the number of bytes in the output buffer.
FIOSETOPTIONS	Sets the device option word.
FIOSYNC	Same as **FIODATASYNC**.

CAUTION: **To change the driver's hardware options (for example, the number of stop bits or parity bits), use the ioctl( ) function SIO_HW_OPTS_SET. Because this command is not implemented in most drivers, you may need to add it to your BSP serial driver. The details of how to implement this command depend on your board's serial chip. The constants defined in the header file sioLib.h provide the POSIX definitions for setting the hardware options.**

7. Pipe Devices Overview

Pipes are virtual devices by which tasks communicate with each other through the I/O system.

Tasks write messages to pipes; these messages can then be read by other tasks. Pipe devices are managed by **pipeDrv** and use the kernel message queue facility to bear the actual message traffic. For more information, see the *VxWorks 7 Application Programmer's Guide: Pipes* and related topics.

WNDRVR

8. Null Devices Overview

VxWorks provides both **/null** and **/dev/null** for null devices. The **/null** device is the traditional VxWorks null device, which is provided by default for backward compatibility. The **/dev/null** device is provided by the **BUNDLE_RTP_POSIX_PSE52** component bundle, and is required for conformance with the POSIX PSE52 profile.

Note that the devs shell command lists **/null** and **/dev/null** with other devices, but the ls command does not list **/dev/null** under the VRFS root directory (because the name violates the VRFS naming scheme). Applications can, in any case, use **/null** or **/dev/null** as required.

For information about POSIX PSE52, see the *VxWorks POSIX Programmer's Guide: VxWorks PSE52 Support in Processes* . For information about VRFS, see VRFS on page 16.